# CJ SYSTEM DESIGN INTERVIEW HANDBOOK

## PART 1 — INTERVIEW FRAMEWORK, CAPACITY ESTIMATION & SCALE 0 → MILLIONS

**Target:** Senior Software Engineer / Backend Engineer — ~8 YOE
**Purpose:** Interview recall, not textbook completeness
**Learning order:** First principles → mental model → interview answer → deep dive → traps → memory hook

> **Priority legend**
> 🟢 **MUST KNOW** — weak answer is a senior-level red flag
> 🟡 **SHOULD KNOW** — explain confidently at conceptual level
> 🔵 **DEEP DIVE** — know when interviewer pushes

This part consolidates the interview template, HLD checklist, capacity-estimation material, and “scale from zero to millions” progression from the source notes.

---

# 1. WHAT IS A SYSTEM DESIGN INTERVIEW?

## ELI5

The interviewer is not primarily asking:

> “Can you draw Netflix?”

They are asking:

> “Can you take a vague business problem, identify what matters, make reasonable assumptions, design something that works, find what will break, and explain your trade-offs?”

A strong system-design candidate does **not immediately start drawing boxes**.

The progression is:

```
UNDERSTAND
   ↓
ESTIMATE
   ↓
CHOOSE REQUIREMENTS / TRADE-OFFS
   ↓
DESIGN
   ↓
TRACE DATA
   ↓
```

FIND BOTTLENECKS
↓
SCALE
↓
HANDLE FAILURES
↓
JUSTIFY

That is the fundamental mental model for the entire interview.

---

# 2. THE SENIOR-ENGINEER SYSTEM DESIGN FRAMEWORK

The source notes divide the interview into:

1. Feature expectations
2. Estimations
3. Design goals
4. High-level design
5. Deep dive
6. Justification.

For interview use, memorize this shorter version:

## R → S → A → D → D → R

R  Requirements
S  Scale
A  API + Data
D  Design
D  Deep Dive
R  Reliability / Review

Or:

> **Requirements → Scale → API/Data → Design → Deep Dive → Review**

---

# 3. STEP 1 — REQUIREMENTS

**🟢 MUST KNOW**

Never design before knowing **what you are designing**.

The source calls this **Feature Expectations** and asks about use cases, excluded scenarios, users, number of users, and usage patterns.

---

## 3.1 Functional Requirements

Functional requirements describe:

> **What must the system do?**

Example: Design WhatsApp.

Possible functional requirements:

1. Send one-to-one messages
2. Receive messages
3. Show delivery/read status
4. Store message history
5. Support offline users

Do not immediately add:

groups
stories
payments
calls
stickers
status
communities

unless the interviewer wants them.

### Senior rule

**Narrow the problem before scaling it.**

---

# 4. NON-FUNCTIONAL REQUIREMENTS

These describe:

> **How well must the system behave?**

Typical dimensions:

Latency
Availability
Consistency
Durability
Scalability

Throughput
Security
Reliability
Cost

---

## 4.1 Example

For a banking transfer system:

Correctness → extremely important
Durability → extremely important
Consistency → strong
Availability → important
Latency → important but not above correctness

For Instagram feed:

Availability → extremely important
Read latency → low
Scalability → extremely important
Consistency → eventual consistency often acceptable

The source explicitly asks candidates to identify latency/throughput requirements and reason about consistency versus availability.

---

# 5. THE FIRST QUESTIONS TO ASK

🟢 **MUST KNOW**

You usually need only a handful of useful questions.

## Functional

What are the core use cases?

Which features should we design today?

What is explicitly out of scope?

## Users

How many users?

DAU or MAU?

Geographically distributed?

## Traffic

Read-heavy or write-heavy?

What is the approximate traffic?

Any large traffic spikes?

## Quality

What latency is expected?

Do we prioritize availability or strong consistency?

Can data be temporarily stale?

---

# 6. WHAT NOT TO DO

### Weak candidate

> “Okay, I'll use Kafka, Redis, Cassandra, Kubernetes…”

### Strong candidate

> “Before choosing technologies, I'd like to clarify the main use cases and scale because those determine our storage, consistency and communication choices.”

That sentence alone communicates maturity.

---

# 7. REQUIREMENTS — 30 SECOND MEMORY CARD

FUNCTIONAL
What does it do?

NON-FUNCTIONAL
How well must it do it?

SCALE
How many users / requests?

TRADE-OFFS
Latency?
Consistency?
Availability?

Durability?

OUT OF SCOPE
What are we deliberately not solving?

---

# 8. STEP 2 — CAPACITY ESTIMATION

🟢 **MUST KNOW**

The purpose of estimation is **not mathematical precision**.

It is to determine whether your architecture needs:

1 machine
10 machines
1,000 machines

10 GB
10 TB
10 PB

100 QPS
100K QPS

Orders of magnitude matter more than exact arithmetic.

The source explicitly estimates throughput, read/write traffic, storage, memory/cache size, latency and machine requirements.

---

# 9. THE FOUR ESTIMATES THAT MATTER MOST

For most interviews calculate only:

1. QPS
2. Peak QPS
3. Storage
4. Bandwidth

Sometimes add:

Cache memory
Number of servers
Read/write ratio

Do **not** spend 15 minutes doing arithmetic unless the interviewer wants it.

---

# 10. NUMBERS YOU SHOULD MEMORIZE

## Decimal magnitude

Thousand = $10^3$
Million = $10^6$
Billion = $10^9$
Trillion = $10^{12}$
Quadrillion = $10^{15}$

Useful storage shorthand:

KB ≈ $10^3$ bytes
MB ≈ $10^6$ bytes
GB ≈ $10^9$ bytes
TB ≈ $10^{12}$ bytes
PB ≈ $10^{15}$ bytes

The source uses the same “3 zeros → thousand/KB, 6 → million/MB, 9 → billion/GB…” memory device.

---

# 11. BINARY NUMBERS WORTH REMEMBERING

$2^{10}$ ≈ 1,000
$2^{20}$ ≈ 1 million
$2^{30}$ ≈ 1 billion

And:

1 byte = 8 bits

For system-design estimation, approximate aggressively.

---

# 12. THE MOST USEFUL SYSTEM-DESIGN NUMBER

## 1 million/day ≈ 12 per second

Because:

1 day
= 24 × 60 × 60
≈ 86,400 seconds
≈ 100,000 seconds

Therefore:

1,000,000 / ~100,000
≈ 10/sec

More accurately ≈ 11.6/sec.

Memorize:

1M/day ≈ 12/sec
10M/day ≈ 120/sec
100M/day ≈ 1,200/sec
1B/day ≈ 12,000/sec

The source explicitly uses this rule and examples such as 5M/day ≈ 60/sec, 30M/day ≈ 360/sec and 100M/day ≈ 1,200/sec.

---

# 13. USERS ARE NOT REQUESTS

This is one of the most common estimation mistakes.

Suppose:

1M users/day

That does **not** mean:

1M requests/day

If each user makes:

50 requests/session

then:

1M × 50
= 50M requests/day

50M × 12 QPS per million/day
≈ 600 requests/sec average

This exact style of calculation appears in the source notes.

---

# 14. AVERAGE QPS IS NOT PEAK QPS

🟢 **MUST KNOW**

Systems fail at peaks, not averages.

If:

Average QPS = 1,000

your design should not necessarily support only:

1,000 QPS

You might assume:

Peak = 2×
Peak = 3×
Peak = 5×

depending on the product.

Example:

Average 1,000 QPS
Peak factor 3×

Peak QPS 3,000

In an interview, say your assumption explicitly:

> “I'll assume peak traffic is approximately 3× average unless you want a different traffic distribution.”

---

# 15. READ VS WRITE QPS

Suppose:

Total QPS = 10,000

Read/write = 90/10

Then:

Read QPS = 9,000
Write QPS = 1,000

Why do we care?

Because architecture changes dramatically.

### Read-heavy

Likely useful:

Cache
CDN
Read replicas
Precomputation

### Write-heavy

Think harder about:

Partitioning
Sequential writes
Batching
Queueing
Write amplification
Index overhead

---

# 16. STORAGE ESTIMATION

Formula:

Storage
=
objects/day
× bytes/object
× retention
× replication factor

---

## Example

Suppose:

10M messages/day

average message = 1 KB

retain = 5 years

Approximate:

10M × 1KB
≈ 10GB/day

10GB × 365
≈ 3.65TB/year

× 5
≈ 18TB

If replicated three times:

≈ 54TB physical storage

Interview conclusion:

> “This clearly exceeds the scale of a single-machine dataset, so horizontal storage and replication become relevant.”

Notice that the **architectural conclusion** matters more than getting 18.25 vs 18.3 TB.

---

# 17. BANDWIDTH ESTIMATION

Formula:

Bandwidth
≈ QPS × average payload size

Example:

10,000 requests/sec
×
100 KB response

Approximately:

1 GB/sec

Then:

≈ 8 Gbps

because:

1 byte = 8 bits

---

# 18. SERVER ESTIMATION

If one application server can safely handle:

2,000 QPS

and peak traffic is:

20,000 QPS

minimum theoretical servers:

20,000 / 2,000

= 10

But you normally don't provision at 100%.

You need:

headroom
failover
deployments
traffic spikes

So perhaps:

15–20 servers

depending on assumptions.

### Important

Do not pretend these numbers are universally known constants.

They depend on:

CPU
payload
business logic
DB calls
network calls
JVM behavior
hardware
latency target

---

# 19. CAPACITY-ESTIMATION CHECKLIST

The source lists peak time, transaction count, read/write counts, throughput, latency, bandwidth, server requirements, and load-balancer requests.

For interviews, compress that into:

USERS
↓
REQUESTS / USER
↓
REQUESTS / DAY
↓
AVERAGE QPS
↓
PEAK QPS
↓
READ / WRITE QPS
↓

PAYLOAD SIZE
↓
BANDWIDTH
↓
RETENTION
↓
STORAGE

---

# 20. CAPACITY ESTIMATION — INTERVIEW ANSWER

### Q: How do you perform back-of-the-envelope estimation?

> “I first estimate users and requests per user to derive daily traffic and QPS. I then apply a reasonable peak multiplier and split traffic into reads and writes. For data-heavy systems, I estimate storage from objects × object size × retention and bandwidth from QPS × payload size. The goal isn't exact forecasting; it's understanding the order of magnitude so I can justify architectural choices.”

That is enough for most interviews.

---

# 21. ESTIMATION TRAPS

## Trap 1 — Excessive precision

Bad:

31,536,000 seconds/year

Usually unnecessary.

Use:

~30M seconds/year

---

## Trap 2 — Forgetting peak traffic

Average is not capacity requirement.

---

## Trap 3 — Confusing users with requests

Always ask:

requests/user/session?

---

## Trap 4 — Doing calculations with no architectural consequence

If the number does not change a design choice, don't spend 10 minutes calculating it.

---

# 22. WHEN CAN YOU SKIP ESTIMATION?

Not every interview requires substantial estimation.

For example:

> Design a distributed rate limiter.

It may be more useful to clarify:

requests/sec
number of clients
number of regions
consistency requirements

than to calculate 5-year storage.

### Senior principle

> **Estimate only what influences architecture.**

---

# 23. STEP 3 — DESIGN GOALS

🟢 **MUST KNOW**

Before designing, establish the important trade-offs.

The source highlights:

Latency
Throughput
Consistency
Availability

as core design goals.

---

# 24. LATENCY VS THROUGHPUT

## Latency

Time for **one operation**.

Example:

request
→ 80 ms
→ response

---

## Throughput

How much work can be performed per unit time.

Example:

50,000 requests/sec

---

## Interview trap

Low latency does not automatically mean high throughput.

A system could process:

one request in 5 ms

but only allow one request at a time.

Or:

10,000 requests simultaneously

but each take 500 ms.

---

# 25. CONSISTENCY VS AVAILABILITY — FIRST PRINCIPLES

For now, use this mental model:

### Consistency

> If data changes, how quickly and reliably does everyone observe the correct/latest value?

### Availability

Can the system continue serving requests when components fail?

We will deep-dive CAP, linearizability, quorum and eventual consistency later.

For Part 1, you need to recognize the trade-off.

---

# 26. EXAMPLE: BANK BALANCE VS SOCIAL FEED

## Bank balance

Suppose:

balance = ₹100

and two concurrent withdrawals arrive.

Showing stale or conflicting values can lose money.

Prefer stronger correctness guarantees.

---

## Social feed

If someone posts a photo and another user sees it:

2 seconds later

instead of:

immediately

that may be acceptable.

Availability and scale may matter more than instantaneous global consistency.

---

# 27. STEP 4 — API DESIGN

🟢 **MUST KNOW**

Before drawing twenty services, define how the outside world interacts with the system.

Example: URL shortener.

POST /urls

Request:

```
{
  "longUrl": "https://example.com/very-long-url"
}
```

Response:

```
{
  "shortCode": "aB7xQ2"
}
```

Read:

GET /aB7xQ2

Response:

301/302 redirect

---

# 28. WHY API DESIGN COMES EARLY

Because APIs expose:

system responsibilities
data requirements
read/write patterns
idempotency requirements

For example:

POST /payments

immediately raises:

> What happens if the client times out and retries?

That becomes an **idempotency** problem.

---

# 29. API INTERVIEW QUESTIONS

🟢 **What are the core APIs?**

🟢 **Which operation creates data?**

🟢 **Which operations must be idempotent?**

🟢 **What does the client send?**

🟡 **Pagination?**

🟡 **Versioning?**

🟡 **Rate limiting?**

🟡 **Authentication?**

---

# 30. STEP 5 — DATA MODEL

🟢 **MUST KNOW**

Before choosing MongoDB, Cassandra, Postgres or DynamoDB, answer:

> **What data exists and how will it be accessed?**

Example: chat.

User
---
user_id
name

Conversation
---
conversation_id
type

Message
---
message_id
conversation_id
sender_id
content
created_at

Then ask:

How are messages read?

By message_id?
By conversation_id?
By user_id?
By timestamp?

### Critical principle

> **Access patterns should drive storage design.**

Not:

> “I know Cassandra, so let's use Cassandra.”

---

# 31. STEP 6 — HIGH-LEVEL DESIGN

The source recommends covering APIs, database schema, a basic algorithm and the read/write HLD before diving deeper.

Start simple.

```
CLIENT
  |
  v
LOAD BALANCER
  |
  v
APPLICATION
  |
  v
DATABASE
```

Explain the request.

Do not begin with:

```
25 microservices
7 Kafka clusters
4 databases
3 caches
```

unless there is a reason.

---

# 32. THE GOLDEN RULE OF HLD

> **Every box must solve a problem.**

Do not add Redis because:

> “system designs have Redis.”

Say:

> “The read path is much hotter than the write path and these objects are repeatedly requested, so I'll introduce Redis to reduce database reads and improve latency.”

Do not add Kafka because:

> “Kafka scales.”

Say:

> “This work does not have to complete synchronously, so a durable queue lets us decouple request latency from downstream processing.”

---

# 33. TRACE THE WRITE PATH

🟢 **MUST KNOW**

Always explain what happens to one write.

Example:

Client
↓
Load Balancer
↓
API Server
↓
Validate
↓
Database Write
↓
Event / Queue
↓
Async Consumers

Ask:

Where can it fail?

Can it retry?

Could retry duplicate work?

When do we acknowledge success?

What if DB succeeds but event publication fails?

That's where senior-level discussion begins.

---

# 34. TRACE THE READ PATH

Example:

Client
↓
CDN
↓
Load Balancer
↓
App
↓
Cache
↓ miss
Database

Explain:

cache hit
cache miss
cache expiration
stale data

---

# 35. STEP 7 — DEEP DIVE

The source specifically recommends scaling the algorithm and individual components, then reasoning about availability, consistency and scale.

You do **not** need to deep-dive every component.

Let the interviewer choose or choose the largest risk yourself.

Good deep dives:

Database partitioning
Cache consistency
Messaging guarantees
Feed generation
Search
Rate limiting
Hot keys
Replication
Failure recovery

---

# 36. THE SENIOR DEEP-DIVE FRAMEWORK

For any component ask:

1. Why is it here?

2. What load does it handle?

3. What state does it hold?

4. How does it scale?

5. What happens if it fails?

6. Is there a SPOF?

7. What consistency does it provide?

8. What is the bottleneck?

9. What happens at 10× traffic?

That sequence is worth memorizing.

---

# 37. STEP 8 — JUSTIFY

The source finishes its interview template by asking candidates to justify throughput, latency at each layer and overall latency.

This is where you prove the boxes weren't random.

Example:

> “I introduced the cache because this is a read-heavy workload and the same records are requested repeatedly. That removes a large fraction of reads from the primary database.”
>
> “I introduced a queue because notification delivery doesn't need to block the user request.”
>
> “I partitioned by user ID because most requests are scoped to one user and that gives stable distribution.”

---

# 38. THE ONE-SENTENCE JUSTIFICATION RULE

For every important component, be able to say:

> **I chose X because Y, accepting Z trade-off.**

Examples:

> “I chose asynchronous processing because the work does not need to complete before responding, accepting eventual completion.”
>
> “I chose replication because we need high availability and additional read capacity, accepting replication lag.”
>
> “I chose sharding because one database can no longer handle the dataset/traffic, accepting cross-shard complexity.”

That is system-design thinking.

---

# 39. SYSTEM DESIGN INTERVIEW MASTER FLOW

Memorize this.

1. REQUIREMENTS
   What are we building?

2. SCALE
   How much traffic/data?

3. DESIGN GOALS
   Latency?
   Availability?
   Consistency?

4. API
   How do clients interact?

5. DATA MODEL
   What do we store?
   How is it queried?

6. HIGH-LEVEL DESIGN
   Client → service → storage

7. READ PATH
   Trace one read

8. WRITE PATH
   Trace one write

9. SCALE
   What breaks first?

10. RELIABILITY
    What happens when components fail?

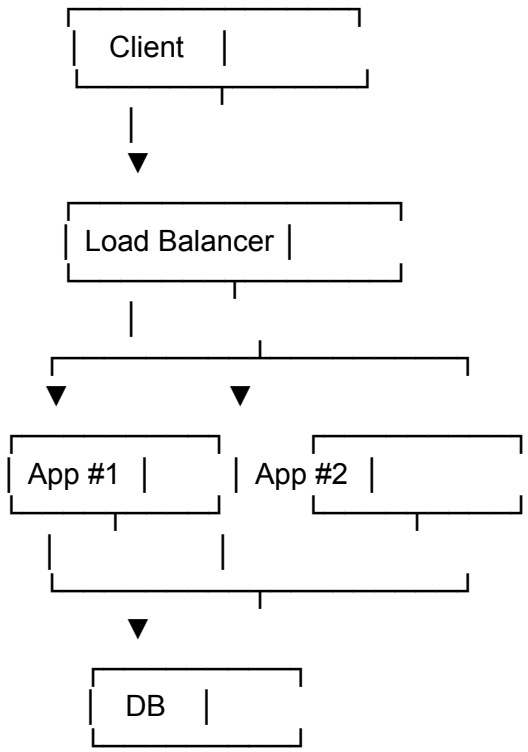
11. DEEP DIVE
    Most important subsystem

12. TRADE-OFFS
    Why this choice?

---

# 40. WHAT SHOULD YOU DRAW FIRST?

A safe starting point:




Then evolve architecture **only when a bottleneck appears**.

That leads naturally into the next major mental model.

---

# 41. SCALE FROM ZERO → MILLIONS

🟢 **MUST KNOW**

Your source has a useful nine-step progression:

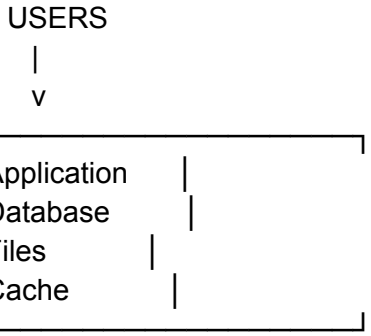
1. Single server
2. Separate application and DB
3. Load balancer + multiple app servers
4. Database replication

5. Cache
6. CDN
7. Multiple data centers
8. Async processing / queues
9. Database scaling / sharding

Instead of memorizing nine technologies, understand **which bottleneck causes each evolution**.

---

# 42. STAGE 0 — ONE MACHINE




For a small product this may be completely fine.

### Benefits

Simple
Cheap
Easy deployment
Easy debugging
Low operational overhead

### Problem

Everything shares:

CPU
RAM
disk
network
failure domain

If the server dies:

everything dies

---

# 43. INTERVIEW LESSON

Do not over-engineer the first version.

### Strong answer

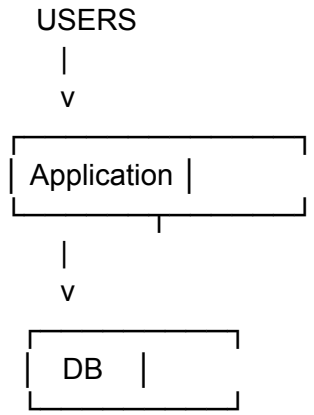
> “I would start with a simple architecture and introduce additional components when scale or reliability requirements justify them.”

This is usually more mature than throwing microservices at a small workload.

---

# 44. STAGE 1 — SEPARATE APPLICATION AND DATABASE

Traffic grows.




The source describes this as the first major scaling step once one application server cannot handle growth.

---

## Why separate them?

Because application and database workloads differ.

Application server may need:

CPU
network
horizontal replicas

Database may need:

RAM
disk IOPS
storage
durability

Now they can scale independently.

---

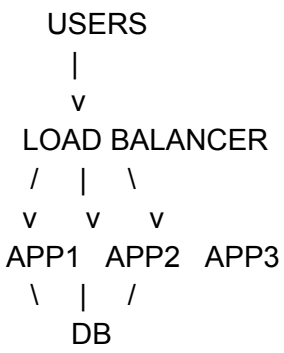
# 45. STAGE 2 — MULTIPLE APPLICATION SERVERS

Soon:

one application server

cannot handle traffic.

Add replicas.




The source explicitly describes clustering application servers and placing a load balancer in front to distribute incoming requests.

---

# 46. WHY A LOAD BALANCER?

**🟢 MUST KNOW**

A load balancer distributes incoming traffic among backend instances.

It can also support:

Health checks
TLS termination
Routing
Failover
Weighted traffic
Rolling deployments

We will deep-dive L4/L7 and algorithms later.

For now remember:

ONE APP SERVER BOTTLENECK
↓
MULTIPLE APP SERVERS
↓
NEED SOMEONE TO DISTRIBUTE REQUESTS

↓
LOAD BALANCER

---

# 47. WHY SHOULD APP SERVERS BE STATELESS?

🟢 **COMMON INTERVIEW QUESTION**

Suppose:

Request 1 → App A
Request 2 → App B

If login/session state lives only in App A's memory:

App B doesn't know the user.

This makes horizontal scaling difficult.

Prefer keeping shared state in something like:

database
distributed cache
external session store
token carried by client

depending on architecture.

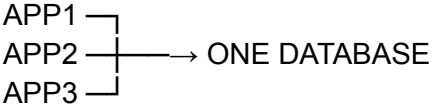
Then any instance can serve the request.

---

# 48. STAGE 3 — DATABASE BECOMES THE BOTTLENECK

Application servers now scale horizontally.

But:




Eventually the database receives too much load.

Possible strategies include:

Better queries

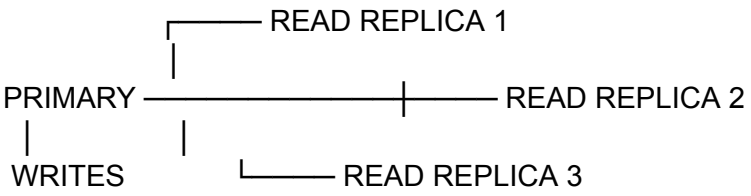
Indexes
Caching
Read replicas
Vertical scaling
Partitioning
Sharding

Do not immediately shard.

---

# 49. READ REPLICAS

Basic pattern:




Writes:

→ primary

Reads:

→ replicas

Useful when:

reads >> writes

---

## 🟨 SOURCE CORRECTION / INTERVIEW NOTE

The source says directing frequent reads to replicas can “greatly increase database write throughput.”

Do **not** memorize that literally.

Read replicas primarily:

> **offload reads from the primary and increase read capacity.**

They may indirectly free primary resources, but they do not fundamentally scale the primary's write path.

That distinction is important in senior interviews.

---

# 50. REPLICATION TRADE-OFF

Replication introduces:

replication lag

Example:

WRITE → Primary

immediately READ → Replica

The replica might still contain the old value.

This is a classic distributed-system trade-off.

We will deep dive later.

---

# 51. STAGE 4 — ADD CACHE

Imagine:

GET product 123

is requested thousands of times.

Without cache:

Request
↓
Database

Request
↓
Database

Request
↓
Database

Repeated identical work.

With cache:

Request
↓
Cache
/ \
hit miss
| |

return DB

---

# 52. WHY CACHE?

Lower latency
Reduce database load
Increase read throughput
Absorb hot reads

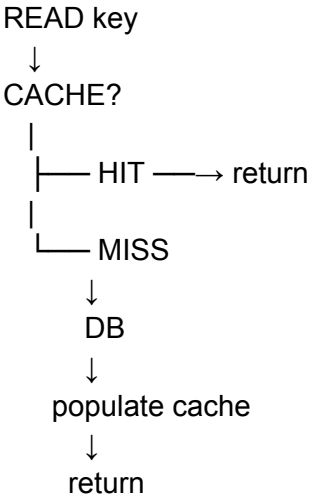
Typical memory systems:

Redis
Memcached

The source places a cache between application servers and the database as an early scaling step.

---

# 53. BASIC CACHE-ASIDE FLOW




Pseudo-code:

```
Value value = cache.get(key);

if (value != null) {
    return value;
}

value = database.get(key);

cache.put(key, value);

return value;
```

---

# 54. CACHE TRADE-OFFS

Caching creates new problems:

stale values
cache invalidation
hot keys
cache stampede
eviction
memory cost

### Senior answer

> “Caching trades freshness and operational complexity for lower latency and reduced backend load.”

---

# 55. STAGE 5 — CDN

Suppose users in India request an image stored in a US data center.

Without CDN:

India
↓
many network hops
↓
US origin

With CDN:

India user
↓
nearby edge
↓
cached image

---

# 56. WHAT IS A CDN?

A CDN is a geographically distributed network of edge servers that caches content closer to users.

Typical content:

Images
CSS
JavaScript
Video chunks

Static downloads

Benefits:

lower latency
lower origin traffic
higher throughput
better geographic distribution

The source places CDN after caching and before multi-data-center scaling.

---

# 57. CACHE VS CDN

### Application cache

Usually accelerates:

application/database data

Example:

user profile
product metadata
permissions

### CDN

Usually accelerates:

content delivered geographically to users

Example:

images
video
CSS
JS
downloads

---

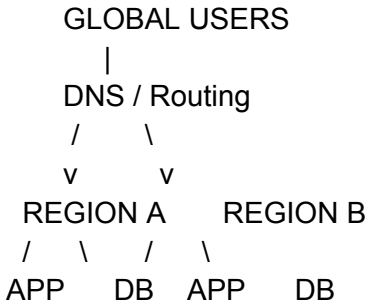
# 58. STAGE 6 — MULTIPLE DATA CENTERS / REGIONS

One data center creates risks:

regional outage
high latency for remote users

capacity limit
disaster

Move toward:




The source describes moving from one data center to multiple data centers and routing users based on geography.

---

# 59. MULTI-REGION CREATES HARD QUESTIONS

Adding another region is not just:

copy servers

It creates distributed-system questions:

Where do writes go?

How is data replicated?

What if regions disagree?

Active-active or active-passive?

How do users fail over?

What latency exists between regions?

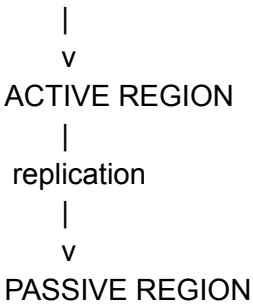
How do we prevent duplicate writes?

This is where system design becomes substantially harder.

---

# 60. ACTIVE-PASSIVE






Passive region primarily waits for failover.

### Advantages

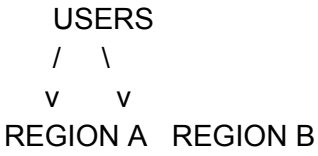
simpler data ownership
simpler conflict handling

### Disadvantages

idle capacity
failover delay

---

# 61. ACTIVE-ACTIVE




Both accept traffic.

### Advantages

lower global latency
better capacity utilization
greater regional availability

### Challenges

conflicting writes
cross-region replication
consistency
data ownership

---

# 62. STAGE 7 — ASYNCHRONOUS PROCESSING

Suppose checkout does:

create order

charge payment
update inventory
send email
send push notification
generate analytics
update recommendation system

Should the user wait for **all of these**?

Usually not.

Separate:

CRITICAL SYNCHRONOUS PATH

from:

ASYNC SIDE EFFECTS

---

# 63. BEFORE QUEUE

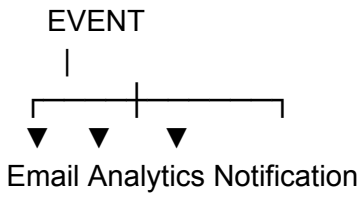
Client
↓
Order Service
↓
Payment
↓
Inventory
↓
Email
↓
Analytics
↓
Response

Latency and failure coupling increase.

---

# 64. AFTER QUEUE

Client
↓
Order Service
↓
Core transaction
↓
Publish Event
↓
Return Response




The source introduces message queueing once fully synchronous processing becomes undesirable.

---

# 65. WHY ASYNCHRONOUS PROCESSING?

Lower user-facing latency
Temporal decoupling
Traffic buffering
Failure isolation
Independent consumer scaling

But you now need to handle:

retries
duplicates
ordering
dead-letter messages
backpressure
eventual consistency

We will cover these deeply in the messaging chapter.

---

# 66. STAGE 8 — SCALE THE DATABASE

Eventually even with:

indexes
cache
read replicas

you may hit limits in:

data size
write throughput
IO
CPU
memory

Two high-level directions:

VERTICAL SCALE

or
HORIZONTAL SCALE

---

# 67. VERTICAL SCALING

Scale **up**.

8 CPU  → 32 CPU
32GB RAM → 256GB RAM
1TB SSD  → 8TB SSD

Benefits:

simple
minimal architecture change

Problems:

hardware ceiling
expensive high-end machines
larger failure blast radius

---

# 68. HORIZONTAL SCALING

Scale **out**.

1 machine
  ↓
many machines

Benefits:

greater theoretical scale
fault tolerance opportunities
incremental capacity

Costs:

coordination
partitioning
distributed failures
network latency
consistency complexity

---

# 69. IMPORTANT TERMINOLOGY CORRECTION

🟨 **INTERVIEW CORRECTION**

The source's scaling example says:

> “Vertical partition — adding more CPU/RAM.”

For interview purposes, distinguish:

### Vertical scaling

bigger machine

### Vertical partitioning

Split by columns/features/tables.

Example:

User core data

separated from:

User profile/blob data

### Horizontal partitioning

Split rows.

Example:

users 1–1M
users 1M–2M
...

### Sharding

Usually horizontal partitioning **across independent nodes/databases**.

We will deep dive later.

---

# 70. SHARDING — FIRST PRINCIPLES

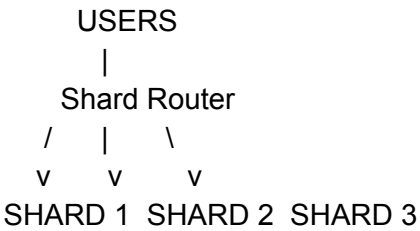
Suppose:

1 database

can no longer handle:

5 TB
100K write QPS

Split data:




Possible shard keys:

user_id
region
tenant_id
customer_id

---

# 71. WHAT MAKES A GOOD SHARD KEY?

🟢 **COMMON SENIOR QUESTION**

A good shard key generally provides:

High cardinality
Even distribution
Stable mapping
Locality for common queries
Low hotspot risk

Bad example:

country

if:

70% users = one country

One shard becomes hot.

---

# 72. SHARDING TRADE-OFFS

Advantages:

Scale storage
Scale writes
Scale query workload

Costs:

Cross-shard queries
Cross-shard joins
Distributed transactions
Rebalancing
Hot shards
Operational complexity

### Memory hook

> **Sharding solves machine limits by buying distributed-systems complexity.**

---

# 73. SCALE 0 → MILLIONS: THE REAL MENTAL MODEL

Do not memorize:

LB
Redis
CDN
Kafka
Cassandra

Memorize the causal chain.

```
ONE MACHINE
   |
   | resource contention / SPOF
   v

SEPARATE APP + DB
   |
   | app capacity
   v

MULTIPLE APP SERVERS
   |
   | need traffic distribution
   v

LOAD BALANCER
   |
   | database reads become hot
   v

CACHE / READ REPLICAS
   |
   | static/geographic delivery
   v
```

CDN
  |
  | regional latency / outages
  v

MULTI-REGION
  |
  | slow/non-critical work
  v

ASYNC QUEUES
  |
  | database size/write limits
  v

PARTITION / SHARD

This is one of the highest-ROI mental models in system design.

---

# 74. IMPORTANT: THIS IS NOT A MANDATORY SEQUENCE

Real systems don't always grow in this exact order.

For example:

A video product may need:

CDN

very early.

A financial system may prioritize:

durability + replication

before huge scale.

A telemetry platform may need:

queues + partitioned storage

almost immediately.

So say:

> “This is a useful scaling progression, not a universal recipe. Components should be introduced to satisfy an actual requirement or bottleneck.”

---

# 75. INTERVIEWER: “HOW WOULD YOU SCALE THIS?”

Do not dump technologies.

Use:

1. Find bottleneck
2. Measure its resource constraint
3. Remove / distribute / cache / async it
4. Identify the new bottleneck

Example:

> “If the application tier becomes CPU-bound, I can scale it horizontally behind a load balancer. If the database then becomes read-bound, I can introduce caching or read replicas. If write throughput or storage becomes the limiting factor, I'd consider partitioning.”

That is a much stronger answer than:

> “Use Kubernetes, Kafka and Cassandra.”

---

# 76. HORIZONTAL VS VERTICAL SCALING — INTERVIEW Q&A

### Q: What is vertical scaling?

> Increasing resources on one machine, such as CPU, RAM or storage.

### Q: Advantages?

> Simpler architecture and usually minimal application changes.

### Q: Disadvantages?

> Hardware limits, expensive large machines and potentially larger failure blast radius.

---

### Q: What is horizontal scaling?

> Adding more machines and distributing workload among them.

### Q: Advantages?

Higher potential scale, incremental capacity and improved fault tolerance.

### Q: Disadvantages?

Distributed coordination, state management, networking and consistency become more complex.

---

# 77. LOAD BALANCER — INTERVIEW Q&A

### 🟢 Q: Why do we need a load balancer?

Once multiple application instances exist, clients need a way to route traffic among healthy instances. A load balancer distributes requests and can also support health checks, failover, TLS termination and routing policies.

---

### 🟢 Q: Can the load balancer itself become a SPOF?

Yes.

Common production approaches include:

multiple LB instances
managed load balancer
active-passive
active-active
DNS / anycast mechanisms

Exact implementation depends on environment.

---

# 78. STATELESS SERVICES — INTERVIEW Q&A

### 🟢 Q: Why are stateless application servers easier to scale?

Because any healthy instance can process any request. We don't need a client to repeatedly return to the same instance just because its session state lives in memory.

---

### Q: Does stateless mean the application has no state anywhere?

No.

It generally means:

> the individual application instance does not need unique local conversational state to serve the next request.

State can still exist in:

database
Redis
object store
client token

---

# 79. READ REPLICA — INTERVIEW Q&A

### 🟢 Q: Why add read replicas?

> To increase read capacity and remove read workload from the primary database.

---

### Q: What's the trade-off?

> Replication lag can cause stale reads.

---

### Q: Do replicas scale writes?

Normally, not the primary write path.

For true write scaling you may need:

partitioning
multi-leader architecture
leaderless architecture
different storage model

depending on requirements.

---

# 80. CACHE — INTERVIEW Q&A

### 🟢 Q: When would you introduce caching?

> When reads are repeated, expensive or significantly outnumber writes and slightly stale data is acceptable.

---

### Q: Why not cache everything?

Because caching adds:

memory cost
staleness
invalidation complexity
eviction
hot-key issues

and data with low reuse may provide little benefit.

---

# 81. QUEUE — INTERVIEW Q&A

### 🟢 Q: When should work become asynchronous?

> When the operation does not need to complete before the client receives its response, or when buffering and decoupling downstream systems improves reliability or throughput.

---

### Q: What new problems does a queue introduce?

duplicate processing
retry policy
ordering
consumer lag
dead-letter handling
eventual consistency

---

# 82. SHARDING — INTERVIEW Q&A

### 🟢 Q: When would you shard?

> When one database can no longer satisfy storage, write-throughput or workload requirements after simpler options such as indexing, caching, replication and vertical scaling are insufficient.

---

### Q: Why not shard early?

> Because sharding introduces cross-shard queries, rebalancing, distributed transactions, hotspots and significant operational complexity.

---

# 83. MICROservices: IMPORTANT SENIOR INTERVIEW CORRECTION

The source HLD cheat sheet says:

> “Start architecting the system with microservices.”

Do **not** turn this into:

> “Every HLD should start as microservices.”

A better interview rule is:

> **Start with logical components. Decide deployment boundaries only when justified.**

Microservices are useful when you need things such as:

independent deployment
independent scaling
clear ownership boundaries
different reliability requirements
different technology/runtime needs

But they cost:

network failures
distributed tracing
deployment complexity
versioning
service discovery
distributed transactions
operational overhead

---

# 84. MONOLITH VS MICROSERVICES — 30 SECOND ANSWER

> “A monolith keeps deployment and in-process transactions simple, while microservices allow independently deployable and scalable components. I wouldn't choose microservices purely because the system is large; I'd use them when service boundaries, ownership, scaling or deployment independence justify the distributed-system complexity.”

Excellent senior-level answer.

---

# 85. THE “WHAT BREAKS NEXT?” METHOD

This is one of the best ways to conduct an interview.

At every design stage ask:

> **What breaks next?**

Example:

One server
↓
server capacity breaks

Many app servers
↓
database breaks

Read replicas
↓
write path breaks

Sharding
↓
cross-shard operations become hard

Multi-region
↓
consistency becomes hard

System design is essentially:

> **Repeatedly move the bottleneck while controlling complexity.**

---

# 86. SPOF — SINGLE POINT OF FAILURE

🟢 **MUST KNOW**

A component is a single point of failure when:

> Failure of that one component can make the entire system unavailable.

Examples:

one DB
one load balancer
one queue broker
one metadata server
one region

Always scan your architecture for SPOFs.

---

# 87. SYSTEM DESIGN IS TRADE-OFF MANAGEMENT

There is almost never a universally perfect design.

Typical tensions:

Consistency ↔ Availability
Latency ↔ Durability
Freshness ↔ Caching
Simplicity ↔ Scalability
Cost ↔ Redundancy
Synchronous ↔ Decoupled
Normalization ↔ Read performance

Interviewers often care more about whether you **see the trade-off** than whether you choose their preferred technology.

---

# 88. THE SENIOR-LEVEL LANGUAGE TO USE

Instead of:

> “Redis is very fast.”

Say:

> “Because this workload is read-heavy with repeated access to the same objects, an in-memory cache can remove significant database load and lower read latency.”

Instead of:

> “Kafka is scalable.”

Say:

> “The downstream processing doesn't need to complete synchronously, and traffic may arrive in bursts, so a durable log/queue can decouple producers from consumers and absorb temporary imbalance.”

Instead of:

> “We should shard MongoDB.”

Say:

> “Before sharding I'd verify whether indexing, caching, replicas or vertical scaling solve the bottleneck, because sharding introduces significant operational and query complexity.”

That sounds like an engineer who has operated systems, not someone reciting a cheat sheet.

---

# 89. QUESTIONS AN 8-YOE ENGINEER SHOULD ANSWER CLEANLY

## 🟢 Interview floor

You should be able to answer all of these without struggling:

**1. What questions do you ask before starting a system design?**

**2. Functional vs non-functional requirements?**

**3. Latency vs throughput?**

**4. How do you estimate QPS?**

**5. Average vs peak QPS?**

**6. How do you estimate storage?**

**7. Why separate app and DB servers?**

**8. Why do app servers ideally remain stateless?**

**9. Why do we need a load balancer?**

**10. Horizontal vs vertical scaling?**

**11. What problem does a cache solve?**

**12. When should you not cache?**

**13. What problem do read replicas solve?**

**14. What is replication lag?**

**15. Why use a CDN?**

**16. Why use asynchronous processing?**

**17. What problem does a message queue solve?**

**18. When would you shard?**

**19. Why is sharding expensive architecturally?**

**20. What makes a good shard key?**

**21. What is a SPOF?**

**22. Why shouldn't you begin every design with microservices?**

**23. What would you do if traffic became 10× tomorrow?**

**24. What would happen if a region failed?**

**25. How do you justify each major component in your design?**

If those 25 answers are automatic, your foundation is strong.

---

# 90. RAPID-FIRE MODEL ANSWERS

## Q1. What are the first things you clarify?

> Core use cases, scale, read/write pattern, latency requirements, consistency/availability requirements and what's out of scope.

---

## Q2. How do you estimate QPS?

> Estimate daily operations and divide by roughly 100K seconds/day. One million operations/day is about 12 QPS. Then apply a peak multiplier.

---

## Q3. What do you calculate for capacity planning?

> Primarily peak QPS, read/write QPS, storage and bandwidth. I calculate additional numbers only when they affect architectural decisions.

---

## Q4. Horizontal vs vertical scaling?

> Vertical scaling gives one machine more resources. Horizontal scaling distributes work across additional machines. Vertical is simpler but bounded; horizontal provides greater scale at the cost of distributed-system complexity.

---

## Q5. Why use read replicas?

To scale reads and reduce read pressure on the primary, accepting potential replication lag.

---

## Q6. Why cache?

To serve frequently accessed data faster and reduce repeated backend work.

---

## Q7. What's the hardest part of caching?

Keeping cache and source-of-truth data acceptably consistent while dealing with expiration, invalidation and concurrent misses.

---

## Q8. Why queue?

To decouple work temporally, buffer bursts, lower synchronous latency and scale producers and consumers independently.

---

## Q9. Why shard?

To distribute storage or write workload once one database can no longer meet requirements.

---

## Q10. Biggest downside of sharding?

Cross-shard operations, data rebalancing, distributed transactions and hot partitions become significantly harder.

---

# 91. THE 5-MINUTE REVISION SHEET

Before an interview, read only this.

SYSTEM DESIGN

1. REQUIREMENTS
   Functional
   Non-functional
   Out of scope

2. SCALE
   Users
   Requests/user
   QPS
   Peak QPS
   Read/write
   Storage
   Bandwidth

3. GOALS
   Latency
   Throughput
   Availability
   Consistency
   Durability

4. INTERFACE
   APIs

5. DATA
   Entities
   Access patterns
   Storage choice

6. HLD
   Client
   LB
   Service
   Cache
   DB
   Queue

7. TRACE
   Read path
   Write path

8. SCALE
   Replicas
   Cache
   CDN
   Partitioning

9. FAILURE
   SPOF
   Retry
   Failover
   Degradation

10. JUSTIFY
    Why X?
    What trade-off?

---

# 92. SCALE 0 → MILLIONS — 60-SECOND MEMORY SHEET

SINGLE SERVER
↓
simple but limited + SPOF

APP / DB SEPARATION
↓
independent resources

MULTIPLE APP SERVERS
↓
horizontal app scale

LOAD BALANCER
↓
traffic distribution

READ REPLICA
↓
read scale

CACHE
↓
lower latency + DB offload

CDN
↓
content near users

MULTI-REGION
↓
global latency + availability

QUEUE
↓
async + buffering + decoupling

SHARDING
↓
storage/write scale

But:

> **Do not introduce a component unless a requirement or bottleneck justifies it.**

---

# 93. THE ONE INTERVIEW MANTRA TO REMEMBER

When stuck, return to:

WHAT ARE WE BUILDING?
↓
HOW BIG IS IT?
↓
WHAT IS THE SIMPLEST DESIGN?
↓
WHAT BREAKS FIRST?
↓
HOW DO WE FIX THAT?
↓
WHAT NEW TRADE-OFF DID WE CREATE?

If you can keep doing that for 45 minutes, you can conduct a strong system-design interview even if you forget the name of an obscure technology.

---

# 94. PART 1 — MEMORY MAP

SYSTEM DESIGN
│
├── Requirements
│ ├── Functional
│ ├── Non-functional
│ └── Out of scope
│
├── Estimation
│ ├── QPS
│ ├── Peak
│ ├── R/W
│ ├── Storage
│ └── Bandwidth
│
├── API
│
├── Data Model
│
├── HLD
│ ├── Read Path
│ └── Write Path
│
├── Scaling
│ ├── Vertical
│ └── Horizontal

```
│
├── Components
│   ├── Load Balancer
│   ├── Cache
│   ├── Replica
│   ├── CDN
│   ├── Queue
│   └── Shard
│
├── Reliability
│   └── SPOF
│
└── Trade-offs
    └── WHY THIS DESIGN?
```

---

# 95. PART 1 — FINAL INTERVIEW CHECK

You are ready to move on when you can explain, **without notes**:

> “I begin by narrowing functional and non-functional requirements. Then I estimate enough traffic and data to understand the order of magnitude. I define latency, availability and consistency goals, sketch the core APIs and data model, and begin with the simplest architecture. I trace the read and write paths and then identify bottlenecks. I introduce scaling mechanisms such as multiple stateless app servers, load balancing, caching, replication, asynchronous processing or sharding only when a requirement justifies them. For each decision I explain the failure mode and trade-off.”

If you can say that naturally and then expand every sentence, **Part 1 is learned**.

---

# CJ SYSTEM DESIGN INTERVIEW HANDBOOK

## PART 2 — NETWORKING FOR SYSTEM DESIGN

**Target:** Senior Software Engineer / Backend Engineer — ~8 YOE
**Purpose:** Understand the network well enough to reason about latency, failures, APIs, load balancers, microservices and distributed systems.

> **Priority legend**
> 🟢 **MUST KNOW** — senior interview floor
> 🟡 **SHOULD KNOW** — explain confidently
> 🔵 **DEEP DIVE** — interviewer-push territory

The source notes move from physical/link/network/transport/application layers through MAC/IP/NAT, TCP/UDP, HTTP, HTTP evolution, DNS and CDN.

# 1. THE NETWORKING MENTAL MODEL

Before memorizing protocols, understand the problem.

Two programs on different machines want to communicate.

They need answers to progressively different questions:

1. HOW ARE BITS PHYSICALLY MOVED?
↓
Physical

2. WHICH DEVICE ON MY LOCAL NETWORK?
↓
MAC / Link

3. WHICH MACHINE ACROSS NETWORKS?
↓
IP

4. WHICH APPLICATION ON THAT MACHINE?
↓
TCP/UDP + Port

5. HOW SHOULD THE APPLICATIONS SPEAK?
↓
HTTP / gRPC / WebSocket / etc.

For backend interviews, think:

APPLICATION
HTTP / gRPC / DNS / WebSocket
↓
TRANSPORT
TCP / UDP / QUIC
↓
NETWORK
IP
↓
LINK
Ethernet / Wi-Fi
↓
PHYSICAL
wire / fiber / radio

The source similarly distinguishes the physical layer, link communication, IP routing, TCP/UDP transport and application-layer protocols such as HTTP.

# 2. HOW MUCH OF OSI SHOULD AN 8-YOE BACKEND ENGINEER KNOW?

🟢 Know the conceptual boundaries.

You generally do **not** need to memorize every OSI-layer trivia question.

For practical backend/system design:

Application HTTP, DNS, gRPC, WebSocket
↓
Transport TCP, UDP
↓
Network IP
↓
Link Ethernet / Wi-Fi / MAC
↓
Physical electrical / optical / radio

### Memory hook

> **IP finds the machine.**
> **Port finds the process.**
> **HTTP tells the process what you want.**

---

# 3. MAC ADDRESS VS IP ADDRESS

🟢 **MUST KNOW CONCEPTUALLY**

## MAC address

Used primarily for communication on a **local network/link**.

Think:

> “Which network interface on this local segment?”

---

## IP address

Used for communication **across interconnected networks**.

Think:

> “Where should this packet be routed?”

The source uses the analogy of MAC as an identity-like address and IP as a changeable location, while emphasizing that IP provides broader network addressing.

---

# 4. INTERVIEW CORRECTION — MAC ADDRESSES

**Model-knowledge clarification beyond the source wording:**

Do not say in an interview:

> “A MAC address is permanently burned into hardware and cannot change.”

Hardware interfaces typically have a factory-assigned MAC address, but operating systems can use **locally administered / randomized MAC addresses**.

The interview-safe distinction is:

MAC → local-link addressing
IP → routable network addressing

That is what matters.

---

# 5. IP ADDRESS — FIRST PRINCIPLES

An IP address identifies a network interface for routing packets.

Examples:

IPv4
192.168.1.10

IPv6
2001:db8::1

The Internet is fundamentally a **network of networks**.

Routers repeatedly answer:

> “What is the next hop toward this destination IP?”

---

# 6. PRIVATE VS PUBLIC IP

🟢 **MUST KNOW**

Your laptop at home might have:

192.168.1.20

That is a private IP.

Your router may expose:

49.x.x.x

to the Internet.

Multiple devices:

Laptop
Phone
TV
Tablet

can share one public IPv4 address through **NAT/PAT**.

---

# 7. NAT — NETWORK ADDRESS TRANSLATION

🟢 **MUST KNOW CONCEPTUALLY**

The source describes NAT as the mechanism allowing devices with private addresses to communicate with public networks through a router with a public-facing address.

Imagine:

```
Laptop      192.168.1.10
Phone       192.168.1.11
TV          192.168.1.12
      \      |      /
          ROUTER
        Public IP
           |
        INTERNET
```

The router tracks connections.

---

# 8. WHY PORTS MATTER WITH NAT

Suppose:

Laptop:50001 → google.com:443
Phone:50002 → google.com:443

Both requests leave through the same public IP.

The router needs to know:

> Which returning packet belongs to which internal connection?

So mappings include ports.

The source explicitly describes modern NAT using ports to distinguish app-to-app connections, often called NAPT or PAT.

---

# 9. PORTS — FIRST PRINCIPLES

🟢 **MUST KNOW**

An IP identifies a machine/interface.

A port identifies an application endpoint on that machine.

Example:

203.0.113.10:443

means approximately:

machine = 203.0.113.10
service/process endpoint = port 443

---

# 10. COMMON PORTS

Know a few:

HTTP 80
HTTPS 443

SSH 22

DNS 53

SMTP 25

You do not need to memorize hundreds.

---

# 11. SOCKET — INTERVIEW MENTAL MODEL

🟡 **SHOULD KNOW**

A network connection is commonly identified by something like:

source IP
source port
destination IP
destination port
protocol

Example:

10.0.0.5:52143
↓
TCP
↓
142.250.x.x:443

This allows many simultaneous connections.

---

# 12. DNS — WHY DOES IT EXIST?

🟢 **MUST KNOW**

Humans want:

google.com

Networks route using addresses.

So DNS provides name resolution:

google.com
↓
IP address

DNS is essentially the Internet's distributed naming system.

---

# 13. DNS RESOLUTION — HIGH LEVEL

Your notes describe the browser checking local DNS information, falling back to a resolver, and eventually reaching authoritative DNS infrastructure before returning the destination address.

The mental model:

Browser
  |
  v
Local cache?
  |
  v
OS / Resolver
  |
  v
Recursive DNS Resolver
  |
  v
Root
  |
  v
TLD (.com)
  |
  v
Authoritative DNS
  |
  v
IP address

---

# 14. DNS PLAYERS

🟢 Know these four.

## Stub/client resolver

Your OS/application starts the lookup.

## Recursive resolver

Does the lookup work on your behalf.

Often provided by:

ISP
enterprise
cloud
public DNS provider

## Root nameserver

Points toward the relevant TLD infrastructure.

## Authoritative nameserver

Has authoritative information for the domain.

---

# 15. DNS EXAMPLE

User enters:

api.example.com

Simplified lookup:

Browser cache?
↓ miss

OS cache?
↓ miss

Recursive Resolver
↓

Root
"Ask .com"

↓

.com server
"Ask example.com's authoritative server"

↓

Authoritative DNS
"api.example.com → 203.0.113.25"

↓

Resolver caches answer

↓

Browser receives IP

---

# 16. DNS CACHING

🟢 **COMMON SYSTEM DESIGN POINT**

Without caching, every request could require multiple DNS lookups.

DNS records therefore have:

TTL = Time To Live

Resolvers cache results until TTL expires.

Trade-off:

LONG TTL
+ fewer DNS lookups
+ less DNS load

- slower propagation when address changes

SHORT TTL
+ changes propagate faster

- more DNS traffic

---

# 17. A RECORD VS CNAME

🟡 **SHOULD KNOW**

Conceptually:

### A record

name → IPv4 address

Example:

example.com
→
203.0.113.10

### AAAA

name → IPv6 address

### CNAME

name → another hostname

Example:

static.example.com
→
example.cdn-provider.com

The source's CDN flow uses DNS aliases/CNAME-like indirection to send clients toward CDN infrastructure.

---

# 18. DNS IN SYSTEM DESIGN

DNS can participate in:

service discovery
geo-routing
multi-region routing
failover
CDN selection
load distribution

But remember:

> DNS is not an instantaneous global control plane.

Caching and TTL mean changes can take time to propagate.

---

# 19. TCP — THE MOST IMPORTANT TRANSPORT PROTOCOL TO UNDERSTAND

🟢 **MUST KNOW**

The source describes TCP as connection-oriented and responsible for reliable, ordered transport, acknowledgements and retransmission.

TCP provides the application with a reliable **byte stream**.

Think:

Application sends bytes
↓
TCP
↓
network may:
lose
reorder
duplicate
delay packets
↓
TCP compensates
↓
Application receives ordered byte stream

---

# 20. WHAT DOES TCP PROVIDE?

Memorize:

Connection-oriented

Reliable delivery

Ordered delivery

Retransmission

Flow control

Congestion control

Full-duplex communication

---

# 21. TCP DOES NOT PRESERVE APPLICATION MESSAGES

🔵 Useful senior nuance.

Suppose application writes:

"HELLO"
"WORLD"

TCP does not promise the receiver sees:

read #1 → HELLO
read #2 → WORLD

TCP is a **byte stream**.

The receiver might observe chunks such as:

HELLOWO
RLD

The application protocol must define framing.

HTTP, WebSocket, gRPC etc. do this above TCP.

---

# 22. TCP 3-WAY HANDSHAKE

🟢 **MUST KNOW**

The source lays out:

SYN
SYN-ACK
ACK

before data transfer.

Diagram:

```
CLIENT                        SERVER
  |                             |
  | -------- SYN ----------------> |
  |                             |
  | <------ SYN + ACK ------------ |
  |                             |
  | -------- ACK ----------------> |
  |                             |
  | ===== connection ready =======  |
```

---

# 23. WHY THREE MESSAGES?

Conceptually, both sides need confidence that communication works in both directions and they need to establish connection state.

Memory:

Client:  Can you hear me?
Server:  Yes. Can you hear me?
Client:  Yes.

---

# 24. TCP CONNECTION COST

A connection is not free.

Before useful application data, TCP may require network round trips.

For geographically distant systems, this matters because:

round-trip latency
×
handshakes
=
user-visible latency

This is one reason connection reuse matters.

# 25. RELIABILITY: SEQUENCE NUMBERS + ACKNOWLEDGEMENTS

Simplified:

Sender
Packet 1 ——————————————→ Receiver

←——— ACK 1 ————————

Packet 2 ——————— X  lost

(no ACK)

Packet 2 ——————————————→ retransmit

TCP uses acknowledgements and retransmission to hide packet loss from many applications.

The source explicitly describes positive acknowledgement with retransmission.

---

# 26. ORDERING

Suppose packets arrive:

1
3
2

TCP provides the application with:

1
2
3

in order.

This reliability is extremely useful.

But it has a consequence:

> Sometimes later data must wait for missing earlier data.

That becomes important when discussing **head-of-line blocking**.

---

# 27. FLOW CONTROL VS CONGESTION CONTROL

🟢 **GOOD SENIOR QUESTION**

These sound similar but solve different problems.

## Flow control

Protects the **receiver**.

Question:

> “Can the receiver consume data this quickly?”

---

## Congestion control

Protects the **network**.

Question:

> “Can the network path carry this much traffic without congestion?”

### Memory

FLOW
sender vs receiver

CONGESTION
sender vs network

---

# 28. UDP

🟢 **MUST KNOW**

The source contrasts UDP with TCP as having less overhead but not providing TCP's reliability behavior.

UDP is datagram-oriented.

It provides much less transport machinery.

Broadly:

No connection setup like TCP

No built-in retransmission guarantee

No built-in ordered delivery guarantee

No built-in reliable delivery guarantee

---

# 29. WHY WOULD ANYONE USE UDP?

Because sometimes:

> **Late data is worse than lost data.**

Example:

Real-time voice.

Suppose packet representing:

audio from 5 seconds ago

is lost.

Would you rather:

A. pause current conversation
and retransmit old audio

B. tolerate tiny packet loss
and continue live

Often **B**.

---

# 30. TCP VS UDP

| Property | TCP | UDP |
|---|---|---|
| Connection | Connection-oriented | Connectionless |
| Reliable delivery | Yes | No built-in guarantee |
| Ordered | Yes | No |
| Retransmission | Yes | No built-in |
| Stream/datagram | Byte stream | Datagrams |
| Overhead | Higher | Lower |
| Typical fit | reliable data | latency-sensitive / protocol-specific |

---

# 31. TCP VS UDP — INTERVIEW ANSWER

> “TCP provides a reliable ordered byte stream using acknowledgements, retransmissions, flow control and congestion control. UDP provides lightweight datagrams without built-in guarantees for delivery or ordering. TCP fits cases where correctness of delivery matters; UDP is useful when low latency or application-controlled reliability is more important.”

---

# 32. IMPORTANT NUANCE

UDP does **not** mean:

fast

and TCP does **not** mean:

slow

Performance depends on:

network conditions
protocol design
packet loss
round trips
application semantics
implementation

UDP simply gives the application more control and fewer guarantees.

---

# 33. IP VS TCP VS HTTP

The source uses a highway/truck/load analogy: IP gets traffic across networks, TCP transports data reliably, and HTTP defines web-level communication.

A cleaner interview mental model:

HTTP
"What does this request mean?"

↓

TCP
"How do I reliably move this byte stream?"

↓

IP
"How do packets reach the destination network?"

---

# 34. ENCAPSULATION

As data moves down the network stack:

HTTP message
↓
TCP segment
↓
IP packet
↓
Ethernet frame

Each layer adds information needed by that layer.

At the receiver the process reverses.

The source explicitly notes that data becomes encapsulated with additional headers as it travels down the layers.

---

# 35. HTTP — FIRST PRINCIPLES

🟢 **MUST KNOW**

HTTP is an **application-layer protocol** built around requests and responses.

Simplified:

CLIENT
|
| HTTP REQUEST
v
SERVER
|
| HTTP RESPONSE
v
CLIENT

The source describes HTTP as a client-server protocol where the client initiates a request and the server returns the requested resource.

---

# 36. HTTP IS STATELESS

🟢 **MUST KNOW**

Stateless means:

> Each HTTP request can be understood independently at the protocol level.

It does **not** mean:

> Web applications cannot maintain state.

Applications maintain state using:

cookies
sessions
tokens
databases
caches

---

# 37. HTTP REQUEST — CONCEPTUAL STRUCTURE

```
GET /users/42 HTTP/1.1
Host: api.example.com
Authorization: Bearer ...
Accept: application/json
```

Potential body for suitable methods:

```
{
  "name": "Alice"
}
```

Think:

METHOD
PATH
HEADERS
BODY

---

# 38. HTTP RESPONSE

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "id": 42,
```

```
  "name": "Alice"
}
```

Think:

STATUS
HEADERS
BODY

---

# 39. HTTP METHODS — INTERVIEW FLOOR

🟢 Know:

GET
POST
PUT
PATCH
DELETE

---

# 40. GET

Used to retrieve a resource.

GET /users/42

Expected semantic properties:

Safe
Idempotent

Safe means:

> The intended operation should not mutate server state.

---

# 41. POST

Typically used for creation or actions.

Example:

POST /orders

POST is not guaranteed to be idempotent.

Two requests could create:

Order 123
Order 124

The source emphasizes this retry problem when contrasting POST with idempotent operations.

---

# 42. PUT

Typically represents creating/replacing a resource at a known URI.

Example:

PUT /users/42

Important property:

idempotent

Repeated identical PUT requests should have the same intended effect.

---

# 43. PATCH

Used for partial modification.

Example:

```
PATCH /users/42
{
  "email": "new@example.com"
}
```

The source distinguishes PATCH as sending only the fields that should be modified rather than replacing the entire resource.

---

# 44. INTERVIEW CORRECTION — PATCH IDEMPOTENCY

**Model-knowledge clarification beyond the source wording:**

The PDF says PATCH is basically non-idempotent.

Do **not** memorize that as a universal rule.

PATCH semantics depend on the operation.

For example:

SET name = Alice

can be idempotent.

But:

INCREMENT balance by 10

is not.

Interview-safe answer:

> “PATCH is not guaranteed to be idempotent; whether it is depends on the patch semantics.”

---

# 45. DELETE

DELETE /users/42

DELETE is defined as idempotent in HTTP semantics.

Calling:

DELETE /users/42

multiple times should have the same intended end state:

user does not exist

Responses may differ, but the intended state does not keep changing.

---

# 46. SAFE VS IDEMPOTENT

🟢 **MUST KNOW**

## Safe

Operation should not change server state as its intended semantic.

Example:

GET
HEAD

## Idempotent

Repeating the same request has the same intended effect as performing it once.

Example:

PUT
DELETE
GET

---

# 47. WHY IDEMPOTENCY MATTERS IN DISTRIBUTED SYSTEMS

This is much more important than memorizing HTTP definitions.

Suppose:

Client
  |
  | POST PAYMENT
  v
Server

Server charges card.

Response gets lost.

Client sees:

timeout

Question:

> Did payment happen?

If client blindly retries:

payment may happen twice

This ambiguity is one of the core problems of distributed systems.

The source uses the same timeout/retry scenario when explaining why idempotency matters for fault-tolerant APIs.

---

# 48. IDEMPOTENCY KEY

🟡 **HIGH-ROI SENIOR CONCEPT — MODEL EXPANSION**

For operations such as payments:

POST /payments
Idempotency-Key: abc123

Server records:

abc123 → payment result

Retry:

same key

returns the existing result instead of performing the operation twice.

Memory:

> **Retries are inevitable. Side effects must be designed for retries.**

---

# 49. HTTP STATUS CODES — MINIMUM SET

🟢 Know these:

200 OK

201 Created

204 No Content

400 Bad Request

401 Unauthenticated / authentication required

403 Authenticated but forbidden

404 Not Found

409 Conflict

429 Too Many Requests

500 Internal Server Error

502 Bad Gateway

503 Service Unavailable

504 Gateway Timeout

You do not need to memorize every HTTP status.

---

# 50. HTTP/1.0 → HTTP/1.1 → HTTP/2 → HTTP/3

🟢 **VERY HIGH ROI**

Do not memorize them as four unrelated versions.

Understand the evolutionary story.

HTTP/1.0
connection overhead
↓

HTTP/1.1
persistent connections
but request-level HOL problems
↓

HTTP/2
multiplex streams over one TCP connection
but TCP-level HOL remains
↓

HTTP/3
HTTP semantics over QUIC
independent transport streams

Your source explicitly presents this progression and identifies application-layer versus TCP-layer head-of-line blocking.

---

# 51. HTTP/1.0

Simplified mental model:

Request
↓
TCP connection
↓
Response
↓
connection closed

Repeated setup creates overhead.

The source describes HTTP/1.0 as creating a separate TCP connection for requests to the same server.

---

# 52. HTTP/1.1

Big improvement:

persistent connections

One TCP connection can be reused.

TCP connection
  |
  ├── Request 1 / Response 1
  ├── Request 2 / Response 2
  └── Request 3 / Response 3

Less connection setup overhead.

But concurrency and ordering remained limiting.

---

# 53. HEAD-OF-LINE BLOCKING

🟢 **MUST KNOW**

ELI5:

Imagine one checkout line.

Person 1 → problem with payment
Person 2 → waits
Person 3 → waits
Person 4 → waits

Even if persons 2–4 are easy, they are stuck behind the blocked item.

That is the intuition behind:

> **Head-of-Line blocking.**

---

# 54. HTTP/1.x HOL PROBLEM

Conceptually:

Request A
Request B

Request C

A slow
↓
B waits
C waits

Parallel TCP connections were often used to work around limitations, but that creates additional connection overhead.

---

# 55. HTTP/2 — MULTIPLEXING

🟢 **MUST KNOW**

HTTP/2 introduces streams.

Instead of:

Request A
wait
Response A

Request B
wait
Response B

you can conceptually have:

ONE TCP CONNECTION

Stream 1 ——— Request A ———————— Response A

Stream 3 ─ Request B ─ Response B

Stream 5 ————— Request C ———————————— Response C

Multiple HTTP exchanges share one TCP connection.

The source explains HTTP/2 as multiplexing multiple HTTP streams through the same TCP connection.

---

# 56. WHAT HTTP/2 FIXES

It greatly reduces **HTTP-layer** head-of-line blocking.

Multiple HTTP streams can progress independently at the application protocol level.

But there is a lower layer:

HTTP/2
↓
TCP

TCP still guarantees ordered delivery of its byte stream.

---

# 57. TCP HEAD-OF-LINE BLOCKING WITH HTTP/2

Suppose:

HTTP stream A packet
HTTP stream B packet
HTTP stream C packet

all travel over one TCP connection.

If a TCP packet is lost, TCP must recover the missing bytes before delivering later stream bytes to the application.

Therefore other HTTP/2 streams can be delayed by packet loss.

The source explicitly notes that HTTP/2 removes application-layer HOL but transport-layer HOL remains because it uses TCP.

---

# 58. WHY NOT JUST REMOVE RELIABILITY FROM TCP?

Because applications still want:

reliable delivery
congestion control
security
connection semantics

The goal is not:

> “Use UDP and accept chaos.”

The goal is:

> Build modern reliable transport semantics **over UDP** while avoiding TCP's connection-wide ordering constraints.

Enter **QUIC**.

---

# 59. QUIC

🟢 **MUST KNOW CONCEPTUALLY**

The source describes QUIC as running over UDP while adding reliability, congestion-control and multiplexing behavior suitable for modern HTTP.

Mental model:

HTTP/3
↓
QUIC
↓
UDP
↓
IP

instead of:

HTTP/2
↓
TCP
↓
IP

---

# 60. WHY UDP UNDER QUIC?

UDP gives QUIC a minimal datagram substrate.

QUIC implements richer behavior in user space/protocol logic rather than inheriting TCP's one ordered byte stream.

The key benefit:

multiple independent streams

Packet loss associated with one stream does not necessarily stall unrelated streams.

The source describes QUIC streams as independently deliverable even while sharing one QUIC connection.

---

# 61. HTTP/3

🟢 Interview answer:

> “HTTP/3 carries HTTP semantics over QUIC rather than TCP. QUIC runs over UDP and provides secure, reliable multiplexed streams without TCP's connection-level head-of-line blocking between streams.”

Good enough for most interviews.

---

# 62. HTTP VERSION MEMORY TABLE

| Version | Core improvement | Remaining issue |
|---|---|---|
| HTTP/1.0 | basic request/response | connection per request |
| HTTP/1.1 | persistent connections | HTTP HOL/concurrency limits |
| HTTP/2 | multiplexing | TCP HOL |
| HTTP/3 | QUIC streams | additional protocol complexity |

Memory hook:

1.0 → CONNECT
1.1 → KEEP
2 → MULTIPLEX
3 → QUIC

---

# 63. LATENCY VS BANDWIDTH VS THROUGHPUT

🟢 **MUST KNOW**

These get confused constantly.

## Latency

How long one unit takes.

Request → 50 ms → Response

---

## Bandwidth

Maximum theoretical communication capacity.

Think:

pipe width

Example:

1 Gbps network link

---

## Throughput

Actual amount successfully transferred/processed per unit time.

Example:

600 Mbps actually achieved

Memory:

BANDWIDTH = capacity

THROUGHPUT = achieved rate

LATENCY = delay

---

# 64. BIG PIPE ≠ LOW LATENCY

Imagine a highway:

100 lanes

but the destination is:

3,000 km away

Huge capacity does not eliminate travel time.

Likewise:

10 Gbps bandwidth

does not automatically produce:

1 ms latency

---

# 65. RTT

🟡 **SHOULD KNOW**

RTT = Round Trip Time.

Client → Server → Client

If one round trip is:

100 ms

then protocols requiring multiple sequential round trips before useful data incur significant latency.

This is why:

DNS lookups
TCP handshake
TLS handshake

matter.

---

# 66. TLS / HTTPS

The PDF contains a later security section describing TLS as the successor to SSL, used to provide encrypted communication and server identity validation through certificates and a TLS handshake. Because this material appears later in the source rather than the networking chapter, the following is a **reorganization of that source material plus clearly marked interview simplification**.

## What is HTTPS?

Conceptually:

HTTP
+
TLS

HTTPS protects HTTP traffic.

---

# 67. WHAT DOES TLS GIVE US?

🟢 **MUST KNOW**

Think:

CONFIDENTIALITY
Who can read it?

INTEGRITY
Was it modified?

AUTHENTICATION
Am I talking to the intended server?

Those three are enough for most interviews.

---

# 68. WHY NOT JUST USE ASYMMETRIC ENCRYPTION FOR EVERYTHING?

🟡 **HIGH ROI**

Public-key cryptography is valuable for authentication and establishing secrets, but symmetric cryptography is efficient for bulk traffic.

Conceptual TLS strategy:

PUBLIC-KEY CRYPTOGRAPHY
↓
authenticate / establish shared secrets
↓
SYMMETRIC SESSION KEYS
↓
fast encrypted application traffic

---

# 69. TLS CERTIFICATE

A certificate connects things such as:

domain identity
+
public key
+
issuer / validity information

The client verifies whether it trusts the certificate chain.

---

# 70. TLS HANDSHAKE — INTERVIEW LEVEL

You do not need to memorize packet-by-packet TLS internals unless the role specifically demands it.

Conceptually:

CLIENT SERVER

ClientHello
supported parameters
----------->

ServerHello
certificate
<----------

verify certificate

establish shared key material

<==========>

encrypted application traffic

### 30-second answer

> “During the TLS handshake, the client and server negotiate cryptographic parameters, the server proves its identity using its certificate, and they derive shared session keys. Subsequent application traffic uses efficient symmetric encryption.”

---

# 71. TLS 1.2 VS TLS 1.3

**🟡 MODEL-KNOWLEDGE INTERVIEW EXPANSION**

For ordinary backend interviews, the main point is:

> TLS 1.3 simplifies the handshake, removes obsolete cryptographic choices and generally reduces connection-establishment latency compared with older TLS versions.

Do not waste high-ROI study time memorizing every cipher suite.

---

# 72. HTTP + TCP + TLS STACK

Traditional HTTPS:

HTTP
↓
TLS
↓
TCP
↓
IP

HTTP/3:

HTTP/3
↓
QUIC
(TLS integrated into QUIC)
↓
UDP
↓
IP

---

# 73. WHAT HAPPENS WHEN YOU TYPE A URL?

🟢🟢🟢 **CLASSIC INTERVIEW QUESTION**

Suppose you enter:

https://example.com/products/42

A good senior answer should connect the entire networking model.

---

# 74. STEP 1 — PARSE URL

Browser identifies roughly:

scheme  https
host  example.com
path  /products/42

HTTPS implies secure HTTP communication.

---

# 75. STEP 2 — DNS

Browser needs the server's IP.

It checks caches and, if necessary, uses DNS resolution:

example.com
  ↓
203.0.113.10

The source places browser cache lookup followed by DNS resolution at the beginning of this flow.

---

# 76. STEP 3 — ROUTING

The operating system determines where to send traffic.

If destination is outside the local network:

machine
↓
gateway/router
↓
ISP / routers
↓
destination network

Potential NAT occurs at network boundaries.

---

# 77. STEP 4 — ESTABLISH TRANSPORT

For traditional HTTPS over TCP:

SYN
SYN-ACK
ACK

TCP connection established.

---

# 78. STEP 5 — TLS HANDSHAKE

Client verifies server identity and establishes encrypted session keys.

Now there is a secure channel.

---

# 79. STEP 6 — HTTP REQUEST

Browser sends something conceptually like:

GET /products/42 HTTP/1.1
Host: example.com

plus headers, cookies etc.

---

# 80. STEP 7 — SERVER-SIDE INFRASTRUCTURE

The request might actually reach:

DNS
↓
CDN / Edge
↓
Load Balancer
↓
Reverse Proxy / Gateway
↓
Application
↓
Cache / Database / Services

Not necessarily one physical web server.

---

# 81. STEP 8 — HTTP RESPONSE

Server returns:

status
headers
body

Example:

HTTP/1.1 200 OK
Content-Type: text/html

---

# 82. STEP 9 — BROWSER RENDERS

Browser parses HTML and may discover:

CSS
JavaScript
images
fonts
API requests

which create additional requests.

Caching/CDN may satisfy many of them.

---

# 83. “TYPE A URL” — 45 SECOND INTERVIEW ANSWER

> “The browser parses the URL and resolves the hostname through DNS, normally using caches before a recursive lookup. Once it has an address, packets are routed toward the destination. For HTTPS over TCP, the client establishes a TCP connection with the three-way handshake, then performs a TLS handshake to authenticate the server and establish encryption keys. The browser sends the HTTP request, which may pass through a CDN, load balancer, reverse proxy or API gateway before reaching the application. The application may access caches, databases or downstream services and sends an HTTP response. The browser then parses the response and may issue further requests for CSS, JavaScript, images and APIs.”

That answer is enough to open many possible interviewer deep dives.

---

# 84. INTERVIEWER: “WHAT IF DNS IS CACHED?”

Then the DNS lookup hierarchy can stop early.

Possible cache layers conceptually include:

browser
OS
recursive resolver

This reduces latency and DNS load.

---

# 85. INTERVIEWER: “WHAT IF CONTENT IS IN CDN?”

Then:

Client
↓
nearby CDN edge
↓
response

The request may never reach the origin application.

The source describes CDN edge servers serving cached content close to users and falling back toward regional/central/origin infrastructure on misses.

---

# 86. CONNECTION TIMEOUT VS READ TIMEOUT

🟢 **BACKEND ENGINEER MUST KNOW**

## Connection timeout

> How long am I willing to wait to establish a connection?

## Read/request timeout

> Once connected/requested, how long am I willing to wait for data/response?

These should not be treated as one magical generic “timeout.”

---

# 87. WHY TIMEOUTS ARE ESSENTIAL

Without timeouts:

Service A
↓
Service B hangs
↓
Thread in A waits forever
↓
More requests
↓
More blocked threads
↓
resource exhaustion
↓
cascading failure

Networking failure often appears as **waiting**, not an immediate exception.

---

# 88. RETRIES

🟢 **MUST KNOW**

Retrying transient network failure can help.

But:

failure
↓
everyone retries immediately
↓
more traffic
↓
system becomes even less healthy

This is a **retry storm**.

Use concepts such as:

limited retries
exponential backoff
jitter
idempotency

We will cover resilience separately.

---

# 89. NETWORK FAILURE IS NOT BINARY

Senior engineers should stop thinking only:

UP
DOWN

Distributed systems experience:

slow response
packet loss
partial failure
connection reset
DNS failure
timeout
stale DNS
one-way connectivity
regional degradation

The network can fail in inconveniently ambiguous ways.

---

# 90. WHY DISTRIBUTED SYSTEMS ARE HARD

In-process call:

result = function();

Distributed call:

Service A
  |
 network
  |
Service B

Now possible outcomes include:

request never arrives

request arrives but response is lost

server executes twice

server executes then crashes

client times out while server succeeds

DNS fails

connection fails

packet loss increases latency

That is why:

> **A remote call is not just a slower function call.**

---

# 91. HTTP KEEP-ALIVE / CONNECTION REUSE

🟢 **SHOULD KNOW**

Creating new network connections repeatedly costs:

handshake time
CPU
kernel resources
TLS setup

Connection reuse allows many requests to avoid repeating this work.

This is particularly important for high-throughput service-to-service communication.

---

# 92. CONNECTION POOLS

**🟢 BACKEND ENGINEER MUST KNOW**

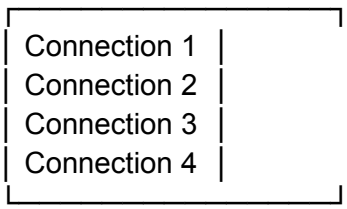
Backend services often maintain reusable connections to:

databases
HTTP services
Redis
message brokers

instead of opening a connection for every request.

Conceptually:

Application




POOL

---

# 93. TOO SMALL VS TOO LARGE CONNECTION POOL

Too small:

requests wait
↓
higher latency

Too large:

downstream overwhelmed
resource usage rises
context switching / contention

Important system-design principle:

> **Concurrency should be bounded by downstream capacity.**

---

# 94. NETWORKING + MICROSERVICES

Consider:

Client
↓
API
↓
Service A
↓
Service B
↓
Service C
↓
Database

Every network hop introduces:

latency
timeout risk
serialization
connection resources
failure probability

Therefore:

> Breaking one process into ten services isn't free.

---

# 95. LATENCY COMPOSITION

Suppose:

Gateway 5 ms
Service A 20 ms
Service B 30 ms
DB 40 ms

A sequential path could already be roughly:

95 ms+

before considering:

network variance
queueing
serialization
retries

This is why reducing unnecessary synchronous hops matters.

---

# 96. PARALLEL CALLS

Suppose service needs:

Profile 50 ms
Orders 70 ms
Preferences 40 ms

Sequential:

50 + 70 + 40
≈ 160 ms

Parallel, conceptually:

max(50, 70, 40)
≈ 70 ms

plus orchestration overhead.

But parallel calls increase:

concurrency
downstream load
failure combinations

Again: trade-offs.

---

# 97. FAN-OUT RISK

Suppose:

1 incoming request

causes:

20 downstream calls

At:

10K incoming QPS

you may create:

200K downstream calls/sec

This is why system designers care about call graphs, not just public API traffic.

---

# 98. P99 MATTERS

**🟢 SENIOR EXPECTATION**

Average latency can hide painful tails.

Example:

p50 = 40 ms
p95 = 100 ms
p99 = 800 ms

Users whose requests involve many downstream calls become increasingly likely to encounter one slow dependency.

When discussing production systems, say:

p95 / p99

not only:

average latency

---

# 99. BANDWIDTH VS QPS

A service can be constrained by either.

Example A:

100K QPS
100-byte responses

High request-processing pressure but relatively little data.

Example B:

100 QPS
100-MB responses

Low QPS but huge network bandwidth.

Therefore:

> QPS alone does not describe network load.

---

# 100. NETWORKING QUESTIONS — 8 YOE INTERVIEW FLOOR

You should answer these without significant hesitation:

**1. MAC vs IP?**

**2. Public IP vs private IP?**

**3. What is NAT?**

**4. Why do we need ports?**

**5. What does DNS do?**

**6. How does DNS resolution work?**

**7. What is DNS TTL?**

**8. TCP vs UDP?**

**9. What does TCP guarantee?**

**10. TCP three-way handshake?**

**11. Flow control vs congestion control?**

**12. HTTP vs TCP?**

**13. Is HTTP stateful or stateless?**

**14. Safe vs idempotent HTTP operation?**

**15. PUT vs POST?**

**16. PUT vs PATCH?**

**17. Why does idempotency matter?**

**18. HTTP/1.1 vs HTTP/2?**

**19. What is multiplexing?**

**20. What is head-of-line blocking?**

**21. Why does HTTP/2 still have transport HOL?**

**22. Why HTTP/3/QUIC?**

**23. What does TLS provide?**

**24. High-level TLS handshake?**

**25. Latency vs bandwidth vs throughput?**

**26. What happens when you type a URL?**

**27. Why reuse network connections?**

**28. What happens if a downstream service becomes slow?**

**29. Why are retries dangerous?**

**30. Why isn't a remote call equivalent to a local method call?**

If these 30 are strong, your networking foundation is appropriate for most senior backend interviews.

---

# 101. RAPID-FIRE MODEL ANSWERS

## Q: IP vs TCP?

> IP routes packets between network endpoints. TCP runs above IP and provides applications with a reliable ordered byte stream.

---

## Q: TCP vs HTTP?

> TCP is a transport protocol concerned with moving bytes reliably. HTTP is an application protocol defining request/response semantics and commonly runs over TCP, although HTTP/3 uses QUIC instead.

---

## Q: Why TCP handshake?

> Both endpoints establish connection state and confirm bidirectional communication before using the connection.

---

## Q: Why UDP?

> When applications need lower-level datagram semantics and want to control reliability or prioritize latency over TCP's ordered reliable byte stream.

---

## Q: What is DNS?

> A distributed naming system that resolves domain names to information such as IP addresses.

---

## Q: What is NAT?

Translation between address spaces, commonly allowing private-network devices to communicate externally through a public-facing address while tracking connection mappings.

---

## Q: What is HTTP/2's main improvement?

Multiplexing multiple HTTP streams over one connection, reducing application-layer head-of-line limitations.

---

## Q: Why HTTP/3?

HTTP/3 uses QUIC so independent transport streams do not inherit TCP's connection-wide head-of-line blocking.

---

## Q: What is TLS?

A security protocol providing encrypted communication, integrity protection and authentication, commonly securing HTTP as HTTPS.

---

## Q: Why idempotency?

Because clients cannot always know whether a timed-out request executed, so safe retry behavior is essential for reliable distributed systems.

---

# 102. COMMON BAD INTERVIEW ANSWERS

### ❌ “HTTP is faster than TCP.”

Wrong abstraction.

HTTP normally relies on a transport protocol.

---

### ❌ “UDP is always faster.”

Not universally.

---

### ❌ “HTTP is one-way.”

HTTP is request-response. Data clearly travels both directions.

What is constrained is **who initiates the request/response exchange**, not physical directionality.

---

### ❌ “TCP has no ports.”

TCP absolutely uses ports.

---

### ❌ “PATCH is always non-idempotent.”

Depends on patch semantics.

These corrections matter because some of those simplified claims appear in the raw source notes around the HTTP/TCP comparison.

---

# 103. ONE-PAGE NETWORKING MEMORY SHEET

NETWORK STACK

APPLICATION
HTTP / DNS / gRPC / WebSocket
↓
TRANSPORT
TCP / UDP / QUIC
↓
NETWORK
IP
↓
LINK
MAC / Ethernet / Wi-Fi

ADDRESSING

DNS
name → IP

IP
machine/network endpoint

PORT
application endpoint

NAT
private ↔ public mapping

TCP

connection-oriented
reliable
ordered
byte stream
ACK + retransmission
flow control
congestion control

HANDSHAKE
SYN
SYN-ACK
ACK

UDP

datagrams
no built-in reliability
no built-in ordering
low protocol overhead

HTTP

request → response
stateless semantics

GET
read

POST
create/action

PUT
replace

PATCH
partial update

DELETE

delete

HTTP EVOLUTION

1.0
new connections

1.1
persistent connections

2
multiplexing
TCP HOL remains

3
QUIC/UDP
independent transport streams

TLS

confidentiality
integrity
authentication

URL FLOW

URL
↓
DNS
↓
IP routing
↓
TCP/QUIC
↓
TLS
↓
HTTP
↓
CDN/LB/Gateway
↓
Service
↓
Cache/DB
↓
Response

---

# 104. 60-SECOND NETWORKING ANSWER

If an interviewer says:

> **“Explain networking from browser to backend.”**

You should be capable of saying:

> “The hostname is first resolved through DNS to an address. IP handles routing packets toward that destination, while TCP or another transport such as QUIC provides the communication semantics. With TCP, the client establishes the connection using the three-way handshake. For HTTPS, TLS then authenticates the server and establishes encryption keys. HTTP runs at the application layer and carries the request and response. The request may pass through a CDN, load balancer, reverse proxy or gateway before reaching an application server. Networked calls introduce latency, timeouts and partial failures, so production systems need bounded connections, timeouts, retries with backoff and idempotent operations.”

If you can expand **every sentence** in that paragraph, this section is learned.

---

# 105. PART 2 — MEMORY MAP

```
NETWORKING
│
├── Addressing
│   ├── MAC
│   ├── IP
│   ├── Port
│   └── NAT
│
├── Naming
│   └── DNS
│       ├── Resolver
│       ├── Root
│       ├── TLD
│       ├── Authoritative
│       └── TTL
│
├── Transport
│   ├── TCP
│   │   ├── Handshake
│   │   ├── Reliability
│   │   ├── Ordering
│   │   ├── Flow Control
│   │   └── Congestion Control
│   │
│   └── UDP
│
├── HTTP
```

```
│   ├── Methods
│   ├── Safe
│   ├── Idempotent
│   └── Status Codes
│
├── HTTP Evolution
│   ├── 1.0
│   ├── 1.1
│   ├── 2 / Multiplexing
│   └── 3 / QUIC
│
├── Security
│   └── TLS
│
└── Production
    ├── Timeouts
    ├── Retries
    ├── Connections
    ├── Pools
    ├── p99
    └── Partial Failure
```

---

# 106. WHAT NOT TO OVER-STUDY

For an ordinary **8-YOE Java/backend/system-design interview**, diminishing returns become steep if you spend hours memorizing:

electrical voltage encoding
all OSI layers verbatim
every TCP flag
TCP header bit fields
every DNS record
every TLS cipher suite
every packet in TLS handshake
kernel TCP implementation

Know the **behavioral consequences** instead:

What guarantees do I get?

What latency does it introduce?

What happens under failure?

What does retry mean?

Can requests duplicate?

Can connections be reused?

What happens at scale?

That is the senior-engineering ROI.

---

# CJ SYSTEM DESIGN INTERVIEW HANDBOOK

## PART 3 — API COMMUNICATION & REAL-TIME SYSTEMS

**Target:** Senior Software Engineer / Backend Engineer — ~8 YOE
**Goal:** Given two components that need to communicate, choose the **simplest communication model that matches the requirement**.

> **Priority legend**
> 🟢 **MUST KNOW** — senior interview floor
> 🟡 **SHOULD KNOW** — explain confidently
> 🔵 **DEEP DIVE** — interviewer-push territory

Your source groups REST, GraphQL and gRPC around the question of how independently implemented software communicates, then separately covers long polling, SSE, WebSocket and WebRTC.

---

# 1. THE FIRST PRINCIPLE: WHAT COMMUNICATION PATTERN DO YOU NEED?

Do **not** start with:

> “Should I use REST or gRPC?”

Start with:

WHO TALKS TO WHOM?
↓
HOW OFTEN?
↓
WHO INITIATES?
↓
ONE-WAY OR TWO-WAY?
↓
REQUEST/RESPONSE OR STREAM?
↓
PUBLIC OR INTERNAL?
↓
HUMAN-READABLE OR BINARY?
↓

LATENCY / THROUGHPUT REQUIREMENTS?

Then choose the protocol.

---

# 2. THE MASTER MAP

CLIENT REQUESTS SOMETHING
```
    │
    ├── ordinary request/response
    │       → REST / HTTP
    │
    ├── client wants flexible fields
    │       → GraphQL
    │
    └── internal high-performance RPC
            → gRPC
```

SERVER MUST PUSH UPDATES
```
    │
    ├── occasional / simple
    │       → Polling
    │
    ├── near-real-time but HTTP-based
    │       → Long Polling
    │
    ├── server → client only
    │       → SSE
    │
    └── client ↔ server real-time
            → WebSocket
```

REAL-TIME AUDIO / VIDEO
```
    ↓
  WebRTC
```

This is the 80:20 map to memorize.

---

# 3. BEFORE REST / GRAPHQL / gRPC: TWO DIFFERENT QUESTIONS

Your source separates two things needed for software written in different languages to communicate:

1. a common representation/interface language

2. a common communication mechanism/protocol.

That distinction is excellent.

Think:

WHAT DOES THE DATA LOOK LIKE?
+
HOW DO WE COMMUNICATE?

Examples of representation:

JSON
XML
Protocol Buffers
Thrift

Examples of API/communication styles:

REST
GraphQL
RPC / gRPC
SOAP

---

# 4. SERIALIZATION — FIRST PRINCIPLES

Suppose Java has:

```
class User {
    long id;
    String name;
}
```

A Python service does not understand a Java object living in JVM memory.

We need a common wire representation.

For example:

```
{
  "id": 42,
  "name": "Alice"
}
```

or a Protocol Buffer binary representation.

This process is broadly:

OBJECT
↓
SERIALIZE
↓

BYTES
↓
NETWORK
↓
DESERIALIZE
↓
OBJECT

---

# 5. JSON VS PROTOCOL BUFFERS

🟢 **HIGH ROI**

## JSON

Human-readable
Text
Widely supported
Easy to debug
Flexible
Generally larger payload
Parsing overhead

## Protocol Buffers

Binary
Schema-based
Compact
Code generation
Efficient
Less human-readable
Requires schema/tooling

Your source emphasizes that gRPC commonly uses Protocol Buffers and native code generation.

---

# 6. REST — FIRST PRINCIPLES

🟢 **MUST KNOW**

REST is an **architectural style** commonly implemented over HTTP.

Think in terms of **resources**.

/users/42

/orders/123

/products/789

Then HTTP methods represent operations.

GET  /users/42

POST  /orders

PUT  /users/42

PATCH  /users/42

DELETE  /users/42

Your source correctly distinguishes REST as an architectural style and HTTP as the communication protocol commonly used to implement it.

---

# 7. REST MENTAL MODEL

CLIENT

"What resource do I want?"

↓

HTTP VERB + RESOURCE URI

↓

SERVER

Example:

GET /products/123

response:

```
{
  "id": 123,
  "name": "Laptop",
  "price": 75000
}
```

---

# 8. REST'S HIGH-VALUE PROPERTIES

The source emphasizes:

ubiquity
statelessness
cache friendliness
broad JSON support

for REST.

Interview version:

### Strengths

Simple mental model
Huge ecosystem
Excellent HTTP tooling
Human-readable JSON commonly used
Easy browser/client interoperability
HTTP caching works naturally for suitable GETs
Loose coupling

### Costs

Potential over-fetching
Potential under-fetching
Multiple round trips
Text payload overhead
API evolution/versioning concerns

---

# 9. WHAT DOES STATELESS REST MEAN?

The server should not require prior conversational request state to understand the current request.

Example:

GET /orders/42
Authorization: Bearer <token>

Everything necessary to process the request should be derivable from the request plus server-side resources.

This does **not** mean:

server has no database
server has no session system
server stores no state

It refers to request/session semantics.

---

# 10. REST — 30 SECOND INTERVIEW ANSWER

> “REST is an architectural style commonly implemented over HTTP where resources are identified by URIs and manipulated using standard HTTP semantics. It is simple, interoperable, cache-friendly for appropriate requests and works well for public APIs. Its trade-offs can include larger text payloads, over- or under-fetching and multiple network calls for complex client views.”

---

# 11. WHEN WOULD YOU CHOOSE REST?

🟢

Good default when:

Public API
Browser/mobile clients
CRUD/resource-oriented operations
Human debuggability matters
Broad interoperability matters
Standard HTTP caching helps

---

# 12. WHEN WOULD YOU NOT DEFAULT TO REST?

Consider something else if the dominant requirement is:

High-volume internal RPC
Strong schema/code generation
Bidirectional streaming
Flexible client-selected response shape
Persistent real-time connection
Real-time media

REST isn't “bad.”

The workload may simply fit another model better.

---

# 13. GRAPHQL — FIRST PRINCIPLES

🟢 **SHOULD KNOW**

The source's key idea is:

> **Get exactly what the client asks for**, reducing over-fetching and under-fetching.

Example REST client may receive:

```
{
  "id": 42,
  "name": "Alice",
  "email": "...",
  "address": "...",
  "preferences": "...",
  "createdAt": "...",
  "lastLogin": "..."
}
```

when it only needs:

name
profilePhoto

GraphQL lets the client ask:

```
query {
  user(id: 42) {
    name
    profilePhoto
  }
}
```

---

# 14. GRAPHQL MENTAL MODEL

REST:

SERVER DEFINES RESPONSE SHAPE

GraphQL:

SERVER DEFINES SCHEMA
+
CLIENT SELECTS FIELDS

This is one of the easiest distinctions to remember.

---

# 15. REST VS GRAPHQL

The source contrasts REST's multiple resource URLs with GraphQL's typical single endpoint and client-driven selection.

Interview table:

| Concern | REST | GraphQL |
|---|---|---|
| Main abstraction | Resources | Graph/schema |
| Response shape | Server-defined | Client-selected |
| Endpoints | Usually many | Usually one |
| Over-fetching | Possible | Reduced |
| Under-fetching | Possible | Reduced |
| HTTP caching | Natural | Less straightforward |
| Complexity | Lower | Higher |
| Client flexibility | Moderate | High |

---

# 16. GRAPHQL PROBLEM: OVER-FETCHING

REST:

GET /users/42

Returns:

20 fields

Client needs:

3 fields

Waste:

17 unnecessary fields

GraphQL can request exactly:

```
user {
  name
  avatar
  status
}
```

---

# 17. GRAPHQL PROBLEM: UNDER-FETCHING

Suppose screen needs:

User

+
Orders
+
Recommendations

REST might require:

GET /users/42

GET /users/42/orders

GET /users/42/recommendations

GraphQL may expose one graph query covering all required data.

---

# 18. GRAPHQL IS NOT “REST BUT FASTER”

Important interview point.

GraphQL buys client flexibility.

That flexibility creates server-side challenges:

complex queries
query-cost control
authorization at field/resolver level
N+1 queries
caching complexity
observability
schema governance

---

# 19. GRAPHQL N+1 PROBLEM

**🟡 MODEL-KNOWLEDGE EXPANSION**

Suppose query requests:

100 users
+
orders for every user

Naive resolver:

1 query → get 100 users

then

100 queries → get orders

Total:

101 DB queries

This is the classic:

> **N+1 query problem**

Typical mitigation:

batching
DataLoader-style techniques
better DB query planning

---

# 20. GRAPHQL — 30 SECOND INTERVIEW ANSWER

> “GraphQL exposes a typed schema and lets clients request exactly the fields they need. It is useful when different clients require different shapes of related data and REST would cause excessive over-fetching, under-fetching or endpoint proliferation. The cost is greater server complexity, query-cost control, caching difficulty and potential N+1 problems.”

---

# 21. WHEN WOULD YOU CHOOSE GRAPHQL?

Good fit when:

Many frontend clients
Different views need different fields
Highly connected data
Rapid frontend iteration
Avoiding repeated endpoint creation matters

Example:

Web
Mobile
Tablet

all need different pieces of:
user
posts
comments
likes
recommendations

---

# 22. WHEN WOULD YOU AVOID GRAPHQL?

If API is:

simple CRUD
few clients
stable payloads
heavy use of HTTP caching
very predictable access patterns

REST may be simpler.

Senior rule:

> **Do not buy GraphQL complexity unless client flexibility pays for it.**

---

# 23. gRPC — FIRST PRINCIPLES

🟢 **MUST KNOW FOR BACKEND**

gRPC is a remote procedure call framework.

Mental model:

LOCAL CODE FEELS LIKE

riskClient.checkRisk(order)

↓

ACTUALLY

serialize request
↓
network
↓
remote service
↓
deserialize
↓
execute
↓
return response

The source associates gRPC with HTTP/2, Protocol Buffers, code generation and internal service-to-service communication.

---

# 24. PROTOBUF CONTRACT

Example:

```
syntax = "proto3";

service RiskService {
  rpc CheckRisk(RiskRequest) returns (RiskResponse);
}

message RiskRequest {
  string client_id = 1;
  double notional = 2;
}

message RiskResponse {
  bool allowed = 1;
  string reason = 2;
}
```

From this schema, tools can generate client/server code.

---

# 25. WHY gRPC IS ATTRACTIVE INTERNALLY

Strong interface contract

Compact binary payloads

Code generation

HTTP/2 multiplexing

Streaming support

Cross-language clients

Efficient service-to-service calls

This matches the source's emphasis on Protocol Buffers and native code generation.

---

# 26. gRPC COMMUNICATION MODES

🟢 **HIGH ROI**

A very useful distinction:

1. Unary
2. Server streaming
3. Client streaming
4. Bidirectional streaming

---

# 27. UNARY RPC

Looks like ordinary request/response.

Client
 |
Request
 |
 v
Server
 |
Response
 |
 v
Client

Example:

GetUser(id)
→ User

---

# 28. SERVER STREAMING

One request:

Client → Server

many responses:

Server → Client
Server → Client
Server → Client

Example:

stream market prices

---

# 29. CLIENT STREAMING

Many client messages:

Client → Server
Client → Server
Client → Server

then one response.

Example:

upload a stream of measurements

---

# 30. BIDIRECTIONAL STREAMING

Both communicate continuously.

CLIENT ⇄ SERVER

independently.

Example:

real-time collaboration
interactive streams

---

# 31. SOURCE CONTRADICTION TO AVOID MEMORIZING

One part of the source says gRPC helps servers communicate “without bidirectional capabilities,” while another section explicitly discusses gRPC allowing the server to send messages and describes full-duplex capabilities.

For interview purposes:

> **gRPC supports bidirectional streaming.**

Treat the “without bidirectional capabilities” sentence as inconsistent source material rather than something to memorize.

---

# 32. REST VS gRPC

| Concern | REST | gRPC |
|---|---|---|
| Common payload | JSON | Protobuf |
| Representation | Text | Binary |
| Human readability | High | Low |
| Contract | OpenAPI optional/common | `.proto` central |
| Code generation | Optional | Core strength |
| Browser friendliness | Excellent | More constrained |
| Streaming | Possible via other HTTP mechanisms | Native RPC streaming model |
| Internal RPC | Good | Excellent fit |
| Public API | Excellent | Less universal |

---

# 33. 30-SECOND REST VS gRPC ANSWER

> “I generally prefer REST for public or browser-facing APIs because it is simple, ubiquitous and easy to inspect. For internal service-to-service communication where strong contracts, efficient serialization, code generation or streaming matter, gRPC is attractive. The trade-off is more tooling and less human-readable traffic.”

---

# 34. IMPORTANT: DON'T SAY “gRPC IS ALWAYS 10× FASTER”

The source includes one benchmark claiming approximately 7×/10× improvements for a specific payload.

Do **not** generalize that benchmark into:

> “gRPC is 10 times faster than REST.”

Performance depends on:

payload
serialization
network
compression
implementation
server processing
connection reuse
request pattern

Interview-safe formulation:

> “Protobuf and HTTP/2 can make gRPC more efficient on the wire, but actual performance should be measured for the workload.”

---

# 35. REST VS GRAPHQL VS gRPC — THE CORE DECISION

PUBLIC / GENERAL-PURPOSE API
↓
REST

CLIENT NEEDS FLEXIBLE GRAPH QUERIES
↓
GraphQL

INTERNAL HIGH-VOLUME RPC
↓
gRPC

Not universal.

Just a strong default heuristic.

---

# 36. 10-SECOND MEMORY HOOK

REST
RESOURCE

GraphQL
CLIENT CHOOSES DATA

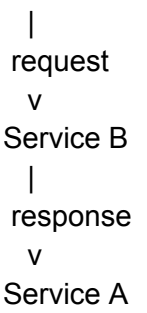
gRPC
CALL REMOTE SERVICE

---

# 37. SYNCHRONOUS REQUEST/RESPONSE VS EVENT-DRIVEN

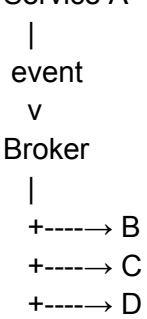
Before proceeding to real-time client connections, distinguish this.

Synchronous:




A waits for B.

Event-driven:



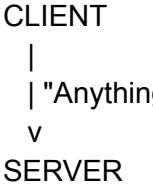

A may not wait for consumers.

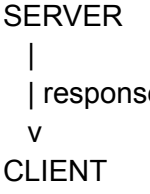
We will deep-dive queues/Kafka later.

---

# 38. REAL-TIME COMMUNICATION: WHAT IS THE PROBLEM?

Normal HTTP:




What if the server gets important data:

1 second later

How does it deliver it immediately?

Possible solutions:

Polling
Long Polling
SSE
WebSocket

---

# 39. SHORT POLLING

🟢 **SHOULD KNOW**

Client repeatedly asks:

Anything new?
Anything new?
Anything new?
Anything new?

Example:

every 5 seconds

Architecture:

CLIENT ---- request ----> SERVER
CLIENT <--- response ---- SERVER

wait

CLIENT ---- request ----> SERVER
CLIENT <--- response ---- SERVER

---

# 40. POLLING ADVANTAGES

Very simple
Plain HTTP
Easy infrastructure
Stateless requests

---

# 41. POLLING DISADVANTAGES

Suppose:

1M clients
poll every 5 sec

Approximate request rate:

1,000,000 / 5
=
200,000 requests/sec

even when nothing changed.

Problems:

wasted requests
server load
bandwidth
update delay

---

# 42. LONG POLLING

The source describes long polling as an HTTP request where the server deliberately delays its response until data becomes available, after which the client issues another request.

Flow:

```
CLIENT
  |
  | "Tell me when something happens"
  v
SERVER

... server waits ...

EVENT ARRIVES

SERVER
  |
 response
  v
CLIENT
```

CLIENT immediately opens another request

---

# 43. WHY LONG POLLING IS BETTER THAN SHORT POLLING

Short polling:

request
empty
request
empty
request
empty

Long polling:

one request
  ↓
wait
  ↓
event
  ↓
response

Far fewer empty requests.

---

# 44. LONG POLLING TRADE-OFFS

+ plain HTTP
+ relatively simple
+ near real-time

- repeated connection/request lifecycle
- server must maintain waiting requests
- reconnect logic
- awkward compared with native push mechanisms

---

# 45. SERVER-SENT EVENTS — SSE

🟢 **MUST KNOW**

The source describes SSE as a long-running HTTP connection allowing the server to continuously push updates to the client.

Direction:

CLIENT  <================  SERVER
          events

Main characteristic:

**Server → Client only**

---

# 46. SSE FLOW

CLIENT

opens HTTP connection
|
v
SERVER

|
| event 1
v
CLIENT

|
| event 2
v
CLIENT

|
| event 3
v
CLIENT

The connection remains open.

---

# 47. WHY SSE?

The source highlights:

unidirectional server push
automatic reconnection
event IDs
status/update use cases

Great examples:

stock-price display
news feed updates
job-progress notifications
server logs
live dashboards
notification stream

where the browser does not need equivalent continuous client→server traffic.

---

# 48. SSE VS POLLING

Polling:

client asks repeatedly

SSE:

client subscribes once
server pushes as events occur

For one-way updates, SSE is often simpler and more efficient.

---

# 49. WEBSOCKET — FIRST PRINCIPLES

🟢 **MUST KNOW**

WebSocket provides a long-lived, full-duplex communication channel.

CLIENT  <================>  SERVER

Either side can send messages whenever necessary.

The source describes WebSocket as bidirectional, full-duplex, long-lived and message-oriented after the initial HTTP-friendly upgrade handshake.

---

# 50. WEBSOCKET CONNECTION FLOW

Simplified:

```
CLIENT
  |
  | HTTP Upgrade request
  v
SERVER
  |
  | Upgrade accepted
  v

========== WebSocket ==========

CLIENT <----------------> SERVER
```

messages can flow both ways

---

# 51. WHY WEBSOCKET EXISTS

Ordinary HTTP request/response:

client initiates
server responds

But chat wants:

Alice sends message
↓
Server
↓
Bob receives immediately

Bob should not have to ask every second:

> “Did Alice message me?”

WebSocket maintains a channel through which the server can immediately push the message.

---

# 52. GOOD WEBSOCKET USE CASES

The source uses examples such as chat and multiplayer gaming.

Typical:

Chat

Multiplayer games

Collaborative editing

Live trading screens

Interactive dashboards

Presence

Real-time bidding

---

# 53. WEBSOCKET DOES NOT AUTOMATICALLY SOLVE EVERYTHING

Important senior point.

WebSocket gives you a connection.

It does **not automatically give you**:

message persistence
exactly-once delivery
offline delivery
business-level ordering
user presence semantics
replay
authorization model
horizontal connection routing

Your application still has to build those.

---

# 54. THE CONNECTION-SCALING PROBLEM

Suppose:

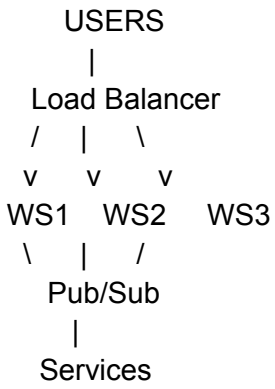
5 million connected users

You now have:

5 million long-lived connections

Unlike ordinary stateless HTTP requests, connection ownership matters.

Architecture may look like:




---

# 55. WHY WEBSOCKET SERVERS NEED SPECIAL SCALE THINKING

Need to manage:

connection count

memory per connection
heartbeat
idle timeout
reconnection storms
connection ownership
routing message to correct node
backpressure
slow clients

---

# 56. WEBSOCKET + PUB/SUB PATTERN

Suppose Bob is connected to:

WebSocket Server 7

Alice's message enters:

WebSocket Server 2

Server 2 needs to reach Server 7.

Possible design:

```
Alice
  |
 WS2
  |
  v
MESSAGE BROKER / PUB-SUB
  |
  v
 WS7
  |
 Bob
```

This is why real-time connection design often intersects with messaging architecture.

---

# 57. SSE VS WEBSOCKET

| Concern | SSE | WebSocket |
|---|---|---|
| Direction | Server → client | Both directions |
| Connection | Long-lived HTTP | Long-lived WebSocket |
| Browser support model | EventSource | WebSocket API |

| | | |
|---|---|---|
| Reconnect | Built-in semantics available | Usually app-managed |
| Data style | Event stream | Messages |
| Best fit | Notifications/feed | Interactive real-time |

---

# 58. INTERVIEW DECISION

Ask:

> **Does the client need to continuously send messages too?**

No:

SSE may be enough

Yes:

WebSocket

Your source makes exactly this distinction: SSE for one-way status/push updates and WebSocket for real-time two-way communication.

---

# 59. POLLING VS LONG POLLING VS SSE VS WEBSOCKET

POLLING

Client:
Anything?
Anything?
Anything?
Anything?

LONG POLLING

Client:
Tell me when something happens...
↓
EVENT
↓
response

SSE

Server:
event →
event →
event →

WEBSOCKET

Client ⇄ Server
Client ⇄ Server
Client ⇄ Server

---

# 60. MASTER COMPARISON

| Pattern | Direction | Connection | Complexity | Best use |
|---|---|---|---|---|
| Polling | request/response | repeated | Low | infrequent changes |
| Long polling | server-delayed response | repeated | Medium | legacy near-real-time |
| SSE | S → C | persistent | Low/medium | server notifications |
| WebSocket | C ↔ S | persistent | Higher | interactive real-time |

---

# 61. WHAT ABOUT gRPC VS WEBSOCKET?

Both can support long-lived bidirectional communication.

But their common roles differ.

### gRPC

Think:

service ↔ service
typed RPC contract

### WebSocket

Think:

browser/client ↔ server
arbitrary application messages

The source compares them around internal microservice communication versus persistent real-time application communication.

---

# 62. gRPC VS WEBSOCKET — INTERVIEW ANSWER

> “Both can provide bidirectional communication, but they solve different primary problems. gRPC is an RPC framework with typed contracts, generated clients and HTTP/2 streaming, so it fits service-to-service communication well. WebSocket gives client and server a persistent full-duplex message channel and is commonly useful for browser-facing interactive systems such as chat.”

---

# 63. PUSH NOTIFICATIONS ARE DIFFERENT

🟡

Do not confuse:

WebSocket

with:

mobile push notification

When an app is backgrounded or terminated, your server usually cannot rely on maintaining its own active WebSocket to the app.

Mobile platforms provide push infrastructure.

Conceptually:

YOUR SERVER
↓
OS PUSH SERVICE
↓
DEVICE

Examples:

APNs
FCM

Your source describes this as an operating-system push service, app publisher and client application flow.

---

# 64. REST VS WEBSOCKET

This is not really:

> “Which protocol is better?”

They address different communication shapes.

REST:

REQUEST
↓
RESPONSE

WebSocket:

persistent conversation

Many real systems use **both**.

Example chat app:

REST
↓
login
load history
update profile

WebSocket
↓
live messages
typing indicators
presence

---

# 65. REAL-TIME MEDIA IS A DIFFERENT PROBLEM

Suppose you want:

Zoom-like video call

Could you just send video frames over REST?

Technically you could move bytes in many ways.

But interactive audio/video has strict requirements:

very low latency
continuous media

packet-loss tolerance
adaptive bitrate
NAT traversal
real-time codecs

This leads to **WebRTC**.

---

# 66. WebRTC — FIRST PRINCIPLES

🟡 **SHOULD KNOW FOR GENERAL BACKEND**
🟢 **MUST KNOW FOR CALLING/VIDEO SYSTEM DESIGN**

The source describes WebRTC as enabling browser/device real-time audio, video and arbitrary data, often using direct peer connectivity when possible.

Think:

```
Browser A
   \
    \  audio/video
     \
      Browser B
```

rather than forcing all real-time media through the ordinary application server.

---

# 67. IMPORTANT WebRTC NUANCE

The raw source first describes it strongly as peer-to-peer, but later correctly notes:

> P2P is possible, not mandatory; relays or media servers may be used.

For interview purposes:

> **WebRTC attempts direct connectivity where appropriate, but real systems often use TURN and/or media servers.**

Do not say:

> “WebRTC means no server.”

---

# 68. SIGNALING VS MEDIA

🟢 **VERY IMPORTANT**

Two separate problems.

## Signaling

Before two users communicate they must exchange information such as:

Who are you?
How can I reach you?
What media capabilities do you support?

This may use:

WebSocket
HTTP
other signaling mechanism

---

## Media

Once connectivity is established:

audio
video
data

flows through the selected path.

Memory:

SIGNALING
= arrange the call

MEDIA
= the call itself

The source explicitly separates signaling and media.

---

# 69. NAT MAKES PEER-TO-PEER HARD

Remember from Part 2:

Laptop
private IP
  |
 NAT
  |
Internet

Peer A often cannot simply contact Peer B's private IP.

We need NAT traversal.

Enter:

STUN
TURN
ICE

---

# 70. STUN — ELI5

🟢 **FOR WEBRTC DESIGN**

STUN answers approximately:

> “What public address/port does the outside world see me as?”

Conceptually:

```
CLIENT
  |
  | What is my public-facing address?
  v
STUN SERVER
  |
  | 203.x.x.x:54321
  v
CLIENT
```

The source describes STUN as being used during connection setup to discover/exchange external host/port information.

---

# 71. TURN — ELI5

What if direct peer connection fails because of:

strict NAT
firewall
enterprise network

Then use relay:

```
Peer A
  |
  v
 TURN SERVER
  |
  v
```

Peer B

The source notes that TURN is used when direct traversal using STUN is not possible.

---

# 72. STUN VS TURN MEMORY HOOK

STUN
"How can peers reach me?"

TURN
"If they can't reach me directly,
relay my traffic."

---

# 73. TURN IS EXPENSIVE

Why?

STUN mostly helps establish connectivity.

TURN carries:

actual media traffic

If 1 Gbps of video flows between users through your TURN infrastructure:

you carry that bandwidth

At scale this becomes expensive.

---

# 74. ICE

🟡 **MODEL-KNOWLEDGE EXPANSION**

ICE coordinates candidate connectivity options.

Simplified:

Try direct local candidate
↓
Try public/STUN-derived candidate
↓
If needed
TURN relay

Think:

**ICE is the connectivity selection process.**

---

# 75. WEBRTC COMPONENTS FROM THE SOURCE

The source identifies:

Signaling server

STUN/TURN

Optional media server

and browser APIs such as:

getUserMedia

PeerConnection

DataChannel

For system design interviews, this is enough.

---

# 76. getUserMedia

Access:

camera
microphone
screen

from browser/device.

---

# 77. PeerConnection

Handles much of the actual WebRTC connectivity/media machinery.

The source mentions responsibilities including:

SDP handling
ICE
TURN fallback

encoding/decoding
media transport
jitter handling
bandwidth estimation
packet-loss concealment

You do not need to memorize every internal algorithm for a normal backend interview.

---

# 78. DataChannel

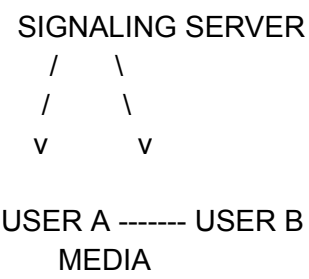
Can carry arbitrary application data.

The source notes that WebRTC DataChannel can be configured with different reliability and ordering characteristics.

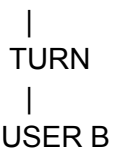
---

# 79. 1:1 VIDEO CALL

Simplified:




If direct path fails:




---

# 80. GROUP VIDEO CHANGES EVERYTHING

Suppose:

10 people

and everyone sends video directly to everyone.

Each participant may need to upload multiple copies.

Peer-to-peer mesh grows badly.

Connections roughly grow toward:

O(n²)

This leads to media-server architectures.

---

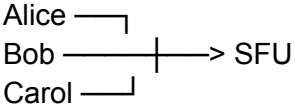
# 81. SFU — HIGH ROI FOR VIDEO SYSTEM DESIGN

**🔵 MODEL-KNOWLEDGE EXPANSION**

SFU:

> Selective Forwarding Unit

Each participant typically sends one stream to media server:




SFU forwards selected streams to other participants.

Benefits:

lower client upload
better group scaling
central quality/routing control

Cost:

server bandwidth
operational complexity

---

# 82. MESH VS SFU

### Mesh

A ↔ B
A ↔ C
B ↔ C

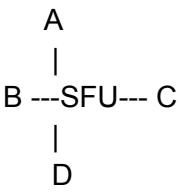
Good for:

very small groups

Problems grow rapidly with group size.

---

### SFU




Better for larger conferences.

---

# 83. WebSocket VS WebRTC

🟢 **COMMON QUESTION**

## WebSocket

Designed for:

application messages

Examples:

chat text
game state
stock updates

---

## WebRTC

Designed for:

real-time media/data

Examples:

audio
video
screen sharing

and deals with:

media codecs
NAT traversal
packet loss
jitter

---

# 84. 30-SECOND ANSWER: WEBSOCKET VS WEBRTC

> “WebSocket gives a persistent reliable client-server message channel and is ideal for interactive application data such as chat or game events. WebRTC is designed for low-latency real-time media and includes peer connectivity, NAT traversal and media handling, making it appropriate for voice, video and screen sharing.”

---

# 85. VIDEO STREAMING VS VIDEO CALLING

Another important distinction.

## Netflix / YouTube playback

Main goal:

scale
quality
buffering tolerance
CDN distribution

Often uses segmented HTTP delivery such as:

HLS
DASH

---

## Zoom / Meet call

Main goal:

interactive low latency

Use:

WebRTC-like real-time media architecture

Your source describes DASH/HLS as adaptive chunk-based delivery, while WebRTC is positioned around low-latency real-time communication.

---

# 86. WHY HLS/DASH CAN USE CDN WELL

Video encoded at multiple quality levels:

360p
720p
1080p
4K

split into chunks.

Player observes:

bandwidth
buffer

and chooses appropriate chunks.

The source describes this adaptive-bitrate behavior and distribution through CDN.

---

# 87. WHY LIVE CALLING CANNOT BUFFER LIKE NETFLIX

Netflix can say:

> “I'll buffer several seconds before playback.”

A conversation cannot.

Imagine:

You: Hello?

five seconds later:

Other person: Hi.

Communication becomes unusable.

So real-time calling trades some perfect delivery for low latency.

---

# 88. COMMUNICATION TECHNOLOGY DECISION TREE

Memorize this.

Do I need ordinary request/response?
|
YES
|
+-- Public/general API?
|  → REST
|
+-- Frontend needs flexible fields?
|  → GraphQL
|
+-- Internal strongly typed high-volume RPC?
  → gRPC

Do I need server push?
|
YES
|
+-- Server → client only?
|  → SSE
|
+-- Both directions?
  → WebSocket

Do I need interactive audio/video?
|
→ WebRTC

Do I need massive one-way video playback?
|
→ HLS / DASH + CDN

---

# 89. COMMUNICATION PATTERN VS TECHNOLOGY

Senior-level thinking:

Don't say:

> “We use Kafka because it's scalable.”

Say:

> “The producer should not wait for the consumer, so we need asynchronous communication.”

Don't say:

> “We use WebSocket because it is real time.”

Say:

> “Both client and server must initiate messages over a long-lived connection.”

Don't say:

> “We use GraphQL because modern companies use GraphQL.”

Say:

> “Different clients need different related fields and fixed REST payloads create significant over-fetching and endpoint proliferation.”

---

# 90. FAILURE BEHAVIOR MATTERS AS MUCH AS HAPPY PATH

For every communication choice ask:

What happens if receiver is down?

What happens if request times out?

Can we retry?

Can retry duplicate side effects?

Is message persisted?

Can messages arrive out of order?

What happens if connection disconnects?

How does client reconnect?

Can receiver apply backpressure?

This is what separates protocol trivia from distributed-system design.

---

# 91. BACKPRESSURE

🟢 **SENIOR CONCEPT**

Suppose producer can send:

100K messages/sec

but consumer handles:

20K/sec

Without control:

queue grows
memory grows
latency grows
system eventually fails

Backpressure means:

> A slower consumer must somehow constrain, buffer or signal the faster producer.

We'll deep-dive this in messaging.

---

# 92. SLOW CLIENT PROBLEM WITH WEBSOCKET

Suppose server generates:

10K events/sec

but one client's network can consume:

100 events/sec

What happens?

Potential:

buffers grow forever

You need policy:

drop?
sample?
disconnect?
buffer to limit?
coalesce?

This is a real senior-system-design concern.

---

# 93. HEARTBEATS

🟡

Long-lived connections need a way to determine whether the other endpoint still exists.

Typical concept:

ping
↓
pong

If heartbeats fail for too long:

connection considered dead

Then reconnect.

---

# 94. RECONNECTION STORM

Imagine:

2 million WebSocket clients

lose connection because one cluster fails.

If all reconnect immediately:

2 million connection attempts

can overwhelm healthy infrastructure.

Mitigation concepts:

exponential backoff
jitter
connection draining
capacity headroom

---

# 95. REST / GRAPHQL / gRPC — 8 YOE QUESTIONS

You should answer these automatically:

**1. REST vs HTTP?**

**2. Why REST?**

**3. What does REST statelessness mean?**

**4. REST vs GraphQL?**

**5. What problem does GraphQL solve?**

**6. What is over-fetching?**

**7. What is under-fetching?**

**8. What new problems does GraphQL create?**

**9. What is gRPC?**

**10. Why Protocol Buffers?**

**11. REST vs gRPC?**

**12. Why is gRPC attractive internally?**

**13. What are unary and streaming RPCs?**

**14. Does gRPC support bidirectional streaming?**

**15. Why isn't gRPC automatically better than REST?**

---

# 96. REAL-TIME — 8 YOE QUESTIONS

**16. Polling vs long polling?**

**17. SSE vs WebSocket?**

**18. When would SSE be enough?**

**19. Why WebSocket for chat?**

**20. How do you scale millions of WebSocket connections?**

**21. What if user A and user B are connected to different WebSocket servers?**

**22. How do you handle disconnected clients?**

**23. How do you detect dead connections?**

**24. What's a reconnection storm?**

**25. How do you handle slow consumers?**

---

# 97. WEBRTC — QUESTIONS

**26. WebSocket vs WebRTC?**

**27. What is signaling?**

**28. What is STUN?**

**29. What is TURN?**

**30. Why is TURN expensive?**

**31. Why doesn't peer-to-peer always work?**

**32. Why does group video need media servers?**

**33. Video streaming vs video conferencing?**

---

# 98. RAPID-FIRE MODEL ANSWERS

## Q: REST vs GraphQL?

> “REST exposes resource-oriented endpoints with server-defined responses. GraphQL exposes a schema and lets clients select the fields and relationships they need. GraphQL reduces over- and under-fetching but adds query, caching and server-complexity concerns.”

---

## Q: REST vs gRPC?

> “REST is simpler and more interoperable for public APIs. gRPC provides strongly typed contracts, generated clients, compact Protobuf serialization and streaming, making it attractive for internal service communication.”

---

## Q: Why WebSocket instead of REST?

> “When both client and server need to initiate low-latency messages repeatedly, a persistent WebSocket avoids repeatedly creating request/response exchanges.”

---

## Q: SSE vs WebSocket?

> “SSE is simpler when communication is only server-to-client. WebSocket is full duplex and better when both sides continuously send messages.”

---

## Q: Why not poll?

> “Polling is simple but wastes requests when nothing changes and adds update latency. Persistent push mechanisms are better when updates are frequent or need low latency.”

---

## Q: WebSocket vs WebRTC?

> “WebSocket is a persistent client-server message channel. WebRTC is specialized for low-latency real-time media and includes connectivity/NAT traversal and media handling.”

---

## Q: STUN vs TURN?

> “STUN helps a peer discover connectivity information for direct communication. TURN relays the actual traffic when direct peer connectivity cannot be established.”

---

# 99. COMMON BAD ANSWERS

### ❌ “GraphQL replaces REST.”

No.

Different trade-off.

---

### ❌ “GraphQL is always faster because it returns fewer fields.”

No.

Complex queries can make the backend more expensive.

---

### ❌ “gRPC is always faster.”

No universal guarantee.

---

### ❌ “gRPC doesn't support bidirectional communication.”

Incorrect; don't memorize the contradictory sentence in the raw source.

---

### ❌ “WebSocket guarantees message delivery.”

A live reliable transport connection does not magically provide application-level durable delivery semantics.

---

### ❌ “WebRTC is always peer-to-peer with no servers.”

The source itself later notes that TURN and media servers may be required.

---

### ❌ “SSE and WebSocket are basically the same.”

No.

SSE → server → client

WebSocket → server ↔ client

---

# 100. ONE-PAGE MEMORY SHEET

API COMMUNICATION

REST
---
resource-oriented
HTTP
usually JSON
simple
ubiquitous
cache-friendly

BEST:
public/general APIs

GRAPHQL
-------
typed schema
client selects fields
reduces over/under-fetching

COST:
query complexity
N+1
caching
governance

BEST:
complex frontend data requirements

gRPC
---
RPC
HTTP/2
Protocol Buffers
code generation
unary + streaming

BEST:
internal service-to-service

REAL-TIME

POLLING
-------
client repeatedly asks

LONG POLLING
------------
request waits until event

SSE
---
persistent HTTP
server → client

BEST:
notifications / feeds

WEBSOCKET
---------
persistent
full duplex
client ↔ server

BEST:
chat / gaming / interactive systems

WEBRTC
---
real-time media
audio/video/data
NAT traversal

SIGNALING
arrange connection

STUN
discover connectivity

TURN
relay when direct path fails

---

# 101. 60-SECOND INTERVIEW DECISION ANSWER

If asked:

> **“How do you choose REST, GraphQL, gRPC, SSE or WebSocket?”**

Answer:

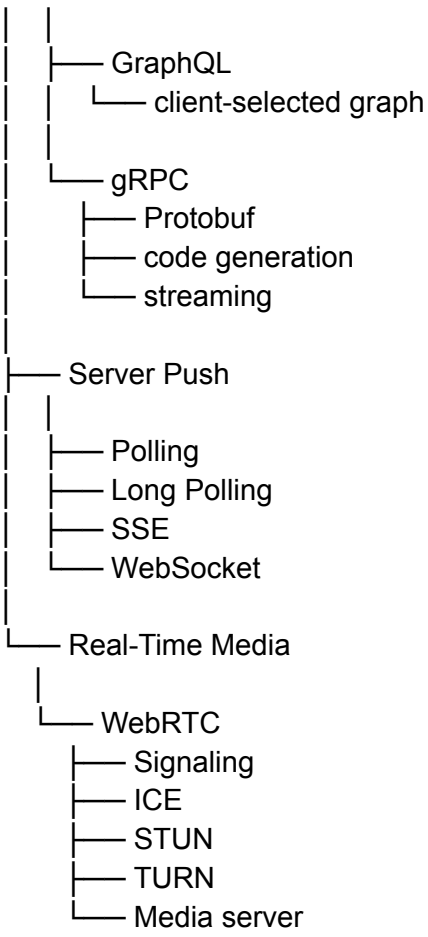
> “I first identify the communication pattern rather than picking a technology. For ordinary public request-response APIs, REST is usually my default because it is simple and interoperable. If frontend clients need flexible related data and fixed REST payloads cause substantial over- or under-fetching, I'd consider GraphQL. For internal service-to-service calls where strong contracts, generated clients, compact serialization or streaming are valuable, I'd consider gRPC. For server-to-client live updates only, SSE is often sufficient. If both client and server need to initiate messages over a persistent connection, WebSocket is a better fit. For interactive audio or video, I would move to WebRTC rather than treating it as ordinary application messaging.”

That is the core of Part 3.

---

# 102. PART 3 — MEMORY MAP

COMMUNICATION
│
├── Request / Response
│   │
│   ├── REST
│   │   └── resource-oriented




---

# 103. WHAT TO ACTUALLY MEMORIZE

If time is limited, memorize these **seven distinctions**:

1. REST vs HTTP

2. REST vs GraphQL

3. REST vs gRPC

4. Polling vs Long Polling

5. SSE vs WebSocket

6. WebSocket vs WebRTC

7. STUN vs TURN

If those seven are crystal clear, most interview follow-ups become derivable instead of memorized.

---

# CJ SYSTEM DESIGN INTERVIEW HANDBOOK

## PART 4 — DATABASES: SQL → INDEXES → REPLICATION → PARTITIONING → SHARDING → CONSISTENT HASHING → CASSANDRA

**Target:** Senior Software Engineer / Backend Engineer — ~8 YOE
**Goal:** Given a workload, choose a storage model and explain **why it works, how it scales, and what breaks next**.

> **Priority legend**
> 🟢 **MUST KNOW** — senior interview floor
> 🟡 **SHOULD KNOW** — explain confidently
> 🔵 **DEEP DIVE** — interviewer-push territory
> 🔷 **INTERVIEW EXPANSION** — added beyond the PDF for interview completeness
> ⚠️ **SOURCE CORRECTION** — source statement that should not be memorized literally

Your source's main data progression is partitioning → sharding → indexing → hashing → distributed hashing → rehashing → consistent hashing → virtual nodes → Cassandra.

---

# 1. THE DATABASE DECISION SHOULD START WITH ACCESS PATTERNS

🟢 **MUST KNOW**

Do not start system design with:

> “SQL or MongoDB?”

Start with:

```
WHAT DATA DO I HAVE?
        ↓
HOW IS IT WRITTEN?
```

↓
HOW IS IT READ?
↓
WHAT MUST BE ATOMIC?
↓
HOW MUCH DATA?
↓
HOW MUCH TRAFFIC?
↓
HOW IMPORTANT ARE JOINS?
↓
HOW IMPORTANT IS CONSISTENCY?

Then choose storage.

---

# 2. THE SENIOR DATABASE QUESTIONS

For every important dataset ask:

1. What is the primary access pattern?

2. Read-heavy or write-heavy?

3. Key lookup or complex query?

4. Do I need joins?

5. Do I need transactions?

6. What is the data size?

7. What is the growth rate?

8. Do I need horizontal scaling?

9. Can data be stale?

10. What happens when one node fails?

This is the database equivalent of the HLD checklist.

---

# 3. SQL VS NoSQL — ELI5

## SQL

Think:

> Structured tables with relationships and strong query/transaction capabilities.

Example:

CUSTOMER
--------
customer_id
name

ORDER
-----
order_id
customer_id
amount

Relationships matter.

---

## NoSQL

Think:

> A family of non-relational storage models designed around particular access/scaling patterns.

Examples:

Key-Value
Document
Wide Column
Graph

There is no single thing called “the NoSQL architecture.”

---

# 4. SOURCE VIEW: WHY NoSQL?

The source emphasizes these motivations:

Flexible schema

Object/document-oriented retrieval

Horizontal partitioning

Scalability / availability

Avoiding relational joins for certain access patterns

It contrasts this with SQL's strengths around updates, transactions and joins. The raw notes, however, make several absolute claims about NoSQL that should not be memorized literally.

---

# 5. ⚠️ SOURCE CORRECTION — “NoSQL CANNOT DO TRANSACTIONS”

The source says, effectively:

> NoSQL databases relax ACID, therefore we cannot have transactions in NoSQL databases.

Do **not** memorize that as a universal rule.

🔷 **Interview correction:**

Different NoSQL databases provide different transaction and consistency guarantees.

The safe answer is:

> “Many NoSQL systems historically traded some relational/transactional guarantees for scale or availability, but modern NoSQL databases vary widely; some support transactions.”

---

# 6. ⚠️ SOURCE CORRECTION — “NoSQL DOESN'T SUPPORT UPDATES”

The source also says updates are not inherently supported and that documents must be deleted/reinserted.

Again, do not generalize that.

🔷 **Interview correction:**

Many document/key-value databases support updates.

The real question is:

> **What update semantics, atomicity scope and consistency guarantees does this database provide?**

---

# 7. SQL VS NoSQL — INTERVIEW TABLE

| Concern | SQL / Relational | NoSQL |
|---|---|---|
| Data model | Tables/relations | KV/document/wide-column/graph |
| Schema | Usually structured | Often more flexible |
| Joins | Major strength | Often limited/avoided |
| Transactions | Major strength | Product-dependent |
| Complex querying | Strong | Model-dependent |
| Horizontal scale | Possible, can be complex | Often a core design goal |
| Data modeling | Normalize relationships | Often access-pattern driven |
| Typical fit | transactional relational data | specialized scale/access patterns |

---

# 8. THE INTERVIEW ANSWER: SQL VS NoSQL

> “I wouldn't choose based purely on scale. If the domain is relational and needs joins, constraints and strong transactional semantics, SQL is usually my default. I'd consider a NoSQL model when the access patterns map naturally to key-value, document, wide-column or graph storage, or when the required distribution and scaling characteristics are better served by that model.”

That is much stronger than:

> “SQL doesn't scale, NoSQL scales.”

---

# 9. NORMALIZATION — FIRST PRINCIPLES

**🔷 INTERVIEW EXPANSION**

Suppose:

ORDER
---
order_id
customer_id
customer_name
customer_email
customer_address

Every order duplicates customer data.

Instead:

CUSTOMER
--------
customer_id
name
email
address

ORDER
-----
order_id
customer_id

This reduces duplication.

That's normalization.

---

# 10. WHY NORMALIZE?

Less duplicated data

Cleaner updates

Referential integrity

Less anomaly risk

But now reads may require:

JOIN

So normalization trades simpler integrity for potentially more complex reads.

---

# 11. DENORMALIZATION

🔷

Sometimes we intentionally duplicate data.

Why?

Fewer joins

Faster reads

Precomputed views

Distributed-read efficiency

Trade-off:

Duplicate data
↓
more write work
↓
consistency becomes harder

Memory:

> **Normalize for integrity.**
> **Denormalize for access patterns.**

Not absolute — but useful.

---

# 12. INDEXING — FIRST PRINCIPLES

🟢 **MUST KNOW**

The source defines an index as a data structure that reduces the number of rows/records that must be examined for a query.

Suppose table contains:

100 million users

and query is:

```
SELECT *
FROM users
WHERE email = 'a@example.com';
```

Without a useful index, database may scan:

row 1
row 2
row 3
...
row 100,000,000

An index provides a much faster path to matching rows.

---

# 13. DATABASE INDEX — ELI5

A textbook has:

500 pages

You want:

"Consistent Hashing"

Without index:

scan every page

With index:

Consistent Hashing → page 312

Same principle.

---

# 14. WHAT IS STORED IN AN INDEX?

Conceptually:

indexed value
+
pointer / row locator / primary key

Example:

alice@example.com → row 918271
bob@example.com → row 125991

Actual implementation depends on database engine.

---

# 15. WHY INDEXES SPEED READS

Instead of:

O(N)-style full scan

you use an index structure to reduce search work.

The source summarizes this as avoiding a full table scan.

---

# 16. INDEX TRADE-OFF

🟢 **MUST KNOW**

Indexes are not free.

Every relevant write may require updating:

table
+
index(es)

Therefore:

MORE INDEXES

+ faster targeted reads

- more storage

- slower writes

- more maintenance

The source correctly notes that writes have additional index-maintenance overhead.

---

# 17. ⚠️ SOURCE CORRECTION — “INDEXES ARE NOT FOR PRODUCTION WRITE DATABASES”

The source says frequent writes can make indexes unusable and suggests indexes are typically more suited to data warehouses than production databases.

Do **not** memorize this.

🔷 **Interview correction:**

Indexes are routinely used in transactional production databases.

The real trade-off is:

> **Each index must justify its read benefit against write/storage cost.**

---

# 18. WHAT SHOULD YOU INDEX?

Usually columns frequently used in:

WHERE

JOIN

ORDER BY

GROUP BY

depending on database and query plan.

But:

> Do not mechanically index every column.

---

# 19. INDEX SELECTIVITY

**🔷 HIGH-ROI INTERVIEW EXPANSION**

Suppose table has:

100M users

Index A:

email

almost every value unique.

Good selectivity.

Index B:

is_active

only:

true
false

Very low cardinality.

An index is usually most valuable when it dramatically narrows the search.

---

# 20. COMPOSITE INDEX

**🟢 COMMON INTERVIEW QUESTION**

Suppose:

SELECT *

```
FROM orders
WHERE customer_id = ?
  AND created_at >= ?
ORDER BY created_at;
```

A composite index might involve:

(customer_id, created_at)

Unlike two unrelated indexes, a composite index represents multiple columns in a particular order.

---

# 21. INDEX COLUMN ORDER MATTERS

🔷

For an index:

(A, B)

it is not generally equivalent to:

(B, A)

The useful prefix/access behavior depends on the database.

Interview principle:

> Design indexes from actual query patterns, not just columns that “look important.”

---

# 22. CLUSTERED VS NON-CLUSTERED INDEX

The source distinguishes clustered and non-clustered indexes, noting that a table can have one clustered organization and multiple non-clustered indexes in systems that support this model.

Conceptually:

## Clustered

The table's physical/logical row organization follows the index key.

Think:

```
DATA ITSELF ORDERED/ORGANIZED
BY THE INDEX
```

## Non-clustered

Separate index structure points toward data rows.

Think:

INDEX
↓
ROW

Exact details are database-engine-specific.

---

# 23. 🔷 B+ TREE — WHY INTERVIEWERS ASK ABOUT IT

The source discusses indexes but does not explain B+ trees.

For interview completeness:

Many relational indexes use tree-like structures such as B+ trees because they support:

Equality lookup

Range lookup

Ordered traversal

Prefix/range scans

Example:

WHERE price BETWEEN 100 AND 200

---

# 24. WHY NOT HASH EVERYTHING?

Hash lookup is excellent for:

key = exact value

But poor for:

key BETWEEN X AND Y

ORDER BY key

prefix ranges

because hashing destroys useful ordering.

This leads to:

B+ TREE
→ equality + ranges

HASH
→ equality

---

# 25. INDEXING VS HASHING

The source distinguishes hashing as computing locations/buckets from a hash function, while indexing keeps structures that point toward relevant records.

Interview mental model:

TREE INDEX

ordered structure
good ranges

HASH

key → bucket
great equality lookup
poor ranges

---

# 26. HASH TABLE — FIRST PRINCIPLES

The source explains collisions and buckets.

Suppose:

hash("Alice") % N = 3

Store Alice in:

bucket 3

Another key may also hash to:

bucket 3

That's a collision.

---

# 27. COLLISION

hash(Alice) → bucket 3

hash(Bob) → bucket 3

Need collision handling.

Possible concept from source:

```
bucket
  |
  +-- Alice
  +-- Bob
```

A well-sized hash table aims for near-constant average lookup.

---

# 28. PRIMARY KEY VS SECONDARY INDEX

🟢

Primary key:

> uniquely identifies the entity/row.

Example:

user_id

Secondary index:

> gives an alternative lookup path.

Example:

email → user

Why secondary indexes become interesting in distributed systems:

> The base data may be partitioned one way, while the index needs a different access path.

That can become expensive.

---

# 29. REPLICATION VS PARTITIONING

🟢🟢🟢 **VERY IMPORTANT**

These solve different problems.

## Replication

COPY DATA

Example:

Node A: Users 1–100M
Node B: Users 1–100M
Node C: Users 1–100M

Each holds copies.

Primary goals:

availability
fault tolerance
read scaling

---

## Partitioning

SPLIT DATA

Example:

Node A: Users 1–30M
Node B: Users 30M–60M
Node C: Users 60M–100M

Primary goals:

storage scaling
write scaling
working-set distribution

---

# 30. MEMORY HOOK

REPLICATION
same data → more machines

PARTITIONING
different data → different machines

You should be able to say this instantly.

---

# 31. PARTITIONING — SOURCE MODEL

The source describes two main forms:

VERTICAL PARTITIONING
split columns

HORIZONTAL PARTITIONING
split rows

---

# 32. VERTICAL PARTITIONING

Example original table:

USER
---
id
name
email
password_hash
bio
profile_photo
preferences

Split:

USER_CORE
---
id
name
email
password_hash

USER_PROFILE
---
id
bio
profile_photo
preferences

Same logical entity, different groups of columns/data.

---

# 33. WHY VERTICAL PARTITION?

Potential reasons:

Different access frequency

Different security requirements

Very large columns

Independent scaling

Service/domain separation

---

# 34. HORIZONTAL PARTITIONING

Rows split by a partition rule.

USERS

Shard A
user_id 1–1M

Shard B
user_id 1M–2M

Shard C
user_id 2M–3M

Same schema.

Different rows.

---

# 35. SHARDING

The source defines sharding as splitting one logical dataset across multiple databases/machines so a cluster can store more data and handle additional requests.

Simplified:

> **Sharding = horizontal partitioning across independently scalable database nodes.**

The source also explicitly distinguishes horizontal partitioning within one DB instance from sharding across database instances.

---

# 36. WHY SHARD?

Suppose one database maxes out at:

storage
CPU
IOPS
write throughput

Then:

```
      ROUTER
     /   |   \
    v    v    v
 SHARD1 SHARD2 SHARD3
```

Each machine serves only part of the dataset.

---

# 37. WHAT DOES SHARDING SCALE?

Potentially:

Storage

Write throughput

Read throughput

Working set

I/O

because load is spread across machines.

---

# 38. WHY NOT SHARD IMMEDIATELY?

Because sharding creates hard problems:

Shard-key choice

Cross-shard queries

Cross-shard joins

Distributed transactions

Rebalancing

Hot shards

Secondary indexes

Operational complexity

Senior rule:

> **Shard when one node is becoming a real bottleneck, not because “large companies shard.”**

---

# 39. SHARD KEY

🟢 **MUST KNOW**

The shard key determines:

> Which shard owns a record?

Example:

shard = hash(user_id) % N

or:

users 1–1M → shard A
users 1M–2M → shard B

---

# 40. WHAT MAKES A GOOD SHARD KEY?

A strong shard key tends to provide:

High cardinality

Even distribution

Stable ownership

Low hotspot risk

Useful query locality

---

# 41. BAD SHARD KEY: LOW CARDINALITY

Suppose:

is_premium

only has:

true
false

Cannot sensibly distribute thousands of machines.

---

# 42. BAD SHARD KEY: HOT VALUE

Suppose you shard by:

country

and:

70% traffic = India

One partition gets 70% traffic.

That's a:

**hot shard / hot partition**

---

# 43. RANGE-BASED SHARDING

🔷 **INTERVIEW EXPANSION**

Example:

1–1M → A
1M–2M → B
2M–3M → C

Advantages:

Range queries easy

Data locality

Disadvantage:

Sequential IDs can create a hot latest shard

---

# 44. HASH-BASED SHARDING

hash(key)
  ↓
shard

Advantage:

better distribution

Disadvantage:

range locality lost

---

# 45. DIRECTORY-BASED SHARDING

The source mentions a lookup table mapping attributes to shards.

Concept:

Customer A → Shard 7
Customer B → Shard 2
Customer C → Shard 9

Advantages:

Flexible placement

Costs:

directory lookup
directory availability
metadata management

---

# 46. QUERY ROUTING

Once sharded, every request must answer:

> “Which shard?”

Example:

GET user 93842
    ↓
compute/lookup shard
    ↓
Shard 7

If query does not contain the shard key:

SELECT *

FROM users
WHERE email = ?

you may not know which shard to ask.

---

# 47. SCATTER-GATHER

🔷

Worst-case approach:

Query
  |
  +----> Shard A
  +----> Shard B
  +----> Shard C
  +----> Shard D

Then combine results.

This can become expensive as shard count grows.

Therefore:

> Query patterns should strongly influence shard-key choice.

---

# 48. CROSS-SHARD JOINS

The source correctly highlights that joins across shards are expensive.

Why?

Instead of:

one DB process

you now have:

network calls
multiple machines
distributed execution
data transfer
coordination

---

# 49. CROSS-SHARD TRANSACTION

Suppose transferring money:

Account A → shard 1

Account B → shard 9

Need:

Debit A
AND
Credit B

atomically.

Now transaction spans multiple databases.

That is dramatically harder than a local database transaction.

We'll cover:

2PC
Saga

later.

---

# 50. CO-LOCATION

🔷 **HIGH ROI**

If entities frequently operate together, try to place them together.

Example:

partition by customer_id

so:

customer
orders
preferences

can potentially be accessed within the same partition/domain.

This is sometimes called improving **data locality**.

---

# 51. REPLICATION + SHARDING TOGETHER

Real distributed databases often do both.

Example:

SHARD 1
├── Replica A
├── Replica B
└── Replica C

SHARD 2
├── Replica A
├── Replica B
└── Replica C

Partitioning answers:

> Which subset of data?

Replication answers:

> How many copies?

---

# 52. DISTRIBUTED HASHING — SOURCE MODEL

The source begins with the problem:

> One machine cannot hold all keys, so distribute keys across multiple servers.

Naive approach:

server =
hash(key) % N

where:

N = number of servers

---

# 53. EXAMPLE

Servers:

S0
S1

S2
S3

For:

hash("alice") = 37

Then:

37 % 4 = 1

Alice belongs to:

S1

Very simple.

---

# 54. THE REHASHING PROBLEM

🟢🟢🟢

The source explains this extremely well.

If:

N = 4

then:

hash(key) % 4

If one server is added:

N = 5

now:

hash(key) % 5

For many keys, the result changes.

The source notes that when server count changes, most modulo mappings change, creating massive remapping/cache misses and load on the source database.

---

# 55. WHY THIS CAN BE DISASTROUS FOR CACHE

Before:

Key A → cache server 1
Key B → cache server 2
Key C → cache server 3

After server count changes:

Key A → different server
Key B → different server
Key C → different server
...

New servers don't have those cached values.

Result:

cache misses
↓
database traffic spike
↓
database overload

---

# 56. CONSISTENT HASHING — THE SOLUTION

🟢 **MUST KNOW**

The source introduces consistent hashing specifically to minimize key relocation when servers are added or removed.

Core idea:

Hash both:

SERVERS
and
KEYS

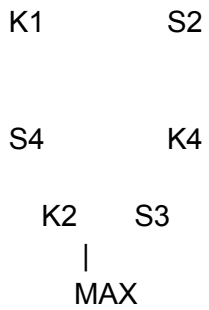
into the same hash space

often visualized as a ring.

---

# 57. THE HASH RING






For a key:

hash(key)

place it on ring.

Then move clockwise until you find a server.

That server owns the key.

---

# 58. KEY OWNERSHIP

Example:

K1
↓
clockwise
↓
S4

So:

K1 → S4

---

# 59. SERVER REMOVAL

Suppose:

S3 crashes

Only keys mapped to S3's range need new ownership.

Other key mappings remain unchanged.

The source emphasizes that keys owned by unaffected servers do not move when another server is removed.

---

# 60. SERVER ADDITION

Add:

S5

between two existing ring locations.

Only keys in the relevant neighboring range move to S5.

Again:

MOST KEYS STAY WHERE THEY WERE

---

# 61. THE MEMORY HOOK

MODULO HASHING

change N
→ remap almost everything

CONSISTENT HASHING

change node
→ remap neighborhood

Memorize this.

---

# 62. CONSISTENT HASHING — 30 SECOND ANSWER

> “Consistent hashing distributes keys across nodes while minimizing remapping when nodes join or leave. Nodes and keys are mapped into the same hash space, usually visualized as a ring. A key is assigned to the next node clockwise. When membership changes, only the neighboring key range needs redistribution instead of recomputing most `hash(key) % N` mappings.”

---

# 63. PROBLEM: UNEVEN SERVER PLACEMENT

Suppose ring looks like:

S1
|
|
|
|
S2-S3-S4

S1 may own huge portion of keyspace.

Result:

S1 overloaded

others underused

The source calls out this imbalance explicitly.

---

# 64. VIRTUAL NODES

🟢 **MUST KNOW**

Instead of one ring position per physical server:

Server A → A

give many:

A1
A2
A3
...
A100

Same for B, C.

The source describes virtual nodes/replicas as a way to spread each physical machine around the ring and improve load distribution.

---

# 65. VIRTUAL NODE RING

A1
  B2
    C3
 A4
      B5
  C6
    A7

B8

Physical server A owns multiple separate ranges.

If A disappears, its ranges get distributed among multiple neighboring servers rather than hammering one node.

---

# 66. WHY VIRTUAL NODES?

Better load balance

Smoother rebalancing

Failure load distributed

Different physical node capacities can be weighted

The source notes that the number of virtual replicas can function like a weight.

---

# 67. CONSISTENT HASHING DOES NOT SOLVE EVERYTHING

🟢 **SENIOR POINT**

It solves:

key placement
+
rebalancing

It does not automatically solve:

Replication

Failure detection

Consistency

Hot keys

Durability

Cross-node transactions

Secondary indexes

Do not oversell it.

---

# 68. CONSISTENT HASHING VS SHARDING

A common confusion.

## Sharding

> We decided to split data across machines.

## Consistent hashing

> One possible method for deciding which machine owns which key.

The source explicitly makes this distinction.

Memory:

SHARDING
= strategy: distribute data

CONSISTENT HASHING
= placement algorithm

---

# 69. HOT KEY VS HOT SHARD

🔷

## Hot shard

Too much workload maps to one partition.

Example:

country = India

---

## Hot key

A single record/key gets enormous traffic.

Example:

celebrity_profile_id = 1

Even perfect hashing doesn't solve one key receiving 1M requests/sec.

Possible approaches depend on use case:

cache
replicate hot item
request coalescing
partition logical key
rate limiting

---

# 70. CASSANDRA — WHY IT APPEARS AFTER CONSISTENT HASHING

The source describes Cassandra as a distributed NoSQL database designed around high availability and scalable distribution, and then connects its data ownership to virtual nodes and consistent hashing.

So the correct learning order is:

Partitioning
↓
Hashing
↓
Distributed Hashing
↓
Consistent Hashing
↓
Virtual Nodes
↓
Cassandra

Now Cassandra makes conceptual sense.

---

# 71. CASSANDRA HIGH-LEVEL MENTAL MODEL

🟡 **SHOULD KNOW**

Think:

many peer nodes

data partitioned across cluster

replicas provide availability

partition key determines placement

clustering columns determine ordering
inside a partition

---

# 72. CASSANDRA IS PEER-ORIENTED

The source contrasts gossip-style distributed membership with central-master communication and describes gossip as a peer-to-peer approach.

Conceptually:

NODE A ↔ NODE B
 ↕ ↕
NODE C ↔ NODE D

not:

MASTER
/ | \
A B C

for every operation.

---

# 73. CASSANDRA PRIMARY KEY

The source distinguishes:

Primary Key

Partition Key

Clustering Key

This is important.

---

# 74. PARTITION KEY

🟢

Determines:

Which partition/node range owns this data?

Example:

user_id

Conceptually:

hash(user_id)
↓
token/range
↓
node

The source says the partitioner computes a hash from the partition key to determine storage location.

---

# 75. CLUSTERING KEY

Determines:

How rows are ordered **inside a partition**.

The source explicitly describes clustering keys as controlling sorted order within a partition.

Example:

PRIMARY KEY ((conversation_id), timestamp)

Conceptually:

conversation_id
→ choose partition

timestamp
→ order messages inside partition

---

# 76. CASSANDRA EXAMPLE — CHAT MESSAGES

Suppose frequent query:

“Get recent messages for conversation 123.”

Model:

conversation_id
↓
PARTITION KEY

and:

message_timestamp
↓
CLUSTERING KEY

Conceptual row organization:

Partition: conversation 123

10:01 message A
10:02 message B
10:03 message C
10:04 message D

This is why:

> **In Cassandra, you model from queries/access patterns.**

---

# 77. CASSANDRA KEY EXAMPLES FROM SOURCE

The source provides patterns such as:

PRIMARY KEY (a)

partition key:

a

and:

PRIMARY KEY (a, b)

where:

a = partition key
b = clustering key

and composite forms where parentheses define a composite partition key.

---

# 78. COMPOSITE PARTITION KEY

Example:

PRIMARY KEY ((tenant_id, region), timestamp)

Partition key:

tenant_id + region

Clustering:

timestamp

Useful when one single field would create partitions that are too large or poorly distributed.

---

# 79. WHY PARTITION SIZE MATTERS

🔷

Suppose partition key:

country

One partition:

India

could contain hundreds of millions of rows.

That's problematic.

A good partition key should avoid:

unbounded partitions

extreme skew

hot partitions

---

# 80. GOSSIP PROTOCOL

🟡 **SHOULD KNOW CONCEPT**

The source describes gossip like spreading a rumor/epidemic through a distributed cluster.

Mental model:

A knows some cluster state
↓
A tells B

B tells C

C tells D

D tells A

Eventually information propagates.

---

# 81. WHY GOSSIP?

Instead of one central machine constantly coordinating everyone:

```
MASTER
├── node A
├── node B
├── node C
├── ...
└── node 10,000
```

peers periodically exchange membership/state information.

Benefits conceptually:

Decentralization

Scalability

Failure information propagation

---

# 82. GOSSIP DOES NOT MEAN CONSENSUS

🔷 **IMPORTANT**

Gossip:

spread information.

Consensus:

agree on one decision/value despite failures.

Do not confuse:

Gossip

with:

Paxos / Raft style consensus

They solve different problems.

---

# 83. DATABASE REPLICATION — FIRST PRINCIPLES

🔷 **INTERVIEW EXPANSION**
The source references replication throughout but does not give a clean standalone progression here.

Replication means:

maintain multiple copies of data

Why?

Availability

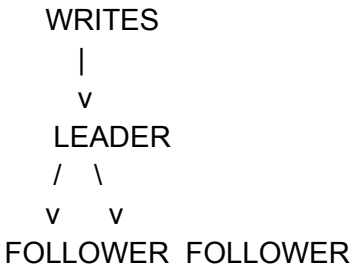
Fault tolerance

Read scaling

Disaster recovery

---

# 84. LEADER-FOLLOWER REPLICATION




Usually:

Writes → Leader

Reads → leader and/or followers

depending on consistency requirements.

---

# 85. REPLICATION LAG

🟢 **MUST KNOW**

Suppose:

WRITE:
balance = 100

to leader.

Immediately:

READ

from follower.

Follower may still show:

balance = 90

because replication has not caught up.

That is replication lag.

---

# 86. READ-AFTER-WRITE PROBLEM

User:

updates profile photo

then immediately:

refreshes profile

Read goes to stale replica.

User sees:

old photo

Possible design concern:

**read-your-write consistency**

We'll cover this deeply in consistency section.

---

# 87. SYNCHRONOUS VS ASYNCHRONOUS REPLICATION

🔷

## Synchronous

Acknowledge write only after required replica confirmation.

+ stronger durability/consistency

- higher write latency
- replica/network failure can affect availability

## Asynchronous

Leader acknowledges before followers catch up.

+ lower write latency

- replication lag
- possible loss during failover depending on guarantees

---

# 88. LEADER-BASED VS LEADERLESS — MENTAL MAP

🔷

LEADER-BASED

one node owns writes
followers replicate

LEADERLESS

client/coordinator may write to
multiple replicas

We'll connect leaderless systems with:

N
R
W
quorum

in Part 5.

---

# 89. THE DATABASE SCALING LADDER

🟢 **MUST KNOW**

Do not immediately shard.

Think:

QUERY SLOW
↓
fix query / index

READ LOAD HIGH
↓
cache / read replicas

MACHINE RESOURCE LIMIT
↓
vertical scale if sensible

DATA / WRITE LOAD EXCEEDS NODE
↓
partition / shard

AVAILABILITY REQUIRED
↓
replication

GLOBAL SCALE
↓
distributed placement
consistency trade-offs

Not universal, but excellent interview reasoning.

---

# 90. DATABASE CHOICE — INTERVIEW EXAMPLE 1

### User/account/payment system

Requirements:

Relationships

Transactions

Constraints

Complex queries

Strong default:

relational database

Reason:

> Data integrity and transactions matter more than schema flexibility.

---

# 91. DATABASE CHOICE — EXAMPLE 2

### Session cache

Access:

session_id → session

Need:

very fast key lookup
TTL

A key-value/in-memory store may fit naturally.

---

# 92. DATABASE CHOICE — EXAMPLE 3

### Product catalog

Products may have variable attributes:

Laptop
RAM
CPU

Shoe
size
material

Book
author
ISBN

Document-oriented storage may be attractive if access patterns align.

But:

> Don't automatically conclude “catalog = MongoDB.”

Still evaluate query/transaction requirements.

---

# 93. DATABASE CHOICE — EXAMPLE 4

### Massive write-heavy event data

Access often:

device_id
+
time range

A partitioned wide-column/time-series-oriented model may be attractive.

Key question:

> How will writes distribute, and how will reads find their partition efficiently?

---

# 94. DATABASE CHOICE — EXAMPLE 5

### Social graph

Queries:

friends of Alice

friends-of-friends

shortest relationship path

Graph storage may fit because relationships themselves are first-class access patterns.

---

# 95. DATABASE INTERVIEW QUESTION: “WHICH DATABASE WOULD YOU USE?”

Weak:

> “Cassandra because it scales.”

Strong:

> “First I'd identify the access patterns and consistency requirements. If this is high-volume key-based access with known partition keys and horizontal write scaling is important, Cassandra may fit. If the workload requires complex joins and multi-row transactional invariants, I'd prefer a relational database.”

# 96. INDEX INTERVIEW QUESTIONS

### 🟢 Q: What is an index?

A secondary data structure that provides a faster access path to rows so the database can avoid scanning the entire table for suitable queries.

### 🟢 Q: Why not index every column?

Because indexes consume storage and each relevant write must maintain them, increasing write cost.

### 🟢 Q: Why does column order in a composite index matter?

Because the index is organized according to that column sequence, affecting which predicates/orderings can efficiently use it.

### 🟡 Q: B-tree vs hash index?

Tree-based indexes support ordered/range access; hash-based indexing is naturally suited to equality lookup.

---

# 97. REPLICATION INTERVIEW QUESTIONS

### 🟢 Q: Why replication?

Availability, fault tolerance, additional read capacity and disaster recovery.

### 🟢 Q: Main downside of read replicas?

Replication lag can produce stale reads.

### 🟢 Q: Does replication solve write scaling?

Not necessarily. Traditional leader/follower replication primarily provides copies and read scaling; write bottlenecks can remain at the leader.

---

# 98. SHARDING INTERVIEW QUESTIONS

### 🟢 Q: Replication vs sharding?

Replication copies the same data to multiple nodes. Sharding splits different subsets of data across nodes.

### 🟢 Q: What makes a good shard key?

High cardinality, even distribution, stable ownership, low hotspot risk and alignment with common access patterns.

### 🟢 Q: Biggest cost of sharding?

Cross-shard queries/transactions, rebalancing, secondary indexes and operational complexity.

### 🟢 Q: When should you shard?

When one database cannot satisfy the required data size or write/workload scale after simpler options are insufficient.

---

# 99. CONSISTENT HASHING INTERVIEW QUESTIONS

### 🟢 Q: Why not `hash(key) % N`?

Because changing `N` remaps a large fraction of keys.

### 🟢 Q: How does consistent hashing help?

Keys and nodes share the same hash space, so adding/removing nodes moves only neighboring ranges.

### 🟢 Q: Why virtual nodes?

To improve load distribution and spread rebalancing/failure load across physical servers.

### 🟡 Q: Does consistent hashing prevent hot keys?

No. A disproportionately popular key can still overload the node serving it.

---

# 100. CASSANDRA INTERVIEW QUESTIONS

### 🟢 Q: Partition key vs clustering key?

The partition key determines which partition/node range stores the data. Clustering keys determine ordering within that partition.

### 🟡 Q: Why is Cassandra access-pattern driven?

Because efficient queries generally need to align with partition and clustering key design rather than arbitrary relational joins.

### 🟡 Q: Why virtual nodes?

A physical node can own many smaller ranges, improving distribution and rebalancing.

### 🟡 Q: What is gossip?

A decentralized mechanism where nodes exchange cluster-state information with peers until it propagates through the cluster.

---

# 101. SENIOR-LEVEL TRAPS

### ❌ “NoSQL is faster than SQL.”

Meaningless without workload.

---

### ❌ “NoSQL scales, SQL doesn't.”

False oversimplification.

---

### ❌ “Indexes always improve performance.”

They improve certain reads while increasing storage/write costs.

---

### ❌ “Sharding makes every query faster.”

Cross-shard queries can become substantially harder.

---

### ❌ “Replication and sharding are the same.”

No.

---

### ❌ “Consistent hashing is sharding.”

Consistent hashing is one placement strategy used with distributed/sharded systems.

---

### ❌ “Consistent hashing balances traffic perfectly.”

No.

Virtual nodes improve distribution, but workload skew and hot keys remain possible.

---

### ❌ “Cassandra partition key is just the SQL primary key.”

No.

Its partition key directly affects physical/distributed data placement.

---

# 102. 8-YOE DATABASE INTERVIEW FLOOR

You should answer these cleanly without notes:

1. SQL vs NoSQL?

2. When would you choose relational?

3. What is normalization?

4. Why denormalize?

5. What is an index?

6. Why do indexes improve reads?

7. Why do indexes slow writes?

8. Composite index?

9. Why does column order matter?

10. B-tree vs hash lookup?

11. Primary vs secondary index?

12. Replication vs partitioning?

13. Vertical vs horizontal partitioning?

14. What is sharding?

15. When should you shard?

16. What makes a good shard key?

17. What is a hot partition?

18. What happens to joins after sharding?

19. What is scatter-gather?

20. What is replication lag?

21. Why read replicas?

22. Does replication scale writes?

23. What is `hash(key) % N`?

24. What is the rehashing problem?

25. How does consistent hashing work?

26. Why virtual nodes?

27. What does consistent hashing NOT solve?

28. Cassandra partition key vs clustering key?

29. Why Cassandra is access-pattern driven?

30. What is gossip?

If these are automatic, your database foundation is strong.

---

# 103. ONE-PAGE DATABASE MEMORY SHEET

DATABASE CHOICE

ACCESS PATTERN FIRST
↓
transactions?
joins?
query type?
scale?
consistency?

SQL
---
relations

joins
transactions
constraints
complex querying

NoSQL
---
KV
document
wide-column
graph

choose by workload
not hype

INDEX
---
faster lookup
avoid full scan

COST:
storage
write amplification

TREE
---
equality + ranges

HASH
---
equality lookup

REPLICATION
---
same data
many machines

GOALS:
availability
read scale
fault tolerance

RISK:
replication lag

PARTITIONING
---

split dataset

VERTICAL
---
columns/features

HORIZONTAL
---
rows

SHARDING
---
horizontal split
across machines

GOOD SHARD KEY
---
high cardinality
even distribution
query locality
low hotspot risk

MODULO HASHING

hash(key) % N

PROBLEM:
change N
→ massive remap

CONSISTENT HASHING
---
nodes + keys on ring

change node
→ small range moves

VIRTUAL NODES
---
many ring positions
per physical server

better balance

CASSANDRA
---
distributed
partition-oriented

PARTITION KEY
→ where data lives

CLUSTERING KEY
→ order inside partition

GOSSIP
→ spread cluster state

---

# 104. 60-SECOND DATABASE ANSWER

If interviewer asks:

> **“How do you think about database scaling?”**

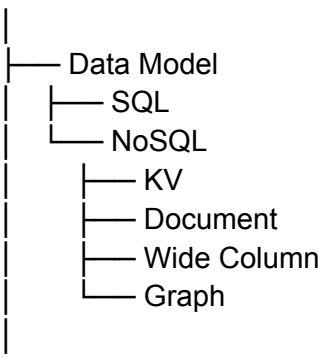
A strong answer:

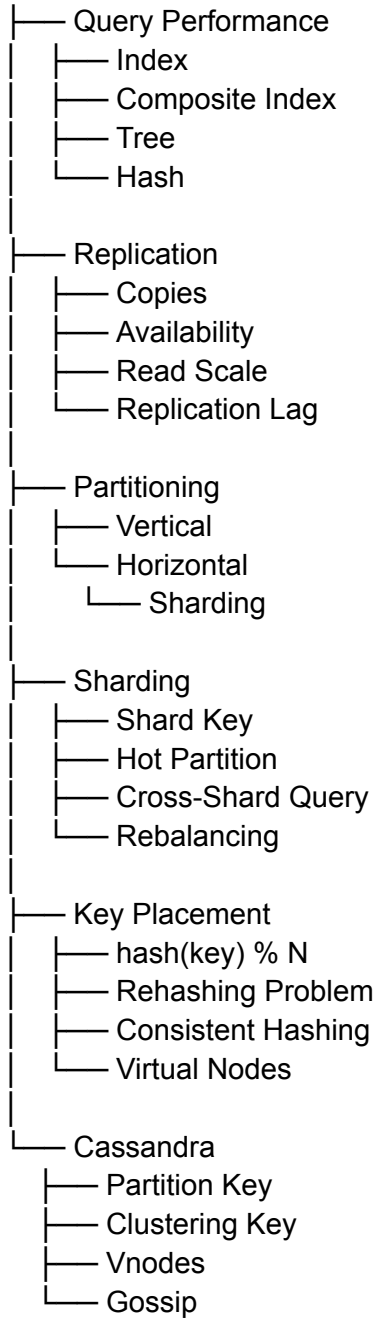
> “I start with access patterns and consistency requirements rather than immediately choosing SQL or NoSQL. If queries become slow, I first look at the schema, query plan and indexing. If the workload is read-heavy, caching or read replicas can reduce primary load. Replication improves availability and read capacity but can introduce stale reads through replication lag. If a single database eventually cannot handle the dataset or write workload, I consider horizontal partitioning or sharding. The shard key must distribute data evenly while matching common queries, otherwise we create hot partitions or expensive scatter-gather operations. If membership changes frequently, placement techniques such as consistent hashing can reduce data movement, with virtual nodes improving balance. Every scaling step buys capacity at the cost of additional distributed-system complexity.”

That paragraph is close to the **8-YOE interview floor**.

---

# 105. PART 4 — MEMORY MAP

DATA





---

# 106. THE ONE DATABASE PRINCIPLE TO REMEMBER

When stuck in an interview:

HOW WILL I QUERY THIS DATA?
↓
HOW WILL I PLACE THIS DATA?
↓
HOW WILL I SCALE IT?
↓
WHAT HAPPENS WHEN A NODE FAILS?
↓

WHAT CONSISTENCY DO I NEED?

That gets you from basic database selection into distributed database design naturally.

---

# CJ SYSTEM DESIGN INTERVIEW HANDBOOK

## PART 5 — TRANSACTIONS, ISOLATION & DISTRIBUTED CONSISTENCY

**Target:** Senior Software Engineer / Backend Engineer — ~8 YOE
**Goal:** Understand what happens when **concurrent operations race, replicas disagree, networks partition, or one business operation spans multiple services/databases**.

> **Priority legend**
> 🟢 **MUST KNOW** — senior interview floor
> 🟡 **SHOULD KNOW** — explain confidently
> 🔵 **DEEP DIVE** — interviewer-push territory
> 🔷 **INTERVIEW EXPANSION** — useful material added beyond what the source explicitly develops
> ⚠️ **SOURCE CORRECTION** — source wording that should not be memorized literally

The source covers dirty reads/writes, atomicity, transaction isolation, 2-phase locking, MVCC, distributed consistency levels, quorum, read-your-writes, 2PC and Saga.

---

# 1. THE BIG PICTURE

You now know how to:

split data
replicate data
shard data
distribute data

The next question is:

> **What happens when multiple copies or multiple writers disagree?**

That creates four major problem families:

CONCURRENT TRANSACTIONS
↓

Isolation

MULTIPLE REPLICAS
↓
Consistency

NETWORK PARTITION
↓
Availability vs consistency

MULTIPLE SERVICES / DATABASES
↓
Distributed transactions

---

# 2. ONE LOCAL DATABASE IS THE EASY CASE

Suppose:

Account A = ₹100

Transaction:

Debit ₹50
Credit another account ₹50

Inside one transactional database:

BEGIN

debit A
credit B

COMMIT

The database can coordinate both operations.

---

# 3. NOW DISTRIBUTE IT

Suppose:

Account Service
↓
Database A

Wallet Service
↓
Database B

Business operation:

Debit Account
+
Credit Wallet

What if:

Debit succeeds
Credit fails

?

You now have:

INCONSISTENT BUSINESS STATE

This is the essence of the distributed transaction problem.

---

# 4. TRANSACTION — FIRST PRINCIPLES

**🟢 MUST KNOW**

A transaction groups operations into one logical unit of work.

Example:

```
BEGIN;

UPDATE accounts
SET balance = balance - 100
WHERE id = 1;

UPDATE accounts
SET balance = balance + 100
WHERE id = 2;

COMMIT;
```

We want this to behave like:

ALL
or
NOTHING

---

# 5. ACID

🟢🟢🟢 **MUST KNOW**

The source separately discusses atomicity, consistency, isolation levels and durability; combine them into the standard ACID memory model.

A  Atomicity
C  Consistency
I  Isolation
D  Durability

---

# 6. ATOMICITY

The source defines atomicity as the transaction happening completely or not happening at all.

Mental model:

ALL OR NOTHING

Example:

Debit Alice
Credit Bob

Bad state:

Debit succeeded
Credit failed

Atomic transaction prevents a partial logical commit.

---

# 7. CONSISTENCY — ACID MEANING

The source explicitly distinguishes **ACID consistency** from distributed/CAP consistency.

For ACID, think:

> A transaction should take the database from one valid state to another valid state.

Example invariant:

balance >= 0

or:

foreign key must reference an existing user

---

# 8. ISOLATION

Isolation asks:

> What should concurrent transactions be allowed to observe about each other?

Suppose:

T1 updates balance

T2 reads balance

Should T2 see:

old committed value?
new uncommitted value?
wait?
snapshot?

The isolation level controls this.

---

# 9. DURABILITY

Once:

COMMIT SUCCESS

is returned, we expect committed changes to survive failures according to the database's durability guarantees.

The source explicitly notes that once a transaction is committed, its updates should be persisted.

---

# 10. ACID MEMORY HOOK

ATOMICITY
all or nothing

CONSISTENCY
valid state → valid state

ISOLATION
concurrent transactions don't corrupt each other

DURABILITY
committed means retained

---

# 11. CONCURRENCY IS WHERE THINGS GET INTERESTING

Suppose:

balance = 100

Two transactions execute concurrently.

T1 reads 100
T2 reads 100

T1 subtracts 30
T2 subtracts 50

Without correct concurrency control, final state might violate what the application expects.

Hence:

locks
MVCC
optimistic concurrency
isolation levels

---

# 12. DIRTY READ

🟢 **MUST KNOW**

The source defines a dirty read as reading data changed by another transaction **before that transaction commits**.

Example:

Initial balance = 100

T1:
UPDATE balance = 0
but not committed

T2:
READ balance
→ sees 0

T1:

ROLLBACK

T2 read a value that never became committed truth.

That is:

**Dirty Read**

---

# 13. DIRTY WRITE

The source also covers dirty writes: one transaction overwrites another transaction's still-uncommitted value.

Example:

T1 writes X = 10
not committed

T2 writes X = 20
not committed

Now rollback/recovery semantics become dangerous.

---

# 14. NON-REPEATABLE READ

🔷 **INTERVIEW EXPANSION**

Suppose:

T1:
SELECT balance
→ 100

T2:
UPDATE balance = 80
COMMIT

T1:
SELECT balance
→ 80

Inside the same T1:

same row
same query
different value

This is a:

> **Non-repeatable read**

---

# 15. PHANTOM READ

The source defines a phantom read as two identical queries returning different **sets of rows**, often because another transaction inserted a matching row.

Example:

```
SELECT *
FROM orders
WHERE amount > 1000;
```

First time:

10 rows

Another transaction inserts a qualifying order.

Second time:

11 rows

The new row is the:

> **phantom**

---

# 16. NON-REPEATABLE VS PHANTOM

NON-REPEATABLE READ

same row
different value

PHANTOM READ

same predicate
different set of rows

Memory hook:

> **Non-repeatable = row changed.**
> **Phantom = population changed.**

---

# 17. TRANSACTION ISOLATION LEVELS

🟢🟢🟢

The source lists:

1. Read Uncommitted
2. Read Committed
3. Repeatable Read
4. Serializable

and orders them from lower to higher isolation.

Memorize:

LOW ISOLATION
↓
Read Uncommitted

Read Committed

Repeatable Read

Serializable
↓
HIGH ISOLATION

---

# 18. READ UNCOMMITTED

Lowest conventional isolation level.

Conceptually:

Transaction may see
another transaction's
uncommitted changes

Therefore:

dirty reads possible

High concurrency, weak isolation.

---

# 19. READ COMMITTED

🟢 **VERY COMMON DEFAULT**

A transaction sees only committed values.

So:

dirty reads prevented

But:

same row may change
between two reads

because another transaction can commit in between.

---

# 20. REPEATABLE READ

Goal:

> If I read the same row again during my transaction, I should continue seeing a consistent version according to the database's isolation semantics.

The source associates Repeatable Read with versioning/snapshot-style behavior.

---

# 21. SERIALIZABLE

🟢 **STRONGEST STANDARD ISOLATION IDEA**

Conceptually:

> Concurrent transactions behave as though they executed one after another in some serial order.

Think:

Concurrent execution

should appear equivalent to

T1
then T2
then T3

This provides strong isolation but typically reduces available concurrency or increases coordination/retry cost.

---

# 22. SOURCE'S ISOLATION TRADE-OFF

The notes summarize:

Efficiency:

Read Uncommitted
>
Read Committed
>
Repeatable Read
>
Serializable

Isolation:

Read Uncommitted
<
Read Committed
<
Repeatable Read
<
Serializable

This is a useful interview intuition, though exact performance depends heavily on database implementation and workload.

---

# 23. ISOLATION LEVEL MEMORY TABLE

| Isolation | Core idea |
|---|---|
| Read Uncommitted | May observe uncommitted changes |
| Read Committed | Read committed values |
| Repeatable Read | Keep repeated-row reads stable |
| Serializable | Behave like serial execution |

For interviews:

> **Stronger isolation generally buys correctness with more coordination/concurrency cost.**

---

# 24. LOCKING — FIRST PRINCIPLES

One way to control concurrent access:

LOCK RESOURCE
↓
use resource
↓
UNLOCK

If another transaction needs incompatible access:

WAIT

---

# 25. SHARED VS EXCLUSIVE LOCK

The source defines two lock types.

## Shared lock — S

Used for:

READ

Multiple compatible readers may hold shared locks.

---

## Exclusive lock — X

Used when exclusive modification access is required.

Think:

WRITE

Other incompatible accesses must wait.

---

# 26. TWO-PHASE LOCKING — 2PL

**🔵 GOOD DATABASE DEEP DIVE**

Do **not** confuse:

2PL

Two-Phase Locking

with:

2PC
Two-Phase Commit

Completely different problems.

---

# 27. TWO-PHASE LOCKING

The source describes two phases:

1. Growing Phase
2. Shrinking Phase

---

# 28. GROWING PHASE

Transaction:

may acquire locks

but should not start releasing them yet.

LOCK A
LOCK B
LOCK C

---

# 29. SHRINKING PHASE

Once transaction begins releasing locks:

UNLOCK A
UNLOCK B

it does not go back to acquiring new locks.

Core idea:

ACQUIRE
ACQUIRE
ACQUIRE

--------- boundary ---------

RELEASE
RELEASE
RELEASE

---

# 30. WHY 2PL?

The source states that two-phase locking can produce conflict-serializable schedules.

Interview interpretation:

> It constrains lock acquisition/release so conflicting transactions can be ordered safely.

---

# 31. 2PL CAN DEADLOCK

🟢 **IMPORTANT**

Example from the source:

T1 holds lock A
T2 holds lock B

T1 waits for B
T2 waits for A

Result:

T1 → waiting for T2
T2 → waiting for T1

Neither progresses.

The source explicitly states that basic 2PL does not guarantee freedom from deadlocks.

---

# 32. DEADLOCK MEMORY HOOK

T1 owns A
wants B

T2 owns B
wants A

→ nobody moves

---

# 33. CASCADING ROLLBACK

Suppose:

T1 writes X

T2 reads T1's uncommitted X

T3 reads something derived from T2

If:

T1 rolls back

then dependent transactions may need rollback too.

The source calls this:

> **cascading rollback**.

---

# 34. STRICT 2PL

The source says strict 2PL keeps exclusive locks until commit/abort, making schedules recoverable/cascadeless, while deadlocks remain possible.

Memory:

X LOCK
held until
COMMIT / ABORT

---

# 35. RIGOROUS 2PL

The source distinguishes rigorous 2PL as keeping:

shared locks
+
exclusive locks

until commit/abort.

For ordinary backend interviews:

Basic 2PL
↓
Strict 2PL
↓

Rigorous 2PL

is enough.

---

# 36. MVCC — MULTI-VERSION CONCURRENCY CONTROL

🟢🟢🟢 **HIGH ROI**

The source explains MVCC as maintaining older/snapshot versions so readers can observe an appropriate committed version while another transaction modifies data.

Mental model:

X version 1
X version 2
X version 3

Different transactions can read appropriate versions.

---

# 37. WHY MVCC?

Without MVCC, one strategy is:

writer locks row

reader waits

With MVCC:

writer works on new version

reader sees valid earlier version

Hence the source's key statement:

> **Writers don't necessarily block readers, and readers don't necessarily block writers.**

---

# 38. MVCC EXAMPLE

Initially:

X = 100

Transaction T1 starts:

UPDATE X = 200

Before T1 commits, another transaction can continue reading an appropriate committed version:

X = 100

rather than consuming T1's unfinished change.

---

# 39. MVCC COST

The source also notes that MVCC is not free.

It requires maintaining:

multiple row versions
undo/history information
cleanup
additional storage

Memory:

MVCC

+ concurrency

- version-management cost

---

# 40. PESSIMISTIC VS OPTIMISTIC CONCURRENCY

🔷 **INTERVIEW SYNTHESIS**

Two broad mindsets.

## Pessimistic

> Conflict is likely. Prevent it.

lock first
then modify

---

## Optimistic

> Conflict is uncommon. Let work proceed, detect conflicts before commit/update.

The source describes optimistic concurrency as expecting concurrent transactions not to change the same key and rolling one back when conflict occurs.

---

# 41. OPTIMISTIC LOCKING EXAMPLE

Typical application-level pattern:

row:
id = 42
balance = 100
version = 7

Read version:

7

Update:

```
UPDATE account
SET balance = 80,
    version = 8
WHERE id = 42
  AND version = 7;
```

If:

0 rows updated

someone changed the row first.

Then:

retry / reject

🔷 This version-column example is interview expansion; the source only provides the higher-level optimistic-conflict idea.

---

# 42. LOCAL CONSISTENCY VS DISTRIBUTED CONSISTENCY

Now shift levels.

Inside one database:

transaction isolation
locks
MVCC

Across copies/nodes:

replication
consistency models
quorums
partitions

The source explicitly warns that **ACID consistency and distributed/CAP consistency mean different things**.

---

# 43. DISTRIBUTED CONSISTENCY — THE PROBLEM

Suppose three replicas:

A: X = 10
B: X = 10
C: X = 10

Write goes to A:

A: X = 20

but replication hasn't completed:

A: 20
B: 10
C: 10

Now a user reads from B.

What should happen?

Possible answer:

return 10

Possible answer:

wait until replicas agree

Possible answer:

query multiple replicas

The chosen semantics define your consistency model.

---

# 44. LINEARIZABILITY

🟢 **MUST KNOW CONCEPTUALLY**

The source's linearizable model says reads and writes appear ordered and reads should reflect preceding completed updates.

Mental model:

> The distributed system behaves as if there were one up-to-date copy observed in real-time order.

Example:

WRITE X = 17
completes

then

READ X

must not return older X = 13

---

# 45. LINEARIZABILITY — MEMORY HOOK

WRITE FINISHED
↓
future reads
must respect it

---

# 46. ⚠️ SOURCE CLARIFICATION — LINEARIZABILITY DOES NOT REQUIRE ONE SINGLE-THREADED SERVER

The source uses a single-threaded single server as an example for obtaining ordered operations.

Do not turn that into:

> “Linearizability requires one server.”

Distributed systems can provide linearizable semantics using coordination/consensus mechanisms.

The single-server explanation is a **simple mental model**, not the only implementation.

---

# 47. EVENTUAL CONSISTENCY

🟢🟢🟢

The source describes eventual consistency as allowing stale values temporarily, with replicas eventually converging.

Example:

Replica A = 20
Replica B = 10
Replica C = 10

time passes

Replica A = 20
Replica B = 20
Replica C = 20

---

# 48. WHAT DOES “EVENTUAL” MEAN?

It does **not** mean:

random forever

It means conceptually:

> If updates stop and the system continues functioning, copies should eventually converge according to that system's guarantees.

---

# 49. EVENTUAL CONSISTENCY EXAMPLES

Potentially acceptable for:

social post like count

view count

some feeds

analytics

search indexing

depending on product requirements.

Less acceptable for:

critical financial invariants

where stale state can produce incorrect business decisions.

---

# 50. EVENTUAL CONSISTENCY INTERVIEW ANSWER

> “Eventual consistency allows replicas to temporarily disagree after a write, but they converge over time. It improves flexibility and availability in distributed systems, but clients must tolerate stale reads.”

---

# 51. CAUSAL CONSISTENCY

🟡 **SHOULD KNOW**

The source's intuition:

> If one operation causally depends on another, the dependent operation must observe the causal predecessor.

Example:

Alice:
"I got the job."

Bob:
"Congratulations!"

It would be strange to observe:

"Congratulations!"

before:

"I got the job."

because the second message depends on the first.

---

# 52. CAUSAL VS UNRELATED EVENTS

If:

A causes B

preserve:

A → B

But unrelated events:

X
Y

may appear in different orders.

That allows more concurrency than requiring a total global order.

---

# 53. CONSISTENCY STRENGTH — MENTAL MODEL

Very roughly:

more ordering / coordination
↓
stronger observable consistency
↓
potentially greater latency /
lower availability during failures

Do not reduce all distributed systems to one universal ranking, but this trade-off appears repeatedly.

---

# 54. QUORUM

🟢🟢🟢 **HIGH ROI**

The source defines quorum around requiring a minimum number of replicas to participate/agree in reads and writes.

Definitions:

N = replication factor

W = replicas required for successful write

R = replicas contacted/required for read

---

# 55. CLASSIC QUORUM MEMORY FORMULA

The source uses:

R + W > N

to create overlap between the read set and write set.

Example:

N = 3
W = 2
R = 2

R + W = 4 > 3

Any read quorum and write quorum must overlap somewhere.

---

# 56. QUORUM EXAMPLE

Three replicas:

A
B
C

Write requires:

W = 2

Suppose updated:

A = 20
B = 20
C = 10

Read:

R = 2

The read should overlap with at least one node that participated in the successful write.

---

# 57. QUORUM TRADE-OFF

Higher:

R
or
W

means more replicas must respond.

Potential benefit:

stronger freshness/durability characteristics

Potential cost:

more latency
lower availability when replicas are unavailable

The source explicitly frames higher quorum requirements as improving consistency while reducing availability.

---

# 58. ⚠️ QUORUM INTERVIEW CLARIFICATION

The source simplifies this to:

R + W > N
→ strongly consistent

Do not treat that formula alone as a universal proof of linearizability.

🔷 **Interview-safe wording:**

> "`R + W > N` gives read/write quorum overlap, but actual consistency also depends on how versions are resolved, concurrent writes, failure handling and the database implementation."

That distinction matters at senior level.

---

# 59. READ-YOUR-WRITES CONSISTENCY

🟢 **HIGH ROI**

Scenario:

User changes profile name

then immediately refreshes

But refresh goes to a lagging read replica:

old name

Terrible UX.

Requirement:

> **A user should observe their own completed write.**

---

# 60. FRAGMENTED PINNING

The source proposes **fragmented pinning** as one solution: after a user performs an important write, route selected related reads to the leader/master for a time window, while unrelated reads can still hit replicas.

Example:

User publishes post
↓
for 10 minutes
↓
GET that post → Leader

other unrelated reads → Replicas

---

# 61. WHY NOT SEND ALL READS TO THE LEADER?

Because that can defeat the point of read replicas.

Fragmented pinning tries to balance:

read-your-write correctness
+
replica read scaling

---

# 62. CAP THEOREM

🔷 **INTERVIEW EXPANSION**

The source mentions CAP and distinguishes CAP consistency from ACID consistency, but it does **not fully develop the CAP theorem**. So the following is additional interview material rather than a direct rewrite of the notes.

CAP:

C = Consistency
A = Availability
P = Partition tolerance

---

# 63. PARTITION — WHAT DOES THAT MEAN?

Suppose:

REGION A
  X
 network broken
  X
REGION B

Both regions are alive.

But they cannot communicate.

That's a network partition.

---

# 64. DURING A PARTITION

Suppose both replicas hold:

X = 10

Then:

Region A gets WRITE X = 20

Region B cannot learn about it.

Now request reaches B.

What should B do?

---

# 65. CHOOSE CONSISTENCY

B refuses to answer/write because it cannot confirm latest state.

+ consistency

- availability

---

# 66. CHOOSE AVAILABILITY

B continues serving requests.

+ availability

- replicas may diverge

Later conflict reconciliation may be required.

---

# 67. CAP MEMORY HOOK

During a **network partition**:

preserve CONSISTENCY
→ some requests may fail

preserve AVAILABILITY
→ some responses may be stale/divergent

Important:

> **P is not normally a feature you casually choose to ignore in a real distributed network.**

The interesting trade-off appears **when partition happens**.

---

# 68. ACID CONSISTENCY VS CAP CONSISTENCY

🟢 **COMMON INTERVIEW TRAP**

The source correctly flags them as different concepts.

## ACID Consistency

database invariants remain valid

## CAP Consistency

Broadly:

distributed clients observe one coherent/latest
view according to the system's consistency guarantee

Same word.

Different concept.

---

# 69. SPLIT BRAIN

🟢 **MUST KNOW**

The source describes split brain as both sides of a network partition continuing to serve reads/writes independently, allowing them to diverge.

Diagram:

```
    CLUSTER

      X
    network
    partition
  /         \
```

Node A Node B
accepts writes accepts writes

A: X = 20 B: X = 30

Now you have two conflicting truths.

---

# 70. HOW DO SYSTEMS AVOID SPLIT BRAIN?

🔷 **INTERVIEW EXPANSION**

Typical approaches involve:

leader election

quorum

majority voting

leases

fencing

consensus

The fundamental idea:

> Do not allow two disconnected sides to both believe they are independently authoritative for the same state.

---

# 71. TWO GENERALS PROBLEM

🟡 **CONCEPTUAL**

The source includes the messenger analogy where two generals need confidence that the other side knows the attack plan, but messages/acknowledgements themselves can be uncertain.

Core lesson:

> Over an unreliable channel, there is always uncertainty about whether the other side received the last message/acknowledgement.

---

# 72. WHY THIS MATTERS IN SOFTWARE

Remember from HTTP retry:

Client → Payment Service

payment succeeds

response lost

Client sees:

timeout

Question:

Did it happen?

Network failure often creates **uncertainty**, not a clean yes/no.

---

# 73. THE DUAL-WRITE PROBLEM

🟢🟢🟢 **VERY IMPORTANT**

The source defines the dual-write problem:

> updating two separate storage systems can leave inconsistent state if one succeeds and the other fails.

Example:

1. Update DB
2. Publish Kafka event

What if:

DB succeeds
Kafka publish fails

?

Now:

database says YES

event consumers know NOTHING

---

# 74. DUAL WRITE EXAMPLE

Order creation:

BEGIN

Insert Order

COMMIT

then:

publish OrderCreated

Failure between the two:

Order exists

but

Inventory/Notification/Analytics
never see event

Classic distributed-systems bug.

---

# 75. POSSIBLE SOLUTIONS TO DISTRIBUTED CONSISTENCY

Different problems call for different approaches:

Distributed transaction
→ 2PC

Long-running business workflow
→ Saga

DB + message publication
→ transactional outbox

Duplicate retries
→ idempotency

Replica freshness
→ quorum / leader reads / pinning

🔷 Transactional outbox is interview expansion; it is not developed in the extracted source section.

---

# 76. 2PC — TWO-PHASE COMMIT

🟢🟢🟢 **MUST KNOW**

Do not confuse:

2PL = locking / concurrency

2PC = distributed commit

The source describes 2PC as a coordinator-controlled distributed transaction with participating nodes running local transactions.

---

# 77. 2PC ACTORS

COORDINATOR

Participant A
Participant B

Participant C

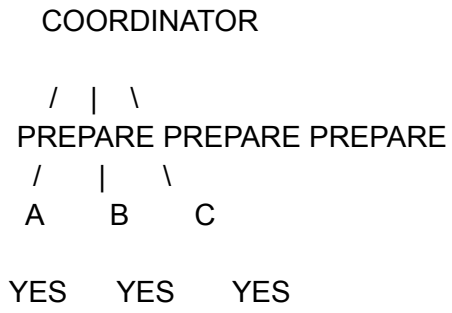
Coordinator decides whether the distributed transaction commits.

---

# 78. PHASE 1 — PREPARE

Coordinator asks:

"Can you commit?"




Participants prepare local state and vote.

The source calls this the **Prepare Phase**.

---

# 79. PHASE 2 — COMMIT

If:

ALL YES

coordinator sends:

COMMIT

If any participant votes:

NO

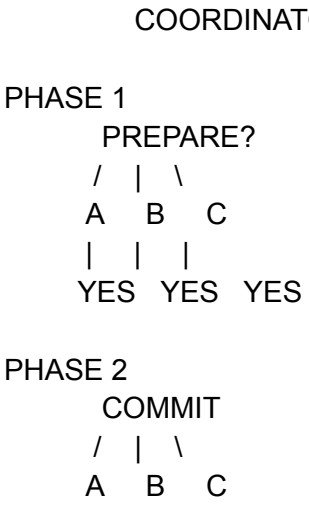
coordinator sends:

ROLLBACK / ABORT

The source describes this exact decision.

---

# 80. 2PC FLOW




---

# 81. 2PC BENEFIT

One logical operation can coordinate:

DB A
DB B
DB C

toward one commit decision.

Useful when:

atomic distributed transaction

is required and the participating systems support the necessary protocol.

---

# 82. 2PC COSTS FROM THE SOURCE

The source identifies major weaknesses:

Coordinator dependency

Participants wait

Slowest participant affects transaction

Chatty

Performance/scalability overhead

---

# 83. THE BLOCKING PROBLEM

Imagine participant says:

YES, I AM PREPARED

Now it may have reserved resources/locks.

Coordinator disappears.

Participant may not know:

COMMIT?
or
ABORT?

It can be stuck waiting.

This is a major conceptual weakness of traditional 2PC.

🔷 The explicit blocking example is interview expansion of the coordinator dependency described in the source.

---

# 84. ⚠️ SOURCE CORRECTION — “2PC IS NOT SUPPORTED IN NoSQL DATABASES”

The source makes this categorical statement.

Do not memorize it universally.

Interview-safe formulation:

> “2PC requires participating resources/protocol support. Whether a SQL or NoSQL system can participate depends on the specific database and transaction infrastructure.”

The important lesson is **capability**, not the SQL/NoSQL label.

---

# 85. SAGA — WHY DOES IT EXIST?

🟢🟢🟢 **VERY HIGH ROI**

Microservice workflow:

Order
↓

Payment
↓
Inventory
↓
Shipping

Holding one global database transaction across all of them can be impractical.

Saga instead uses:

sequence of LOCAL transactions

The source defines a Saga exactly this way.

---

# 86. SAGA EXAMPLE

T1 Create Order

T2 Reserve Inventory

T3 Charge Payment

T4 Create Shipment

Each service commits its own local transaction.

---

# 87. WHAT IF T3 FAILS?

We cannot simply do:

ROLLBACK DATABASE TRANSACTION

because T1 and T2 may already be independently committed.

Instead:

Compensate T2
Release Inventory

Compensate T1
Cancel Order

The source says failed Saga steps trigger compensating transactions that counteract earlier committed work.

---

# 88. COMPENSATING TRANSACTION

🟢

A compensation is a business operation that semantically reverses earlier work.

Example:

Reserve Seat

compensation:

Release Seat

Another:

Charge Card

compensation might be:

Refund Card

Important:

> Compensation is not always identical to database rollback.

---

# 89. SAGA FLOW

```
CREATE ORDER
  |
  v
RESERVE STOCK
  |
  v
CHARGE PAYMENT
  |
  X FAIL
  |
  v
RELEASE STOCK
  |
  v
CANCEL ORDER
```

---

# 90. TWO SAGA STYLES

The source explicitly gives:

1. Choreography
2. Orchestration

---

# 91. CHOREOGRAPHY

No central workflow controller.

Order Service
  |
  | OrderCreated
  v

Inventory Service
  |
  | InventoryReserved
  v

Payment Service
  |
  | PaymentCompleted
  v

Shipping Service

Services react to events.

---

# 92. CHOREOGRAPHY BENEFIT

The source emphasizes:

loose coupling
no central coordinator
natural event-based flow

particularly for relatively small workflows.

---

# 93. CHOREOGRAPHY PROBLEM

As workflow grows:

A listens B

B listens C

C listens D

D listens A

Understanding:

who triggered what?

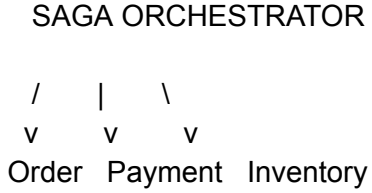
becomes difficult.

The source notes increasing complexity, event dependency/cycles and testing difficulty as more Saga steps are added.

---

# 94. ORCHESTRATION

Central coordinator controls workflow.




The source says the orchestrator:

tells participants what to execute

stores/interprets workflow state

handles failure recovery

---

# 95. CHOREOGRAPHY VS ORCHESTRATION

| Choreography | Orchestration |
|---|---|
| No central workflow controller | Central orchestrator |
| Event-driven | Command/workflow driven |
| Looser coupling | Explicit workflow |
| Simple for small flows | Easier to reason about large flows |
| Can become event spaghetti | Coordinator becomes important component |

---

# 96. 2PC VS SAGA

🟢🟢🟢 **COMMON SENIOR QUESTION**

## 2PC

One distributed atomic commit decision

Participants prepare and commit together.

Good when:

strong atomic distributed transaction
+
protocol support

Cost:

coordination
blocking
latency

---

## Saga

multiple local commits
+
compensations

Good when:

business workflow spans services

Cost:

temporary inconsistency
compensation complexity
more application logic

---

# 97. MEMORY HOOK

2PC
EVERYONE COMMIT TOGETHER

SAGA
COMMIT STEP BY STEP;
UNDO BUSINESS STEPS IF NEEDED

---

# 98. SAGA IS NOT MAGIC ROLLBACK

Suppose:

Email sent

What is rollback?

You cannot “unsend” an email.

Possible compensation:

send correction email

Suppose:

package already shipped

Compensation could require:

return/refund workflow

Therefore:

> **Business compensation must be explicitly designed.**

---

# 99. PIVOT / COMPENSABLE / RETRYABLE TRANSACTIONS

🔵 **DEEP DIVE**

The source defines three useful Saga concepts:

Compensable transaction

Pivot transaction

Retryable transaction

A compensable transaction can be reversed by another operation.

A pivot transaction is a go/no-go point.

Retryable transactions occur after the pivot and are designed to eventually succeed.

You do not need this vocabulary for every interview, but it is good deep-dive material.

---

# 100. CQRS

🟡 **SHOULD KNOW**

The source introduces:

> **Command Query Responsibility Segregation**

around separating read and write concerns and maintaining a read-oriented view from domain events.

Core mental model:

WRITE MODEL
↓
events
↓
READ MODEL

---

# 101. NORMAL MODEL

One database/model handles:

writes
+
reads

---

# 102. CQRS MODEL

COMMAND
↓
WRITE MODEL
↓
EVENT
↓
READ MODEL
↓
QUERY

You can optimize each side differently.

---

# 103. WHY CQRS?

Possible reasons:

Read and write workloads differ

Read model needs denormalization

Complex cross-service query views

Independent scaling

Different storage models

The source specifically discusses creating a read-only/view database and keeping it updated through domain events.

---

# 104. CQRS TRADE-OFF

Now you have:

WRITE STORE

and:

READ STORE

They may temporarily disagree.

So CQRS commonly introduces:

eventual consistency

between write and read models.

---

# 105. CQRS IS NOT REQUIRED FOR EVERY MICROSERVICE

🔷 **INTERVIEW EXPANSION**

Do not use:

CQRS
+
Kafka
+
separate DBs

for simple CRUD without a requirement.

It adds:

more infrastructure
event synchronization
failure handling
eventual consistency
debugging complexity

---

# 106. CONSISTENCY DECISION BY BUSINESS DATA

A senior engineer doesn't say:

> “Strong consistency is best.”

Ask what the data means.

Examples:

BANK TRANSFER
correctness dominates

SOCIAL LIKE COUNT
temporary stale value may be okay

USER PROFILE AFTER EDIT
read-your-write is important

SEARCH INDEX
eventual consistency often acceptable

---

# 107. THE CENTRAL TRADE-OFF

STRONGER COORDINATION
↓
more consistent observation
↓
more waiting / failure coupling

LESS COORDINATION
↓
greater independence / availability
↓

temporary disagreement

This single idea explains a large portion of distributed-systems design.

---

# 108. 8-YOE INTERVIEW FLOOR — TRANSACTIONS

You should answer all of these cleanly:

**1. What is a transaction?**

**2. Explain ACID.**

**3. What is a dirty read?**

**4. What is a dirty write?**

**5. Non-repeatable read vs phantom read?**

**6. Read Uncommitted vs Read Committed?**

**7. Repeatable Read?**

**8. Serializable?**

**9. Why not always use Serializable?**

**10. What is optimistic concurrency control?**

**11. Pessimistic vs optimistic locking?**

**12. What is MVCC?**

**13. Why does MVCC improve concurrency?**

---

# 109. 8-YOE INTERVIEW FLOOR — DISTRIBUTED SYSTEMS

**14. ACID consistency vs distributed consistency?**

**15. What is linearizability?**

**16. What is eventual consistency?**

**17. When is eventual consistency acceptable?**

**18. What is causal consistency?**

**19. What are N, R and W?**

**20. Why `R + W > N`?**

**21. What is read-your-write consistency?**

**22. What is split brain?**

**23. What happens during a network partition?**

**24. Explain CAP theorem.**

---

# 110. 8-YOE INTERVIEW FLOOR — DISTRIBUTED TRANSACTIONS

**25. What is the dual-write problem?**

**26. What is 2PC?**

**27. Prepare vs commit phase?**

**28. Why can 2PC hurt availability/latency?**

**29. Two-phase locking vs two-phase commit?**

**30. What is Saga?**

**31. What is a compensating transaction?**

**32. Choreography vs orchestration?**

**33. Saga vs 2PC?**

**34. What is CQRS?**

**35. Why does CQRS often introduce eventual consistency?**

If these 35 are automatic, you're in strong shape.

---

# 111. RAPID-FIRE MODEL ANSWERS

## Q: What is ACID?

“Atomicity means all-or-nothing execution, consistency preserves database invariants, isolation controls what concurrent transactions observe, and durability means committed changes survive according to the database's durability guarantees.”

---

## Q: Dirty read?

“Reading another transaction's changes before that transaction commits.”

---

## Q: Non-repeatable vs phantom read?

“A non-repeatable read means the same existing row changes between reads; a phantom means the set of rows matching a predicate changes.”

---

## Q: Why not Serializable everywhere?

“Stronger isolation generally requires more coordination, blocking, serialization or retries, which can reduce throughput and increase latency.”

---

# 112. RAPID FIRE — MVCC

## Q: What is MVCC?

“MVCC keeps multiple versions of data so transactions can read an appropriate snapshot while concurrent writers create newer versions. This can reduce reader-writer blocking at the cost of version storage and cleanup.”

---

# 113. RAPID FIRE — EVENTUAL CONSISTENCY

## Q: What does eventual consistency mean?

“Replicas may temporarily disagree after a write, but they converge over time according to the system's replication guarantees.”

---

# 114. RAPID FIRE — LINEARIZABILITY

> “Linearizability makes operations appear as if they occur atomically in a single real-time order, so once a write completes, a later read should not return an older value.”

---

# 115. RAPID FIRE — QUORUM

> “With replication factor N, W defines how many replicas participate in a successful write and R defines how many participate in a read. `R + W > N` creates read/write overlap, although actual consistency still depends on the database's conflict and failure semantics.”

---

# 116. RAPID FIRE — 2PL VS 2PC

> “Two-phase locking is a concurrency-control protocol governing when locks are acquired and released. Two-phase commit is a distributed-transaction protocol coordinating whether multiple participants commit or abort. The similar names are accidental; they solve different problems.”

Very important answer.

---

# 117. RAPID FIRE — 2PC

> “In 2PC, the coordinator first asks all participants to prepare. If everyone votes yes, it sends commit; if anyone rejects, it aborts. The benefit is atomic coordination across participants, but it adds latency, coordinator dependency and possible blocking.”

---

# 118. RAPID FIRE — SAGA

> “A Saga breaks a distributed business transaction into local transactions. Each step commits independently, and if a later step fails, compensating transactions undo the business effects of earlier steps.”

---

# 119. RAPID FIRE — SAGA VS 2PC

> “2PC tries to preserve one atomic distributed commit and requires tight coordination. Saga accepts a sequence of local commits and achieves business consistency through compensation. 2PC gives stronger atomicity but more blocking and coordination; Saga scales service workflows better but introduces temporary inconsistency and compensation complexity.”

---

# 120. COMMON BAD ANSWERS

### ❌ “Atomicity means thread safety.”

No.

Atomicity here means:

transaction happens completely
or not at all

---

### ❌ “Read Committed prevents every concurrency anomaly.”

No.

It mainly prevents dirty reads; weaker anomalies can still occur depending on the DB.

---

### ❌ “MVCC means no locking ever.”

No.

MVCC reduces certain reader/writer conflicts; databases may still use locks for other purposes.

---

### ❌ “Eventual consistency means the data may remain wrong forever.”

Not the intended consistency model.

---

### ❌ “CAP means choose any two of three all the time.”

Too simplistic.

The meaningful trade-off is during a network partition.

---

### ❌ “`R + W > N` automatically proves linearizability.”

Too strong.

---

### ❌ “2PL and 2PC are basically the same.”

Completely different.

---

### ❌ “Saga rolls back database transactions.”

Not exactly.

It runs **new compensating business operations**.

---

### ❌ “Saga gives the exact same semantics as one ACID transaction.”

No.

Intermediate committed states can exist.

---

# 121. ONE-PAGE MEMORY SHEET

TRANSACTIONS

ACID

A = Atomic
all or nothing

C = Consistency
valid state

I = Isolation
concurrent transactions

D = Durability
committed survives

ANOMALIES

Dirty Read
read uncommitted value

Dirty Write
overwrite uncommitted value

Non-repeatable Read
same row → changed value

Phantom
same query → changed row set

ISOLATION

Read Uncommitted
↓
Read Committed
↓
Repeatable Read
↓
Serializable

CONCURRENCY

LOCKING
prevent conflict

MVCC
multiple versions

OPTIMISTIC
detect conflict

2PL

Growing:
acquire locks

Shrinking:
release locks

Risk:
deadlock

DISTRIBUTED CONSISTENCY

Linearizable
latest completed write respected

Eventual
temporary stale data
eventual convergence

Causal
preserve causal order

Read-your-writes
see your own update

QUORUM

N = replicas
W = write replicas
R = read replicas

classic overlap:
R + W > N

CAP

Partition happens

choose behavior:
Consistency
vs
Availability

SPLIT BRAIN

two disconnected sides
both act authoritative

DISTRIBUTED TRANSACTION

Dual Write
DB succeeds
event fails

2PC

1 Prepare
2 Commit

Atomic coordination
but blocking/latency

SAGA

local transaction

↓
local transaction
↓
local transaction

failure
↓
compensate

SAGA STYLES

Choreography
events

Orchestration
central workflow

CQRS

Write Model
↓ events
Read Model

---

# 122. THE 60-SECOND SENIOR ANSWER

If asked:

> **“How do you reason about consistency and transactions in distributed systems?”**

A strong answer:

> “I separate local transaction isolation from distributed consistency. Within one database, ACID transactions, isolation levels, locking or MVCC control concurrent access. Once data is replicated, we must define what readers are allowed to observe—such as linearizable, eventual or read-your-write consistency—and decide how much coordination the workload can tolerate. Quorums can create overlap between reads and writes, while network partitions force us to decide whether to reject requests or tolerate temporary divergence. When one business operation spans several services, simple dual writes are unsafe because one operation can succeed while another fails. If true distributed atomic commit is supported and required, 2PC is one option, but it introduces coordination and blocking. In microservice workflows, Saga is often used instead: each service commits locally and failures are handled through compensating transactions.”

If you can expand every sentence, Part 5 is learned.

---

# 123. PART 5 MEMORY MAP

CONSISTENCY & TRANSACTIONS

- ACID
  - Atomicity
  - Consistency
  - Isolation
  - Durability
- Concurrency Anomalies
  - Dirty Read
  - Dirty Write
  - Non-repeatable Read
  - Phantom
- Isolation
  - Read Uncommitted
  - Read Committed
  - Repeatable Read
  - Serializable
- Concurrency Control
  - Locks
  - 2PL
  - MVCC
  - Optimistic
- Distributed Consistency
  - Linearizable
  - Eventual
  - Causal
  - Read-your-write
  - Quorum
- Network Failure
  - CAP
  - Partition
  - Split Brain
  - Two Generals
- Distributed Transactions
  - Dual Write
  - 2PC
  - Saga
    - Choreography
    - Orchestration
  - CQRS

---

# CJ SYSTEM DESIGN INTERVIEW HANDBOOK

## PART 6 — CACHING: LOCAL, DISTRIBUTED, WRITE POLICIES, EVICTION & THUNDERING HERD

**Target:** Senior Software Engineer / Backend Engineer — ~8 YOE
**Goal:** Be able to explain **why caching helps, where it belongs, how reads/writes flow, and what new failure modes caching introduces.**

> **Priority legend**
> 🟢 **MUST KNOW** — senior interview floor
> 🟡 **SHOULD KNOW** — explain confidently
> 🔵 **DEEP DIVE** — interviewer-push territory
> ⚠️ **SOURCE NOTE** — wording/claim appears in the notes but should be treated in the context given there rather than as a universal rule

Your notes explicitly cover distributed vs local cache, TTL, write policies, thrashing, Redis vs Memcached, thundering herd, and the Promise/request-collapsing solution. They also list cache-aside, write-through, write-behind and refresh-ahead in the HLD checklist.

---

# 1. WHY CACHE AT ALL?

**🟢 MUST KNOW**

The source's basic argument is:

RAM access
↓
faster than
↓
disk / SSD access

So:

REQUEST
↓

CACHE HIT
↓
avoid database query
↓
lower response time
+
lower database load

The source also notes the cost side: RAM is more expensive than disk/SSD for the same capacity.

---

# 2. CACHE — ELI5

Imagine a library.

Database:

all books in warehouse

Cache:

the 100 books people ask for most
already sitting beside librarian

If requested book is beside librarian:

fast

Otherwise:

go to warehouse
retrieve
bring back
possibly keep nearby

That is the core caching idea.

---

# 3. CACHE HIT VS CACHE MISS

🟢

## Cache hit

Client
↓
Cache
↓

FOUND
↓
return

No database call required.

---

## Cache miss

The notes describe the pattern:

check cache
↓
not found
↓
query database
↓
cache result
↓
return result

This is the basic cache-aside style flow described elsewhere in the notes' distributed-hashing section.

---

# 4. CACHE-ASIDE

🟢 **HIGH ROI**

The HLD section explicitly lists:

Cache aside

among cache strategies.

The source's read flow elsewhere is:

Application
↓
Check Cache
|
├── HIT
| ↓
| return
|
└── MISS
↓
Database
↓
Cache result

↓
return

The application is responsible for loading the missing value.

---

# 5. WHY CACHE-ASIDE IS EASY TO REMEMBER

CACHE FIRST

MISS?
↓
DATABASE

THEN
↓
CACHE IT

Memory hook:

> **Look aside at cache before going to source.**

---

# 6. CACHE IS NOT THE SOURCE OF TRUTH BY DEFAULT

A typical architecture in the notes is:

CACHE
↓ miss
DATABASE

with the database retaining the durable source data.

This matters because if cache disappears:

performance may degrade

but underlying data can still be retrieved.

---

# 7. WHERE CAN CACHING EXIST?

The source's HLD checklist lists several cache layers:

Client caching

CDN caching

Web server caching

Database caching

Application caching

Query-level caching

Object-level caching

So caching is not synonymous with:

Redis

It is a general architectural technique.

---

# 8. LOCAL CACHE

🟢 **MUST KNOW**

The source defines the key advantage clearly:

LOCAL CACHE
↓
in-process access
↓
no network call
↓
very fast

Example architecture:

APP NODE A
[local cache]

APP NODE B
[local cache]

APP NODE C
[local cache]

---

# 9. LOCAL CACHE PROBLEM — DUPLICATION

Suppose all three app instances cache:

User 42

Now you may have:

A → User42 copy

B → User42 copy

C → User42 copy

The source highlights:

cache duplication

and:

update fan-out

as consequences of per-instance caching.

---

# 10. UPDATE FAN-OUT

Suppose:

User42.name

changes.

If there are:

50 application instances

each with a cached copy, you may need to invalidate/update many copies.

Conceptually:

```
DATABASE UPDATE
     |
     +-----→ Cache A
     +-----→ Cache B
     +-----→ Cache C
     +-----→ ...
```

That is the update fan-out problem referenced in the notes.

---

# 11. LOCAL CACHE CONSISTENCY

The source points out another problem:

Node A cache = new value

Node B cache = old value

Now requests can produce different answers depending on which app node handles them.

---

# 12. DISTRIBUTED / GLOBAL CACHE

The alternative in the notes:

```
      APP A
        |
       APP B
        |
       APP C
        |
        v
  GLOBAL CACHE
        |
        v
    DATABASE
```

Instead of each service instance carrying its own full cache, they access a shared/distributed cache.

---

# 13. LOCAL VS GLOBAL CACHE

| Local cache | Global/distributed cache |
|---|---|
| In-process | Network access |
| Very low latency | Additional network hop |
| Duplicate data across app nodes | Shared cache layer |
| Update fan-out problem | Centralized/shared cache state |
| Instance-specific | Multi-client |

The source's main trade-off is exactly:

LOCAL
faster

GLOBAL
reduces duplication / fan-out

while consistency remains something that must be handled.

---

# 14. WHY NOT PUT EVERYTHING IN LOCAL MEMORY?

The source identifies another issue:

> cache memory competes with the application's own memory needs.

Large local caches can make the service itself:

bloated

because application state and cached state compete for memory.

---

# 15. CACHE SIZE IS BOUNDED

Your HLD notes classify:

RAM → bounded size

with examples such as Redis and Memcached.

This naturally raises:

> What happens when cache fills up?

Answer:

EVICTION

---

# 16. WRITE POLICY VS REPLACEMENT POLICY

🟢 **VERY GOOD INTERVIEW DISTINCTION**

The source explicitly separates them.

## Write policy

Triggered when:

WRITE happens

Question:

> How should cache/database be updated?

---

## Replacement policy

Triggered when:

cache has no room

Question:

> Which existing item should be removed?

This distinction is worth memorizing.

---

# 17. TTL — TIME TO LIVE

🟢 **MUST KNOW**

The source defines TTL as a timestamp/expiration associated with cache entries.

When the lifetime is exceeded:

entry expires

Conceptually:

Key inserted
at 10:00

TTL = 5 minutes

10:05+
↓
expired

---

# 18. WHY TTL?

The source doesn't separately enumerate motivations, but its cache section uses TTL as an expiration mechanism.

From the source's framing:

cache entry
↓
lifetime ends
↓
eviction

So for the handbook, memorize simply:

> **TTL gives cached data a bounded lifetime.**

---

# 19. CACHE WRITE POLICIES IN YOUR NOTES

The source explicitly describes three:

Write Around

Write Through

Write Back

The HLD checklist separately uses the label:

Write Behind

rather than `Write Back`. It does not explicitly reconcile those two names, so keep the source terminology rather than assuming they are identical.

---

# 20. WRITE-AROUND — SOURCE FLOW

According to the notes:

WRITE
↓
DATABASE

instead of updating cache.

The notes then warn that an existing cached copy can remain stale.

Conceptually:

Cache: X = 10

DB write:
X = 20

Cache still:

X = 10

Potential result:

stale read

---

# 21. WRITE-AROUND — MEMORY HOOK

WRITE GOES AROUND CACHE
↓
DATABASE

The problem in the source's described version:

old cache entry may remain

---

# 22. WRITE-THROUGH — SOURCE DESCRIPTION

The notes describe write-through as coordinating the database/cache write path so persisted data remains consistent, and on the next read the cache can obtain the updated database entry.

Source summary table characterizes it as:

Consistency → High
Persistence → High
Efficiency → Low

For memorization, keep the source idea:

WRITE
↓
persistent store/cache coordination
↓
fresh read

---

# 23. WRITE-BACK — SOURCE FLOW

The notes describe:

WRITE
↓
CACHE

with the database update happening later rather than immediately.

Main benefit according to source:

efficient

Main risk:

cache fails
before database persistence

which can leave the database with stale/missing updates.

---

# 24. WRITE-BACK FAILURE EXAMPLE

Database:
balance = 100

Cache:
balance = 80

The new value exists only in cache.

Before persistence:

CACHE CRASHES

Database still contains:

100

That illustrates the durability risk described in the source.

---

# 25. SOURCE WRITE-POLICY MEMORY TABLE

| Policy | Source's main idea | Source-highlighted risk/trade-off |
|---|---|---|
| Write Around | Update DB, not cache | Cached value may be stale |
| Write Through | Keep DB/cache update path coordinated | Lower efficiency |
| Write Back | Update cache first, DB later | Data may be lost/stale if cache fails before persistence |

These are directly derived from the source's cache-policy section.

---

# 26. REFRESH-AHEAD

The HLD checklist lists:

Refresh ahead

as another cache strategy.

However, the notes excerpt available here **does not explain how refresh-ahead works**.

So for a source-faithful handbook:

REFRESH AHEAD
→ listed in source
→ detailed behavior not supplied in source

Do not memorize an invented definition from this document alone.

---

# 27. EVICTION

When cache is full:

new object arrives
↓
no free capacity
↓
choose an existing entry
↓
evict it

Your notes call the mechanism choosing this object the:

**replacement policy**.

---

# 28. LRU

The notes reference:

LRU

and state that, in most cases in their framing, it works better than LFU.

Memory meaning:

LRU
=

Least Recently Used

The source doesn't provide a separate formal algorithm here, so this is the level supported by this section.

---

# 29. LFU

Likewise:

LFU
=
Least Frequently Used

The notes contrast it with LRU but do not develop the full LFU algorithm.

For this source-based pass:

LRU → recency-oriented
LFU → frequency-oriented

is enough to remember the naming distinction.

---

# 30. THRASHING

🟢 **MUST KNOW FROM SOURCE**

The notes define thrashing as a situation with:

high miss ratio
+
many loads
+
many evictions

because cache capacity is too small relative to the working dataset.

---

# 31. THRASHING FLOW

REQUEST A
↓
MISS
↓
load A
↓

evict B

REQUEST B
↓
MISS
↓
load B
↓
evict C

REQUEST C
↓
MISS
↓
load C
↓
evict A

Cache keeps doing work:

load
evict
load
evict

while hit ratio remains poor.

---

# 32. WHY THRASHING IS BAD

You built cache to:

reduce backend work

but thrashing creates:

many misses
↓
backend reads
↓
cache insertion
↓
eviction
↓
more misses

So the cache provides little benefit while still adding overhead.

---

# 33. CACHE HIT RATIO

The source frames thrashing using:

miss ratio

Therefore the natural metric to keep in mind is:

hits
vs
misses

At interview level:

> A cache that rarely hits may not be worth its memory and operational cost.

---

# 34. REDIS VS MEMCACHED — SOURCE FRAMING

The source explains their original design intent differently.

## Memcached

Described as:

large high-speed in-memory cache

originally pooling available RAM for caching use cases.

---

## Redis

Described as:

persistent in-memory data-structure store

supporting multiple data types.

---

# 35. MEMCACHED — SOURCE DETAILS

The notes describe Memcached using a segmented-LRU style scheme with:

hot
warm
cold

regions and eviction based on last-use information.

---

# 36. REDIS VS MEMCACHED — MEMORY VERSION

Based only on the source framing:

MEMCACHED
simple high-speed cache

REDIS
in-memory data-structure store
with richer capabilities/persistence framing

The notes also state that Memcached can be easier to set up/maintain when operating the cluster yourself.

---

# 37. DON'T TURN PRODUCT HISTORY INTO INTERVIEW PRIORITY

The source contains historical details such as:

Memcached → 2003

Redis → 2009

and examples of companies using them.

For interview memorization, these are **low ROI**.

Remember instead:

What problem does each solve?

What data model does it expose?

How will my cache fail?

---

# 38. DISTRIBUTING CACHE KEYS

Your earlier database chapter covered:

hash(key) % N

and consistent hashing.

The cache notes connect naturally to that.

If distributed cache has:

Cache A
Cache B
Cache C

you need to decide:

Which cache node owns key X?

The source's distributed hashing section uses caching as the main example.

---

# 39. WHY CONSISTENT HASHING MATTERS FOR CACHE

With:

server = hash(key) % N

changing N remaps many keys.

The source explains the consequence:

keys not found
↓
mass cache misses
↓
requests hit origin database
↓
database load spike

This is a particularly important cache failure mode described in the notes.

---

# 40. CACHE MISS CAN BECOME A SYSTEM FAILURE

One cache miss:

fine

100,000 simultaneous cache misses:

potential disaster

This leads to the source's:

> **Thundering Herd problem**

---

# 41. THUNDERING HERD

🟢🟢🟢 **VERY HIGH ROI**

The notes define the general thundering herd problem as many waiting processes/threads all waking and competing for a resource when an event occurs.

The caching example is more useful for system design.

---

# 42. LIVE-VIDEO CACHE MISS EXAMPLE

The notes describe millions of users tuning into a live broadcast.

Potentially:

hundreds of thousands

of requests see:

EDGE CACHE MISS

at the same time.

Without protection:

100K clients
↓
CACHE MISS
↓
100K origin requests

↓
origin/cache-processing overload

---

# 43. THUNDERING HERD DIAGRAM

R1 ─┐
R2 ─┤
R3 ─┤
R4 ─┤
... │
RN ─┘
↓
CACHE MISS
↓
DATABASE / ORIGIN

EVERYONE ATTACKS BACKEND

Exactly the opposite of why the cache was introduced.

---

# 44. SOURCE SOLUTION — REQUEST QUEUING / COLLAPSING

The source's live-video solution:

many requests miss
↓
queue them
↓
allow ONE request through
↓
fetch content
↓
fill edge cache
↓
serve waiting requests

---

# 45. VISUALIZE IT

Instead of:

1000 misses
↓

1000 DB calls

do:

1000 misses
↓
ONE backend load
↓
cache result
↓
1000 consumers reuse result

This is an extremely useful system-design pattern.

---

# 46. PROMISE-BASED SOLUTION IN THE NOTES

The source describes another approach used when a new cluster has an empty cache:

MISS
↓
insert Promise into cache
↓
ONE worker loads backend value

Other concurrent requests:

find Promise
↓
wait for same result

rather than independently going to the backend.

---

# 47. PROMISE FLOW

Request A
↓
MISS
↓
create Promise P
↓
backend load

Request B

↓
find Promise P
↓
WAIT

Request C
↓
find Promise P
↓
WAIT

Backend completes
↓
P fulfilled
↓
A, B, C receive result

---

# 48. WHY THIS WORKS

Instead of:

N concurrent misses
=
N backend calls

you get:

N concurrent misses
=
1 backend call
+
N waiters

That is the central insight in the source's Promise/thundering-herd example.

---

# 49. EMPTY CACHE / COLD CACHE

The source's Promise example is specifically motivated by:

new cluster
+
empty cache

This is useful operationally because a cache can become dangerous precisely when:

deployment
restart
new cluster
mass expiry

causes many misses together.

Only the new-cluster/empty-cache case is explicitly developed in this source; the other examples are not separately developed here.

---

# 50. TTL CAN INTERACT WITH LOAD

The source gives TTL as an expiration mechanism and separately gives mass cache misses as a thundering-herd scenario. It does **not explicitly connect simultaneous TTL expiry to thundering herd**, so don't treat that connection as source text.

For the source-grounded memory path, remember separately:

TTL
→ expiration

Thundering Herd
→ many requests competing after a miss/event

---

# 51. CACHE CONSISTENCY

Caching creates two copies:

DATABASE
+
CACHE

Therefore the obvious question is:

Can they disagree?

Your source repeatedly highlights stale cached values in its write-policy examples.

---

# 52. STALE CACHE EXAMPLE

Database
price = 120

Cache

price = 100

Read cache:

returns 100

even though durable source says:

120

That's the central cache-consistency problem.

---

# 53. CACHE CONSISTENCY VS PERFORMANCE

The source's write-policy table effectively shows the trade-off:

more immediate persistence/consistency
↕
efficiency

For example, its write-back policy is characterized as efficient but weaker in persistence/consistency, while write-through is characterized as stronger but less efficient.

---

# 54. ⚠️ SOURCE NOTE — `R + W > N`

The cache section begins by saying:

R + W > N

for consistency and:

R + W <= N

may return stale data.

The notes do **not explain the replicated-cache topology or assumptions behind applying quorum notation here**.

Therefore, for this cache chapter:

> Remember `R/W/N` as part of the distributed-consistency chapter, but do not treat it as a universal formula for every ordinary cache.

That preserves the source without pretending it supplies more context than it does.

---

# 55. CACHE EVICTION VS EXPIRATION

Keep them conceptually separate.

## Expiration

Source mechanism:

TTL elapsed

---

## Eviction

Source mechanism:

no room for new key
↓
replacement policy chooses key

This distinction is interview-friendly.

---

# 56. CACHE INTERVIEW FLOW

When interviewer says:

> **“Should we add Redis?”**

Do not simply answer yes.

Walk through:

What data are we caching?

How often is it reused?

What's the read/write ratio?

How stale can it be?

What's the TTL?

What happens on miss?

How is cache updated?

What happens if cache disappears?

Could many misses overload the DB?

The exact wording of this checklist is a restructuring of the issues covered by the source's cache section rather than a quoted checklist.

---

# 57. WHAT DATA MAKES SENSE TO CACHE?

The source doesn't give a universal selection rule.

What it does establish is:

cache hit
→ avoid DB query

Therefore the source supports reasoning about data where repeated reads make avoiding repeated database access valuable.

Do not invent a blanket:

> “Always cache X.”

Tie it to the access pattern.

---

# 58. CACHE CAPACITY PLANNING

The system-design template asks:

What data will be cached?

How much RAM?

How many machines?

when doing memory estimation. This is part of the original interview framework.

So a cache isn't merely:

put Redis box in diagram

You should be able to reason about:

working set
memory requirement
cache node count

---

# 59. WHAT HAPPENS IF CACHE FAILS?

Based on the source's typical cache-miss flow:

cache unavailable / missing data
↓
backend/database

This can restore functionality but increase database load.

And the source's consistent-hashing discussion shows that a sudden mass miss can severely overload the origin database.

---

# 60. CACHING CHANGES YOUR FAILURE MODE

Before cache:

database serves normal traffic

After cache:

database serves only misses

So database capacity might quietly depend on:

cache remaining healthy

If cache abruptly disappears:

traffic shifts back to origin

The notes' rehashing and thundering-herd sections both demonstrate this danger.

---

# 61. THRASHING VS THUNDERING HERD

🟢 **GOOD INTERVIEW DISTINCTION**

## Thrashing

Source definition:

cache too small / poor fit
→ repeated misses + loads + evictions

---

## Thundering Herd

Source definition:

many concurrent actors wake/miss
→ all compete for same backend/resource

Memory:

THRASHING
cache keeps replacing

THUNDERING HERD
everyone rushes backend together

---

# 62. CACHE-ASIDE VS WRITE POLICIES

Another important distinction.

## Cache-aside

Primarily explains:

READ MISS PATH
cache
↓ miss
DB
↓
populate cache

---

## Write-through / write-around / write-back

Explain:

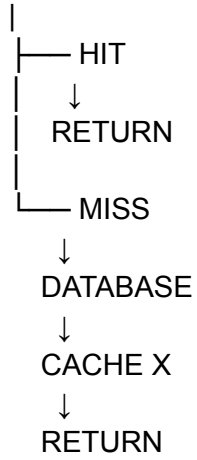
WRITE PATH

Your source contains both concepts in different sections.

---

# 63. SIMPLE READ PATH

GET X
↓
CACHE




That is the core read-path diagram to memorize.

---

# 64. SIMPLE WRITE QUESTIONS

Whenever data changes, ask:

Write DB first?

Write cache too?

Leave cache stale?

Update DB later?

Evict old cache?

When does fresh value appear?

Your source's three write policies are different answers to these questions.

---

# 65. REDIS / MEMCACHED ARE IMPLEMENTATIONS, NOT THE CONCEPT

Important structural point:

CACHING

is the architectural pattern.

Redis
Memcached

are technologies that can participate in that architecture.

The source itself first describes caching conceptually, then discusses Redis/Memcached as examples.

---

# 66. 8-YOE CACHING INTERVIEW FLOOR

You should answer these without hesitation:

**1. Why do we cache?**

**2. Cache hit vs cache miss?**

**3. What is cache-aside?**

**4. Local vs distributed cache?**

**5. Why is local cache faster?**

**6. What problem does local-cache duplication create?**

**7. What is update fan-out?**

**8. What is TTL?**

**9. Expiration vs eviction?**

**10. Write policy vs replacement policy?**

**11. What is write-around?**

**12. What is write-through?**

**13. What is write-back?**

**14. What can go wrong with write-back?**

**15. What is LRU?**

**16. What is LFU?**

**17. What is cache thrashing?**

**18. Redis vs Memcached according to these notes?**

**19. What happens if a distributed cache node is added/removed under modulo hashing?**

**20. Why is consistent hashing useful for distributed caches?**

**21. What is a thundering herd?**

**22. How does request collapsing help?**

**23. How does the Promise-based solution work?**

**24. What happens to your database when cache disappears?**

If those are automatic, this source's caching section is learned.

---

# 67. RAPID-FIRE MODEL ANSWERS

## Q: Why cache?

> “To serve frequently requested data from faster memory and avoid repeatedly querying the database, reducing response time and database load.”

Supported by the source's RAM-vs-disk and DB-load explanation.

---

## Q: Local vs distributed cache?

> “Local cache avoids a network call and is therefore faster, but when the service has many instances it duplicates data and makes updates fan out across instances. A global/distributed cache centralizes that cached state but requires network access.”

---

# 68. RAPID FIRE — CACHE-ASIDE

> “Check the cache first. On a miss, read the database, place the result in cache, then return it.”

This behavior is described in the source's cache/distributed-hashing material.

---

# 69. RAPID FIRE — TTL

> “TTL gives an entry an expiration time; once the configured lifetime is exceeded, the cache entry can be removed.”

---

# 70. RAPID FIRE — WRITE POLICY

“A write policy defines how cache and persistent storage are handled when data changes; it is different from a replacement policy, which decides what to evict when the cache is full.”

---

# 71. RAPID FIRE — WRITE-BACK

“In the source's write-back policy, the cache is updated first and the database is updated later. This is efficient, but if the cache fails before persistence, the database may remain stale.”

---

# 72. RAPID FIRE — THRASHING

“Thrashing is when the cache has a high miss ratio and continually loads and evicts entries, typically because the cache is too small for the working set described by the notes.”

---

# 73. RAPID FIRE — THUNDERING HERD

“A thundering herd occurs when many concurrent requests compete for the same resource at once. In the cache example, many simultaneous misses can overwhelm the origin.”

---

# 74. RAPID FIRE — HOW TO REDUCE THE HERD?

“The source shows request collapsing: let one request fetch the missing value while the rest wait, then share the fetched result through the cache.”

---

# 75. RAPID FIRE — PROMISE SOLUTION

“On the first cache miss, insert a Promise representing the in-flight backend request. Concurrent callers find that same Promise instead of issuing more backend calls, so they all wait on one fetch.”

---

# 76. COMMON BAD ANSWERS

### ❌ “Cache always makes the system faster.”

The source itself shows:

thrashing
mass misses
thundering herd
stale data

so caching can introduce new failure modes.

---

### ❌ “Local cache is always better because it's faster.”

The source highlights:

duplication
update fan-out
consistency issues
memory pressure

---

### ❌ “Cache and DB can never disagree.”

The source's write-around/write-back examples explicitly show stale/inconsistent states.

---

### ❌ “Write policy and eviction policy are the same.”

No.

The notes explicitly separate them.

---

### ❌ “Redis = caching.”

Too narrow.

The HLD notes include client, CDN, webserver, database, application, query and object caching.

---

# 77. ONE-PAGE MEMORY SHEET

CACHING

WHY?

RAM fast
↓
cache hit
↓
avoid DB
↓
lower latency
lower DB load

READ PATH

GET
↓
CACHE
├─ HIT → return
└─ MISS
↓
DB
↓
cache
↓
return

LOCAL CACHE

+ in-process
+ fastest

- duplicate data
- update fan-out
- consistency issues
- application memory usage

GLOBAL / DISTRIBUTED CACHE

shared cache layer
↓
network call required

TTL

entry lifetime
↓
expiration

WRITE POLICY

what happens on write?

REPLACEMENT POLICY

what gets evicted
when cache is full?

WRITE AROUND

write DB
cache may remain stale

WRITE THROUGH

coordinate persistent/cache state
higher consistency in source framing

WRITE BACK

write cache first
DB later

RISK:
cache failure before persistence

EVICTION

LRU
Least Recently Used

LFU
Least Frequently Used

THRASHING

miss
load
evict
miss
load
evict

DISTRIBUTED CACHE

hash(key) % N

change N
↓
mass remapping
↓
mass misses

consistent hashing
↓
reduce remapping

THUNDERING HERD

many misses
↓
everyone hits backend
↓
overload

REQUEST COLLAPSING

many misses
↓
one backend fetch
↓
others wait
↓
shared result

PROMISE PATTERN

cache in-flight Promise
↓
concurrent callers wait on same work

---

# 78. 60-SECOND SENIOR ANSWER

If interviewer asks:

> **“How do you think about caching?”**

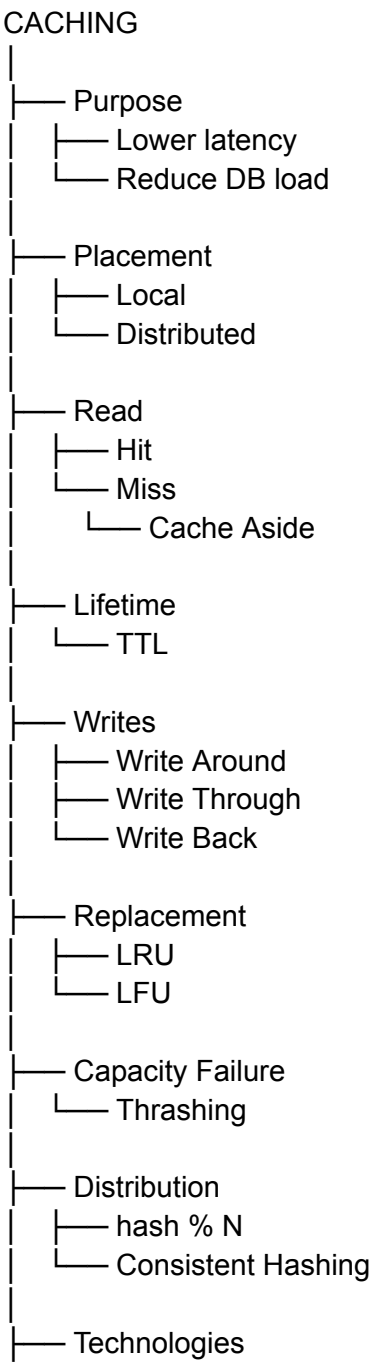
A source-grounded answer is:

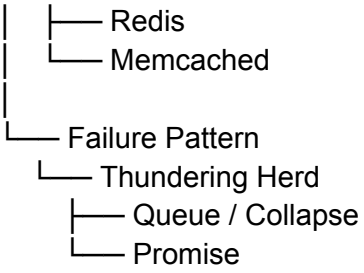
> “Caching keeps frequently needed data in faster memory so a hit can avoid database access and reduce latency and backend load. I first decide where the cache should live: a local in-process cache is faster but duplicates state across service instances and creates update fan-out, while a distributed cache adds a network hop but provides a shared cache layer. I then define the read-miss path, TTL and write policy—such as write-around, write-through or write-back in these notes—and separately define the replacement policy. I also need to plan failure behavior: a poor working-set fit can cause thrashing, and simultaneous cache misses can cause a thundering herd that overwhelms the origin. The source shows request collapsing or caching an in-flight Promise so multiple callers share one backend fetch.”

That captures almost the entire cache section of the notes.

---

# 79. PART 6 MEMORY MAP




Redis
Memcached
Failure Pattern
Thundering Herd
Queue / Collapse
Promise

# CJ SYSTEM DESIGN INTERVIEW HANDBOOK

## PART 7 — ASYNCHRONOUS PROCESSING, EVENT-DRIVEN ARCHITECTURE, QUEUES, PUB/SUB & EVENT STREAMING

**Target:** Senior Software Engineer / Backend Engineer — ~8 YOE
**Goal:** Understand **when work should leave the synchronous request path, how producers and consumers are decoupled, and how queueing, pub/sub and event streams differ.**

> **Priority legend**
> 🟢 **MUST KNOW** — senior interview floor
> 🟡 **SHOULD KNOW** — explain confidently
> 🔵 **DEEP DIVE** — source-specific detail

This part stays within the material in your notes. One important boundary: the notes have a heading **"Kafka VS SQS VS SNS VS Kinesis"**, but the retrieved section actually explains Kafka, SQS and SNS; it does **not** provide substantive Kinesis material. I therefore won't invent a Kinesis section from outside knowledge here.

---

# 1. SYNCHRONOUS VS ASYNCHRONOUS

**🟢 MUST KNOW**

The notes define synchronous processing as waiting for the operation to complete, while asynchronous processing lets the requester continue doing other work.

Simplest mental model:

```
SYNCHRONOUS

Client
  |
  | request
  v
Server
  |
  | processing...
  |
  | response
  v
Client

CLIENT WAITS
```

versus:

ASYNCHRONOUS

Client
  |
  | submit work
  v
System
  |
  | acknowledge
  v
Client continues

meanwhile...

Worker processes work

---

# 2. THE PIZZA-SHOP ANALOGY FROM THE NOTES

The source gives a pizza shop example:

Customer orders pizza
↓
order accepted
↓
customer does not wait at kitchen counter
↓
order goes into queue
↓
kitchen takes orders
↓
pizza is prepared
↓
customer is notified

The important idea is:

> **The customer does not need the final work to finish before the order can be acknowledged.**

---

# 3. WHEN DOES ASYNCHRONOUS PROCESSING MAKE SENSE?

The notes explicitly mention background work such as:

Send SMS

Validate email

Approve posts

Process orders

Pass information between services

as examples where asynchronous processing can be useful.

Memory question:

> **Does the caller need the final work to finish before continuing?**

If not, async processing becomes a candidate.

---

# 4. SYNCHRONOUS REQUEST-RESPONSE COUPLING

Imagine:

```
Service A
  |
  | request
  v
Service B
  |
  | response
  v
Service A
```

A needs B **now**.

The services participate in the same immediate request path.

---

# 5. ASYNCHRONOUS DECOUPLING

```
Producer
  |
  | message
  v
```

Queue
  |
  |
  v
Consumer

The producer does not require the consumer to finish immediately.

The source describes this as allowing producer and consumer to operate without needing to interact at the same time.

---

# 6. TWO SIDES OF ASYNCHRONOUS PROCESSING

The notes reduce async processing to two broad actors:

1. Service(s) creating processing tasks

2. Service(s) consuming and processing those tasks

Interview terms:

PRODUCER
creates work

CONSUMER
processes work

---

# 7. WHY NOT JUST USE A DATABASE TABLE AS A QUEUE?

🟢 **HIGH ROI**

The source spends substantial space on this.

Naive design:

PRODUCER
  |
  | INSERT job
  v
DATABASE TABLE
  ^
  |
  | SELECT pending jobs

CONSUMER

At small scale, this can look attractive.

But the notes identify several problems.

---

# 8. DATABASE-AS-QUEUE PROBLEM #1 — POLLING

Consumers repeatedly ask:

Any job?

Any job?

Any job?

Any job?

The notes say frequent polling adds significant database load.

Conceptually:

```
100 consumers
     ↓
poll
poll
poll
poll
     ↓
DATABASE
```

Much of the work can be checking whether work exists rather than actually processing it.

---

# 9. DATABASE-AS-QUEUE PROBLEM #2 — COMPETING OPERATIONS

A job table may simultaneously receive:

INSERT
UPDATE
SELECT
DELETE

while many producers and consumers compete over it.

The notes associate this with:

race conditions

locking

deadlocks

performance degradation

as volume grows.

---

# 10. DATABASE-AS-QUEUE PROBLEM #3 — MANUAL CLEANUP

Completed jobs accumulate.

JOB TABLE

pending
completed
completed
completed
completed
...

The source says old completed tasks need periodic cleanup, otherwise the table grows significantly.

---

# 11. DATABASE-AS-QUEUE PROBLEM #4 — FAILURE HANDLING

The source groups queue-like responsibilities such as:

polling

cleanup

concurrency

failed processing

as additional application complexity when implementing background processing with a normal database table.

Its conclusion is:

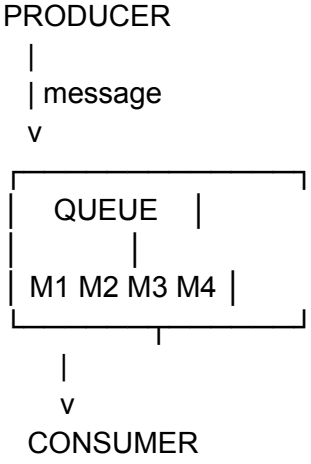
> A traditional RDBMS is not the best fit for this kind of queue-based background processing at scale.

---

# 12. MESSAGE QUEUE — FIRST PRINCIPLES

🟢🟢🟢

The notes define message queuing as applications communicating by sending messages, with the queue temporarily storing them when the destination is busy or unavailable.




---

# 13. WHY THE QUEUE EXISTS

Suppose:

incoming rate = HIGH
processing rate = LOWER

Without queue:

Producer
|
v
Consumer overwhelmed

With queue:

Producer
|
v

QUEUE
M1
M2
M3
M4
M5
  |
  v
Consumer processes progressively

The source describes the queue retaining incoming work so the receiving web service can remain available rather than blocking on processing.

---

# 14. QUEUE AS TEMPORARY STORAGE

The notes explicitly say:

> The queue provides temporary message storage when the destination program is busy or not connected.

This enables:

Producer available
Consumer temporarily busy/down
        ↓
    queue retains work

instead of requiring both to be ready at exactly the same moment.

---

# 15. TEMPORAL DECOUPLING

🟢 **EXCELLENT INTERVIEW TERM**

The source describes the idea even though it does not use this exact phrase:

Producer operates at Time A

Consumer can process later at Time B

The two do not need simultaneous interaction.

Memory hook:

> **Queue separates “work accepted” from “work completed.”**

---

# 16. QUEUE AND TRAFFIC BURSTS

The notes give an example where many requests arrive in a short period.

Instead of forcing processing service to handle them immediately:

```
BURST
||||||||||||||||||
      ↓
    QUEUE
      ↓
   CONSUMER
```

the queue retains the requests while processing catches up.

---

# 17. SCALE CONSUMERS

The source makes the scaling step explicit:

> If workload grows, add more consumers to work through the queue faster.

```
    QUEUE
   /  |  \
  v   v   v
 C1   C2  C3
```

This is one of the easiest horizontal-scaling patterns to explain in a system-design interview.

---

# 18. QUEUE — 30 SECOND INTERVIEW ANSWER

> “A message queue decouples the producer that accepts work from the consumer that performs it. Messages can be stored temporarily when the consumer is busy or unavailable, so the producer doesn't need to wait for processing to finish. If backlog grows, the notes suggest scaling by adding more consumers.”

---

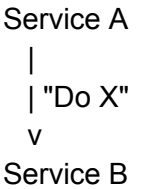
# 19. REQUEST-RESPONSE VS EVENT-DRIVEN

🟢🟢🟢

The notes' central distinction is:

### Request-response

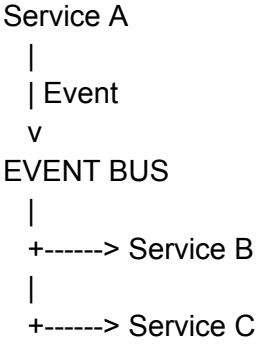
Services communicate directly.




### Event-driven

Services communicate by publishing that:

> **Something happened.**




The source describes event-driven services as not talking directly to each other; instead they publish events representing changes that interested consumers can consume.

---

# 20. COMMAND MENTALITY VS EVENT MENTALITY

Request:

"Inventory Service,
reserve item 123."

Event-style statement:

"OrderCreated"

The source's framing is:

> An event records that something has happened.

---

# 21. EVENT CHARACTERISTICS IN THE NOTES

The source describes events as:

A record that something happened

Immutable

Potentially persistable

Consumable multiple times

It also distinguishes:

EVENT
=
the action/change

EVENT NOTIFICATION
=
message saying that action happened

---

# 22. WHY IMMUTABILITY MATTERS IN THE SOURCE'S MODEL

The notes say events can be shared among multiple services without one service modifying/removing information that other services need.

Conceptually:

```
OrderCreated
    |
    +----> Inventory
    |
    +----> Analytics
    |
    +----> Notifications
```

Each consumer observes the recorded event.

---

# 23. ADVANTAGE — DECOUPLING

The event producer does not need direct knowledge of every consumer.

BEFORE

Service A
 ├── calls B
 ├── calls C
 └── calls D

EVENT MODEL

Service A
  |
 publish
  v
Event Bus
 ├── B
 ├── C
 └── D

This loose coupling appears throughout the source's EDA/pub-sub discussion.

---

# 24. ADVANTAGE — PARALLEL PROCESSING

The notes give this explicit EDA use case:

> One event can trigger multiple processes that execute asynchronously from each other.

Example:

OrderCreated
    |
    +----> Inventory
    |
    +----> Email
    |
    +----> Analytics

These downstream actions need not execute one by one.

---

# 25. ADVANTAGE — DATA REPLICATION

The source gives another use case:

ONE EVENT
  |
  +----> Service A database
  |

+----> Service B database
|
+----> Service C database

where multiple services need copies of event-derived data.

---

# 26. ADVANTAGE — REAL-TIME MONITORING

The notes say systems can emit events when state changes so those events can be examined for:

anomalies

suspicious activities

---

# 27. ADVANTAGE — INTEROPERABILITY

The notes describe events as a way to propagate information independently of the programming language used by each service.

Java Service
|
event
|
+----> Python
|
+----> Go

---

# 28. ADVANTAGE — REDUNDANCY

The source says that if a service is unavailable, events can be retained in the routing layer until the service becomes available to consume them.

This supports a key EDA objective:

Producer availability
doesn't have to equal
Consumer availability

---

# 29. ADVANTAGE — MICROSERVICE DECOUPLING

The source explicitly pairs EDA with microservices for sharing information between decoupled systems at scale.

---

# 30. SOURCE'S EDA CAPABILITIES LIST

The notes summarize EDA/microservice benefits with concepts such as:

Loose coupling

Independent component scaling

Independent deployment

Asynchronous processing

Workload throttling

Elastic scalability

Fault tolerance / resilience

Processing pipelines

Polyglot programming

Cloud-native affinity

For memorization, compress to:

DECOUPLE
SCALE
ASYNC
RESILIENCE
PIPELINES

---

# 31. EVENT-DRIVEN ARCHITECTURE IS NOT FREE

🟢 **MUST KNOW**

The source spends substantial space on disadvantages.

The most important one:

> Each service can possess its own copy/state, so some data may become stale.

---

# 32. EDA CONSISTENCY PROBLEM

Example:

Service A:
User.name = Alice

event delayed

Service B:
User.name = Alicia

Temporarily:

A and B disagree

This is the kind of stale-data issue described by the notes.

---

# 33. EDA FLOW CAN BECOME HARD TO REASON ABOUT

The notes explicitly say:

> Looking at a publisher alone may not tell you where its events ultimately go.

In synchronous architecture:

A → B → C

is visually explicit.

In event architecture:

A → Event Bus → ???

you need to understand subscriber relationships.

---

# 34. EVENT AUTHORIZATION COMPLEXITY

The source notes that if:

some services may consume Event X

others may not

the event bus becomes more complex.

So event-driven architecture also creates:

security
+
access-control

questions.

---

# 35. EVENT REPLAY AND EXTERNAL SIDE EFFECTS

One of the more interesting cautions in the notes concerns replaying an event through a service that depends on an external, time-dependent system.

Example:

event
↓
Email Service
↓
External provider

Replaying the old event later might not recreate the exact original external response.

The notes use this to explain why replay isn't automatically equivalent to perfectly recreating history.

---

# 36. SOURCE'S EDA TRADE-OFF SUMMARY

The notes summarize EDA as:

higher availability
↕
lower/imperfect immediate consistency

in the architecture they describe, because services maintain local event-derived state.

Keep that as the source's framing.

---

# 37. MESSAGE QUEUE VS EVENT-DRIVEN ARCHITECTURE

Do not confuse the terms.

### Message Queue

A communication/storage mechanism:

Producer
↓
Queue
↓
Consumer

### Event-Driven Architecture

A wider architectural style in which components communicate through events rather than direct synchronous calls.

A queue/broker can be part of EDA.

---

# 38. THE DUAL-WRITE PROBLEM APPEARS AGAIN

🟢🟢🟢

Your notes place the dual-write problem immediately after the queue example:

> If two separate storage updates are needed and one succeeds while the other fails, the distributed system can become inconsistent.

Example:

WRITE DATABASE
↓
SUCCESS

PUBLISH MESSAGE
↓
FAIL

Now:

Database says operation happened

Consumers never receive the message

---

# 39. DIRECT SERVICE FAN-OUT PROBLEM

The source describes an architecture where:

```
Server 1
 ├── Server 2
 └── Server 3
     ├── Server 4
     └── Server 5
```

and notes:

> Server 1 must know about Servers 2 and 3, and Server 3 must know about Servers 4 and 5.

That creates strong coupling.

---

# 40. FAILURE PROPAGATION IN DIRECT CALL CHAINS

The notes describe a case where some servers process successfully but a later server fails, leaving stale/inconsistent states on retries.

Conceptually:

S1 success

S2 success

S4 FAILS

Now replay/retry becomes difficult because not every component is in the same state.

---

# 41. PUBLISHER-SUBSCRIBER MODEL

🟢🟢🟢

The notes introduce Pub/Sub specifically to reduce direct coupling.

```
PUBLISHER
   |
   | publish
   v
```

TOPIC / BROKER
  /  |  \
 v   v   v
S1  S2  S3

Publisher does not need the concrete details of every downstream server.

---

# 42. PUB/SUB ADVANTAGE #1 — DECOUPLING

Source:

> Server 1 doesn't need to know exact details of Servers 2 and 3; it only publishes to the messaging layer.

Memory:

PUBLISHER KNOWS TOPIC
NOT EVERY SUBSCRIBER

---

# 43. PUB/SUB ADVANTAGE #2 — EASIER EXTENSION

The notes say a new service can be added by subscribing to the message queue/topic rather than requiring changes to every publisher.

BEFORE

Publisher → A
Publisher → B

ADD C

Publisher must know C

versus:

Publisher
  ↓
 Topic
/ | \
A  B  C

---

# 44. PUB/SUB — SOURCE'S PERSISTENCE CLAIM

The source states that if a message is published to the message queue, the messaging layer persists it until it reaches its destination.

Because that behavior is broker/product dependent, memorize this as:

> **This is the guarantee assumed by the notes' Pub/Sub model.**

Do not assume every messaging product has identical persistence semantics unless the particular product says so.

---

# 45. PUB/SUB DISADVANTAGES IN THE NOTES

The source explicitly lists:

Possible data inconsistency

No inherent idempotency guarantee

Late subscribers cannot replay older events
in the subscription model being described

This last point leads directly to:

> **Event Streaming**

---

# 46. QUEUE VS PUB/SUB — SOURCE-BASED MENTAL MODEL

The notes frame them approximately as:

QUEUE

Producer
↓
Queue
↓
processing consumer(s)

versus:

PUB/SUB

Publisher
  ↓
Topic
 / | \
Sub Sub Sub

Key Pub/Sub motivation in the source:

> One publisher can communicate with multiple interested services without directly knowing them.

---

# 47. EVENT STREAMING

🟢🟢🟢

The source gives a clean definition:

> Events are written to a **log**, and consumers can read from any part of that stream, including past events.

EVENT LOG

E1
E2
E3
E4
E5
E6

Consumer A might be at:

E6

Consumer B might start from:

E2

---

# 48. PUB/SUB VS EVENT STREAMING

From the notes:

### Subscription-style model

Late subscriber:

joins now
↓
receives new events

Older events are not replayable in the model described.

### Event streaming

events stored in log
↓
consumer can choose position
↓
past events can be replayed

---

# 49. THE MOST IMPORTANT EVENT-STREAM IDEA

QUEUE-LIKE VIEW
message is work to process

EVENT-LOG VIEW
event is history to retain/read

The second formulation is a compact reorganization of how the source contrasts subscription with an event log; it isn't a separate quote from the notes.

---

# 50. REPLAY

Because event streaming retains the log:

E1
E2
E3
E4

a consumer can:

start again from E1

and rebuild/process earlier history.

The source explicitly identifies replay as the distinguishing event-streaming capability.

---

# 51. WHY WOULD REPLAY MATTER?

Within the source's own EDA framing, event history is used for ideas such as:

rebuilding service state

moving to an earlier point in history

replacing a service

reprocessing events

The notes discuss event logs and historical movement/replay as EDA advantages.

---

# 52. KAFKA — SOURCE MENTAL MODEL

**🟢 MUST KNOW FROM THESE NOTES**

The notes describe Kafka as:

```
Producer
  |
  | event
  v
Topic
  |
  + partitions
  |
  + consumer groups
```

with multiple consumers/groups capable of reading topic data.

---

# 53. KAFKA TOPIC

According to the notes:

```
PRODUCER
    |
    v
  TOPIC
```

An event/message is sent to a topic.

Consumers subscribe/read from that topic.

---

# 54. KAFKA PARTITIONS

The notes explicitly say a topic contains:

PARTITIONS

for parallel streaming.

Mental image:

TOPIC

Partition 0: E1 E4 E7

Partition 1: E2 E5 E8

Partition 2: E3 E6 E9

The exact placement example is just a visualization; the source's supported point is that partitions enable parallel streaming.

---

# 55. WHY PARTITIONS MATTER IN THE SOURCE

Without partitions:

ONE stream

With partitions:

multiple parallel streams

So the source associates partitions with parallel consumption/streaming.

---

# 56. KAFKA CONSUMER GROUP — SOURCE IDEA

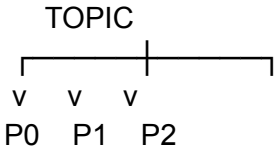
The notes introduce:

consumer group

and explain that when a message is read from a topic partition, its consumption position is recorded/committed for that group, while another consumer group can still read the same message.

---

# 57. CONSUMER-GROUP MENTAL MODEL




GROUP A
A1 / A2 / A3

GROUP B
B1 / B2

The important source-backed concept is:

> Different consumer groups can consume the same topic independently.

---

# 58. WHY THAT MATTERS

Example:

```
OrderCreated
    |
    v
Kafka Topic
    |
    +---- Consumer Group: Inventory
    |
    +---- Consumer Group: Analytics
```

Both groups can process the same event for different purposes.

This example is only illustrating the source's statement that separate groups can read the same topic records.

---

# 59. SQS — WHAT THE NOTES SAY

The source contrasts Amazon SQS with Kafka by describing SQS as a queue-based, pull-oriented service and names two queue types:

Standard

FIFO

Because these are product-specific statements, keep this section explicitly tied to the notes rather than treating it as independently verified current AWS documentation.

---

# 60. SQS SOURCE MENTAL MODEL

PRODUCER
  |
  v
SQS QUEUE
  |
 pull
  v
CONSUMER

That's the basic architecture presented by the source.

---

# 61. SNS — WHAT THE NOTES SAY

The notes describe Amazon SNS as a:

publish / subscribe

service aimed at push-style messaging, giving examples such as:

email

SMS

Again, these are source notes rather than a current AWS verification pass.

---

# 62. SNS MENTAL MODEL

PUBLISHER
   |
   v
 SNS TOPIC
  /  |  \
 v   v   v
Sub1 Sub2 Sub3

The important conceptual contrast in the notes is:

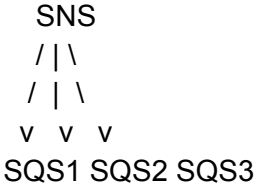
SQS
→ queue

SNS
→ publish/subscribe

---

# 63. SNS + SQS FAN-OUT

The source describes a pattern:




A message published once to the SNS topic is distributed to several SQS queues.

The notes call this:

**Fanout pattern**

---

# 64. WHY COMBINE THEM IN THE SOURCE'S MODEL?

The source's reasoning is:

SNS
→ one-to-many distribution

SQS
→ queued/persisted processing

Therefore:

SNS
↓ fan-out
multiple SQS queues
↓
independent consumers

---

# 65. KINESIS — SOURCE GAP

The heading says:

Kafka VS SQS VS SNS VS Kinesis

but the retrieved content proceeds through:

Kafka

SQS

SNS

SNS + SQS

and then immediately enters the next topic, two-phase locking.

Therefore:

KINESIS
→ named in heading
→ not explained in this source section

For a source-faithful handbook, stop there rather than filling it from memory.

---

# 66. PRODUCT-COMPARISON CAUTION

The source contains several concrete AWS-product statements about:

subscriber behavior

retention

message loss

SQS + SNS replacing Kafka

Those claims are **part of the notes**, but we have not done the external/current verification step in this handbook pass.

So for interview memorization:

### Safest source-supported conceptual distinctions

Kafka
→ topic + partitions + consumer groups

SQS
→ queue

SNS
→ publish / subscribe

SNS + SQS
→ fan-out into independent queues

If we later build the final interview handbook, current cloud-product details should be verified separately.

---

# 67. MESSAGE QUEUE VS PUB/SUB VS EVENT STREAM

This is the highest-ROI conceptual comparison from this chapter.

| Model | Source's core idea |
|---|---|
| Message Queue | Store work until consumer processes it |
| Pub/Sub | Publisher broadcasts to interested subscribers without direct coupling |
| Event Streaming | Persist events in a log so consumers can read/replay from positions |

Supported across the source's queue, Pub/Sub and event-stream sections.

---

# 68. MEMORY HOOK

QUEUE
"DO THIS WORK"

PUB/SUB
"SOMETHING HAPPENED — TELL EVERYONE INTERESTED"

EVENT STREAM
"KEEP THE HISTORY SO CONSUMERS CAN READ/REPLAY IT"

---

# 69. QUEUE OR DIRECT SYNCHRONOUS CALL?

Ask:

Does caller need result immediately?

If yes:

request-response may fit

If no and work can happen later:

queue / async processing may fit

This is a direct restatement of the synchronous/asynchronous distinction in the source.

---

# 70. QUEUE OR PUB/SUB?

Ask:

Is this one processing workflow?

or

Should multiple independent services react?

The source positions simple queueing around a producer and processing service, whereas Pub/Sub is introduced to decouple one publisher from multiple downstream services.

---

# 71. PUB/SUB OR EVENT STREAM?

Ask:

Do future consumers need old events?

In the source's Pub/Sub model:

late subscriber
→ no historical replay

In event streaming:

consumer
→ can read old log positions

---

# 72. THE ASYNCHRONOUS SCALING STORY

Memorize the progression:

SLOW WORK IN REQUEST PATH
↓
CLIENT WAITS
↓
MOVE WORK TO QUEUE
↓
REQUEST CAN FINISH EARLIER
↓
CONSUMER PROCESSES WORK
↓
TRAFFIC INCREASES
↓
QUEUE BUILDS BACKLOG
↓
ADD CONSUMERS

The source explicitly gives the queue-buffering and additional-consumer scaling steps.

---

# 73. EVENT-DRIVEN SCALING STORY

SERVICE A DIRECTLY CALLS
B, C, D
↓
strong knowledge/coupling
↓
PUBLISH EVENT
↓
BROKER
↓
B / C / D SUBSCRIBE
↓
new service?
↓
add subscriber

This mirrors the direct-service and Pub/Sub progression in the notes.

---

# 74. WHAT DOES ASYNC BUY YOU?

Source-derived summary:

Caller doesn't wait for final work

Producer and consumer decoupled in time

Queue buffers work

Consumer can be unavailable temporarily

Consumer tier can scale independently

Multiple services can react to events

Supported by the queue and EDA sections.

---

# 75. WHAT DOES ASYNC COST YOU?

From the source's EDA discussion:

Stale / inconsistent local copies

Harder-to-follow control flow

More event-bus complexity

Authorization/security concerns

Replay may not reproduce external side effects exactly

Idempotency is not inherently guaranteed

This is the trade-off an 8-YOE candidate should be able to articulate.

---

# 76. DELIVERY SEMANTICS — WHAT YOUR NOTES SAY

The EDA section states that sent messages can have:

at-least-once

or

at-most-once

delivery guarantees.

The notes do not provide a detailed delivery-semantics chapter here, so don't expand beyond that yet.

For this part, memorize the vocabulary.

---

# 77. IDEMPOTENCY — SOURCE CONNECTION

The Pub/Sub section explicitly says:

does not guarantee idempotency

So if processing could happen more than once under your messaging semantics, application design must pay attention to repeated effects.

The notes do not provide the detailed idempotent-consumer implementation here, so stop at that conceptual warning.

---

# 78. BACKLOG — SOURCE CONCEPT

The notes don't use the specific term “consumer lag” in this section, but they do describe:

many incoming requests
↓
queue retains them
↓
consumers work through them

and scaling by adding more consumers.

You can remember that pending queued work is the backlog you are trying to drain, without inventing product-specific metrics.

---

# 79. WHY A MESSAGE BROKER REDUCES APPLICATION COMPLEXITY

The source argues that using a purpose-built message queue avoids manually implementing much of the queueing/concurrency/cleanup behavior on top of a database table.

So the architectural principle is:

> **Use infrastructure specialized for the communication pattern instead of rebuilding that infrastructure inside application/database logic.**

---

# 80. 8-YOE ASYNC INTERVIEW FLOOR

You should answer these cleanly:

**1. Synchronous vs asynchronous processing?**

**2. When should work leave the request path?**

**3. What problem does a message queue solve?**

**4. Producer vs consumer?**

**5. Why not use a DB table as a queue?**

**6. Why is polling expensive?**

**7. How does a queue absorb traffic bursts?**

**8. How do you scale queue processing?**

**9. Request-response vs event-driven architecture?**

**10. What is an event?**

**11. Why are events immutable in the source's model?**

**12. What are advantages of EDA?**

**13. What are disadvantages of EDA?**

**14. What is the dual-write problem?**

**15. Why does direct service-to-service fan-out create coupling?**

**16. What is Pub/Sub?**

**17. Queue vs Pub/Sub?**

**18. What does Pub/Sub not inherently guarantee according to the notes?**

**19. What is event streaming?**

**20. Pub/Sub vs event streaming?**

**21. Why is replay useful?**

**22. What is a Kafka topic?**

**23. Why partitions?**

**24. What is a consumer group in the notes' model?**

**25. SQS vs SNS conceptually?**

**26. What is SNS + SQS fan-out?**

**27. What does the source actually say about Kinesis?**

Answer to #27:

Only the heading names it; the retrieved notes do not explain it.

---

# 81. RAPID-FIRE MODEL ANSWERS

## Q: Synchronous vs asynchronous?

> “Synchronous processing makes the requester wait until the operation completes. In asynchronous processing, work can be acknowledged or queued so the requester continues while another component completes it later.”

---

# 82. Q: Why use a queue?

> “A queue temporarily stores work between the producer and consumer, allowing them to operate independently in time. It can absorb bursts and lets the consumer tier process work at its own rate.”

Supported by the source's queue example.

---

# 83. Q: Why not use a database as a queue?

> “The notes highlight frequent polling, concurrent inserts/updates/reads, locking/race-condition concerns, manual cleanup and failure handling. A purpose-built message queue is better suited to this background-processing pattern.”

---

# 84. Q: Request-response vs event-driven?

> “Request-response has one service directly ask another service to perform work. In the event-driven model in these notes, a service publishes that something happened to an event bus, and interested consumers react without the publisher directly knowing all of them.”

---

# 85. Q: Why Pub/Sub?

> “Pub/Sub reduces direct coupling. A publisher sends to the messaging layer rather than explicitly calling each downstream service, and new consumers can subscribe without the publisher knowing their implementation details.”

---

# 86. Q: Pub/Sub vs event stream?

> “In the subscription model described by the notes, a subscriber receives events from the point it joins and cannot replay older ones. Event streaming writes events to a log, allowing consumers to read and replay from earlier positions.”

---

# 87. Q: What is Kafka according to these notes?

> “Kafka is described as a publish/subscribe event system where producers send events to topics, topics contain partitions for parallel streaming, and consumer groups track their own consumption while separate groups can independently read the same topic.”

---

# 88. Q: SQS vs SNS according to these notes?

> “The notes frame SQS as queue-based/pull-oriented processing and SNS as publish/subscribe push distribution. They then combine SNS with multiple SQS queues as a fan-out pattern.”

---

# 89. Q: What is dual write?

> “It is the consistency problem that occurs when one logical operation requires updates to two separate storage systems and one succeeds while the other fails.”

---

# 90. COMMON BAD ANSWERS

### ❌ “Async means faster execution.”

The source only establishes that the requester does not have to wait for completion.

The background work still has to happen.

---

### ❌ “Queue means no failure.”

No.

The source motivates queues partly because failures and temporary unavailability exist.

---

### ❌ “Event-driven means all services are automatically consistent.”

The notes explicitly describe stale copies and reduced immediate consistency as disadvantages.

---

### ❌ “Pub/Sub automatically guarantees idempotency.”

The source says the opposite.

---

### ❌ “Pub/Sub and event streaming are identical.”

Not in these notes.

The source specifically differentiates them by **historical log/replay**.

---

### ❌ “The notes explain Kinesis.”

They don't, beyond naming it in the comparison heading.

---

# 91. ONE-PAGE MEMORY SHEET

ASYNC

SYNCHRONOUS

request
↓
process
↓
response

caller waits

ASYNCHRONOUS

submit work
↓
acknowledge
↓
caller continues

worker finishes later

QUEUE

Producer
↓
Queue
↓
Consumer

WHY?

decouple
buffer work
handle busy consumer
scale consumers

DATABASE AS QUEUE?

Problems in notes:

polling
locks
race conditions
cleanup
failed-processing complexity
scaling

EVENT-DRIVEN

Service
↓
Event Bus
/ \
B C

EVENT

"something happened"

EDA BENEFITS

decoupling
parallel processing
data replication
monitoring
redundancy
microservices
independent scaling

EDA COSTS

stale copies
harder reasoning
security/access complexity
replay limitations with external effects

DUAL WRITE

DB success
+
message failure
=
inconsistent state

PUB/SUB

Publisher
↓
Topic
/ | \
A B C

publisher doesn't know
every subscriber

PUB/SUB LIMITS
IN NOTES

possible inconsistency
no inherent idempotency
no historical replay

in model described

EVENT STREAM

E1 E2 E3 E4 E5

persistent log
↓
consumer position
↓
replay

KAFKA
IN NOTES

Producer
↓
Topic
↓
Partitions
↓
Consumer Groups

SQS
IN NOTES

Queue
pull-oriented

SNS
IN NOTES

Publish / Subscribe

SNS + SQS

SNS
/ | \
Q1 Q2 Q3

fan-out

KINESIS

named
but not explained
in retrieved source

---

# 92. THE 60-SECOND SENIOR ANSWER

If asked:

> **“When would you use asynchronous/event-driven architecture?”**

A strong answer grounded in these notes is:

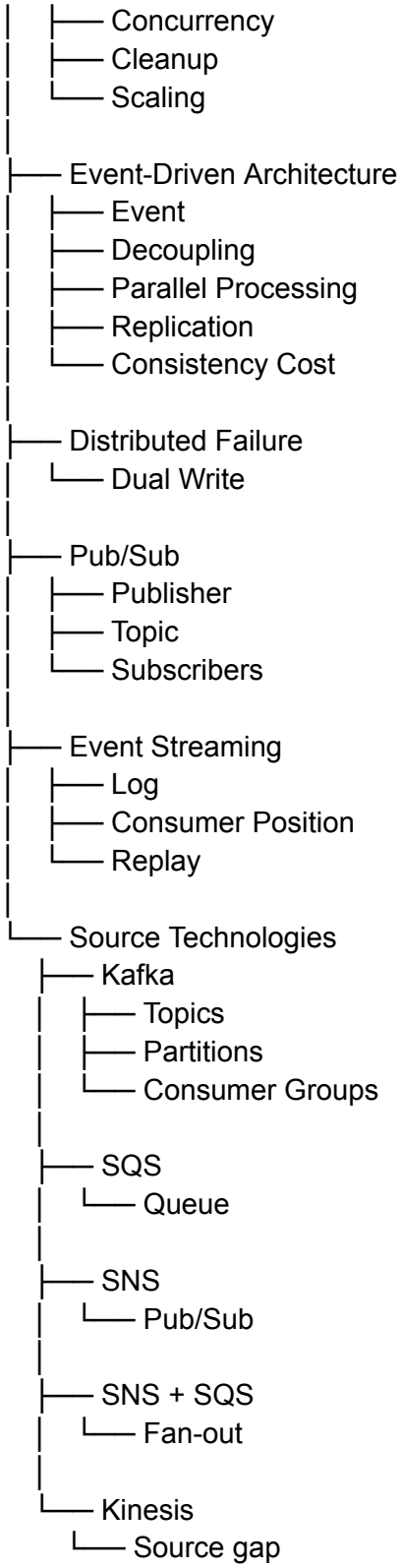
> “I would move work out of the synchronous path when the requester does not need the final processing result immediately. A message queue lets the producer submit work and continue while consumers process it later, and it can buffer bursts or temporary consumer unavailability. If processing demand grows, consumers can be scaled independently. For broader service decoupling, an event-driven model lets a producer publish that something happened without directly knowing all downstream consumers. Pub/Sub supports one-to-many notification, while the event-streaming model in these notes persists events in a log so consumers can replay history. The trade-off is increased consistency and reasoning complexity: services can hold stale local state, replay can be tricky when external side effects are involved, and idempotency is not automatically guaranteed.”

If you can naturally expand each sentence, **Part 7 is learned**.

---

# 93. PART 7 MEMORY MAP

ASYNC & EVENTS
│
├── Synchronous
│ └── Caller waits
│
├── Asynchronous
│ └── Caller continues
│
├── Queue
│ ├── Producer
│ ├── Temporary Storage
│ ├── Consumer
│ └── Add Consumers
│
├── DB-as-Queue Problems
│ ├── Polling




---

# CJ SYSTEM DESIGN INTERVIEW HANDBOOK

## PART 8 — LOAD BALANCERS, API GATEWAYS, SERVICE DISCOVERY & SERVICE MESH

**Target:** Senior Software Engineer / Backend Engineer — ~8 YOE
**Goal:** Stop mixing up four different infrastructure problems:

LOAD BALANCER
Where among equivalent instances should traffic go?

API GATEWAY
Which backend/API should this client request reach?

SERVICE DISCOVERY
Where is that service currently running?

SERVICE MESH
How should internal services communicate?

> **Priority legend**
> 🟢 **MUST KNOW** — senior interview floor
> 🟡 **SHOULD KNOW** — explain confidently
> 🔵 **SOURCE-SPECIFIC / DEEP DIVE**

This section follows the notes' progression: **Load Balancer → Gateway → Nginx → Sticky Sessions → DNS Load Balancing → ELB types → Service Discovery → Service Mesh → comparison.**

---

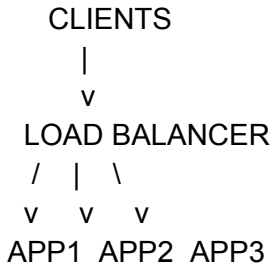
# 1. LOAD BALANCER — FIRST PRINCIPLES

### 🟢 MUST KNOW

The source defines the heart of load balancing as:

> distributing traffic effectively across healthy servers.

Basic architecture:




Without it:

All traffic
↓
Server A
↓
overload

With it:

Traffic
 ↓
LB
├── A
├── B
└── C

---

# 2. WHY DO WE NEED A LOAD BALANCER?

The notes give three motivations:

1. Distribute requests
   so one server is not overworked

2. Allow servers to be removed/added
   → scalability + fault tolerance

3. Avoid performance degradation
   caused by overloaded servers

Memory:

> **Spread load + route around unhealthy capacity + scale horizontally.**

---

# 3. LOAD BALANCING APPEARS AFTER HORIZONTAL SCALING

One application server:

Client
 ↓
Server

Scale out:

Server 1
Server 2
Server 3

Now the new problem is:

> **Which server receives each request?**

That is why the original scale-up path places a load balancer in front of multiple application servers.

---

# 4. HEALTHY SERVERS MATTER

The source specifically says traffic should be distributed across:

healthy servers

not merely all configured servers.

Conceptually:

APP1 ✓
APP2 ✗
APP3 ✓

LB should route toward:

APP1 / APP3

rather than APP2.

---

# 5. STATIC VS DYNAMIC LOAD-BALANCING ALGORITHMS

The notes distinguish:

## Static

Do not consider current server state.

Examples given:

Round Robin

Consistent Hashing

## Dynamic

Consider current state.

Example:

Least Connections

which chooses the server with the fewest active connections.

---

# 6. ROUND ROBIN

Source classification:

STATIC

Conceptually:

Request 1 → A
Request 2 → B
Request 3 → C
Request 4 → A

Simple cycling.

---

# 7. LEAST CONNECTIONS

Source classification:

DYNAMIC

Suppose:

A → 100 active connections
B → 20
C → 60

Next request:

→ B

because B currently has the fewest active connections.

---

# 8. CONSISTENT HASHING AS LOAD-BALANCING STRATEGY

The notes classify consistent hashing among static load-balancing approaches.

You already learned its deeper role in Part 4:

key
↓
hash
↓
stable node assignment

Here, simply remember:

> It can also be used to keep related requests mapped consistently toward particular nodes.

---

# 9. LOAD BALANCER PERSISTENCE

The source says that with persistence enabled:

first client connection
→ load balanced normally

subsequent connections/requests
→ same server

while persistence remains active.

This leads naturally to:

> **Sticky sessions**

---

# 10. STICKY VS NON-STICKY SESSIONS

🟢 **MUST KNOW**

The notes spend significant space on this because session state becomes problematic once multiple servers exist.

---

# 11. SINGLE SERVER SESSION

With one web server:

Client
↓
Server A

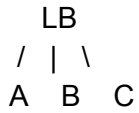
Session object lives in A's memory

Every request reaches A, so the server can continue using that in-memory session.

---

# 12. MULTIPLE SERVERS CREATE THE SESSION PROBLEM

Now:




Client requests might go:

Request 1 → A
Request 2 → B
Request 3 → C

But:

Session on A
≠
Session on B
≠
Session on C

because they are independent machines.

---

# 13. STICKY SESSION

The source's solution:

first request
↓
LB chooses B
↓
all later session requests
↓
B

So the user continues interacting with the same physical server.

Memory:

STICKY
same client/session
→ same server

---

# 14. NON-STICKY SESSION

The notes summarize non-sticky behavior as:

Load balancer
may choose
any server
for each request

---

# 15. STICKY WITHOUT SESSION REPLICATION

The source gives this failure case:

User pinned to A
↓
A crashes/restarts
↓
LB sends user to B
↓
B does not have user's session
↓
session is lost

This is the major weakness of relying only on server-local session state.

---

# 16. STICKY + SESSION REPLICATION

The notes then describe:

User pinned to A

session replicated
A → B/C

A crashes

request goes to B

B already has session

so user can continue.

---

# 17. STICKY SESSION MEMORY CARD

NON-STICKY
request can hit any node

STICKY
same session → same node

STICKY + NO REPLICATION
node failure → session loss

STICKY + REPLICATION
node failure → another node has session

---

# 18. REVERSE PROXY

The source does not give a standalone textbook definition, but repeatedly positions a reverse proxy as infrastructure sitting before backend/upstream servers.

It specifically says **NGINX** is commonly used as both:

Reverse Proxy
+
Load Balancer

to manage incoming connections and distribute them to upstream systems.

---

# 19. NGINX — SOURCE ROLE

🟡

According to the notes:

CLIENT
↓
NGINX
↓
UPSTREAM SERVERS

NGINX can sit in front of:

microservices
legacy upstream systems

other servers

and manage a high volume of incoming connections.

---

# 20. API GATEWAY — FIRST PRINCIPLES

🟢🟢🟢

The source distinguishes a gateway from a load balancer approximately like this:

API GATEWAY

Which backend/service
matches this API request?

LOAD BALANCER

Which instance
of the selected backend
should receive traffic?

The notes say gateways connect/rout among microservices, whereas load balancers distribute requests across multiple instances of the same scaled service.

---

# 21. API GATEWAY REQUEST ROUTING

Imagine:

GET /users/42
↓
API Gateway
↓
User Service

while:

POST /orders
↓
API Gateway
↓
Order Service

The source emphasizes routing to different backend resources according to the requested destination rather than merely distributing evenly across equivalent nodes.

---

# 22. LOAD BALANCER VS API GATEWAY

Source mental model:

CLIENT
↓
API GATEWAY
↓
choose SERVICE
↓
LOAD BALANCER
↓
choose INSTANCE

They are not necessarily mutually exclusive.

The notes explicitly say API gateways and reverse proxies/load balancers can be used together.

---

# 23. API GATEWAY FUNCTIONS IN THE NOTES

The source associates gateway behavior with:

Authentication

Transport security

Load balancing

Rate restriction

Dynamic configuration

Service discovery

Aggregation

Monitoring/security functionality

Another later section says an API gateway can handle:

load balancing
caching
access control
API metering
monitoring

---

# 24. API AGGREGATION

One benefit emphasized in the notes is:

Client
  ↓
Gateway
  ├── Service A
  ├── Service B
  └── Service C
  ↓
aggregate response

The gateway can invoke multiple backend systems and combine their results.

---

# 25. API GATEWAY AS SPECIALIZED REVERSE PROXY

The source explicitly says:

> Think of an API gateway as a special form of reverse proxy.

Its framing is:

BASIC REVERSE PROXY

+ authentication
+ rate restriction
+ dynamic config
+ service discovery
+ API-specific features

≈ API GATEWAY

---

# 26. API PROXY VS API GATEWAY — SOURCE DISTINCTION

The notes say:

API PROXY
lighter
minimal security/monitoring

API GATEWAY

richer security/monitoring/features

and suggest a proxy can be sufficient when requirements are modest.

---

# 27. SOURCE-SPECIFIC API GATEWAY CLAIMS

🔵

The notes contain specific pricing/timeout/throughput claims about an API Gateway product, including a **29-second duration** and comparisons with load-balancer pricing.

These are product/version-specific statements.

Since this pass is based on your notes rather than current vendor verification, **do not make those numeric claims part of the timeless interview core**.

Memorize the architectural distinction instead.

---

# 28. DNS-BASED LOAD BALANCING

🟡

The notes define DNS-based LB as returning different IP addresses in DNS responses to distribute traffic across servers.

Conceptually:

```
      DNS
   /   |   \
  IP1  IP2  IP3
```

Different clients/queries can receive different destination IPs.

---

# 29. ROUND-ROBIN DNS

Basic DNS setup:

example.com
→ IP1

Round-robin setup:

example.com

→ IP1
→ IP2
→ IP3

The notes say multiple A records are associated with a domain and addresses rotate as DNS queries arrive.

---

# 30. WEIGHTED DNS LOAD BALANCING

The source gives:

Server A capacity = 2 × Server B

so a weighted algorithm can direct proportionally more traffic toward A.

Mental model:

A weight = 2
B weight = 1

roughly:
A gets more traffic than B

---

# 31. LEAST-CONNECTION DNS EXAMPLE

The notes also describe a dynamic option using server monitoring:

find server
with fewest open connections
↓
return its IP

Keep this as the source's example.

---

# 32. ELB TYPES — SOURCE SECTION

The notes list four AWS Elastic Load Balancer types:

Classic Load Balancer

Application Load Balancer

Network Load Balancer

Gateway Load Balancer

This is vendor-specific material from the notes.

---

# 33. APPLICATION LOAD BALANCER — SOURCE

The notes associate ALB with:

HTTP / HTTPS

Application layer

Host-based routing

Path-based routing

Dynamic host-port mapping

---

# 34. NETWORK LOAD BALANCER — SOURCE

The notes associate NLB with:

Transport layer

TCP traffic

Very high request volume

---

# 35. GATEWAY LOAD BALANCER — SOURCE

The notes describe it as supporting deployment/scaling/management of virtual appliances such as:

firewalls

while combining gateway behavior with traffic distribution.

---

# 36. CLASSIC LOAD BALANCER — SOURCE

The source describes Classic LB as the older/traditional AWS option and notes that it does not provide the host/path routing capabilities attributed to ALB.

For interview ROI:

ALB
→ application/HTTP routing

NLB
→ transport/TCP-oriented

GWLB
→ virtual appliances

Classic
→ legacy/traditional category in notes

---

# 37. SERVICE DISCOVERY — WHY DOES IT EXIST?

🟢🟢🟢

The source gives the key microservices problem:

> Running service instances change dynamically and have dynamically assigned network locations.

Imagine:

User Service

10.0.0.7:8080  today

dies

10.0.1.18:8080  replacement

Hard-coding:

10.0.0.7

doesn't work in a dynamic environment.

---

# 38. SERVICE DISCOVERY QUESTION

The problem is simply:

**Where is Service X running right now?**

"User Service"
↓
Service Registry
↓
10.0.1.18:8080

---

# 39. THREE SERVICE-DISCOVERY COMPONENTS

The source explicitly gives:

1. Service Provider

2. Service Registry

3. Service Consumer

---

# 40. SERVICE PROVIDER

SERVICE PROVIDER

is the service instance providing functionality over the network.

Example:

Payment Service
instance 17

---

# 41. SERVICE REGISTRY

The source calls the registry a database containing:

locations of available service instances

Conceptually:

SERVICE REGISTRY

UserService
├── 10.0.1.10
├── 10.0.1.11
└── 10.0.1.12

PaymentService
├── 10.0.2.20
└── 10.0.2.21

---

# 42. SERVICE CONSUMER

The consumer:

needs Service X
↓
finds X's location
↓
communicates with an instance

This is the third component in the source's model.

---

# 43. REGISTRATION AND DEREGISTRATION

The notes say a service registry provides:

Management API

Query API

Service instances:

register

deregister

through the management API, while clients/components query available locations via the query API.

---

# 44. CLIENT-SIDE SERVICE DISCOVERY

The source defines:

Client
↓
query registry
↓
receive instances
↓
client selects one

↓
call service

The client itself participates in selecting the destination.

---

# 45. SERVER-SIDE SERVICE DISCOVERY

The source's second pattern:

Client
↓
Router
↓
Registry
↓
Service instance

The router queries the registry and forwards to an available service instance.

---

# 46. CLIENT-SIDE VS SERVER-SIDE DISCOVERY

CLIENT-SIDE

Client
→ Registry
→ chooses instance
→ Service

SERVER-SIDE

Client
→ Router
→ Registry
→ Router chooses instance
→ Service

That distinction is directly supported by the notes.

---

# 47. LOAD BALANCER VS SERVICE DISCOVERY

🟢🟢🟢 **VERY COMMON INTERVIEW QUESTION**

The source's distinction:

SERVICE DISCOVERY
knows/finds current endpoints

LOAD BALANCER
distributes traffic across endpoints

The notes explicitly say both can coexist: service discovery tracks dynamically changing endpoints, while a load-balancing tool such as NGINX directs traffic to them.

---

# 48. WHY THEY WORK TOGETHER

Imagine containers changing:

Old:
A B C

C dies

New:
A B D

Service registry updates:

A
B
D

Load balancer needs that updated endpoint set to route correctly.

The source uses Consul + NGINX as its concrete example of this cooperation.

---

# 49. SOURCE EXAMPLES OF SERVICE-DISCOVERY TOOLS

The notes mention:

ZooKeeper

Consul

Eureka

SkyDNS

in the gateway/service-discovery discussion.

Treat these as examples from the source, not as a current product-ranking recommendation.

---

# 50. API GATEWAY + SERVICE DISCOVERY

The source describes:

Client
↓
API Gateway
↓
asks service-discovery:
"Where is Service X?"
↓
Registry returns address
↓
Gateway forwards request

This answers two different questions:

Gateway
→ Which logical service?

Discovery
→ Where is its current instance?

---

# 51. SERVICE MESH — FIRST PRINCIPLES

🟢🟢🟢

The notes define a service mesh as:

> an infrastructure layer handling inter-service communication in a microservice architecture.

Think:

SERVICE A
↕
MESH
↕
SERVICE B

rather than every service implementing all network concerns itself.

---

# 52. WHY SERVICE MESH?

The source says it:

reduces complexity in microservice code

reduces duplication

moves common infrastructure concerns
into another layer

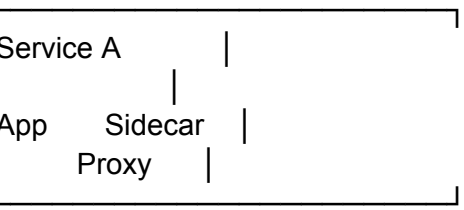
loosens coupling between app code
and underlying platform

---

# 53. SIDECAR PATTERN

🟢 **MUST KNOW**

The source's key service-mesh model:




A sidecar runs alongside each application/service.

---

# 54. TRAFFIC THROUGH SIDECAR

The notes say incoming and outgoing service traffic flows through the sidecar proxy.

Service A
↓
Sidecar A
↓
network
↓
Sidecar B
↓

Service B

The sidecar can:

manage traffic

collect telemetry

enforce policies

---

# 55. SERVICE CODE KNOWS LESS ABOUT NETWORKING

This is the important mental model from the source:

WITHOUT MESH

Service code
├── network handling
├── routing
├── policies
├── telemetry
└── service concerns

WITH SIDECAR

Service code
→ app logic

Sidecar
→ network/infrastructure concerns

The notes describe the service as effectively depending on the local sidecar while the sidecar abstracts network concerns.

---

# 56. DATA PLANE VS CONTROL PLANE

**🟢 MUST KNOW FOR SERVICE MESH**

The source explicitly separates the two.

---

# 57. DATA PLANE

Source responsibilities include:

service communication

service discovery

load balancing

traffic management

health checks

The data plane handles the actual service traffic.

---

# 58. CONTROL PLANE

The source says the control plane:

manages/configures sidecar proxies

enforces policies

collects telemetry

Memory:

DATA PLANE
moves traffic

CONTROL PLANE
configures how traffic should be handled

---

# 59. ISTIO / ENVOY / LINKERD — SOURCE EXAMPLES

The notes mention:

Istio

Linkerd

Envoy

and say Istio uses Envoy as a sidecar proxy in the model being described.

These are examples from the notes.

---

# 60. API GATEWAY VS SERVICE MESH

🟢🟢🟢 **MEMORIZE THIS**

The source gives a particularly clean distinction:

API GATEWAY
external traffic

SERVICE MESH
internal service-to-service traffic

---

# 61. EXTERNAL VS INTERNAL

Mobile App
Web App
    |
    v
API GATEWAY
    |
    v
------------------------
internal system
------------------------
    |
 Service A
    |
  MESH
    |
 Service B

Gateway is the client-facing entry layer in the source.

Mesh focuses on internal service communication.

---

# 62. MASTER COMPARISON

| Component | Main question in the notes |
| --- | --- |

| | |
|---|---|
| Load Balancer | Which healthy instance should receive traffic? |
| API Gateway | Which API/backend service should this external request reach? |
| Service Registry/Discovery | Where are the current instances of that service? |
| Service Mesh | How should internal service-to-service traffic be handled? |

This table consolidates the distinctions made across the source sections.

---

# 63. PUTTING ALL FOUR TOGETHER

Source-consistent conceptual architecture:

```
          EXTERNAL CLIENT
                |
                v
           API GATEWAY
                |
      "I need User Service"
                |
                v
          SERVICE DISCOVERY
                |
          returns endpoints
                |
                v
           LOAD BALANCER
            /       \
           v         v
    User Service   User Service
           |         |
        Sidecar    Sidecar
           \         /
            SERVICE MESH
                |
           Other Services
```

Not every architecture needs every box.

But each solves a different problem.

---

# 64. THE FASTEST MEMORY MODEL

DISCOVERY

WHERE is the service?

GATEWAY
WHICH service?

LOAD BALANCER
WHICH instance?

MESH
HOW do services communicate internally?

If you memorize only one thing from this chapter, memorize that.

---

# 65. SERVICE DISCOVERY + EPHEMERAL CONTAINERS

The source specifically motivates discovery with containers whose addresses change because instances can be:

created

evicted

scaled

at runtime.

Therefore:

static IP list

becomes unreliable.

A registry provides current endpoint information.

---

# 66. LOAD BALANCER NEEDS ENDPOINT KNOWLEDGE

The source explicitly states:

> Load balancers need the endpoints of the resources whose traffic they balance.

So:

Discovery
provides/churns endpoints

LB
uses endpoints to route traffic

This is why the two systems coexist rather than replace one another.

---

# 67. GATEWAY DOES NOT REPLACE SERVICE DISCOVERY

The source's architecture has the gateway query service-discovery software to determine where a backend service is currently located.

Thus:

Gateway knows:
request → logical service

Discovery knows:
logical service → current endpoint

---

# 68. SERVICE MESH DOES NOT MEAN API GATEWAY

The source explicitly separates them by traffic direction/domain:

Gateway
→ external clients entering system

Mesh
→ communication between internal services

---

# 69. INTERVIEW QUESTION — “WHY NOT JUST USE NGINX FOR EVERYTHING?”

The source itself shows overlapping responsibilities:

NGINX

→ reverse proxy / load balancer

API Gateway
→ richer API/security/routing/aggregation features

Service Discovery
→ dynamic endpoint knowledge

Mesh
→ distributed internal communication infrastructure

So the answer is not:

> “One replaces all the others.”

The source repeatedly describes them as potentially coexisting.

---

# 70. 8-YOE INTERVIEW FLOOR

You should answer these immediately:

**1. Why do we need a load balancer?**

**2. Round robin vs least connections?**

**3. Static vs dynamic LB algorithm?**

**4. What are sticky sessions?**

**5. Sticky vs non-sticky?**

**6. What happens if the sticky server dies?**

**7. What does session replication solve?**

**8. What is DNS-based load balancing?**

**9. What does NGINX do in these notes?**

**10. Load balancer vs API gateway?**

**11. Why would you use both?**

**12. What functions does API Gateway provide?**

**13. What is API aggregation?**

**14. Why is service discovery required?**

**15. What is a service registry?**

**16. Provider vs registry vs consumer?**

**17. Client-side vs server-side service discovery?**

**18. Load balancer vs service discovery?**

**19. What is a service mesh?**

**20. What is the sidecar pattern?**

**21. Data plane vs control plane?**

**22. API gateway vs service mesh?**

---

# 71. RAPID-FIRE MODEL ANSWERS

## Q: What does a load balancer do?

> “It distributes incoming traffic across healthy backend servers so load is spread and individual server failure or overload does not become the sole path for requests.”

---

# 72. Q: Round robin vs least connections?

> “The notes classify round robin as static because it doesn't consider current server state, while least connections is dynamic because it selects the server with the fewest active connections.”

---

# 73. Q: What is sticky session?

> “After the load balancer selects a server for the session, subsequent requests from that user/session continue going to the same physical server.”

---

# 74. Q: API gateway vs load balancer?

> “The gateway routes an API request toward the appropriate backend service and may provide authentication, rate limiting, aggregation and monitoring. A load balancer primarily spreads traffic across backend instances.”

---

# 75. Q: What is service discovery?

> “It solves the problem of locating dynamically changing service instances. A service registry stores available instance locations, and clients or routers query it before communicating with a service.”

---

# 76. Q: Load balancer vs service registry?

> “The service registry tells the system which service endpoints currently exist; the load balancer decides how to distribute traffic across those endpoints.”

---

# 77. Q: What is service mesh?

> “The notes define a service mesh as an infrastructure layer for inter-service communication. Sidecar proxies handle traffic and infrastructure concerns around each service, while a control plane configures those proxies.”

---

# 78. Q: Data plane vs control plane?

> “The data plane handles actual service communication and traffic concerns such as discovery, load balancing and health checks. The control plane manages and configures the proxies and policies.”

---

# 79. Q: Gateway vs mesh?

> “The notes position the API gateway primarily at the external entry point for web/mobile clients, while the service mesh primarily manages internal service-to-service traffic.”

---

# 80. COMMON BAD ANSWERS

❌ **“Load balancer and service discovery are the same.”**

No.

Discovery → what instances exist?

LB → which one receives traffic?

---

### ❌ “API gateway is just another name for load balancer.”

The notes give the gateway broader API-routing, security, aggregation and service-discovery responsibilities.

---

### ❌ “Service mesh replaces API gateway.”

Not in the source's model.

Gateway → external

Mesh → internal

---

### ❌ “Sticky sessions survive server failure automatically.”

Not if the session only exists on the failed server.

The notes explicitly distinguish sticky sessions with and without session replication.

---

### ❌ “Service addresses stay fixed.”

The source's entire motivation for service discovery is that microservice instances can have dynamically changing network locations.

---

# 81. ONE-PAGE MEMORY SHEET

LOAD BALANCER
==========

CLIENTS
↓
LB
/|\
A B C

GOAL
spread traffic
healthy servers
scale horizontally

ALGORITHMS

Round Robin
→ static

Consistent Hashing
→ static in notes

Least Connections
→ dynamic

STICKY SESSION

Client
↓
same server

WITHOUT REPLICATION
server dies
→ session lost

WITH REPLICATION
another server can continue

DNS LOAD BALANCING

domain
↓
multiple IPs
↓
distributed traffic

NGINX

reverse proxy
+
load balancer

API GATEWAY
==========

external request
↓
gateway
↓
correct backend service

FEATURES IN NOTES

auth
security
rate restriction
aggregation
monitoring
service discovery

SERVICE DISCOVERY
==========

WHY?
service locations change

Provider
Registry
Consumer

REGISTRY
service → endpoints

CLIENT-SIDE
client queries registry

SERVER-SIDE
router queries registry

SERVICE MESH
==========

internal
service-to-service traffic

Service
+
Sidecar Proxy

DATA PLANE
traffic

CONTROL PLANE
proxy configuration/policy

MASTER DISTINCTION
==========

Gateway

→ WHICH service?

Discovery
→ WHERE is service?

Load Balancer
→ WHICH instance?

Mesh
→ HOW services communicate internally?

---

# 82. 60-SECOND SENIOR ANSWER

If asked:

> **“Explain load balancer vs gateway vs service discovery vs service mesh.”**

Use this:

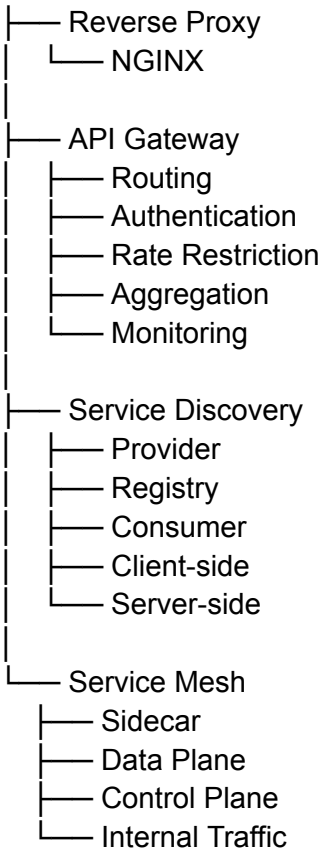
> “A load balancer distributes traffic across healthy instances of a backend. An API gateway is the external API entry layer and routes requests toward the appropriate backend service while potentially handling concerns such as authentication, rate restrictions, aggregation and monitoring. In a dynamic microservice environment, service discovery maintains the current locations of service instances, so a client, router or gateway can find them. A service mesh moves common internal service-to-service networking concerns into infrastructure such as sidecar proxies. In the notes' model, the simplest distinction is: the gateway chooses the logical service, discovery tells us where it is, the load balancer chooses an instance, and the mesh manages internal service communication.”

That is the core of Part 8.

---

# 83. PART 8 MEMORY MAP

TRAFFIC INFRASTRUCTURE
│
├── Load Balancer
│ ├── Healthy Servers
│ ├── Round Robin
│ ├── Least Connections
│ ├── Consistent Hashing
│ └── Sticky Sessions
│
├── DNS Load Balancing
│ ├── Multiple A/AAAA
│ ├── Round Robin
│ └── Weighted
│




---

# CJ SYSTEM DESIGN INTERVIEW HANDBOOK

## PART 9 — MONOLITHS, MICROSERVICES, CIRCUIT BREAKERS & RATE LIMITING

**Target:** Senior Software Engineer / Backend Engineer — ~8 YOE
**Goal:** Understand what happens **after a system is split into services**: more network calls, more failure modes, more deployment/observability needs, and the need to protect services from overload.

> **Priority legend**
> 🟢 **MUST KNOW** — senior interview floor
> 🟡 **SHOULD KNOW** — explain confidently
> 🔵 **SOURCE-SPECIFIC / DEEP DIVE**
> ⚠️ **SOURCE BOUNDARY** — the notes mention a topic but do not provide enough detail to safely expand it further

This chapter follows the source sequence:

Monolith vs Microservices
↓
Microservice boundaries
↓
Infrastructure overhead
↓
Circuit Breaker
↓
Rate Limiting
↓
Leaky Bucket
↓
Token Bucket
↓
Sliding Window
↓
Timer Wheel
↓
Request Collapsing
↓
Thundering Herd
↓
Internal Rate Limiting

The notes treat these as one connected resilience block.

---

# 1. MONOLITH — SOURCE MENTAL MODEL

🟢

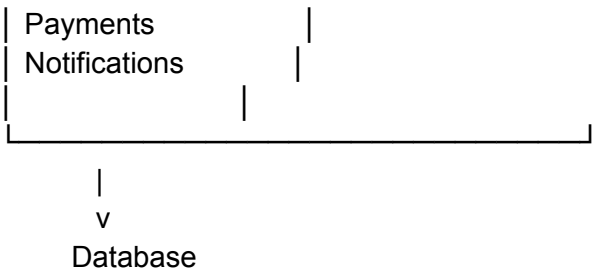
The notes characterize a monolith as having:

code
+
expectations
+
changes

in one place, which reduces some kinds of cross-service breaking-change risk.

Conceptually:

```
┌──────────────────────────────────────┐
│        APPLICATION                   │
│                                      │
│  Users                               │
│  Orders                              │
```




One deployable application contains multiple business capabilities.

---

# 2. MICROSERVICES — SOURCE MENTAL MODEL

The notes describe each microservice as:

self-contained

and say different services may use different:

languages
technology stacks

Conceptually:

User Service
|
Order Service
|
Payment Service
|
Notification Service

with separate service boundaries.

---

# 3. THE FIRST IMPORTANT LESSON

The source does **not** say:

> “Microservices are simply a better monolith.”

It immediately spends several pages on the additional infrastructure required.

That is the important interview framing:

MICROSERVICES

+ separation
+ independent services
+ technology choice

BUT

+ contracts
+ discovery
+ deployments
+ network calls
+ logging complexity
+ operational cost

---

# 4. MICROSERVICE OVERHEAD #1 — CONTRACTS

🟢

The source says reusable services need a clear definition of:

request
response

through a service contract.

Its recommendation:

**Contract-first approach**

Meaning:

update contract
↓
then implement code changes

rather than modifying behavior invisibly.

---

# 5. WHY CONTRACTS MATTER

Suppose:

Order Service
↓
Payment Service

Payment previously expects:

```
{
  "amount": 100
}
```

and Order suddenly starts sending:

```
{
  "totalAmount": 100
}
```

A service boundary creates an external dependency.

The source's answer is:

define/update the contract explicitly

before changing implementation.

---

# 6. MICROSERVICE OVERHEAD #2 — SERVICE LOCATION

The source says services need to be located and therefore require mechanisms such as:

Load Balancer
Service Registry

Hard-coding:

Payment Service = 10.0.0.7

is considered difficult to maintain.

The notes recommend a registry that updates when service instances come up.

This directly connects Part 9 to Part 8.

---

# 7. MICROSERVICE OVERHEAD #3 — MANY DEPLOYMENTS

The notes say:

multiple services
↓
multiple deployments

and that staggered service deployments can make teams feel as though they are deploying constantly.

Source remedy:

CI/CD

to make deployment management less taxing.

---

# 8. MICROSERVICE OVERHEAD #4 — NETWORK CALLS

🟢🟢

The source explicitly says:

> Lots of service calls add to overall response time.

Monolith:

```
method()
  ↓
memory/local call
```

Microservice:

```
Service A
  ↓ network
Service B
  ↓ network
Service C
```

Every boundary potentially adds communication cost.

---

# 9. CHOOSE THE RIGHT COMMUNICATION STYLE

The notes recommend asking whether communication should be:

request-response

event-driven

batch messaging

Specifically:

No synchronous response needed
↓
consider event-driven

and:

Bulk processing
↓
consider batch messaging

---

# 10. MICROservices DO NOT REMOVE COUPLING AUTOMATICALLY

You can physically create:

20 services

while still having:

tight dependencies

The source's service-boundary discussion asks:

> Can a service be maintained without affecting others?

That is more important than merely counting services.

---

# 11. MICROSERVICE OVERHEAD #5 — LOGGING

🟢 **VERY HIGH ROI**

The notes call centralized logging necessary because a transaction can cross many services.

Example:

Client
↓
Gateway
↓
Order
↓
Payment
↓
Inventory

↓
Notification

One logical request generated logs on:

5+ systems

How do you reconstruct it?

---

# 12. CORRELATION ID

The source explicitly recommends:

unique correlation ID

that travels through all service calls belonging to one transaction/request.

Example:

request-id = ABC123

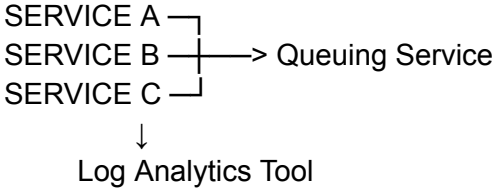
Then:

Gateway ABC123
Order ABC123
Payment ABC123
Inventory ABC123

Centralized logs can retrieve the whole request journey using that identifier.

---

# 13. SOURCE CENTRALIZED-LOGGING FLOW




The notes explicitly recommend pushing service logs toward a centralized analytics system and querying them with the correlation ID.

---

# 14. MICROSERVICE BOUNDARY — WHERE DO YOU STOP?

🟢 **IMPORTANT DESIGN QUESTION**

The source asks:

> **Breaking down the services — where to stop?**

Its main criteria include:

Reusability

Clear functionality differentiation

Decoupling

Independent maintainability

Different business needs

---

# 15. SOURCE EXAMPLE — GOOD SEPARATION

The notes give:

getImages

vs

getProfiles

as clearly different functionality and therefore potentially justified as separate services.

---

# 16. SOURCE EXAMPLE — QUESTIONABLE SEPARATION

The notes compare:

getLinkedInProfiles

vs

getProfiles

and argue that if most logic is shared and changes in one are likely to affect the other, separating them may not produce real independence.

---

# 17. SOMETIMES A LIBRARY IS ENOUGH

The source explicitly proposes:

library
or
common project

instead of a new microservice when there is no clear justification for an independently maintained service.

This is excellent interview reasoning:

> Not every reusable piece of code needs a network boundary.

---

# 18. DUPLICATE MICROSERVICES — SOURCE ANTI-PATTERN

The notes call it an anti-pattern to create duplicated microservices that:

perform essentially the same role
and call the same backend

merely to improve efficiency/response time.

The intended lesson:

> Split around meaningful responsibility, not arbitrary duplication.

---

# 19. ARGUMENTS FOR A MONOLITH IN THE NOTES

The source gives several reasons:

Many microservices
→ infrastructure cost

More services
→ more maintenance

More connections
→ more integration problems

Small teams

→ harder to maintain many services

It says startups often begin with:

monolith
or
minimal number of microservices

because small teams have limited operational capacity.

---

# 20. “SCALE MICROSERVICES AS YOU GROW”

That is literally the heading used in the notes.

Source reasoning:

organisation grows
↓
more dedicated maintenance/development resources
↓
greater ability to operate many services

---

# 21. SOURCE-SPECIFIC TEAM-SIZE RULE

🔵

The notes give a specific suggestion for medium-scale organizations:

2 people per service

so that there is a backup owner.

Treat this as a **source-specific heuristic**, not a timeless industry rule.

The more useful principle is:

> A service needs sufficient ownership capacity to maintain it.

---

# 22. WHO SHOULD DECIDE SERVICE BOUNDARIES?

The source says:

Mostly:
business needs / business-component separation

Sometimes:
engineering complexity / maintainability

Good memory hook:

BUSINESS BOUNDARY FIRST

ENGINEERING PRESSURE SECOND

---

# 23. POLYGLOT SERVICES

The notes discuss using different technologies for different service needs.

Example reasoning:

highly optimized search
→ one language/tool

dashboard
→ another technology

They call the database equivalent:

> **polyglot persistence**

when services choose different databases suited to their needs.

---

# 24. DATABASE PER SERVICE — SOURCE FRAMING

The notes strongly favor a separate database/schema per service to reduce coupling:

Service A
→ DB A

Service B
→ DB B

and warn that multiple services referencing the same table is a coupling smell.

Keep this as the source's microservice design principle.

---

# 25. SINGLE SOURCE OF TRUTH

The notes say a service should be the:

single source of truth

for its owned domain data.

Example:

Order Service

owns the full order.

Other services may use:

order_id

to obtain order information rather than directly treating another shared table as their own.

---

# 26. WHY MICROSERVICES CREATE RESILIENCE PROBLEMS

Once:

Service A
↓
Service B
↓
Service C

exists, the system can experience:

B slow

B failing

B overloaded

C overloaded

and upstream systems may continue sending traffic.

This is exactly where the source next introduces:

**Circuit Breaker Pattern**.

---

# 27. CIRCUIT BREAKER — FIRST PRINCIPLES

🟢🟢🟢

The source says:

1. Wrap a protected function call.
2. Monitor failures.
3. If failures reach a threshold, trip the circuit.
4. Further calls fail without invoking the protected dependency.

---

# 28. NORMAL CALL

Service A
  |
  | request
  v
Service B
  |
 response
  v
Service A

All good.

---

# 29. DEPENDENCY STARTS FAILING

A → B ❌
A → B ❌
A → B ❌
A → B ❌

Without protection:

A continues calling B

even though B is already unhealthy.

---

# 30. CIRCUIT BREAKER TRIPS

After the configured failure threshold:

A

|
v
CIRCUIT BREAKER
|
X
B

New calls:

fail immediately

without invoking B.

That is the behavior explicitly described in the source.

---

# 31. WHY FAIL FAST?

The source's mechanism prevents repeatedly calling the failing protected function after the threshold is reached.

Memory:

KNOWN-BAD DEPENDENCY

don't repeatedly pay
for another doomed remote call

The notes do not provide a separate deep explanation of resource pools/timeouts here, so don't add more semantics than the source supplies.

---

# 32. CIRCUIT BREAKER ALERTING

The source explicitly says you generally also want:

monitor alert

when the breaker trips.

So the circuit breaker is both:

runtime protection
+
operational signal

in this section.

---

# 33. ⚠️ SOURCE BOUNDARY — CIRCUIT STATES

Many interview resources discuss named states such as:

closed
open
half-open

But **this source excerpt does not explain those states**.

For this source-faithful handbook chapter:

remember threshold → trip → fail fast

and do not pretend the uploaded notes gave the full state-machine detail.

---

# 34. OVERLOAD DOES NOT REQUIRE A PROCESS TO DIE

🟢 **IMPORTANT**

The notes point out that a server may still technically be running while becoming practically unusable:

too many requests
↓
processing slows
↓
latency increases
↓
resource consumption increases
↓
system stalls

This is an important resilience mental model.

---

# 35. TIMEOUTS + RATE LIMITERS

The source says production systems use mechanisms such as:

Timeouts
+
Rate Limiters

to prevent overload from turning into full system failure.

---

# 36. LOAD BALANCER VS RATE LIMITER

🟢🟢🟢 **VERY GOOD INTERVIEW DISTINCTION**

The notes explicitly explain this.

## Load Balancer

1,000,000 requests arrive
↓
spread among servers

It answers:

> **Where should load go?**

---

## Rate Limiter

too much load arrives
↓
reject/exclude some requests

It answers:

> **Should this request be admitted at all?**

---

# 37. WHY LOAD BALANCING ALONE IS NOT ENOUGH

Suppose:

10 servers

can each safely process:

1,000 requests/sec

But suddenly:

1,000,000 requests/sec

arrive.

A perfect load balancer can distribute them evenly:

100,000/server

Every server can still collapse.

This is exactly the source's point:

> Load balancing distributes a spike; it does not necessarily make the spike safe.

---

# 38. RATE LIMITER — SOURCE DEFINITION

The source describes configuring a maximum allowed request rate.

Example:

Limit = 1000 requests / second

After that threshold:

additional requests rejected

to protect servers.

---

# 39. SOURCE STATUS-CODE EXAMPLE

🔵

The uploaded notes use:

503 Service Unavailable

as their example response after exceeding the configured rate limit.

For this source-based chapter, retain that as:

> **the source's example**

rather than turning the particular status code into a universal rate-limiting rule.

---

# 40. DISTRIBUTED RATE LIMITING — SOURCE SCOPE

The heading in the notes is:

Distributed Rate Limiting

but the material that follows mainly explains:

Leaky Bucket

Token Bucket

Sliding Window

Timer Wheel

⚠️ The notes do **not** provide a detailed distributed coordination design for maintaining counters across many limiter nodes.

So this handbook should not invent one.

---

# 41. LEAKY BUCKET

**🟢 MUST KNOW FROM SOURCE**

The notes give four steps:

1. Packet/request enters bucket.

2. Bucket leaks at constant rate.

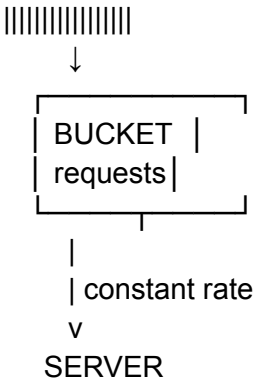
3. Bursty input becomes uniform output.

4. Bucket behaves like a finite queue.

---

# 42. LEAKY BUCKET — VISUAL

BURSTY INPUT



Source memory:

> **Bursty traffic in → smoother fixed-rate traffic out.**

---

# 43. WHY FINITE QUEUE MATTERS

The source explicitly calls the practical bucket a:

finite queue

with finite output rate.

If incoming work exceeds what the queue can absorb/process:

capacity eventually runs out

which is why the source later contrasts packet/request dropping behavior with token bucket.

---

# 44. TOKEN BUCKET

🟢🟢

The notes describe:

tokens generated at regular intervals

bucket has maximum capacity

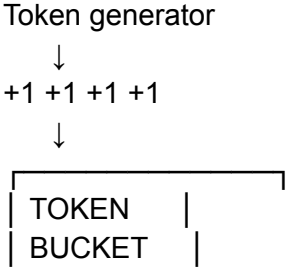
request can proceed if token exists

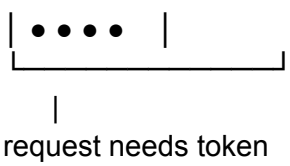
sending consumes token

no token
→ request cannot be sent

---

# 45. TOKEN BUCKET — VISUAL






---

# 46. TOKEN BUCKET BURST BEHAVIOR

The source says a key advantage over leaky bucket is:

> Token bucket can allow larger bursts at a faster rate, whereas leaky bucket emits at a constant rate.

Why, in the source's model?

Tokens can accumulate up to bucket capacity.

Then several requests can consume saved tokens quickly.

---

# 47. LEAKY BUCKET VS TOKEN BUCKET

| Leaky Bucket | Token Bucket |
|---|---|
| Finite request queue | Finite token bucket |
| Output constant rate | Requests require tokens |
| Smooths bursts | Can allow bursts when tokens accumulated |
| Source says packets may be discarded when full | Source says excess tokens may be discarded when token bucket full |

Derived from the notes' comparison.

---

# 48. MEMORY HOOK

LEAKY BUCKET

requests stored
↓
drip out steadily

TOKEN BUCKET

tokens stored
↓
requests spend tokens

---

# 49. SLIDING WINDOW

🟢 **MUST KNOW FROM SOURCE**

The notes define:

allow at most N requests
within a time interval

If count exceeds limit:

do not process extra requests

---

# 50. SLIDING-WINDOW EXAMPLE

Limit:

5 requests / 10 seconds

Requests:

t=1 R1
t=2 R2
t=3 R3
t=5 R4
t=6 R5
t=7 R6

Within the relevant interval, once the allowed number is reached:

R6
→ not processed

This example only illustrates the source's rule.

---

# 51. SOURCE ADVANTAGE — SLIDING WINDOW

The notes say:

simple to implement

---

# 52. SOURCE DISADVANTAGE #1 — MEMORY

The notes' specific sliding-window representation keeps request information in memory.

If window size is:

N

then it may need to retain up to:

N requests

---

# 53. SOURCE DISADVANTAGE #2 — CLEANUP

As time moves forward, old requests become irrelevant.

The source says each new request may require finding entries that are no longer valid, potentially checking up to N slots.

The notes call this:

Garbage collection

in their sliding-window discussion.

---

# 54. TIMER WHEEL

**🔵 SOURCE-SPECIFIC DEEP DIVE**

The notes describe timer wheel as:

variation of token bucket

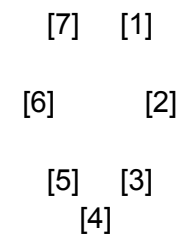
with a fixed number of tokens per time slot and use it to make timeout/expiration management easier.

---

# 55. TIMER-WHEEL SOURCE INTUITION

Imagine a circular set of time buckets:

[0]




A pointer advances with time.

Requests are assigned to time slots.

The notes emphasize:

clock movement
→ handles token/request expiry

---

# 56. TIMER-WHEEL SOURCE FORMULA

The notes give:

Number of buckets
=
incoming request timeout

with buckets numbered:

0 ... Timeout - 1

and requests placed using:

Time % NumberOfBuckets

---

# 57. TIMER-WHEEL TIMEOUT BEHAVIOR

According to the notes:

request remains in slot
↓
timeout reached
↓
wheel evicts it
↓
server never processes it

If server consumes it before timeout:

request removed from slot

so it won't later be expired.

---

# 58. TIMER-WHEEL CONCURRENCY ISSUE

The source explicitly asks what happens when:

server pulls request

at the same time that:

wheel expires request

and suggests locking as one possible way to avoid ambiguity.

This is a source-specific deep dive rather than core rate-limiter knowledge.

---

# 59. TIMER WHEEL VS CONSISTENT HASHING

The notes explicitly separate these:

Consistent Hashing
→ reduce data migration in caches

Timer Wheel
→ find expired requests efficiently

Good interview distinction because both can appear in the same HLD.

---

# 60. FAILED REQUESTS — DEAD LETTER QUEUE

The notes say failed requests can be placed into a:

Dead Letter Queue

for:

diagnosis
auditing

Alternative mentioned in the source:

drop/ignore and expect client retry later

when load is more manageable.

---

# 61. BAD ACTORS

The later rate-limiting section defines bad actors as:

malicious users/bots

or

faulty clients/nodes

that produce excessive requests and overload resources.

The source wants well-behaved traffic protected from such flooding.

---

# 62. INTERNAL SERVICE BUG CAN BE A BAD ACTOR TOO

Important source point:

> A bad actor is not necessarily malicious.

It can be:

bug in an internal service
↓
massive burst of requests

This is very relevant to microservices.

---

# 63. EXPONENTIAL BACKOFF — SOURCE MENTION

The notes say:

after failure/retry

increase time interval
between successive attempts

and after some number of retries:

stop retrying

This is the extent of exponential-backoff detail in this source section.

---

# 64. HOW DOES A SERVICE KNOW IT IS OVERLOADED?

The source gives three indicators:

1. Average response time

2. Age of queued requests

3. Dead-letter queue growth

This is an excellent source-derived interview checklist.

---

# 65. SIGNAL #1 — AVERAGE RESPONSE TIME

If:

average response time
keeps increasing

the notes say either:

service is malfunctioning

or

load is beyond processing capacity

---

# 66. SIGNAL #2 — AGE OF QUEUED REQUESTS

Example from source:

normal processing time ≤ 1 sec

but requests are waiting:

> 2 sec

That suggests:

incoming rate
>>
processing rate

---

# 67. SIGNAL #3 — DEAD LETTER QUEUE

If the service cannot process requests and sends them to a DLQ, then:

DLQ keeps growing

is a signal that the service is failing to keep up or successfully process traffic.

---

# 68. HEAD-OF-LINE-LIKE QUEUE PROBLEM IN THE SOURCE

The notes then describe:

Queue:
R1
R2
R3
R4

where:

R1 = bad request

and it can block the others.

Their proposed solution:

partition the queue.

---

# 69. SOURCE QUEUE-PARTITIONING SOLUTION

Split one queue into:

Q0
Q1
Q2
...

Qx-1

Then route:

q_num = h(request_id) % x

Now one bad request affects only the queue/partition containing it rather than necessarily stopping all traffic.

---

# 70. SOURCE LIMITATION OF QUEUE PARTITIONING

The source itself admits:

worst case
→ every queue contains one bad request

so all queues could still become blocked.

This is good reasoning:

> Partitioning reduces blast radius; it does not mathematically eliminate all failure.

---

# 71. REQUEST COLLAPSING

🟢 **HIGH ROI**

The notes define request collapsing as:

> combining multiple requests for the same object into one request to origin and using the resulting response for the waiting requests.

---

# 72. WITHOUT REQUEST COLLAPSING

Request A ——> Origin
Request B ——> Origin
Request C ——> Origin
Request D ——> Origin

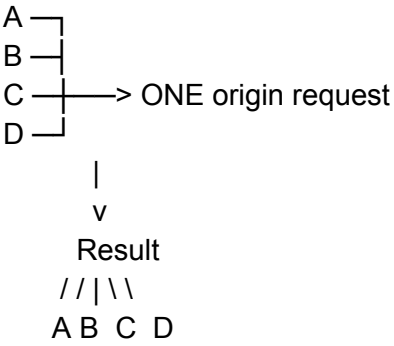
All ask for:

same object

Wasteful.

---

# 73. WITH REQUEST COLLAPSING




This reduces concurrent backend work.

---

# 74. SOURCE MOTIVATION — CACHE EXPIRY

The notes specifically give:

very popular cached object expires
↓
many requests miss simultaneously
↓
origin gets flood

Request collapsing allows one request to fetch it and satisfy the others.

---

# 75. COLLAPSING REDUCES RESOURCE USE

The source says it can reduce:

threads

network connections

origin traffic

when many concurrent calls can be grouped.

---

# 76. REQUEST COLLAPSING COST

The notes explicitly warn:

collapsing waits for a batch window
↓
extra latency

Maximum extra cost in their model:

batch-window size

---

# 77. WHEN REQUEST COLLAPSING IS NOT WORTH IT

The source says if concurrency is only:

1 or 2 requests

there may be little benefit.

Instead, callers pay the batching delay for almost no reduction in backend traffic.

---

# 78. WHEN REQUEST COLLAPSING IS VALUABLE

If one command receives:

dozens
or
hundreds

of concurrent equivalent calls, then the reduction in:

threads
connections
backend calls

can outweigh batching latency.

---

# 79. THUNDERING HERD

🟢🟢🟢

The notes define the general problem as:

many processes/threads
waiting for event
↓
event occurs
↓
everyone wakes
↓
everyone competes for resource

---

# 80. CACHE THUNDERING HERD

The source's live-video example:

many viewers arrive
↓
edge cache miss
↓
huge number of requests
↓
origin/cache-processing servers

which can overwhelm the origin.

This was already introduced in Part 6; here it belongs as a **resilience pattern**.

---

# 81. SOURCE SOLUTION — ONE REQUEST THROUGH

The notes' live-video solution:

many requests
↓
queue at edge
↓
ONE request goes to origin
↓
content returned/cached
↓
serve waiting requests

That is request collapsing in action.

---

# 82. PROMISE-BASED COLLAPSING

The notes describe another form:

first miss
  ↓
create Promise
  ↓
cache Promise
  ↓
one backend request

Later callers:

find same Promise
      ↓
wait for it

instead of each performing another backend load.

---

# 83. THE BIG RESILIENCE CHAIN

This chapter is easier if you see the full progression:

MICROSERVICES
    ↓
MORE NETWORK DEPENDENCIES
    ↓
DEPENDENCY CAN FAIL
    ↓
CIRCUIT BREAKER

TRAFFIC CAN SPIKE
    ↓
LOAD BALANCER ONLY SPREADS IT
    ↓
RATE LIMITER

BURSTS NEED SMOOTHING
    ↓
LEAKY / TOKEN BUCKET

REQUEST RATE OVER WINDOW
    ↓
SLIDING WINDOW

EXPIRING REQUESTS
↓
TIMER WHEEL

SAME BACKEND WORK REPEATED
↓
REQUEST COLLAPSING

MANY CALLERS WAKE/MISS TOGETHER
↓
THUNDERING HERD

---

# 84. LOAD BALANCER VS RATE LIMITER VS CIRCUIT BREAKER

🟢🟢🟢 **MEMORIZE THIS**

LOAD BALANCER
"Where should this request go?"

RATE LIMITER
"Should this request be admitted?"

CIRCUIT BREAKER
"Should we even call this failing dependency?"

This distinction can answer a surprising number of system-design follow-ups.

---

# 85. REQUEST COLLAPSING VS RATE LIMITING

RATE LIMITING

too many requests
↓
restrict/reject some

REQUEST COLLAPSING

many equivalent requests
↓
combine into one backend call

The source treats both as ways to protect downstream systems, but through different mechanisms.

---

# 86. CIRCUIT BREAKER VS RATE LIMITER

CIRCUIT BREAKER

trigger:
dependency failure

RATE LIMITER

trigger:
too much traffic

Circuit breaker in the notes watches failures and trips after a threshold. Rate limiter enforces request-volume limits.

---

# 87. 8-YOE MICROSERVICES INTERVIEW FLOOR

You should answer these without hesitation:

**1. Monolith vs microservices?**

**2. What extra infrastructure do microservices require?**

**3. Why do microservices need contracts?**

**4. Why service discovery?**

**5. Why do network calls increase latency?**

**6. Why centralized logging?**

**7. What is a correlation ID?**

**8. How do you choose a service boundary?**

**9. When should code remain a library instead of a service?**

**10. Why is a shared DB/table a coupling concern in the source?**

**11. What does “single source of truth” mean?**

**12. Why might a small team prefer fewer services?**

# 88. 8-YOE RESILIENCE INTERVIEW FLOOR

**13. What is circuit breaker?**

**14. What triggers it according to the notes?**

**15. Why fail fast?**

**16. Load balancer vs rate limiter?**

**17. What is leaky bucket?**

**18. What is token bucket?**

**19. Token bucket vs leaky bucket?**

**20. What is sliding-window rate limiting?**

**21. What costs does the source identify for sliding window?**

**22. What is timer wheel in these notes?**

**23. What is request collapsing?**

**24. When is request collapsing useful?**

**25. What is its latency cost?**

**26. What is thundering herd?**

**27. How does the Promise approach reduce it?**

**28. How can you detect service overload according to the source?**

---

# 89. RAPID-FIRE — MONOLITH VS MICROSERVICES

> “The notes frame a monolith as keeping code and expectations together, while microservices split capabilities into self-contained services that can use different technology stacks. The benefit comes with infrastructure overhead: contracts, service discovery, deployments, network latency and centralized logging all become important.”

---

# 90. RAPID-FIRE — HOW DO YOU CHOOSE A MICROSERVICE BOUNDARY?

“The source asks whether the functionality is clearly differentiated, independently maintainable and meaningfully reusable. If there isn't a clear independent-service justification, it suggests considering a library or common project rather than another network service.”

---

# 91. RAPID-FIRE — CIRCUIT BREAKER

“A circuit breaker wraps a dependency call and monitors failures. Once failures cross a threshold, the breaker trips and subsequent calls fail immediately without invoking the protected dependency.”

---

# 92. RAPID-FIRE — WHY RATE LIMIT?

“A load balancer can spread an incoming spike but cannot make unlimited traffic safe. The source uses a rate limiter to reject requests beyond a configured rate so backend servers are protected from overload.”

---

# 93. RAPID-FIRE — LEAKY BUCKET

“Requests enter a finite bucket or queue and leave at a constant rate, converting bursty input into a smoother output rate.”

---

# 94. RAPID-FIRE — TOKEN BUCKET

“Tokens are generated periodically up to a maximum bucket capacity. A request can proceed only when it consumes a token. The notes highlight that accumulated tokens allow bursts.”

---

# 95. RAPID-FIRE — SLIDING WINDOW

“Allow only a configured number of requests within a moving time interval. The source calls it simple but notes memory and expired-request-cleanup costs.”

---

# 96. RAPID-FIRE — REQUEST COLLAPSING

> “Combine concurrent requests for the same object into one origin request and share the returned result. It reduces backend calls, threads and network connections, at the cost of waiting for the batching window.”

---

# 97. RAPID-FIRE — THUNDERING HERD

> “Many processes or requests become active together and compete for the same constrained resource. In the cache example, many simultaneous misses can flood the origin, so the notes queue/collapse them behind one backend fetch.”

---

# 98. RAPID-FIRE — HOW DO YOU KNOW A SERVICE IS OVERLOADED?

Source answer:

> “Watch increasing average response time, increasing age of queued requests, and growth in the dead-letter queue.”

---

# 99. COMMON BAD ANSWERS

### ❌ “Microservices are automatically more scalable.”

The notes spend substantial space on the operational overhead they create.

---

### ❌ “Every independent class/module should become a microservice.”

The source explicitly says some functionality should remain a library/common project when a separate service is not justified.

---

### ❌ “Load balancer protects you from unlimited traffic.”

No.

The source specifically explains why a spike can still overload all balanced servers.

---

### ❌ “Circuit breaker and rate limiter are the same.”

No.

breaker
→ dependency failure

limiter
→ incoming traffic volume

---

### ❌ “Leaky bucket and token bucket handle bursts identically.”

The notes explicitly contrast them: leaky bucket outputs at a constant rate while token bucket can allow accumulated-token bursts.

---

### ❌ “Request collapsing is always faster.”

The source explicitly says it introduces batching-window latency and can be counterproductive when concurrency is low.

---

# 100. ONE-PAGE MEMORY SHEET

MONOLITH
========

one application
fewer service boundaries

SOURCE ARGUMENTS
simpler maintenance for small teams
fewer integration points

MICROSERVICES
=============

self-contained services

BENEFITS IN NOTES
technology freedom

independent boundaries
polyglot persistence

COSTS
contracts
discovery
CI/CD
network calls
logging
infra cost

SERVICE BOUNDARY

clear business function?
independent maintenance?
reusable?

NO?
→ maybe library

CORRELATION ID

one logical request
↓
same ID across services
↓
centralized debugging

CIRCUIT BREAKER
==========

dependency calls fail
↓
failure threshold
↓
TRIP
↓
future calls fail fast

LOAD BALANCER
==========

WHERE should traffic go?

RATE LIMITER
==========

SHOULD traffic be admitted?

LEAKY BUCKET
==========

bursty input
↓
finite queue
↓
constant output

TOKEN BUCKET
==========

tokens generated
↓
request spends token

accumulated tokens
→ bursts possible

SLIDING WINDOW
==========

N requests
within interval

SOURCE COSTS
memory
cleanup

TIMER WHEEL
==========

time buckets
↓
request expiry

DEAD LETTER QUEUE
==========

failed requests
↓
diagnosis / auditing

OVERLOAD SIGNALS

==========

response time ↑
request age ↑
DLQ size ↑

REQUEST COLLAPSING
==========

many equivalent requests
↓
ONE backend request
↓
share result

COST
batch latency

THUNDERING HERD
==========

many callers wake/miss
at once
↓
backend flood

PROMISE PATTERN
==========

first caller
→ creates in-flight Promise

others
→ wait on same Promise

---

# 101. THE 60-SECOND SENIOR ANSWER

If asked:

> **“What new problems do microservices introduce, and how do you make them resilient?”**

A source-grounded answer is:

> “The notes treat microservices as independently maintained service boundaries, but that independence comes with infrastructure cost. Services need explicit contracts, service discovery, repeated deployments, centralized logging and correlation IDs, and every remote

call adds latency and failure potential. Service boundaries should therefore be justified by clearly different business responsibilities or independent maintainability rather than splitting everything. Once calls cross the network, I need failure and overload protection. A circuit breaker stops repeatedly calling a dependency once failures cross a threshold. A load balancer distributes traffic, while a rate limiter restricts traffic when the incoming rate itself is unsafe. The notes then use leaky bucket, token bucket and sliding window as limiting approaches. For many equivalent concurrent requests, request collapsing can reduce downstream work, and it also helps prevent a thundering herd where large numbers of requests flood the same backend at once.”

If you can expand each sentence, **Part 9 is learned**.

---

# 102. PART 9 MEMORY MAP

MICROSERVICES & RESILIENCE
│
├── Monolith
│ └── Fewer distributed boundaries
│
├── Microservices
│ ├── Contracts
│ ├── Service Discovery
│ ├── CI/CD
│ ├── Network Calls
│ ├── Correlation IDs
│ ├── Central Logs
│ ├── Service Boundaries
│ ├── DB Ownership
│ └── Polyglot Persistence
│
├── Dependency Failure
│ └── Circuit Breaker
│
├── Overload
│ ├── Rate Limiter
│ ├── Leaky Bucket
│ ├── Token Bucket
│ └── Sliding Window
│
├── Expiration
│ └── Timer Wheel
│
├── Failed Work
│ └── Dead Letter Queue
│
├── Overload Detection
│ ├── Response Time
│ ├── Request Age
│ └── DLQ Growth

```
│
└── Duplicate Concurrent Work
    ├── Request Collapsing
    ├── Thundering Herd
    └── Promise
```

---

# CJ SYSTEM DESIGN INTERVIEW HANDBOOK

## PART 10 — BATCH vs STREAM PROCESSING, ETL, TELEMETRY, AVAILABILITY & LEADER ELECTION

**Target:** Senior Software Engineer / Backend Engineer — ~8 YOE
**Goal:** Understand **when data should be processed later vs continuously, how operational information is collected, how availability is expressed, and when distributed nodes need one coordinator.**

> **Priority legend**
> 🟢 **MUST KNOW** — senior interview floor
> 🟡 **SHOULD KNOW** — explain confidently
> 🔵 **SOURCE-SPECIFIC / DEEP DIVE**
> ⚠️ **SOURCE GAP** — heading/term exists in the notes, but the notes do not explain it enough to safely expand from the source alone.

The source progression is:

```
Batch Processing
      vs
Stream Processing
      ↓
ETL
      ↓
Monitoring & Telemetry
      ↓
Availability / Five Nines
      ↓
SLI < SLO < SLA
      ↓
Leader Election
```

---

# 1. BATCH PROCESSING — FIRST PRINCIPLES

🟢 **MUST KNOW**

The notes define batch processing as processing **blocks of data that have already accumulated over a period of time**.

Mental model:

Data arrives
Data arrives
Data arrives
Data arrives
↓
stored
stored
stored
↓
WAIT
↓
process the whole batch

---

# 2. SOURCE EXAMPLES OF BATCH WORK

The notes list examples such as:

Line-item invoices

Payroll processes

Transactions accumulated
over a week

The important common property:

> **The result does not need to be produced immediately for every individual event.**

---

# 3. BATCH PROCESSING EXAMPLE

Source-style example:

Monday transactions
Tuesday transactions
Wednesday transactions
...
Sunday transactions
↓
stored
↓
process together

The notes specifically use a financial firm processing a large stored set of transactions later for analysis.

---

# 4. WHEN DOES BATCH PROCESSING FIT?

The source says batch works well when:

Real-time result
is NOT required

and

processing large volumes
is more important
than immediate analytics

---

# 5. SOURCE E-COMMERCE BATCH EXAMPLE

The notes describe:

all sales records for one day
↓
batch processing
↓
statistics

such as:

users per country

average amount spent

---

# 6. SOURCE'S HADOOP / MAPREDUCE STATEMENT

🔵 **SOURCE-SPECIFIC**

The notes state:

> **Hadoop MapReduce is the best framework for processing data in batches.**

That is the source's wording.

For this source-grounded handbook, remember:

Batch Processing
↓
source associates it with
Hadoop MapReduce

rather than silently replacing that claim with outside material.

---

# 7. STREAM PROCESSING — FIRST PRINCIPLES

🟢🟢🟢

The source contrasts batch processing with processing that needs access to data:

in real time

or nearly immediately as events occur.

Mental model:

Event
↓
process

Event
↓
process

Event
↓
process

instead of waiting to accumulate a large batch.

---

# 8. BATCH VS STREAM — VISUAL

BATCH

E1 E2 E3 E4 E5
\ | | | /
STORE
↓
PROCESS

STREAM

E1 → PROCESS
E2 → PROCESS
E3 → PROCESS
E4 → PROCESS

---

# 9. SOURCE STREAM-PROCESSING USE CASES

The notes explicitly mention:

Cybersecurity

Fraud detection

because risks may need to be detected immediately.

---

# 10. FRAUD EXAMPLE

Suppose:

Card transaction
₹2,00,000
↓
fraud analysis

If you wait until:

end of day

the detection may be too late for the source's use-case framing.

Therefore:

event arrives
↓
analyze now

---

# 11. SOURCE EXPLANATION FOR STREAM SPEED

🔵

The notes say stream processing is fast because it:

analyzes the data before it hits disk.

Keep this as the source's explanation.

---

# 12. BATCH VS STREAM — CORE QUESTION

In an interview, according to this section's framing, first ask:

CAN THIS RESULT WAIT?

If:

YES
→ Batch may fit

If:

NO
→ Stream may fit

---

# 13. BATCH VS STREAM — SOURCE COMPARISON

| Batch | Stream |
|---|---|
| Stored data | Continuously arriving events |
| Process later | Process near real time |
| Large blocks | Individual/continuous events |
| Good when result can wait | Good when fast reaction matters |
| Source examples: payroll, invoices, analysis | Source examples: fraud, cybersecurity |

Supported by the notes' batch/stream section.

---

# 14. SOURCE-CLAIMED BATCH ADVANTAGE #1 — SMALL MACHINES

The notes state:

Batch processing
can run on small machines

Keep this as a source-specific claimed advantage.

---

# 15. SOURCE-CLAIMED BATCH ADVANTAGE #2 — COMPLEX OPERATIONS

The notes say batch processing can perform:

complex operations

with higher precision than streams.

Again, preserve this as the source's comparison.

---

# 16. SOURCE-CLAIMED BATCH ADVANTAGE #3 — COST

The source says batch can be:

cheap

because it may use commodity hardware and run during non-peak hours.

---

# 17. SOURCE-CLAIMED BATCH ADVANTAGE #4 — RECOMPUTATION

This is one of the most useful source points.

The notes say:

input data can be versioned

↓
batch can be recomputed
↓
process becomes repeatable

---

# 18. BATCH REPEATABILITY — MENTAL MODEL

INPUT V1
↓
PROCESS
↓
RESULT V1

Need to rerun?

INPUT V1
↓
PROCESS AGAIN
↓
RESULT

The source specifically emphasizes versioned input and repeatability.

---

# 19. BATCH VS STREAM — MEMORY HOOK

BATCH
SAVE → WAIT → PROCESS

STREAM
ARRIVE → PROCESS

---

# 20. 30-SECOND INTERVIEW ANSWER

> “The notes define batch processing as operating on data accumulated and stored over time, which suits workloads where the result can wait, such as payroll or end-of-day analytics. Stream processing instead analyzes continuously arriving data in near real time, which the source associates with use cases like fraud detection and cybersecurity.”

---

# 21. ETL

🟢 **MUST KNOW FROM SOURCE**

The notes define:

E = Extract

T = Transform

L = Load

---

# 22. ETL FLOW

SOURCE DATA
↓
EXTRACT
↓
TRANSFORM
↓
LOAD
↓
DATA PLATFORM / TARGET

---

# 23. EXTRACT

Within the acronym:

EXTRACT

means obtaining the input data.

The notes do not provide a deeper standalone explanation beyond naming the ETL stages.

⚠️ So keep it at this level for this source-only pass.

---

# 24. TRANSFORM

TRANSFORM

is the processing stage in the ETL acronym.

Again, the source does not enumerate specific transformation operations here.

---

# 25. LOAD

LOAD

is the final named step.

The source connects processed data to a shared data platform/data lake context used by backend teams/services.

---

# 26. PURPOSE OF ETL — SOURCE ANSWER

The exact conceptual purpose in the notes is:

> provide a data platform and engineering functionality that multiple backend teams can use.

---

# 27. ETL SOURCE ARCHITECTURE

Data Sources
↓
Extract
↓
Transform
↓
Load
↓
Data Lake / Platform
↓
Multiple Services / Teams

The source says information obtained after processing data from the data lake can be used by multiple services.

---

# 28. ETL MEMORY HOOK

GET IT

CHANGE IT

PUT IT

maps to:

Extract

Transform

Load

---

# 29. BATCH / STREAM / ETL — DON'T MIX THEM

The source places these concepts together, but they answer different questions:

BATCH vs STREAM

WHEN / HOW CONTINUOUSLY
should processing occur?

ETL

WHAT PIPELINE STAGES
does data pass through?

That distinction is a reorganization of the source's structure.

---

# 30. MONITORING & TELEMETRY

🟢🟢

The source uses a **motor-racing analogy**.

Fans see measurements such as:

top speed

G-forces

lap times

race position

The displayed measurements are described as:

telemetry

while the overall process of installing sensors, collecting information and exposing selected metrics is described as:

monitoring

---

# 31. TELEMETRY — SOURCE DEFINITION

The notes define telemetry as:

automatic recording

+

transmission of data

from remote/inaccessible sources

to another system

for monitoring and analysis

---

# 32. TELEMETRY — SIMPLE MENTAL MODEL

RUNNING SYSTEM
↓
measurements
↓
transmitted elsewhere
↓
analysis / monitoring

---

# 33. WHY “TELEMETRY”?

The notes derive the word from:

tele
→ remote

metron
→ measure

and connect this to remotely collected/transmitted operational data.

This is low interview ROI but present in the source.

---

# 34. TELEMETRY IN SOFTWARE

The source says telemetry can reveal:

Which features are used

Bugs / issues

Performance information

User engagement

Crash context

---

# 35. SOURCE EXAMPLES OF TELEMETRY QUESTIONS

The notes explicitly ask questions like:

Are customers using expected features?

How frequently do users engage?

For how long?

Which settings do they select?

What happens around crashes?

---

# 36. TELEMETRY VS LOGGING — SOURCE FRAMING

🟡

The notes distinguish them approximately like this:

LOGGING
→ diagnose errors and code flow
→ internal structure

TELEMETRY
→ automatically collect information
from real-world product use
→ performance / usage insight

Preserve this as the source's framing.

---

# 37. SOURCE'S LOGGING MENTAL MODEL

Something goes wrong
↓
developer examines logs
↓
diagnose error / code path

---

# 38. SOURCE'S TELEMETRY MENTAL MODEL

Users operate product
↓
data automatically collected
↓
transmitted
↓
analytics
↓
product/performance insight

The source explicitly links telemetry to using V1 data to prioritize improvements for later releases.

---

# 39. MONITORING VS TELEMETRY — MEMORY HOOK

Using the source's racing analogy:

TELEMETRY
= measurements/data

MONITORING
= process/system that observes
those measurements

---

# 40. WHY MONITORING MATTERS IN SYSTEM DESIGN

The source connects telemetry with understanding whether software is:

working correctly

performing well

used as expected

crashing under particular conditions

That is the operational side of your HLD.

---

# 41. AVAILABILITY

🟢🟢🟢

The notes introduce availability through:

Five Nines
99.999%

and convert percentages into approximate yearly downtime.

---

# 42. FIVE NINES

The source states:

99.999%
≈ 5 minutes 15 seconds
of downtime per year

---

# 43. FOUR NINES

The source gives:

99.99%
≈ 52 minutes 36 seconds
downtime/year

---

# 44. TWO NINES

The source gives:

99%

≈ 15 hours 40 minutes
downtime/year

---

# 45. SOURCE AVAILABILITY TABLE

| Availability | Downtime/year in notes |
|---|---|
| 99.999% | ~5m 15s |
| 99.99% | ~52m 36s |
| 99% | ~15h 40m |

---

# 46. WHY AN EXTRA NINE MATTERS

The source's numbers make the point visually:

99%
→ hours

99.99%
→ tens of minutes

99.999%
→ only minutes

So:

> “just one more nine”

can imply a dramatically tighter downtime budget.

---

# 47. SOURCE DOWNTIME-COST EXAMPLE

🔵 **SOURCE-SPECIFIC**

The notes cite a Ponemon Institute figure of:

$336,000 average cost
per hour of downtime

and use it to illustrate the business consequences of availability loss.

Treat this as a sourced example from the notes, not a timeless universal cost.

---

# 48. AVAILABILITY IS A BUSINESS CONSTRAINT TOO

The source directly connects downtime to:

customer satisfaction

trust

financial impact

So availability is not merely an infrastructure percentage.

---

# 49. SLI < SLO < SLA

⚠️ **SOURCE GAP**

The notes contain only:

SLI < SLO < SLA

at this point.

They do **not define**:

SLI

SLO

SLA

in the retrieved source block.

Therefore, in a source-faithful handbook:

SLI < SLO < SLA
→ TERM PRESENT

Definitions
→ NOT PROVIDED HERE

Do not silently fill those definitions from outside knowledge in this source-only pass.

---

# 50. WHAT YOU SHOULD MEMORIZE FROM THE SOURCE FOR SLI/SLO/SLA

For now:

Availability section
↓
Five Nines
↓
SLI < SLO < SLA

That is all this source supports.

If we later do an **expanded/verified handbook**, this is a natural place to add outside material explicitly labeled as such.

---

# 51. LEADER ELECTION — FIRST PRINCIPLES

🟢🟢🟢 **MUST KNOW**

The notes start with an important contrast:

Sometimes horizontal scaling means:

Node A handles some requests

Node B handles some requests

Node C handles some requests

and no special leader is necessary.

But other tasks require:

more precise coordination

so one node is selected as leader.

---

# 52. LEADER ELECTION — SOURCE DEFINITION

The notes define a leader-election algorithm as allowing a cluster with no leader to:

communicate

↓
choose exactly one member
↓
make it leader

---

# 53. WHEN DOES ELECTION RUN?

According to the source:

cluster starts

or:

current leader goes down

---

# 54. WHAT SPECIAL POWER DOES THE LEADER HAVE?

The source gives examples:

Assign work

Modify a piece of data

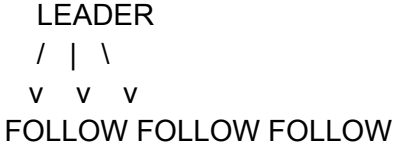
Handle requests

Coordinate followers

The exact responsibility depends on the distributed system.

---

# 55. LEADER-FOLLOWER MENTAL MODEL




Leader may coordinate work that otherwise requires more distributed agreement.

---

# 56. WHY ELECT A LEADER?

The source says leader election can help with:

Efficiency

Reduced coordination

Simpler architecture

Reduced operational coordination

---

# 57. BUT LEADER ELECTION ADDS COMPLEXITY

The same source immediately warns of:

new failure modes

scaling bottlenecks

harder correctness reasoning

So:

> **Leader election is not automatically a simplification overall.**

---

# 58. SOURCE LEADER-ELECTION OPTIONS

The notes mention several possibilities:

Paxos

Apache ZooKeeper

Custom hardware

Leases

The source does not explain Paxos here.

Do not invent a Paxos chapter from this section.

---

# 59. LEASE-BASED LEADER ELECTION

The notes provide more detail on leases than on the other approaches.

Source model:

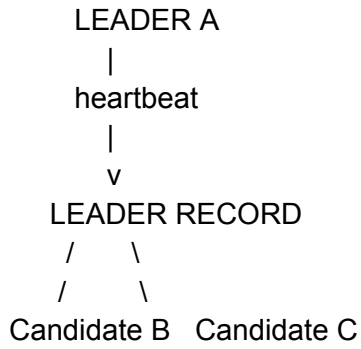
database
stores current leader

leader
heartbeats periodically

heartbeat stops
↓
other candidates
can try to take over

---

# 60. LEASE VISUAL




As long as A continues its expected heartbeat:

A remains leader

If heartbeat stops long enough:

B/C may attempt takeover

This follows the mechanism described by the source.

---

# 61. SOURCE AMAZON CLAIM

🔵

The notes state that leases are the most widely used leader-election mechanism at Amazon.

Keep that as a source-specific statement.

---

# 62. LEADER FAILURE

Leader alive
↓
heartbeats

Leader fails
↓
heartbeat stops
↓
other candidates
attempt takeover

This is the core failure-recovery path described in the notes.

---

# 63. SPLIT BRAIN

🟢 **VERY IMPORTANT**

The source warns that poor leader-election implementation can create:

Leader A thinks:
"I am leader"

Leader B thinks:
"I am leader"

simultaneously.

That is the source's:

**split brain** example.

---

# 64. SPLIT-BRAIN VISUAL

NETWORK PROBLEM

LEADER A LEADER B
| |
accepts control accepts control

BOTH THINK
THEY ARE LEADER

This overlaps conceptually with the split-brain discussion from Part 5.

---

# 65. OTHER LEADER-ELECTION FAILURE

The notes identify the opposite situation too:

NO leader elected

which can leave:

cluster unable to coordinate

So failures include:

TOO MANY LEADERS

or

NO LEADER

---

# 66. WHEN DOES THE SOURCE SAY TO USE LEADER ELECTION?

The notes say it should be used when there is a need for:

complex coordination

or

strong consistency

and alternatives do not fit.

---

# 67. LEADER AS BOTTLENECK

🟢

The source directly warns:

one leader
↓
single node
↓
can become bottleneck

---

# 68. LEADER AS TEMPORARY SPOF

The source also describes the leader as a possible:

temporary single point of failure

That is why election/failover logic matters.

---

# 69. BAD LEADER DECISIONS

Interesting source point:

Even if the leader is technically alive, it can make:

bad decisions

and followers may simply execute those assignments, harming the cluster.

So leader failure is not only:

process crashed

but can also involve:

incorrect coordination behavior

in the source's framing.

---

# 70. LEADER ELECTION — 30 SECOND ANSWER

> “Leader election is used when a cluster needs one member to have special coordination responsibility. The nodes select exactly one leader, typically when the cluster starts or the previous leader fails. The source describes a lease model where the current leader heartbeats against a stored leader record, and if the heartbeat stops, another candidate can take over. The costs are complexity, possible split brain, periods with no leader, and the leader itself becoming a bottleneck.”

---

# 71. BATCH / STREAM / LEADER ELECTION — WHY THESE ARE IN ONE OPS BLOCK

They solve different problems:

BATCH / STREAM
→ WHEN data is processed

ETL
→ HOW data moves through pipeline stages

TELEMETRY
→ WHAT operational/product data is collected

AVAILABILITY
→ HOW MUCH downtime is tolerated

LEADER ELECTION
→ WHO coordinates distributed nodes

---

# 72. 8-YOE INTERVIEW FLOOR — DATA PROCESSING

You should answer these cleanly:

**1. What is batch processing?**

**2. When would you use it?**

**3. What examples do the notes give?**

**4. What is stream processing?**

**5. Batch vs stream?**

**6. Why does fraud detection fit stream processing in the notes?**

**7. What batch advantages does the source list?**

**8. What does ETL stand for?**

**9. What purpose does the source give for ETL?**

---

# 73. 8-YOE INTERVIEW FLOOR — OPERATIONS

**10. What is telemetry?**

**11. Monitoring vs telemetry according to the notes?**

**12. Telemetry vs logging according to the notes?**

**13. What kinds of questions can telemetry answer?**

**14. What is five-nines availability?**

**15. Roughly how much yearly downtime is 99.999%?**

**16. How much is 99.99%?**

**17. What does the source say about SLI/SLO/SLA?**

Correct answer for #17:

> It only writes `SLI < SLO < SLA`; definitions are not provided in this source block.

---

# 74. 8-YOE INTERVIEW FLOOR — LEADER ELECTION

**18. What is leader election?**

**19. Why might a cluster need one leader?**

**20. When does election happen?**

**21. What responsibilities can a leader have?**

**22. What election mechanisms does the source name?**

**23. How does the source's lease mechanism work?**

**24. What is split brain?**

**25. What if no leader is elected?**

**26. Why can a leader become a bottleneck?**

---

# 75. RAPID-FIRE — BATCH PROCESSING

> “Batch processing operates on blocks of data that have accumulated and been stored over time. The notes use workloads such as payroll, invoices and end-of-day or weekly analysis where real-time results are unnecessary.”

---

# 76. RAPID-FIRE — STREAM PROCESSING

> “Stream processing handles continuously arriving data in near real time. The source highlights fraud detection and cybersecurity as examples because the system may need to react immediately.”

---

# 77. RAPID-FIRE — ETL

> “ETL stands for Extract, Transform and Load. The notes describe its purpose as providing a shared data platform and engineering capability whose processed information can be consumed by multiple backend teams or services.”

---

# 78. RAPID-FIRE — TELEMETRY

> “The source defines telemetry as automatically recording measurements or other data and transmitting them to another system for monitoring and analysis.”

---

# 79. RAPID-FIRE — FIVE NINES

> “In the notes, 99.999% availability corresponds to roughly five minutes and fifteen seconds of downtime per year.”

---

# 80. RAPID-FIRE — LEADER ELECTION

> “Leader election lets a cluster choose exactly one node with special coordination responsibilities. The source says election occurs when the cluster starts or when the current leader goes down.”

---

# 81. RAPID-FIRE — LEASE

> “In the lease mechanism described by the notes, a database stores the current leader and the leader periodically heartbeats. If it stops heartbeating for long enough, other candidates can attempt to take over.”

---

# 82. RAPID-FIRE — SPLIT BRAIN

> “Split brain is the source's failure case where two nodes both act as leader at the same time.”

---

# 83. COMMON BAD ANSWERS

### ❌ “Batch means slow and stream means fast.”

Too vague.

The source distinction is primarily:

accumulated stored data
vs
continuously processed real-time data

---

### ❌ “Stream processing is always better.”

The source explicitly lists several claimed benefits of batch processing, including repeatability, recomputation and non-peak processing.

---

### ❌ “Telemetry and logging are exactly the same.”

Not according to these notes.

The source distinguishes diagnostic/code-flow logging from broader automatically collected real-world telemetry.

---

### ❌ “99.99 and 99.999 are basically the same.”

The source's downtime numbers show a significant difference.

---

### ❌ “Every distributed cluster needs a leader.”

The source explicitly begins by saying some horizontally scaled systems can simply let nodes independently handle subsets of requests.

---

### ❌ “Leader election removes all coordination problems.”

The notes say it can itself introduce:

split brain

no leader

bottlenecks

new failure modes

---

# 84. ONE-PAGE MEMORY SHEET

DATA PROCESSING
==========

BATCH

store data
↓
accumulate
↓
process later

USE WHEN
real-time result not needed

SOURCE EXAMPLES
payroll
invoices
daily/weekly analytics

STREAM

event arrives
↓
process now

SOURCE EXAMPLES
fraud
cybersecurity

BATCH vs STREAM

BATCH
SAVE → WAIT → PROCESS

STREAM
ARRIVE → PROCESS

ETL
===

Extract
↓
Transform
↓
Load

PURPOSE IN NOTES
shared data platform
for teams/services

TELEMETRY
=========

measure
↓
record
↓
transmit
↓
analyze

SOURCE USES
feature usage
performance
bugs
crashes
engagement

LOGGING vs TELEMETRY
====================

Logging
→ errors / code flow

Telemetry
→ real-world usage
and operational data

AVAILABILITY
==========

99.999%
≈ 5m 15s/year

99.99%
≈ 52m 36s/year

99%
≈ 15h 40m/year

SLI < SLO < SLA

definitions:
NOT PROVIDED
in source section

LEADER ELECTION
==========

Cluster
↓
choose exactly ONE leader

WHY?

assign work
modify data
coordinate requests

SOURCE OPTIONS

Paxos
ZooKeeper
Custom hardware
Leases

LEASE

leader record
↓
heartbeat
↓
heartbeat stops

↓
candidate takeover

FAILURES

Split brain
→ 2 leaders

No leader
→ no coordination

Single leader
→ bottleneck / temporary SPOF

---

# 85. 60-SECOND SENIOR ANSWER

If asked:

> **“Explain batch vs stream processing and how you operate a distributed processing system.”**

A source-grounded answer:

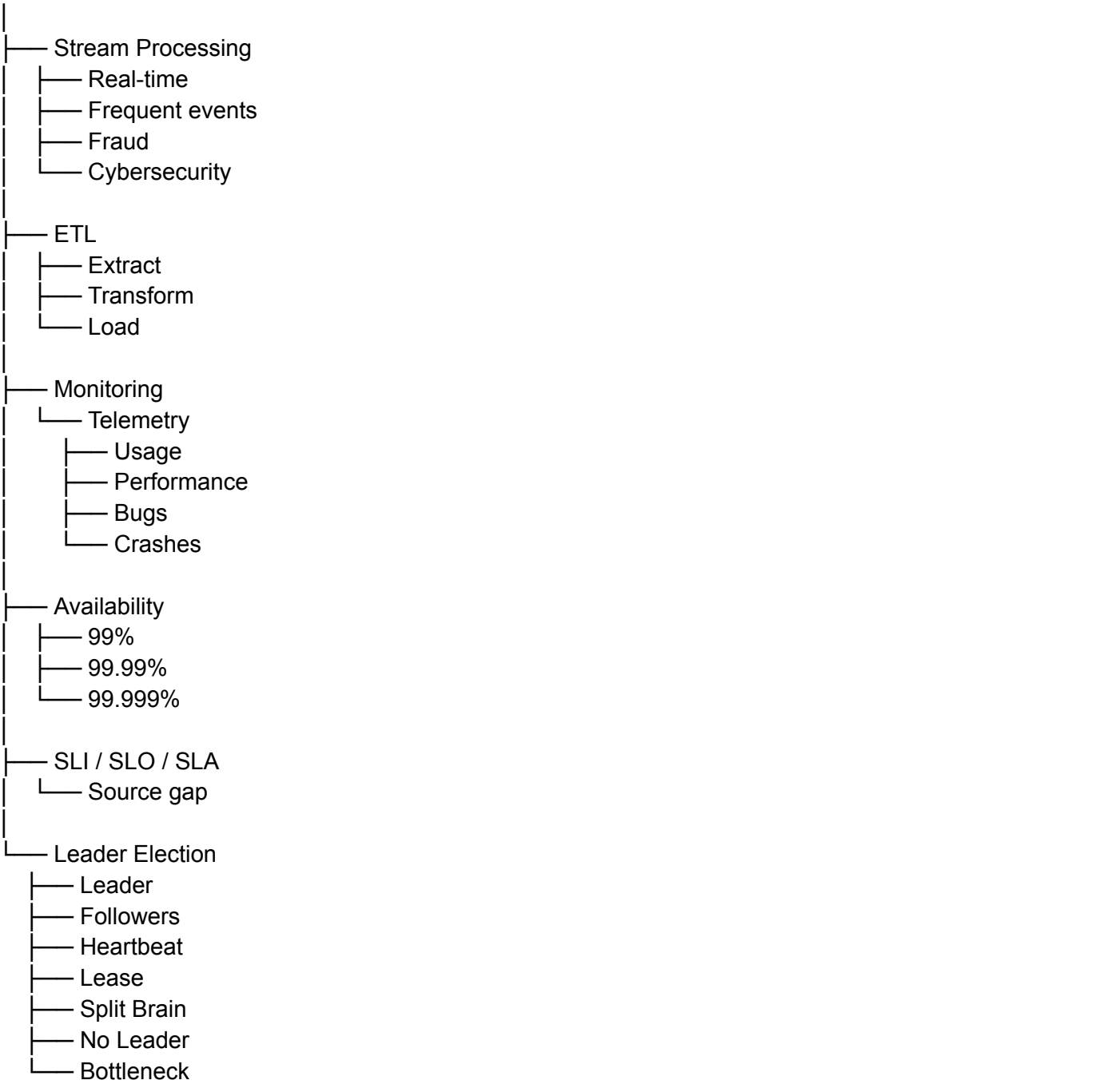
> “The notes distinguish batch processing as operating on data that has accumulated and been stored over time, while stream processing analyzes data continuously when low-latency reaction is important. Batch fits workloads such as payroll or end-of-day analytics, whereas the source uses fraud detection and cybersecurity as streaming examples. ETL organizes data movement as Extract, Transform and Load so processed information can be reused across teams and services. Operationally, telemetry automatically records and transmits measurements so the system can be monitored and analyzed. Availability targets can be expressed using nines—the source gives 99.999% as roughly five minutes of yearly downtime. In distributed clusters that need one coordinator, leader election chooses a single node with special responsibilities; the notes describe lease-based leadership using heartbeats, while warning about split brain, periods with no leader and the leader becoming a bottleneck.”

If you can expand every sentence, **Part 10 is learned**.

---

# 86. PART 10 MEMORY MAP

OPERATIONS & DATA PROCESSING
│
├── Batch Processing
│ ├── Stored data
│ ├── Process later
│ ├── Large volumes
│ └── Recomputable


Stream Processing
Real-time
Frequent events
Fraud
Cybersecurity
ETL
Extract
Transform
Load
Monitoring
Telemetry
Usage
Performance
Bugs
Crashes
Availability
99%
99.99%
99.999%
SLI / SLO / SLA
Source gap
Leader Election
Leader
Followers
Heartbeat
Lease
Split Brain
No Leader
Bottleneck

---

# CJ SYSTEM DESIGN INTERVIEW HANDBOOK

## PART 11 — CASE STUDY PLAYBOOK I: WHATSAPP, TINDER & GMAIL

**Target:** Senior Software Engineer / Backend Engineer — ~8 YOE
**Goal:** Stop learning components in isolation and start recognizing **why a particular component appears in an actual design**.

> **Priority legend**
> 🟢 **MUST KNOW** — reusable interview lesson
> 🟡 **SHOULD KNOW** — useful design reasoning
> 🔵 **SOURCE-SPECIFIC** — exact approach taken by these notes
> ⚠️ **SOURCE GAP / CORRECTION** — the uploaded notes do not support a fuller design, or their wording should not be generalized

The source begins its case-study block with:

WhatsApp
↓
Tinder
↓
Instagram
↓
Gmail
↓
Chess
↓
Calling App
↓
Live Video
↓
Analytics
↓
BookMyShow
↓
TinyURL

---

# 1. HOW TO STUDY CASE STUDIES

Do **not** memorize:

WhatsApp = Kafka
Tinder = sharding
Gmail = inverted index

Instead extract the recurring design question:

PROBLEM
↓
WHY naive design fails
↓
DESIGN DECISION
↓
TRADE-OFF
↓
REUSABLE PATTERN

Example:

Store connection mapping in Gateway?
↓
fast local access
BUT
gateway nodes duplicate state
↓
Session Service
↓
centralized ownership of mapping

That reasoning transfers to many systems.

---

# 2. CASE STUDY 1 — WHATSAPP / CHAT SYSTEM

The source focuses primarily on five design problems:

Connection/session mapping

Gateway responsibilities

Group routing

Reliable delivery / retries

Idempotency + ordering

---

# 3. CONNECTION MAPPING

A messaging system needs to know:

USER
↓

which live connection?

Conceptually:

userId
↓
connectionId

This lets the system route a message toward the currently connected user.

---

# 4. OPTION A — STORE MAPPING IN GATEWAY

The notes identify the advantage:

Gateway
already has mapping
↓
in-process/local access
↓
no extra network call

But gateways have limited memory and are themselves distributed.

---

# 5. PROBLEM — GATEWAY IS DISTRIBUTED

Suppose:

Gateway 1
Gateway 2
Gateway 3
Gateway 4

If each stores:

userId → connectionId

you create:

duplicate state
+
update fan-out

Every mapping change may need synchronization across gateway nodes. That duplication/fan-out concern is explicit in the notes.

---

# 6. SOURCE DECISION — SESSION SERVICE

The notes therefore conclude:

Gateway
↓
Session Service
↓
userId → connectionId

rather than placing the mapping independently inside every gateway.

---

# 7. TRADE-OFF

GATEWAY-LOCAL MAPPING

+ faster
- duplicated
- gateway memory
- update fan-out

SESSION SERVICE

+ dedicated state owner
+ avoids gateway duplication

- network call

That is a reusable design trade-off.

---

# 8. KEEP THE GATEWAY LIGHTWEIGHT

The notes explicitly say the gateway should not parse application messages because the goal is to:

minimize gateway memory use
+
maximize TCP connections

So a separate:

Message Parser

converts incoming raw/unparsed messages into usable application messages.

---

# 9. REUSABLE LESSON — DON'T BLOAT CONNECTION-TIER SERVICES

Source reasoning:

Gateway's important job
=
maintain many connections

Adding unrelated processing
=
more memory / responsibility

Therefore:

Gateway
→ connection handling

Parser / downstream service
→ application processing

This same separation appears later in the Gmail notes, where the source repeatedly warns against putting too many responsibilities into the gateway.

---

# 10. GROUP MESSAGING

For groups, the source needs a mapping:

groupId
  ↓
many userIds

This is described as a one-to-many relationship.

---

# 11. DISTRIBUTED GROUP-MESSAGING SERVICE

Suppose:

Group Service 1
Group Service 2
Group Service 3

The design needs to answer:

Which server owns/processes messages for Group X?

The source chooses:

consistent hashing(groupId)

to route that group to a server.

---

# 12. GROUP ROUTING FLOW

Message for Group 42
↓
hash(groupId)
↓
chosen Group Service node
↓
process members/messages

This connects directly to your Part 4 consistent-hashing chapter.

---

# 13. WHY USE THE GROUP ID AS ROUTING KEY?

According to the source design:

same groupId
↓
same routing decision

so related group work can be handled consistently by the selected node.

The notes describe this specifically as reducing redundancy in the distributed group-messaging service.

---

# 14. MESSAGE QUEUE FOR DELIVERY FAILURES

The notes introduce a message queue because delivery may fail.

Source flow:

Send message
↓
Message Queue
↓
attempt delivery
↓
failure?
↓
retry

If maximum retries are reached:

notify sender of failure

---

# 15. THREE GROUP-MESSAGING REQUIREMENTS

The source explicitly lists:

1. Retries

2. Idempotency

3. Ordering

These three are extremely reusable in messaging interviews.

---

# 16. RETRIES

The source assigns retry handling to the message queue:

temporary failure
↓
retry delivery

---

# 17. WHY RETRY CREATES A NEW PROBLEM

Suppose message M1 is delivered.

But acknowledgment/result is lost.

System retries:

M1
↓
M1 again

Now the receiver might process it twice.

That brings us to:

IDEMPOTENCY

---

# 18. SOURCE IDEMPOTENCY APPROACH

The notes assign every message a:

unique message ID

and use an at-least-once style queue interaction.

If a consumer has already seen that ID:

ignore duplicate

---

# 19. DEDUPLICATION MENTAL MODEL

Receive:
messageId = ABC

Seen ABC before?
|
YES ——→ ignore duplicate
|
NO
↓
process
↓
remember ABC

---

# 20. ⚠️ SOURCE CLARIFICATION — IDEMPOTENCY

The notes phrase idempotency as:

> “each message should be sent only once”

but the mechanism they actually describe is:

at-least-once delivery
+
unique message ID
+
duplicate detection

For interview memorization, keep the mechanism rather than the literal “sent once” wording.

---

# 21. ORDERING

The notes say messages within a group should remain ordered by timestamp.

Source strategy:

one particular group
↓
one particular thread
from thread pool

so its messages are processed through the same execution path.

---

# 22. ORDERING LESSON

The design principle is:

Parallelize ACROSS independent keys

Serialize WITHIN an ordered key

In the source:

key = groupId

So:

Group A → execution path A

Group B → execution path B

while individual group ordering is preserved.

---

# 23. WHATSAPP CASE — MEMORY SHEET

CHAT SYSTEM

==========

Gateway
→ maintain connections
→ keep lightweight

Session Service
→ userId → connectionId

Group Service
→ groupId → users

Routing
→ consistent hash(groupId)

Message Queue
→ retries

Idempotency
→ unique messageId
→ dedupe retries

Ordering
→ same group
→ same processing path

---

# 24. 60-SECOND WHATSAPP ANSWER

> “The notes keep the gateway focused on maintaining many TCP connections rather than storing all session state or parsing application messages. User-to-connection mappings are moved to a session service because gateways are distributed and local copies would create duplication and update fan-out. For group messaging, group IDs are consistently hashed to route related work to a particular service node. Delivery goes through a queue to support retries. Because retries can create duplicates, every message carries a unique ID and already-seen IDs are ignored. The final concern is ordering: messages belonging to the same group are sent through the same processing path so their relative order can be maintained.”

Supported by the WhatsApp case-study notes.

---

# 25. CASE STUDY 2 — TINDER

The source's Tinder section mainly develops:

Image storage

Client vs server ownership of match data

Caching connection mapping

Database sharding

Indexes

Replication

API surface

---

# 26. IMAGES: DATABASE BLOB VS FILE STORAGE

The source asks:

> Should profile images be stored as BLOBs inside the database or as files?

It evaluates database features:

Mutability
Transactions
Indexes
Access control

and argues that most of these do not provide enough value for profile-image bytes themselves.

---

# 27. SOURCE REASONING — MUTABILITY

Database BLOB:

can update image value

But the source argues that profile-image changes generally replace the image rather than update a few bits in-place.

Therefore the mutability benefit is not considered useful for this use case.

---

# 28. SOURCE REASONING — TRANSACTIONS

The notes say:

we are not performing
an atomic operation
on image contents

so transaction support is not a compelling reason to put the raw image itself in the relational DB.

---

# 29. SOURCE REASONING — INDEXES

The source points out that we are not searching the binary image content itself.

So database indexes over the raw BLOB are not considered useful for this design.

---

# 30. SOURCE REASONING — ACCESS CONTROL

Database storage offers access control, but the notes say equivalent access mechanisms can be created around file storage.

---

# 31. FILE-STORAGE ADVANTAGES IN SOURCE

The notes list:

comparatively cheap

better suited to large files

static files can use CDN

and conclude:

user images
→ Distributed File System

---

# 32. IMAGE DELIVERY FLOW

Source-derived architecture:

Database
→ profile metadata

Distributed File Storage
→ image bytes

CDN
→ faster static-image delivery

---

# 33. REUSABLE RULE FROM THE SOURCE

LARGE STATIC BINARY OBJECT
↓
file/object-style storage + CDN

STRUCTURED QUERYABLE METADATA
↓
database

That is the core lesson the Tinder section wants you to retain.

---

# 34. CONNECTION CACHE

The notes say they do not want to query the database every time a client sends a message.

So cache:

userId
→ connectionId

This is the same access-pattern reasoning you already saw in WhatsApp.

---

# 35. MATCH DATA — CLIENT OR SERVER?

Source asks:

Where should a user's matches live?

Option 1:

Client

Option 2:

Server

---

# 36. CLIENT-SIDE BENEFIT

The notes identify:

server storage saved

as the client-storage advantage.

But the source immediately says the saving is not particularly meaningful because only user IDs need storing.

---

# 37. CLIENT-SIDE FAILURE

If match information lives only on the device:

user uninstalls app
↓
local match data disappears

---

# 38. SERVER-SIDE BENEFITS

The source identifies:

Persistence

Source of Truth

Harder for client to tamper with

and therefore decides:

store relevant match details
on server

---

# 39. VERY REUSABLE PRINCIPLE

**Do not make an untrusted client the authoritative owner of important business state.**

In source terms:

Server
=
Source of Truth

This same principle later appears in the chess design for validating legal moves.

---

# 40. TINDER SHARDING

The notes propose:

horizontal partitioning / sharding
based on location

This reflects the Tinder access pattern:

find nearby/relevant profiles

so location becomes an important database dimension in this source design.

---

# 41. INDEXES

The notes suggest indexing:

name

age

for efficient queries.

Keep this specifically as the source's design.

---

# 42. REPLICATION

For each database partition, the notes propose:

master / slave

replication so the application can continue if the primary database fails.

This connects to Part 4:

Sharding
→ split dataset

Replication
→ duplicate shard for availability

---

# 43. TINDER API SURFACE — SOURCE

The notes explicitly list:

POST /user/signup

GET /user/login

GET /user/:userID

PUT /user/:userID

DELETE /user/:userID

plus session, match and recommendation APIs.

---

# 44. SESSION APIs — SOURCE

GET /session/users/:connectionID

DELETE /session/connection/:connectionID

POST /session/connection/:userID1/:userID2

---

# 45. MATCH / RECOMMENDATION APIs

GET /match

DELETE /match/:userID

GET /recommendation

---

# 46. TINDER CASE — MEMORY SHEET

TINDER
======

Profile images
→ distributed file storage
→ CDN

Structured profile data
→ database

Match state
→ SERVER
→ source of truth

Connection lookup
→ cache

User DB
→ shard by location

Queries
→ indexes

Each shard
→ replication

Core APIs
→ user
→ session
→ match
→ recommendation

---

# 47. 60-SECOND TINDER ANSWER

> “The Tinder notes separate large static image content from queryable application data. Profile images are placed in distributed file storage and can be delivered through a CDN, while structured user and match data remains server-side. Match state is not made client-authoritative because local data can disappear when the app is uninstalled and can be tampered with, so the server acts as the source of truth. User data is horizontally partitioned by location in the source design, indexed for relevant queries, and each partition is replicated for availability. Frequently needed user-to-connection mappings can be cached rather than queried from the database for every message.”

---

# 48. INSTAGRAM — SOURCE GAP

The uploaded notes contain the heading:

Instagram System Design

but the retrieved page contains **no substantive Instagram design content** before the Gmail section begins.

Therefore:

Instagram Case Study
→ heading exists

Architecture/details
→ not supported by source text here

I will not fabricate an Instagram design and present it as source-derived.

---

# 49. CASE STUDY 3 — GMAIL

The Gmail section is much denser.

It covers:

Service registry

Authentication

Service boundaries

Gateway responsibilities

Session/token state

Distributed cache

Virus/spam detection

Email persistence

Delivery retries

Metadata vs body

Pagination

Search

Inverted index

Search-service separation

Contracts registry

---

# 50. SERVICE REGISTRY — SINGLE VS MULTIPLE INSTANCES

The source says:

Profile Service has 1 node
→ registry stores 1 IP

Profile Service has many nodes
→ registry stores multiple IPs

and its design may also associate a load-balancing algorithm with that service.

---

# 51. SOURCE TRADE-OFF — SEPARATE LOAD BALANCER PER SERVICE

The notes say you could give each distributed system its own load balancer and put that LB address in the registry, but that:

increases complexity

---

# 52. PASSWORD STORAGE

The source clearly states:

Passwords
→ HASH

rather than reversible encryption.

The reasoning:

if original password can be easily recovered,
that reversibility also increases exposure risk

---

# 53. SERVICE BREAKDOWN

The Gmail notes revisit the microservice-boundary question.

If one service handles:

multiple unrelated business use cases

there may be scope to split it.

But the notes explicitly say the decision depends on:

data

communications

APIs

time investment

---

# 54. AUTHENTICATION FLOW IN SOURCE

The notes prefer subsequent authenticated calls to use:

username + auth_token

rather than resending the password.

They also say the auth token can be sent in an:

HTTP header

instead of exposing it in the URL.

---

# 55. OTP VS AUTH TOKEN

According to the source:

OTP
→ login

auth_token / secret
→ authenticate later requests

---

# 56. SERVICE DATA OWNERSHIP

The source states:

> Services should share their data through APIs rather than directly reading each other's databases.

This reinforces the Part 9 principle:

Service
owns its data

---

# 57. SERVICE REGISTRY CACHE STALENESS

The notes describe a failure scenario:

service goes down
↓
registry/cache must update

If the gateway reads stale routing information:

gateway routes to dead node
↓
requests fail

---

# 58. SOURCE'S GATEWAY / REGISTRY ARRANGEMENT

The notes ultimately prefer putting the registry in a dedicated:

Service Manager

instead of making the gateway own it.

Reason:

Gateway-local registry
→ faster

BUT

more gateway responsibility

whereas:

Service Manager

→ network call

BUT

decouples responsibilities

---

# 59. GLOBAL CACHE VS GATEWAY LOCAL CACHE

The source again asks:

in-memory cache inside each gateway?

or

global/distributed cache?

Local cache:

+ faster in-process access

- extra gateway memory/code
- duplicate copies
- update fan-out

The notes conclude:

global cache

is preferable in this design.

---

# 60. TOKEN LOGOUT

The notes describe:

login
↓
Auth Service generates token
↓
client stores token

Subsequent requests include:

userId + token

On logout, local token data is deleted. The notes also discuss server-side invalidation using a validity flag or expiration time.

---

# 61. VIRUS / SPAM CHECKING

Source warns against simply stuffing virus checking into the gateway because:

Gateway
already has many responsibilities
↓
virus scanning there
→ gateway becomes bloated

---

# 62. EMAIL PERSISTENCE BEFORE DELIVERY

🟢 **VERY GOOD DESIGN LESSON**

The notes specify:

1. Persist email in DB

2. ACK sender

3. Deliver to recipient
in parallel

The source explicitly says this helps prevent:

"ghost" emails

that were sent but have no stored server record.

---

# 63. FAILED DELIVERY RETRIES

The notes propose periodically checking failed deliveries and retrying them.

Source flow:

Persist
↓
ACK
↓
delivery attempt
↓

failure
↓
retry later

---

# 64. MESSAGE QUEUE OR PERIODIC CHECK

For virus/spam processing, the notes explicitly mention two possible approaches:

Message Queue

or

Cron / periodic routine

---

# 65. TAGGING

After an email is persisted, the source proposes a separate table:

email_id
+
tags

---

# 66. EMAIL BODY VS METADATA

🟢 **HIGH ROI**

The notes recommend separating:

BODY CONTENT

from:

METADATA

because many queries need only metadata.

Source metadata examples:

sender

receiver

timestamp

---

# 67. WHY SEPARATE THEM?

Suppose inbox listing needs:

sender
subject/metadata
time

It does not need to load the full body for every row.

Source idea:

common query
→ lightweight metadata

full body
→ load only when required

---

# 68. PAGINATION

The notes suggest reading only:

first / next X entries

rather than every email.

Example from source:

```
select *
from email
where 'to' = 'kiran'
limit 10 offset 50;
```

The reusable principle is:

large collection
→ paginate

---

# 69. INDEX ON RECIPIENT

The notes propose an index on:

to

for selecting emails belonging to a recipient efficiently.

---

# 70. SIMPLE SEARCH VS COMPLEX SEARCH

Source distinguishes:

Email Service
→ simple searches
  receiver/date/etc.

from:

Search Engine
→ complex full-text queries

---

# 71. INVERTED INDEX

The notes define an inverted index as mapping:

word / sentence
    ↓
location(s)

so full-text search can find content efficiently.

---

# 72. SOURCE SEARCH PROBLEM

Complex email queries may require expensive joins if implemented entirely through the normal email CRUD database.

Therefore the notes recommend:

separate complex search operations
from
email CRUD operations

---

# 73. SEARCH INDEX CONSISTENCY TRADE-OFF

Once search is separate:

Email DB
  ↓ event/update
Search Engine

the search system may receive the email update later.

The source explicitly identifies the drawback:

Email Service state
and
Search Engine state

may temporarily disagree

That's eventual consistency appearing in a real design.

---

# 74. THIS CONNECTS PART 5 TO A REAL CASE

Part 5 theory:

eventual consistency

Gmail case:

mail persisted now

search index updated later

That is exactly why system-design theory matters.

---

# 75. CLIENT CACHE + ASYNC SERVER SEARCH

The source says recent mail may often already be cached on the client, so the client can use local information for a faster initial result while a server request asynchronously refreshes it.

The notes themselves flag the consistency problem:

> What happens if mail was deleted on desktop while mobile still has stale cached information?

---

# 76. CONTRACT REGISTRY

The Gmail-related notes then introduce a:

Contracts Registry

because services receive data in JSON and need an agreed contract for converting it into program objects.

---

# 77. WHY VERSION CONTRACTS?

The source says contract versioning supports:

backward compatibility

gradual client/service migration

avoiding breaking changes

so services can temporarily continue communicating with an older API contract while changes are rolled out.

---

# 78. GMAIL CASE — MEMORY SHEET

GMAIL
=====

Auth
→ password hashed
→ login gives token
→ later calls use token

Service ownership
→ services communicate via APIs
→ don't directly own each other's DB

Registry
→ Service Manager

Gateway
→ don't overload with unrelated work

Cache
→ global cache preferred in source
over duplicate gateway caches

Email creation

Persist DB

↓
ACK sender
↓
deliver

Delivery failure
→ retry

Spam / malware
→ async or scheduled processing

Email storage

Metadata
separate from
Body

Inbox query
→ index
→ pagination

Search
→ inverted index
→ separate Search Service

Trade-off
→ search index may lag Email DB

Contracts
→ Registry
→ Versioning
→ backward compatibility

---

# 79. 60-SECOND GMAIL ANSWER

> “The Gmail notes separate concerns aggressively. Authentication uses hashed passwords for login and an auth token for later requests. Service locations are managed through a service registry rather than hard-coded into the gateway, and services communicate through APIs instead of reading one another's databases. Email is first persisted so the system has a source-of-truth record; the sender can then be acknowledged while delivery proceeds separately and failures are retried. Email metadata is separated from body content because many inbox queries need only metadata, and recipient indexes plus pagination reduce query

> cost. Complex full-text search is split into a dedicated search component using an inverted index, accepting that the search view may lag the primary email record.”

---

# 80. CROSS-CASE PATTERN #1 — SOURCE OF TRUTH

Appears repeatedly:

Tinder
→ Match state on server

Gmail
→ Email persisted before delivery

Chess later
→ Server validates moves

The source's recurring instinct is:

> Put authoritative state in trusted server-side components.

---

# 81. CROSS-CASE PATTERN #2 — KEEP GATEWAY THIN

WhatsApp:

Don't parse messages there.

Gmail:

Don't overload it with registry/
virus-scanning/etc.

Source lesson:

connection / edge layer
should not become
the entire application

---

# 82. CROSS-CASE PATTERN #3 — CACHE THE HOT MAPPING

Examples in source:

userId → connectionId

because this mapping is consulted frequently and repeated DB access would be wasteful.

---

# 83. CROSS-CASE PATTERN #4 — PARTITION BY A DOMAIN KEY

WhatsApp:

groupId
→ consistent hash
→ group-service node

Tinder:

location
→ database shard

The key should relate to how the workload is accessed.

---

# 84. CROSS-CASE PATTERN #5 — RETRY REQUIRES IDEMPOTENCY

WhatsApp:

retry
+
messageId
+
duplicate detection

Calling App later uses the same pattern.

This is one of the strongest reusable system-design patterns in your source.

---

# 85. CROSS-CASE PATTERN #6 — STATIC MEDIA OUTSIDE PRIMARY DB

Tinder:

image
→ distributed file storage
→ CDN

Live Video later similarly puts video output in distributed file storage.

---

# 86. CROSS-CASE PATTERN #7 — SEPARATE CRUD FROM SPECIALIZED SEARCH

Gmail:

Email Service
→ CRUD/simple query

Search Engine
→ full-text / inverted index

with eventual consistency accepted between them.

---

# 87. INTERVIEW CASE-STUDY CHECKLIST

After reading these source cases, ask these questions in any interview:

1. What is the authoritative state?

2. Which data is accessed most often?

3. What can be cached?

4. What is the natural partition key?

5. What happens when delivery/call fails?

6. Will retries duplicate side effects?

7. Is ordering required?

8. Is any data static media?

9. Should it use file storage/CDN instead of DB?

10. Does one service have too many responsibilities?

11. Can specialized search be separated?

12. What becomes eventually consistent?

13. What happens if a node disappears?

14. Which component owns each piece of data?

Those questions are synthesized directly from the trade-offs repeatedly raised in the source case studies.

---

# 88. CASE STUDY I — MASTER MEMORY MAP

CASE STUDIES I
│
├── WhatsApp
│ ├── Gateway
│ ├── Session Service
│ ├── Message Parser
│ ├── Group Service
│ ├── Consistent Hashing
│ ├── Queue
│ ├── Retry
│ ├── Idempotency
│ └── Ordering
│
├── Tinder
│ ├── Profile DB
│ ├── Image File Storage
│ ├── CDN
│ ├── Match Source of Truth
│ ├── Connection Cache
│ ├── Location Sharding
│ ├── Indexes
│ └── Replication
│
├── Instagram
│ └── Source gap
│
└── Gmail
├── Authentication
├── Auth Token
├── Service Registry
├── Gateway
├── Global Cache

├── Persist-before-send
├── Retry
├── Spam / Malware
├── Metadata / Body
├── Pagination
├── Search Service
├── Inverted Index
├── Eventual Consistency
└── Contract Registry

---

# 89. THE MOST IMPORTANT CASE-STUDY LESSON

The source is repeatedly asking:

WHERE SHOULD THIS STATE LIVE?

and:

WHO SHOULD OWN THIS RESPONSIBILITY?

Examples:

connection mapping
→ Session Service

image bytes
→ File Storage

match state
→ Server

email source of truth
→ Email DB

full-text lookup
→ Search Service

API schemas
→ Contract Registry

That is the real system-design skill:

> **Choose the component that owns each responsibility, then defend why it belongs there.**

# CJ SYSTEM DESIGN INTERVIEW HANDBOOK

## PART 12 — CASE STUDY PLAYBOOK II: CHESS, CALLING APP, LIVE VIDEO, ANALYTICS & TINYURL

**Target:** Senior Software Engineer / Backend Engineer — ~8 YOE
**Goal:** Learn how previously studied components combine into real interview designs.

> **Priority legend**
> 🟢 **MUST KNOW** — reusable interview principle
> 🟡 **SHOULD KNOW** — useful design reasoning
> 🔵 **SOURCE-SPECIFIC** — approach specifically used by these notes
> ⚠️ **SOURCE GAP** — source does not contain enough design detail to safely expand

---

# 1. CASE STUDY — ONLINE CHESS

The Chess section exercises:

Cache TTL
Matching
Global cache
Server-authoritative validation
Game-state persistence
Sharding/stickiness by gameId
Consistency

The notes keep active challenges in memory/global cache, validate moves on the server, store move history relationally, and route all moves for one game to the same cache/database instance.

---

# 2. CACHE TTL PROBLEM

The notes ask:

> How should expired cache entries be removed?

They reject simply clearing the whole cache periodically because:

old entry → should expire
recent entry → should NOT expire

A global clear removes both.

Source suggestions:

Cron periodically removes expired entries

OR

check/delete expired entry during GET

---

# 3. MATCHMAKING

The source says the matching service needs to keep:

active challenge requests

available so new challenges can be matched against them.

Conceptually:

```
Player A ─┐
Player B ─┤
Player C ─┤ → MATCHING SERVICE
Player D ─┘
```

---

# 4. DISTRIBUTED MATCHING SERVICE + GLOBAL CACHE

The matching service itself has multiple instances.

Instead of every instance independently owning all challenges:

```
Matching Node 1
Matching Node 2
Matching Node 3
      |
      v
  GLOBAL CACHE
```

The notes place challenge objects in a global cache and say scatter-gather is handled by the cache cluster coordinator.

---

# 5. WHY VALIDATE MOVES ON SERVER?

🟢🟢🟢

Source answer:

> **Server is the source of truth and ensures fair play.**

If validation were trusted to the client:

modified client
↓
illegal move
↓
server accepts it?

the user could manipulate the application.

---

# 6. REUSABLE PRINCIPLE

CLIENT
= untrusted input

SERVER
= authoritative validation

Never make an easily modified client authoritative for rules that matter.

This pattern also appeared in the Tinder case.

---

# 7. STORE CHESS MOVES

The source chooses a relational model containing attributes such as:

matchId
playerId
move
timestamp

Conceptually:

| matchId | playerId | move | timestamp |
|---|---|---|---|

---

# 8. GAME-ID STICKINESS

🟢 **VERY HIGH ROI**

The notes hash:

gameId

and route all moves belonging to that game toward the same cache/database instance.

```
Game 42 Move 1 ─┐
Game 42 Move 2 ─┼─ hash(gameId) → Node B
Game 42 Move 3 ─┘
```

---

# 9. WHY STICKINESS?

Source reasoning:

```
all live state of Game 42
    ↓
one place
    ↓
easier consistency
```

The notes specifically say this helps prevent illegal moves by keeping one live copy of the game's state instead of spreading it across multiple instances.

---

# 10. WITHOUT STICKINESS

Move 1 → Node A

Move 2 → Node B

Move 3 → Node C

Now several nodes may need coordination to determine the valid current board state.

The notes say that if live game state existed on different instances, maintaining correctness would require consensus-like coordination.

---

# 11. CHESS MEMORY CARD

CHESS
=====

Challenges

→ Global Cache

Matching
→ distributed matching service

Moves
→ validate on SERVER

Server
→ Source of Truth

Move history
→ relational DB

gameId
→ hash
→ sticky cache/DB node

Goal
→ one authoritative live game state

---

# 12. 45-SECOND CHESS ANSWER

> “The notes make the server authoritative for all game moves so a modified client cannot submit illegal state. Active matchmaking challenges are stored in a shared cache for a distributed matching service. Live game requests are sticky by game ID: the game ID is hashed so all moves for a game reach the same cache/database instance, making live-state consistency easier. Completed move history can be stored relationally using match ID, player ID, move and timestamp.”

---

# 13. CASE STUDY — CALLING APP

This source section exercises many distributed-system concepts at once:

Acknowledgements
Retries
Idempotency
Sessions/connections
Finite-state machines
Message queues

Replication
Billing
Event-driven state

---

# 14. ACKNOWLEDGEMENT LOSS

Suppose sender sends:

M1

Receiver processes it and sends:

ACK(M1)

But the ACK is lost.

Sender sees:

no acknowledgement

and retries:

M1 again

The source explicitly uses this scenario.

---

# 15. IDEMPOTENCY

🟢🟢🟢

The retry uses the same:

messageId

If receiver already processed that ID:

do NOT process again

send ACK again

---

# 16. CORE DISTRIBUTED-SYSTEM LESSON

Network uncertainty
↓
retry
↓

possible duplicate
↓
idempotency / deduplication

This exact pattern appeared in WhatsApp as well.

---

# 17. SESSION ID AND CONNECTION ID

The notes distinguish:

User
↓
Session ID
↓
Connection ID

A new session ID is generated when the app connects and is assigned to an available connection. When disconnected, the session is marked inactive; reconnecting creates a new session that may map to a different connection.

---

# 18. SOFT DELETE / AUDIT

Instead of deleting disconnected session history entirely:

session.status = INACTIVE

The source frames this as a soft delete useful for audit purposes.

---

# 19. CALL LIFECYCLE AS FINITE STATE MACHINE

🟢 **MUST KNOW**

The source explicitly models the call as an FSM.

Example progression in the notes:

REQUESTED
↓
VALIDATED
↓
OTHER PARTY LOCATED

↓
CONNECTIONS BRIDGED

---

# 20. WHY FSM FITS CALLS

Because a call is not simply:

ON
or
OFF

It moves through a controlled series of states.

The source also mentions SIP as a protocol containing codes that represent call-state changes.

---

# 21. MESSAGE QUEUES

The detailed source design uses **two queues**.

### Queue 1 — Call State Change Queue

Events such as:

call requested
user picked up
state changed

go here.

Call State Manager consumes them.

---

# 22. QUEUE 2 — CONNECTION EVENT QUEUE

Used when:

connection created

connection terminated

Session Service consumes these events and stores:

user
↓
call session

↓
connection

---

# 23. SWITCH

The notes define the switch as the component that receives:

sender address
receiver address

and bridges the two connections.

---

# 24. CALL STATE MANAGER

Source calls this:

> **the brain of the system.**

Its responsibilities include:

consume call events

get users' connection IDs

tell switch what to bridge

track whether call is active/terminated

provide state to billing

---

# 25. CALL-STATE DATABASE FAILURE

The notes ask:

> What if call-state DB is damaged?

Source answer:

Replication

for example a master/slave arrangement.

It also says call logs may be recreated if copies remain in:

Switch

or

Message Queue

---

# 26. BILLING

The notes place billing logic with the Call State Manager because charging can depend on:

state transitions

who started call

duration

Source flow:

Call State
↓
Call State Manager
↓
Billing Service

---

# 27. EVENT LOG + CURRENT STATE

The notes later describe the calling system as suitable for an event-driven design:

EVENT LOG
→ persisted cheaply

CURRENT CALL STATE
→ RDBMS for fast queries

This is an important architectural split:

HISTORY
vs
CURRENT VIEW

---

# 28. SOURCE COMPONENT SUMMARY

CLIENTS
↓
SWITCH
|

+---- Call State Events ──→ Queue ──→ Call State Manager
|
+---- Connection Events ──→ Queue ──→ Session Service

Call State Manager
↓
Billing Service

This structure comes directly from the notes.

---

# 29. CALLING APP MEMORY CARD

CALLING APP
==========

Retry
→ same messageId

Idempotency
→ dedupe repeated message

User
→ Session
→ Connection

Call
→ Finite State Machine

Switch
→ bridges connections

Queue 1
→ call-state events

Queue 2
→ connection events

Call State Manager
→ system brain
→ current call state
→ billing

Session Service
→ user/session/connection mapping

DB failure
→ replication

---

# 30. 60-SECOND CALLING ANSWER

> “The notes model a call as a finite-state machine rather than one stateless request. A switch bridges the caller and receiver connections, while a Call State Manager consumes call-state events, tracks the current call state and provides billing information. A separate Session Service maps users to call sessions and sessions to connections. State and connection changes flow through separate message queues. Because lost acknowledgements cause retries, messages retain the same ID and duplicate IDs are not processed twice. Call-state persistence is replicated, and the notes also distinguish an event history from a queryable current-state view.”

---

# 31. CASE STUDY — LIVE VIDEO STREAMING

The source focuses on:

Encoding
Thundering herd
File storage
Chunking
Parallel processing
Distributed storage
Hot chunks
Adaptive bitrate
HLS vs WebRTC

---

# 32. FFMPEG

The source identifies FFMPEG as a command-line tool that can:

convert audio/video formats

capture media

encode in real time

---

# 33. VIDEO PROCESSING PIPELINE

Source architecture:

ORIGINAL VIDEO
↓
split into equal chunks
↓
worker nodes
↓
different resolutions / codecs
↓
distributed file storage

---

# 34. WHY CHUNK VIDEO?

The notes distribute video chunks among multiple workers.

Video
├── Chunk 1 → Worker A
├── Chunk 2 → Worker B
├── Chunk 3 → Worker C
└── Chunk 4 → Worker D

Then processed chunks are stored back into distributed storage.

---

# 35. MULTIPLE RESOLUTIONS / CODECS

A chunk may be processed into multiple forms:

Chunk
├── Resolution A
├── Resolution B
├── Resolution C
└── different codec

The source then stores these outputs in distributed file storage such as S3.

---

# 36. WHY FILE STORAGE?

The notes give two reasons:

File storage tends to be cheaper

Video normally does not need DB-style in-place updates

This repeats the same storage principle used for Tinder images.

---

# 37. HOT CHUNKS

The source says live viewers are particularly interested in approximately:

last 60 seconds

of the video, referring to those as the hot chunks.

That reveals an important access pattern:

not every video byte
has equal demand

---

# 38. THUNDERING HERD — SOURCE RESPONSES

For overloaded live streaming, the notes suggest:

Drop requests waiting too long

Reduce video quality

Horizontally scale edge servers

These are the source's explicit responses.

---

# 39. PIPELINE TRANSMISSION CHOICES

The notes say processing latency depends partly on how data moves between processing stages:

pull

push

job scheduler

along with fault tolerance and waiting time for worker allocation.

---

# 40. SOURCE LIVE-PROCESSING FLOW

For live video the source proposes workers:

process chunk
↓
push to next processing layer

while:

intermediate results
→ asynchronously persisted

---

# 41. ADAPTIVE BITRATE STREAMING

🟢 **MUST KNOW FROM SOURCE**

The notes describe varying video bitrate according to:

network capability
+
device capability

---

# 42. ADAPTIVE BITRATE FLOW

Strong connection
↓
higher quality

Weak connection
↓
lower bitrate / quality

The browser/client can adapt quality based on what it can currently handle.

---

# 43. SOURCE HLS VS WEBRTC FRAMING

🔵 **SOURCE-SPECIFIC**

The notes frame:

HLS
→ TCP
→ more reliable
→ slower

WebRTC
→ UDP
→ less reliable
→ faster / lower latency

Keep this as the source's simplified comparison for this handbook pass.

---

# 44. LIVE VIDEO MEMORY CARD

LIVE VIDEO
==========

Capture / encode
→ FFMPEG

Video
→ chunks

Chunks
→ parallel workers

Workers
→ multiple resolutions/codecs

Output
→ distributed file storage

Most-demanded area
→ hot recent chunks

Traffic spike
→ edge scaling
→ drop overdue requests
→ lower video quality

Client/network varies

→ adaptive bitrate

---

# 45. 45-SECOND LIVE VIDEO ANSWER

> “The notes split the incoming video into chunks and distribute them across workers, which generate different resolutions and codecs before writing the outputs to distributed file storage. Recent chunks are hottest because live viewers are concentrated near the live edge. Adaptive bitrate lets the client change video quality according to device and network capability. During a thundering-herd event, the source suggests horizontally scaling edge servers, dropping requests that have waited too long and potentially reducing video quality.”

---

# 46. CASE STUDY — ANALYTICS ENGINE

🟢🟢🟢 **VERY REUSABLE**

This case focuses on:

Pipeline stages
Intermediate persistence
Fault tolerance
Job scheduling
Production DB isolation
Data lake
Batch analytics
Real-time analytics
Lambda Architecture

---

# 47. MULTI-STAGE PIPELINE

Imagine:

Stage A
↓
Stage B
↓
Stage C
↓
Stage D

What happens if Stage C fails?

If everything is only in memory:

earlier work may need to be repeated

---

# 48. SOURCE SOLUTION — INTERMEDIATE STORAGE

The notes place an intermediate database/table between stages.

STAGE A
↓
TABLE A
↓
STAGE B
↓
TABLE B
↓
STAGE C

---

# 49. WHY STORE INTERMEDIATE OUTPUTS?

The notes give two reasons:

Decouple stages

Increase fault tolerance

If Stage B fails:

Stage A output
still exists

so successful earlier processing is not automatically lost.

---

# 50. TRADE-OFF — STORAGE COST

Persisting every intermediate result costs storage.

The source proposes several ways to reduce that cost:

Compression

Consume temporary results faster

Recompute on failure instead of storing everything

Reduce assignment/execution delays

---

# 51. STORE VS RECOMPUTE

Classic design trade-off:

STORE INTERMEDIATE RESULT

+ faster recovery
+ less recomputation

- storage cost

versus the source's alternative:

RECOMPUTE AFTER FAILURE

+ less temporary storage

- repeat computation

---

# 52. JOB SCHEDULER

The notes say a job scheduler can:

send work between stages in batches

divide tasks

decouple stages

---

# 53. WHY NOT RUN ANALYTICS DIRECTLY ON PRODUCTION DATABASES?

🟢🟢🟢

The source gives a very important answer:

Analytics queries
↓
consume production I/O
↓
compete with main application workload

---

# 54. PAYMENTS EXAMPLE

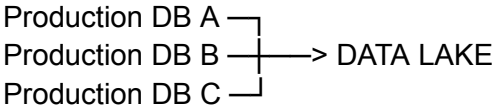
A payments database might use strong isolation and row locking.

The notes argue that heavy analytics queries running against the same database could consume resources or interfere with operational transactions.

---

# 55. DATA LAKE

Source solution:




Rather than repeatedly querying operational systems, copy their data into a heterogeneous data lake.

---

# 56. SOURCE DATA-LAKE CHARACTERISTICS

The notes describe the pulled data lake as:

immutable

appendable

not updated/deleted

and propose cheap distributed file storage such as:

HDFS

---

# 57. BATCH ANALYTICS

For analysis that can wait:

Production systems
↓
Data Lake
↓
Batch / MapReduce-style processing

This connects directly to Part 10.

---

# 58. REAL-TIME ANALYTICS

But if the requirement is:

ALERT NOW

the source says querying the data lake is too slow.

Instead:

Services
↓
Event Bus
↓
Analytics Engine

provides streaming data.

---

# 59. BATCH + STREAM

The notes call the combination:

MapReduce
+
Streaming
=
Lambda Architecture

Keep this as the source terminology.

---

# 60. ANALYTICS MEMORY CARD

ANALYTICS ENGINE
==========

Pipeline
→ multiple stages

Between stages
→ persist intermediate output

Why?
→ decoupling
→ fault tolerance

Cost?
→ storage

Reduce cost
→ compression
→ faster consumption
→ recomputation

Do NOT overload
production DB

Instead
→ copy data
→ Data Lake

Historical analytics
→ batch

Immediate alerts
→ event bus
→ stream analytics

Batch + Stream
→ Lambda Architecture

---

# 61. 60-SECOND ANALYTICS ANSWER

> “The notes decouple analytics pipeline stages by persisting each stage's intermediate output so a later-stage failure does not destroy completed work. The trade-off is storage cost, which can be reduced through compression, quicker downstream consumption or recomputing results after failure. Analytics should not continuously compete with production databases for I/O and locking resources, so source data is copied into a heterogeneous data lake for offline analysis. For alerts requiring immediate processing, services instead publish events into a streaming path. The source calls the combination of MapReduce-style batch processing and streaming Lambda Architecture.”

---

# 62. BOOKMYSHOW — SOURCE GAP

The document contains the heading:

BookMyShow Movie Ticket Design

but no substantive architecture follows before the TinyURL section starts.

Therefore:

BookMyShow
→ heading present

Seat locking
→ not explained here

Booking concurrency
→ not explained here

Payment flow
→ not explained here

Those topics should **not** be fabricated as source-derived notes.

---

# 63. CASE STUDY — TINYURL / UNIQUE-ID GENERATION

The TinyURL section shifts heavily toward one core problem:

> **How do we generate compact unique IDs in a distributed system?**

The source first rejects naive single-node approaches, then discusses Base62, UUID, ZooKeeper sequences and Snowflake.

---

# 64. NAIVE ID #1 — AUTO INCREMENT

On one machine:

1
2
3
4
5

looks easy.

But with many independent machines:

Machine A → 101

Machine B → 101

you can create duplicates unless the sequence itself is coordinated.

The notes identify this distributed uniqueness problem explicitly.

---

# 65. NAIVE ID #2 — TIMESTAMP

Likewise, using timestamp alone can appear unique on a single machine.

The source groups timestamps with auto-increment IDs as approaches that may fail to guarantee uniqueness when multiple machines independently generate IDs.

---

# 66. BASE62

🟢 **HIGH ROI**

The notes define the alphabet:

0–9
A–Z
a–z

Total:

62 characters

---

# 67. WHY BASE62 FOR SHORT URLS?

If the short key is seven characters:

_____

and every position has 62 possibilities:

62^7

possible strings exist according to the source calculation.

---

# 68. SOURCE HASHING NOTES

The source lists:

MD5
128 bits
16 bytes
32 characters

SHA1
160 bits
20 bytes
40 characters

before introducing Base62.

The section itself does not build a full collision-resolution architecture from these hashes.

---

# 69. UUID

The notes describe UUID/GUID as a:

128-bit identifier

commonly represented as hexadecimal text and having an extremely large identifier space.

---

# 70. UUID DRAWBACKS IN SOURCE

The notes list concerns including:

long representation

larger storage requirement

not naturally increasing/sortable

unfriendly to indexing in source framing

no business meaning

---

# 71. ZOOKEEPER SEQUENCE GENERATOR

🔵 **SOURCE-SPECIFIC**

The source next describes ZooKeeper as a centralized sequence generator and names two sequential znode modes:

EPHEMERAL_SEQUENTIAL

PERSISTENT_SEQUENTIAL

Both append monotonically increasing sequence numbers; the source differentiates them by whether the znode automatically disappears when the client disconnects.

---

# 72. EPHEMERAL SEQUENTIAL

Source behavior:

client connects
↓
sequential znode created
↓
client disconnects
↓
node deleted automatically

while the name contains an increasing sequence number.

---

# 73. PERSISTENT SEQUENTIAL

Source behavior:

client connects
↓
sequential znode created
↓
client disconnects
↓
node remains

again with a monotonically increasing suffix.

---

# 74. SNOWFLAKE

🟢 **IMPORTANT SOURCE TOPIC**

The notes describe Snowflake as a service/algorithm for generating unique IDs at high scale.

The source's later breakdown uses a 64-bit long identifier containing fields for:

unused/sign bit

timestamp

machine identity

per-millisecond sequence

---

# 75. SOURCE SNOWFLAKE BIT BREAKDOWN

The notes give:

1 bit
→ unused

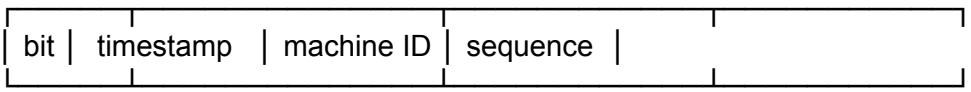
41 bits
→ timestamp

10 bits
→ working-machine ID

12 bits
→ serial/sequence number

---

# 76. SNOWFLAKE MENTAL MODEL




The source's goal is that multiple machines can independently generate IDs using their machine identity and local sequence information.

---

# 77. SOURCE SNOWFLAKE ADVANTAGE

Because timestamp increases over time, the source says generated values tend toward an:

increasing trend

while generation can occur independently on machines following the bit-layout rules.

---

# 78. CLOCK-ROLLBACK PROBLEM

The source explicitly identifies a Snowflake issue:

machine's observed time
moves backward

If IDs depend heavily on timestamp, this creates a uniqueness/ordering problem.

---

# 79. SOURCE CLOCK-ROLLBACK RESPONSES

The notes mention:

Small rollback
→ wait until time catches up

and for larger rollback:

report error / deny generation

or

use an extra expansion position

to continue distinguishing IDs.

---

# 80. UNIQUE-ID APPROACHES — SOURCE COMPARISON

| Approach | Source focus |
|---|---|
| Auto increment | Simple single-machine sequence; distributed coordination problem |

| | |
|---|---|
| Timestamp | Distributed duplicate risk in naive use |
| Base62 | Compact URL alphabet |
| UUID | Huge identifier space but long/non-increasing |
| ZooKeeper sequence | Centralized monotonically increasing sequence |
| Snowflake | High-scale structured ID generated from time + machine + sequence |

---

# 81. TINYURL MEMORY CARD

TINYURL / UNIQUE IDS
==========

Problem
→ distributed uniqueness

Naive auto-increment
→ multiple machines conflict

Timestamp alone
→ distributed uniqueness risk

Base62
→ 0-9 A-Z a-z
→ 62 symbols
→ 7 chars = 62^7

UUID
→ huge space
→ long
→ not naturally increasing

ZooKeeper
→ centralized sequence
→ sequential znodes

Snowflake
→ timestamp
→ machine ID
→ sequence

Snowflake risk
→ clock rollback

---

# 82. 60-SECOND UNIQUE-ID ANSWER

> “The TinyURL notes focus on generating compact unique IDs in a distributed environment. Auto-increment or naive timestamp-based IDs work much more easily on one machine but require coordination when multiple nodes generate IDs concurrently. Base62 gives a URL-friendly alphabet of 62 characters, so a seven-character key has 62762^7 possible values. UUID provides a huge identifier space but the source highlights its length and lack of natural ordering. ZooKeeper can provide centralized monotonically increasing sequential znodes. Snowflake instead constructs a 64-bit ID from timestamp, machine identity and a local sequence number, with clock rollback becoming one of its important failure cases.”

---

# 83. CROSS-CASE PATTERN — PARTITION RELATED STATE TOGETHER

Chess:

gameId
→ same node

Calling:

session
→ mapped to connection

Analytics:

stage output
→ persistent handoff

TinyURL:

machine identity
→ part of distributed ID generation

The recurring question:

> **What information must remain together so the system can reason locally instead of coordinating globally?**

---

# 84. CROSS-CASE PATTERN — RETRIES CHANGE SEMANTICS

Calling App:

lost ACK
→ retry
→ possible duplicate
→ messageId dedupe

WhatsApp from Part 11 used the same pattern.

This is a system-design reflex worth memorizing:

RETRY?
↓
ASK ABOUT IDEMPOTENCY

---

# 85. CROSS-CASE PATTERN — CURRENT STATE VS HISTORY

Calling App:

Event Log
→ history

RDBMS
→ current call state

Analytics:

Data Lake
→ historical dataset

Streaming pipeline
→ immediate alerts

These systems need different representations of the same world for different access patterns.

---

# 86. CROSS-CASE PATTERN — STORAGE DEPENDS ON DATA TYPE

Chess moves

→ relational records

Call current state
→ RDBMS

Video
→ distributed files

Analytics
→ data lake / distributed file system

Short URL
→ compact identifier mapping

Do not choose a database type before understanding:

data shape
+
access pattern
+
consistency requirement

---

# 87. CROSS-CASE PATTERN — BUFFER BETWEEN STAGES

Calling:

Switch
→ Queue
→ Call State Manager

Analytics:

Stage A
→ intermediate table
→ Stage B

Different technologies, same broad reason:

decouple producer
from downstream processing

---

# 88. CROSS-CASE PATTERN — HOT DATA

Live Video:

recent ~60 sec
→ hot chunks

Chess:

active game state
→ hot live state

Analytics:

real-time alert events
→ immediate path

Historical/cold data can often take a different path from active data.

---

# 89. CASE-STUDY MASTER QUESTIONS

For almost any new HLD question, ask:

1. What is the source of truth?

2. What state is active/hot?

3. What state is historical?

4. What must be ordered?

5. What can be retried?

6. If retried, how do we dedupe?

7. What key should related work partition on?

8. What should be cached?

9. Which operations need real-time processing?

10. Which can be asynchronous/batch?

11. Where do we need intermediate persistence?

12. What should live in DB vs file storage?

13. What happens when one component crashes?

14. Where could a thundering herd occur?

15. What state can tolerate eventual consistency?

That is the reusable value of these case studies.

---

# 90. PART 12 MASTER MEMORY MAP

CASE STUDIES II

- Chess
  - Matching Cache
  - Server Validation
  - Source of Truth
  - Move DB
  - gameId Stickiness
- Calling App
  - ACK
  - Retry
  - Idempotency
  - Session
  - Connection
  - Switch
  - FSM
  - Queues
  - Call State Manager
  - Billing
- Live Video
  - FFMPEG
  - Chunking
  - Worker Processing
  - Distributed Storage
  - Hot Chunks
  - Edge Scaling
  - Adaptive Bitrate
- Analytics
  - Pipeline
  - Intermediate Storage
  - Fault Tolerance
  - Job Scheduler
  - Data Lake
  - Batch
  - Streaming
  - Lambda Architecture
- BookMyShow
  - Source gap

└── TinyURL / IDs
  ├── Auto Increment
  ├── Timestamp
  ├── Base62
  ├── UUID
  ├── ZooKeeper Sequence
  └── Snowflake

---

# 91. HLD CORE SECTION COMPLETE

At this point, the handbook has covered the main HLD source material:

PART 1
Interview Framework + Scale

PART 2
Networking

PART 3
APIs + Real-Time Communication

PART 4
Databases + Sharding + Consistent Hashing

PART 5
Transactions + Distributed Consistency

PART 6
Caching

PART 7
Queues + Events + Kafka

PART 8
LB + Gateway + Discovery + Mesh

PART 9
Microservices + Resilience + Rate Limiting

PART 10
Batch/Stream + ETL + Operations + Leader Election

PART 11
Case Studies I

PART 12
Case Studies II

# CJ SYSTEM DESIGN INTERVIEW HANDBOOK — GAP-FILL SUPPLEMENTS

**Status:** 🔷 **INTERVIEW EXPANSION — added beyond the original source where needed**
**Purpose:** Fill only the highest-ROI gaps for an **8-YOE Java / Backend / System Design interview**.

---

# HLD SUPPLEMENT 1 — SECURITY

## 1. THE SECURITY MENTAL MODEL

Every backend must answer four questions:

WHO ARE YOU?
→ Authentication

WHAT ARE YOU ALLOWED TO DO?
→ Authorization

CAN SOMEONE READ / ALTER THE TRAFFIC?
→ TLS

CAN I TRUST THE INPUT / REQUEST?
→ Validation + application security

Memory:

AUTHN = identity
AUTHZ = permission
TLS = transport protection

---

## 2. AUTHENTICATION VS AUTHORIZATION

### Authentication

"Prove who you are."

Examples:

password
OTP
session
JWT/access token
certificate

### Authorization

"Now that I know who you are,
are you allowed to do this?"

Example:

Alice authenticated successfully.

DELETE /users/42

Is Alice an admin?
→ authorization decision

---

## 3. 401 VS 403

🟢 Memorize:

401
→ authentication missing / invalid

403
→ identity known,
but operation is forbidden

---

# 4. SESSION-BASED AUTHENTICATION

Flow:

User sends credentials
↓
Server validates
↓
Server creates session
↓
sessionId stored server-side
↓
sessionId sent to browser cookie
↓
future requests send cookie

Architecture:

CLIENT
|
Cookie: sessionId
|
v
SERVER
|

SESSION STORE

With multiple application servers, session state should usually not live only in one application's local memory.

Possible shared store:

Redis
DB
distributed session store

---

# 5. JWT — FIRST PRINCIPLES

JWT is a signed token containing claims.

Conceptually:

HEADER
.
PAYLOAD
.
SIGNATURE

Claims may include:

userId
roles
issuer
expiry
audience

Important:

JWT SIGNED
≠
JWT ENCRYPTED

The payload is generally not secret merely because it is inside a JWT.

---

# 6. SESSION VS JWT

| Session | JWT-style token |
| --- | --- |
| Server holds session state | Claims carried by token |
| Easy immediate invalidation | Revocation can be harder |

| | |
|---|---|
| Session lookup may be required | Verification can often be local |
| Small cookie/session ID | Token may be larger |
| Central state useful for control | Useful for distributed auth |

Do not say:

> “JWT is always better because it is stateless.”

Statelessness trades away some centralized control.

---

# 7. ACCESS TOKEN VS REFRESH TOKEN

ACCESS TOKEN
→ short-lived
→ sent to APIs

REFRESH TOKEN
→ longer-lived
→ used to obtain new access token

Security principle:

Short-lived credentials
limit exposure window.

---

# 8. PASSWORD STORAGE

Never store:

plaintext password

Do not normally store passwords using reversible encryption.

Use:

slow password hash
+
unique salt

Conceptual flow:

password
+
salt

↓
password-hashing function
↓
stored hash

---

# 9. COOKIES — THREE HIGH-ROI FLAGS

HttpOnly
→ JavaScript cannot directly read cookie

Secure
→ send over HTTPS

SameSite
→ helps control cross-site sending

---

# 10. CSRF

Problem:

User authenticated to bank.com

User visits evil.com

evil.com causes browser
to submit request to bank.com

browser automatically
includes bank cookie

This is why cookie-authenticated state-changing requests need CSRF consideration.

Memory:

CSRF
→ abuse victim's existing authenticated browser session

---

# 11. CORS

CORS is mainly a **browser-enforced cross-origin access policy**.

Do not say:

"CORS secures my backend from attackers."

Non-browser clients are not magically blocked by CORS.

---

# 12. TLS

TLS provides:

Confidentiality
Integrity
Server authentication

TLS does **not** replace application authorization.

TLS
→ protects communication

AuthN/AuthZ
→ protects application actions

---

# 13. SECURITY — 30 SECOND ANSWER

> “I separate authentication from authorization: authentication proves identity, authorization decides what that identity may do. User credentials should be stored using proper password hashing and salts. Browser applications typically use secure session cookies or tokens, with HTTPS protecting traffic in transit. For tokens I keep access tokens short-lived and separate them from refresh tokens. I also consider CSRF for cookie-authenticated state-changing requests, CORS for browser-origin policy, input validation and least-privilege authorization.”

---

# HLD SUPPLEMENT 2 — OBSERVABILITY / SRE

# 14. OBSERVABILITY MENTAL MODEL

Three pillars:

METRICS
What is happening?

LOGS
What happened?

TRACES
Where did this request spend time?

---

# 15. METRICS

Examples:

request rate
error rate
latency
CPU
memory
queue depth
consumer lag
cache hit ratio
DB connection usage

Metrics are ideal for:

dashboards
alerts
trends
aggregations

---

# 16. RED METRICS

🟢 Very useful for services:

R = Rate
requests/sec

E = Errors
failed requests

D = Duration
latency

Memory:

RATE
ERRORS
DURATION

---

# 17. LOGS

Logs tell you individual events.

Example:

2026-09-16T12:01:02
orderId=123
paymentId=456
status=FAILED
reason=TIMEOUT

Useful fields:

timestamp
service
severity
requestId
traceId
user/entity ID
operation
error

Avoid logs that are only:

"Something failed"

---

# 18. DISTRIBUTED TRACING

Suppose request crosses:

Gateway
  ↓ 10ms
Order Service
  ↓ 30ms
Risk Service
  ↓ 500ms
Database
  ↓ 20ms

Trace tells you:

WHERE the latency occurred.

---

# 19. TRACE ID AND SPAN

Trace
= complete distributed request

Span
= one operation/hop inside trace

Conceptually:

Trace ID = ABC

Span Gateway
Span Order Service
Span Risk Service
Span DB

---

# 20. CORRELATION ID

At minimum, propagate something like:

requestId / correlationId

through every service.

Then logs from:

Gateway
Service A
Service B
DB adapter

can be connected to one request.

---

# 21. P50 / P95 / P99

Do not monitor only average latency.

Example:

p50 = 40 ms
p95 = 150 ms
p99 = 2 sec

Interpretation:

99% of requests
complete within ~2 sec

Tail latency matters especially when one request fans out to many downstream services.

---

# 22. SLI / SLO / SLA

🟢🟢🟢

### SLI

**Service Level Indicator**

What are we measuring?

Examples:

request success rate
availability
p99 latency

### SLO

**Service Level Objective**

What target do we want?

Example:

99.9% successful requests
over 30 days

### SLA

**Service Level Agreement**

A contractual/business commitment, potentially with consequences if violated.

Memory:

SLI = measurement

SLO = target

SLA = promise/agreement

---

# 23. ERROR BUDGET

If:

SLO = 99.9%

then tolerated failure budget is approximately:

0.1%

Error budgets let teams balance:

reliability

vs
shipping/change velocity

---

# 24. MONITORING VS OBSERVABILITY

Monitoring:

"Is the known thing broken?"

Observability:

"Can I understand what is happening,
including failures I did not predict?"

Do not treat the terms as perfectly interchangeable.

---

# 25. WHAT SHOULD PAGE SOMEONE?

Bad alert:

CPU > 70%

with no customer impact.

Better alerts usually focus on symptoms:

error rate rising
latency SLO violated
requests being dropped
queue lag dangerously high

Then infrastructure metrics help diagnose why.

---

# 26. OBSERVABILITY — 30 SECOND ANSWER

> “I instrument services with metrics, structured logs and distributed traces. Metrics tell me system health and trends, logs explain individual events, and traces show where a distributed request spends time. I propagate correlation or trace IDs through services. I monitor rate, errors and latency, including p95/p99 rather than only averages. Reliability should be expressed using SLIs, SLOs and error budgets, with alerting focused primarily on user-visible symptoms.”

---

# HLD SUPPLEMENT 3 — PRODUCTION RESILIENCE

# 27. THE RESILIENCE CHAIN

REMOTE CALL
↓
TIMEOUT / DEADLINE
↓
FAILURE?
↓
RETRY SAFE?
↓
BACKOFF + JITTER
↓
DEPENDENCY STILL FAILING?
↓
CIRCUIT BREAKER

At overload:

RATE LIMIT
↓
LOAD SHED
↓
BULKHEAD
↓
GRACEFUL DEGRADATION

---

# 28. TIMEOUT

Without timeout:

Service A
↓
Service B hangs
↓
A waits
↓
threads/connections fill
↓
A also becomes unhealthy

Every remote call needs a bounded waiting strategy.

---

# 29. TIMEOUT VS DEADLINE

TIMEOUT
How long may this particular operation wait?

DEADLINE
Absolute/end-to-end time
by which the overall work must finish

Example:

Client deadline = 1 sec

Gateway uses 100 ms
Service A uses 400 ms

Only ~500 ms remains downstream.

---

# 30. RETRIES

Retry helps:

temporary/transient failure

but harms:

persistent overload

Worst pattern:

service overloaded
↓
requests time out
↓
every client retries
↓
more traffic
↓
service becomes even more overloaded

= retry storm.

---

# 31. EXPONENTIAL BACKOFF

Instead of:

retry immediately
retry immediately
retry immediately

use approximately:

100 ms
200 ms
400 ms
800 ms
...

---

# 32. JITTER

If 100,000 clients all retry at:

exactly 1 sec

you create a synchronized spike.

Jitter adds randomness:

~0.8 sec
~1.1 sec
~1.4 sec
...

---

# 33. RETRY ONLY IDEMPOTENT/SAFE OPERATIONS

Before retrying:

Can this operation execute twice?

Payment example:

request executed
response lost
client retries

Without idempotency:

double charge

---

# 34. CIRCUIT BREAKER

States:

CLOSED
requests flow normally

OPEN
fail fast

HALF-OPEN
allow limited test requests

Flow:

failures exceed threshold
↓
OPEN
↓
wait
↓
HALF-OPEN
↓
success?
/ \
YES NO
↓ ↓
CLOSED OPEN

---

# 35. BULKHEAD

Think ship compartments.

Without isolation:

Dependency X hangs
↓
uses every worker/thread
↓
Dependency Y requests also fail

With bulkhead:

Pool X
Pool Y
Pool Z

Failure of X has a bounded blast radius.

---

# 36. LOAD SHEDDING

Sometimes the correct action is:

REJECT WORK

rather than accept everything and collapse.

Examples:

429 Too Many Requests
503 Service Unavailable
drop low-priority requests

---

# 37. GRACEFUL DEGRADATION

If recommendation service fails:

Bad:

entire homepage fails

Better:

serve homepage
without recommendations

Other examples:

cached stale result
default result
reduced quality
disable optional feature

---

# 38. RATE LIMITER VS CIRCUIT BREAKER VS BULKHEAD

| Mechanism | Solves |
|---|---|
| Rate limiter | Too much incoming work |
| Circuit breaker | Repeated calls to unhealthy dependency |
| Bulkhead | Failure-resource isolation |
| Timeout | Bound waiting |

| | |
|---|---|
| Retry | Recover transient failure |
| Load shedding | Reject excess work |
| Fallback | Degrade instead of fail completely |

---

# 39. RESILIENCE — 45 SECOND ANSWER

> “Every remote call needs bounded waiting through timeouts or an end-to-end deadline. I retry only failures that are likely transient and only where the operation is safe to repeat, using exponential backoff and jitter to avoid retry storms. A circuit breaker stops repeatedly calling an unhealthy dependency. Bulkheads bound the blast radius by isolating resource pools. Under overload I prefer load shedding or rate limiting rather than accepting unlimited work, and for optional functionality I design graceful degradation.”

---

# HLD SUPPLEMENT 4 — TRANSACTIONAL OUTBOX / CDC

# 40. THE DUAL-WRITE PROBLEM

Suppose Order Service does:

1. INSERT order into DB
2. Publish OrderCreated to Kafka

Failure:

DB commit succeeds
↓
process crashes
↓
Kafka publish never happens

Now:

Database
→ order exists

Downstream systems
→ never hear about it

---

# 41. WHY NOT JUST REVERSE THE ORDER?

1. Publish event
2. Write DB

Now the opposite failure exists:

event succeeds
DB fails

Consumers believe something happened that the source-of-truth DB never committed.

That is the **dual-write problem**.

---

# 42. TRANSACTIONAL OUTBOX

Store business change and event record in the **same local DB transaction**.

BEGIN TRANSACTION

INSERT INTO orders ...

INSERT INTO outbox ...

COMMIT

Because one database transaction covers both:

both commit
or
both rollback

---

# 43. OUTBOX FLOW

CLIENT
↓
ORDER SERVICE
↓
DATABASE TRANSACTION
├── Orders
└── Outbox
↓
OUTBOX PUBLISHER / CDC
↓
BROKER
↓
CONSUMERS

---

# 44. OUTBOX DOES NOT MEAN EXACTLY-ONCE BUSINESS PROCESSING

Publisher could:

publish event
↓
crash before marking outbox delivered
↓
publish again

Therefore consumer still needs:

IDEMPOTENCY

---

# 45. INBOX / DEDUPLICATION

Consumer stores processed event IDs:

eventId = ABC

Flow:

Receive ABC
↓
Already processed?
/ \
YES NO
↓ ↓
ignore apply operation
↓
remember ABC

---

# 46. CDC — CHANGE DATA CAPTURE

CDC captures committed database changes and emits them downstream.

Conceptual flow:

DATABASE WAL / CHANGE LOG
↓
CDC
↓

EVENT STREAM
↓
CONSUMERS

---

# 47. OUTBOX + CDC

A common architecture:

Service transaction
↓
Business table + Outbox
↓
DB commit
↓
CDC observes outbox
↓
Kafka

Advantage:

The application does not need to perform:

DB write
+
broker write

as two unrelated distributed operations.

---

# 48. OUTBOX VS SAGA

Do not confuse them.

OUTBOX
→ reliably publish event
corresponding to local DB transaction

SAGA
→ coordinate business transaction
across multiple services

They are complementary.

---

# 49. OUTBOX — 30 SECOND ANSWER

> “The dual-write problem occurs when a service needs to update its database and publish an event but one succeeds while the other fails. Transactional Outbox solves this by writing the business record and an outbox record in the same local database transaction. A separate publisher or CDC process sends committed outbox records to the broker. Since publication can still be retried, consumers should remain idempotent and may maintain processed event IDs.”

---

# HLD SUPPLEMENT 5 — DR / RPO / RTO

# 50. DISASTER RECOVERY

Disaster recovery asks:

What if we lose:
a server?
a database?
an availability zone?
an entire region?

Two crucial numbers:

RPO
RTO

---

# 51. RPO — RECOVERY POINT OBJECTIVE

How much data can we afford to lose?

Examples:

RPO = 0
No committed data loss acceptable.

RPO = 5 minutes
Losing latest 5 minutes may be acceptable.

---

# 52. RTO — RECOVERY TIME OBJECTIVE

How long can the system remain unavailable?

Example:

RTO = 15 minutes

means recovery should occur within approximately that target.

---

# 53. MEMORY HOOK

RPO
POINT in data/history

RTO
TIME to recover

---

# 54. RPO VS RTO

Failure occurs at 12:00

Latest recoverable backup = 11:55
→ RPO impact ~5 min

System online again at 12:20
→ RTO impact ~20 min

---

# 55. REPLICATION IS NOT BACKUP

Extremely important.

Suppose:

DELETE FROM customers;

is accidentally executed.

Replication may faithfully copy:

the DELETE

to every replica.

Backup gives you an older recoverable copy.

Therefore:

Replication
→ availability / failover

Backup

→ recovery from corruption,
deletion, ransomware, etc.

---

# 56. ACTIVE-PASSIVE

Users
↓
ACTIVE REGION
↓ replication
PASSIVE REGION

On disaster:

Passive
→ promoted

Advantages:

simpler write ownership

Trade-offs:

unused capacity
failover time
possibly stale replica

---

# 57. ACTIVE-ACTIVE

Users
/ \
Region A
Region B

Both serve traffic.

Benefits:

lower geographic latency
capacity utilization
regional resilience

Costs:

write conflicts
cross-region consistency
data ownership
conflict resolution

---

# 58. BACKUP POLICY QUESTIONS

For critical systems ask:

How frequently are backups taken?

Where are they stored?

Are they cross-region?

Are they immutable?

How long retained?

Are restores tested?

A backup that has never been restored is an assumption, not a proven recovery mechanism.

---

# 59. DR — 30 SECOND ANSWER

> “I start disaster-recovery design with RPO and RTO. RPO defines how much recent data we can lose; RTO defines how long recovery may take. Replication improves availability but is not a substitute for backups because logical corruption or deletion can replicate too. Based on requirements I'd choose active-passive or active-active regional architecture, maintain tested backups and regularly validate the failover and restoration process.”

---

# APPENDIX A — SQL / DBMS INTERVIEW REVISION

# 60. SQL INTERVIEW FLOOR

Know confidently:

Primary key
Foreign key
Unique constraint
Index
Composite index
Normalization
Transactions
Isolation
JOIN

GROUP BY
HAVING
Window functions
ROW_NUMBER
RANK
DENSE_RANK
CTE

---

# 61. INNER VS LEFT JOIN

INNER JOIN
→ only matching rows

LEFT JOIN
→ every left row
+ matching right rows
+ NULL where no match

---

# 62. GROUP BY VS WINDOW FUNCTION

### GROUP BY

Collapses rows:

```
SELECT department_id, COUNT(*)
FROM employee
GROUP BY department_id;
```

### Window function

Keeps individual rows:

```
SELECT
    employee_id,
    department_id,
    COUNT(*) OVER (
        PARTITION BY department_id
    )
FROM employee;
```

Memory:

GROUP BY
→ collapse

WINDOW

→ retain rows + calculate across group

---

# 63. ROW_NUMBER VS RANK VS DENSE_RANK

For values:

100
90
90
80

### ROW_NUMBER

1
2
3
4

### RANK

1
2
2
4

### DENSE_RANK

1
2
2
3

---

# 64. NTH HIGHEST SALARY

Typical:

```
SELECT salary
FROM (
   SELECT
      salary,
      DENSE_RANK() OVER (
         ORDER BY salary DESC
      ) rnk
   FROM employee
) x
WHERE rnk = 3;
```

Use `DENSE_RANK` when duplicate salary values should share a rank.

---

# 65. WHERE VS HAVING

WHERE
→ filters rows before grouping

HAVING
→ filters groups after GROUP BY

---

# 66. INDEX INTERVIEW ANSWER

> “An index trades additional storage and write maintenance for faster reads. I design indexes from query access patterns, paying attention to selectivity and composite-column ordering rather than indexing every column.”

---

# APPENDIX B — JAVA LLD / CONCURRENCY REVISION

# 67. COMPOSITION VS INHERITANCE

Prefer composition when the relationship is:

HAS-A

Inheritance should represent:

IS-A

and must satisfy LSP.

Example:

```
class Car {
   Engine engine;
}
```

Car:

HAS-A Engine

not:

---

# 68. IMMUTABILITY

Typical immutable class:

fields final
no setters
state initialized in constructor
do not expose mutable internals directly

Benefits:

easier reasoning
thread safety
safe sharing
stable hash keys

---

# 69. `equals()` AND `hashCode()`

Critical contract:

if a.equals(b) == true
then
a.hashCode() == b.hashCode()

Otherwise hash-based collections break logically.

Relevant:

HashMap
HashSet
ConcurrentHashMap

---

# 70. THREAD SAFETY

Thread safe means concurrent usage does not violate the object's correctness invariants.

Do not confuse:

atomic variable

with:

atomic business operation

---

# 71. VOLATILE

`volatile` provides visibility/order guarantees around the variable.

It does **not** make compound operations atomic.

volatile int count;

count++;

is still:

read
modify
write

and can lose updates.

---

# 72. ATOMIC INTEGER

AtomicInteger count = new AtomicInteger();

count.incrementAndGet();

Useful when the operation fits an atomic primitive/CAS-style update.

---

# 73. CONCURRENT HASHMAP DOES NOT MAKE BUSINESS TRANSACTIONS ATOMIC

Bad assumption:

ConcurrentHashMap<UserId, Balance> map;

Balance b = map.get(id);
b.withdraw(100);
map.put(id, b);

The container can be thread-safe while the multi-step invariant is not.

Memory:

Thread-safe collection

≠
thread-safe workflow

---

# 74. LOCK WHEN AN INVARIANT SPANS MULTIPLE OPERATIONS

Example:

availableBalance >= withdrawal

Need:

check
+
modify

as one protected critical section.

---

# 75. STATE MACHINES + CONCURRENCY

Useful pattern:

ORDER:

CREATED
→ PAID
→ SHIPPED
→ DELIVERED

Invalid transitions:

CREATED
→ DELIVERED

Keep transition logic centralized.

When multiple threads/events can transition state, concurrency control becomes part of the design.

---

# 76. DEPENDENCY INJECTION

Prefer:

class OrderService {

```
    private final PaymentGateway gateway;

    OrderService(PaymentGateway gateway) {
        this.gateway = gateway;
    }
}
```

over:

gateway = new StripeGateway();

Benefits:

lower coupling
replaceable implementation
testability

---

# APPENDIX C — 5-MINUTE REVISION SHEETS

# 77. HLD — 5 MINUTES

REQUIREMENTS
↓
Functional
Non-functional
Out of scope

SCALE
↓
Users
QPS
Peak
R/W
Storage
Bandwidth

API
↓
Core endpoints
Idempotency

DATA
↓
Entities
Access patterns
SQL/NoSQL

HLD
↓
Client
LB
Service
Cache
DB
Queue

TRACE
↓
Read path
Write path

SCALE
↓
Stateless apps
Replicas
Cache
CDN
Partitioning

FAILURES
↓
Timeout
Retry
Idempotency
Circuit breaker
SPOF

TRADE-OFF
↓
"I chose X because Y,
accepting Z."

---

# 78. DISTRIBUTED SYSTEMS — 5 MINUTES

Replication
→ copy data

Partitioning
→ split data

Consistency
→ what value reads observe

Availability
→ can requests still complete?

CAP
→ during partition choose
consistency vs availability

Quorum
→ R + W > N mental model

2PC
→ distributed atomic commit

Saga
→ local transactions
+ compensation

Outbox
→ DB write + event record
same transaction

Idempotency
→ retries safe

Leader Election
→ exactly one coordinator

---

# 79. NETWORK / COMMUNICATION — 5 MINUTES

DNS
name → address

TCP
reliable ordered byte stream

UDP
datagrams, fewer guarantees

HTTP/1.1
keep alive

HTTP/2
multiplex over TCP

HTTP/3
QUIC

REST
resource API

gRPC
typed RPC

SSE
server → client

WebSocket
client ↔ server

WebRTC
real-time media

---

# 80. DATA — 5 MINUTES

SQL
relationships + transactions

NoSQL
access-pattern-specific models

Index
faster reads
costs writes/storage

Replication
read availability/scale

Sharding
write/storage scale

B+Tree
equality + ranges

Hash
great exact lookup

Cache
low latency
staleness problem

Cassandra
partition key
clustering key
query-driven model

---

# 81. LLD — 5 MINUTES

SRP
one cohesive responsibility

OCP
extend rather than constantly modify

LSP
subtype keeps parent contract

ISP
small client-focused interfaces

DIP
depend on abstractions

PATTERNS

Singleton
one instance

Factory
choose product

Builder
step-by-step construction

Decorator
add behavior dynamically

Proxy
control access

Strategy
interchange algorithm

State
behavior changes with state

Observer
publish updates to subscribers

Chain of Responsibility
pass request through handlers

---

# APPENDIX D — FINAL INTERVIEW QUESTION BANK

# SYSTEM DESIGN FOUNDATION

1. What questions do you ask before designing?
2. Functional vs non-functional requirements?
3. How do you estimate QPS?
4. Average vs peak QPS?
5. How do you estimate storage?
6. Latency vs throughput?
7. Horizontal vs vertical scaling?
8. Why stateless application servers?
9. What is a SPOF?
10. How do you identify the next bottleneck?

# NETWORKING

11. DNS resolution?
12. TCP vs UDP?
13. What does TCP guarantee?
14. Flow control vs congestion control?
15. HTTP/1.1 vs HTTP/2 vs HTTP/3?
16. Head-of-line blocking?
17. TLS handshake conceptually?
18. What happens when you type a URL?
19. Why connection pools?
20. Why can retries make failures worse?

# API / COMMUNICATION

21. REST vs HTTP?
22. POST vs PUT?
23. PUT vs PATCH?
24. What is idempotency?
25. REST vs GraphQL?
26. REST vs gRPC?
27. Polling vs long polling?
28. SSE vs WebSocket?
29. WebSocket vs WebRTC?
30. How would you scale millions of WebSocket connections?

# DATABASES

31. SQL vs NoSQL?
32. What drives storage choice?
33. Why use indexes?
34. Composite-index ordering?
35. B+Tree vs hash?
36. Replication vs partitioning?
37. What makes a good shard key?
38. Range vs hash sharding?

39. What is consistent hashing?
40. Why virtual nodes?
41. What is replication lag?
42. Normalization vs denormalization?

## TRANSACTIONS / CONSISTENCY

43. ACID?
44. Isolation levels?
45. Dirty vs non-repeatable vs phantom read?
46. Optimistic vs pessimistic locking?
47. MVCC?
48. Linearizability?
49. Eventual consistency?
50. CAP?
51. 2PL vs 2PC?
52. 2PC failure modes?
53. Saga choreography vs orchestration?
54. What is compensation?
55. What is dual-write failure?
56. Transactional Outbox?
57. Why must consumers remain idempotent?

## CACHE

58. Cache-aside?
59. Write-through?
60. Write-behind?
61. Write-around?
62. LRU vs LFU?
63. Cache invalidation?
64. Cache stampede / thundering herd?
65. Hot key?
66. Cold cache?
67. Redis vs Memcached conceptually?

## MESSAGING

68. Queue vs Pub/Sub?
69. Kafka topic and partition?
70. Consumer group?
71. Ordering guarantees?
72. At-most-once vs at-least-once?
73. Why duplicates occur?
74. Consumer lag?
75. DLQ?
76. Poison message?
77. Backpressure?

## SERVICE ARCHITECTURE

78. LB vs API Gateway?
79. Service discovery?
80. Service mesh?
81. Client-side vs server-side discovery?
82. Monolith vs microservices?
83. When should you not use microservices?
84. How do you choose service boundaries?

## RESILIENCE

85. Timeout vs deadline?
86. Why retry?
87. Why exponential backoff?
88. Why jitter?
89. Circuit breaker states?
90. Rate limiter vs circuit breaker?
91. Bulkhead?
92. Load shedding?
93. Graceful degradation?

## OBSERVABILITY / SRE

94. Metrics vs logs vs traces?
95. Trace ID vs span?
96. RED metrics?
97. Why p99?
98. SLI vs SLO vs SLA?
99. Error budget?
100. What should trigger an alert?

## SECURITY

101. Authentication vs authorization?
102. Session vs JWT?
103. 401 vs 403?
104. Access vs refresh token?
105. Password hashing vs encryption?
106. CSRF?
107. CORS?
108. What does TLS solve?

## DR

109. RPO?
110. RTO?
111. Backup vs replication?
112. Active-active vs active-passive?

113. What happens if an entire region disappears?

## LLD / JAVA

114. Explain SOLID.
115. SRP vs cohesion?
116. LSP violation example?
117. Composition vs inheritance?
118. Singleton concurrency problem?
119. Factory vs Builder?
120. Decorator vs Proxy?
121. Strategy vs State?
122. Observer?
123. Chain of Responsibility?
124. Why dependency injection?
125. What makes a class immutable?
126. `equals()` / `hashCode()` contract?
127. Why is `volatile count++` unsafe?
128. AtomicInteger vs lock?
129. Does ConcurrentHashMap make compound operations atomic?
130. How do you protect an invariant spanning several fields?

---

# FINAL 60-SECOND MASTER CHECK

Before an interview, you should be able to say naturally:

> “I start by clarifying requirements and scale, then identify latency, availability, consistency and durability goals. I design the API and data model from access patterns before choosing technologies. I begin with the simplest architecture, trace read and write paths and identify the first bottleneck. I use caching, replication, asynchronous messaging or sharding only when the workload justifies them. For every remote operation I consider timeout, retry, idempotency and failure isolation. For distributed writes I consider dual-write failure and patterns such as Outbox. I define observability using metrics, logs, traces and SLOs, and disaster recovery using RPO and RTO. For LLD, I assign cohesive responsibilities, depend on abstractions, preserve invariants under concurrency and apply patterns only when they solve a concrete problem.”

If every phrase in that paragraph is something you can expand for **2–5 minutes**, the handbook is at a strong senior-backend interview floor.

# CJ SYSTEM DESIGN INTERVIEW HANDBOOK

## PART 13 — LOW LEVEL DESIGN FOUNDATION: SOLID PRINCIPLES

**Target:** Senior Java / Backend Engineer — ~8 YOE
**Goal:** Be able to look at a class design and explain **why it will become difficult to change**, then identify which SOLID principle is being violated.

> **Priority legend**
> 🟢 **MUST KNOW** — interview floor
> 🟡 **SHOULD KNOW** — recognize in code
> 🔵 **SOURCE-SPECIFIC** — terminology/examples used by your notes
> ⚠️ **SOURCE GAP** — the notes state the principle but do not provide a deeper example; I will not silently invent one.

The uploaded notes introduce LLD with:

SOLID
│
├── S — Single Responsibility Principle
├── O — Open/Closed Principle
├── L — Liskov Substitution Principle
├── I — Interface Segregation Principle
└── D — Dependency Inversion Principle

and separately discuss **cohesion** immediately after SRP.

---

# 1. WHY SOLID EXISTS

The source's recurring concern is:

CHANGE
↓
How many unrelated pieces of code
must also change?

Good object design tries to avoid:

one change
↓
class A changes
↓
class B breaks
↓
class C needs modification
↓

tests explode

The source therefore emphasizes:

high cohesion
+
lower coupling
+
clear responsibilities
+
substitutable abstractions

---

# 2. SOLID — 10-SECOND MEMORY

S
One responsibility

O
Extend instead of repeatedly modifying

L
A subtype must work wherever
its parent type is expected

I
Don't force clients to depend on
methods they don't need

D
Depend on interfaces / abstractions,
inject concrete implementations

---

# 3. S — SINGLE RESPONSIBILITY PRINCIPLE

🟢🟢🟢

The notes describe SRP using:

> **Only one reason to change.**

They also say that if a class performs several **unrelated functions**, it should be split.

The source does **not** mean:

ONE CLASS
=
ONE METHOD

It explicitly says one purpose may require several methods.

---

# 4. SRP — CORRECT SOURCE MENTAL MODEL

ONE CLASS
=
ONE COHESIVE PURPOSE

That purpose can involve:

methodA()
methodB()
methodC()

as long as all three support the **same responsibility**.

---

# 5. WRONG INTERPRETATION

❌

Every method
must have its own class

That is not what the source says.

The notes specifically say multiple methods may perform different operations while still serving:

one single purpose

---

# 6. SOURCE'S DECOMPOSITION RULE

An especially useful line in the notes is effectively:

> Decompose only when the resulting classes have meaningful independent usability/responsibility.

So do not split blindly.

Bad decomposition:

OrderNameGetter

OrderAmountGetter
OrderDateGetter
OrderPrinter
OrderSetter

merely because there are several methods.

The source wants **cohesive responsibility**, not maximum number of classes.

---

# 7. SRP — BAD SHAPE

Conceptually:

UserService

```
├── createUser()
├── calculateTax()
├── sendEmail()
├── generatePDF()
├── saveAuditFile()
└── resizePhoto()
```

Ask:

> Do these methods belong to one reason for change?

If several unrelated responsibilities are mixed together, that is the kind of structure the source says should be split.

---

# 8. SRP — THE QUESTION TO ASK

Don't ask:

> “How many methods does this class have?”

Ask:

> **“How many unrelated reasons could cause this class to change?”**

That is much closer to the source's definition.

---

# 9. SOURCE BENEFIT #1 — TESTING

The notes say a class with one responsibility tends to have:

fewer test cases

because there are fewer unrelated behaviors bundled together.

---

# 10. SOURCE BENEFIT #2 — LOWER COUPLING

The notes connect:

less unrelated functionality
↓
fewer dependencies
↓
lower coupling

That is important for LLD interviews.

---

# 11. SOURCE BENEFIT #3 — ORGANIZATION

The source also says smaller, well-organized classes are easier to find and understand than monolithic classes.

So SRP is not merely philosophical.

It directly affects:

testability
maintainability
navigation
coupling

---

# 12. SRP — INTERVIEW ANSWER

> “Single Responsibility Principle means a class should have one cohesive responsibility, or one reason to change. It does not mean one method per class—a responsibility can require several methods. If a class contains several unrelated functions, the source recommends decomposing it into independently meaningful classes.”

---

# 13. COHESION

🟢🟢

The notes discuss cohesion immediately after SRP.

Definition in the source:

> Cohesion measures how strongly the methods of a class are related to each other.

Mental model:

HIGH COHESION
methods work together
toward one purpose

LOW COHESION
methods manipulate unrelated concerns

---

# 14. HIGH COHESION

Conceptually:

ShoppingCart

addItem()
removeItem()
calculateTotal()
getItems()

All operations revolve around:

Shopping Cart

This illustrates the source's idea of related methods serving one function.

---

# 15. LOW COHESION

Conceptually:

UtilityManager

sendEmail()
calculateSalary()
compressImage()
validatePassword()
generateInvoice()

The methods have little conceptual relationship.

That resembles what the source calls a:

non-cohesive class

which may need restructuring.

---

# 16. HOW THE NOTES THINK ABOUT COHESION

The source uses class-level data as one indicator.

If methods:

m1()
m2()
m3()

operate on the same related fields, they are more likely to belong together.

If:

m1()
→ works only on A/B

m2()
→ works only on X/Y

m3()
→ works only on P/Q

the class may actually contain multiple unrelated responsibilities.

---

# 17. SRP + COHESION

These concepts reinforce one another:

SRP
one responsibility
↓
methods naturally relate
↓
HIGH COHESION

Memory:

> **SRP tells you what the class should own; cohesion tells you how well its contents belong together.**

---

# 18. COUPLING VS COHESION

The source mentions both concepts.

Desired direction:

HIGH COHESION
inside a class/module

LOWER COUPLING
between classes/modules

Memory:

Things that belong together
→ keep close

Things that do not
→ don't unnecessarily bind together

---

# 19. O — OPEN/CLOSED PRINCIPLE

🟢🟢🟢

The source states the principle essentially as:

> When a new requirement comes, we should prefer extension rather than repeatedly modifying existing classes.

Memory:

OPEN
for extension

CLOSED
for modification

---

# 20. THE PROBLEM OCP IS TARGETING

Imagine:

calculatePrice(type)

if type == REGULAR ...
else if type == PREMIUM ...
else if type == CORPORATE ...

Every new requirement means:

open existing code
↓
add another condition
↓
risk changing working behavior

The source's principle says new requirements should ideally be accommodated by:

EXTENSION

rather than repeatedly editing stable code.

⚠️ The exact pricing example above is illustrative; the source itself only states the extension-vs-modification principle.

---

# 21. OCP MENTAL MODEL

OLD REQUIREMENT
↓
existing implementation

NEW REQUIREMENT
↓
new extension

rather than

NEW REQUIREMENT
↓
rewrite existing working class

---

# 22. WHY THIS MATTERS

Every modification to existing working code potentially affects:

existing behavior

existing callers
existing tests

The source's desired direction is therefore:

new functionality
→ add extension

instead of:

new functionality
→ keep rewriting central class

---

# 23. OCP IS NOT “NEVER MODIFY CODE”

The wording in the notes is strong:

extension should not lead
to existing class modification

For memorization, preserve that source framing.

Do not reinterpret it as a literal guarantee that production code can never be edited.

The source's principle is about **designing extension points so new requirements do not constantly force modification of existing behavior**.

---

# 24. OCP — INTERVIEW ANSWER

> “Open/Closed Principle says software should be open for extension but closed for modification. In the notes' framing, when a new requirement arrives we should ideally extend the design rather than repeatedly modify existing stable classes.”

---

# 25. L — LISKOV SUBSTITUTION PRINCIPLE

🟢🟢🟢 **VERY IMPORTANT**

The source gives two core statements:

A subclass should extend,
not narrow down, its parent.

and:

If code expects A,
an object of subtype B
should be usable there.

---

# 26. LSP — SIMPLEST MENTAL MODEL

If:

class B extends A

then code that works with:

A

should still work when given:

B

without special hacks.

Memory:

> **A subtype must honor the expectations of the parent type.**

---

# 27. BAD INHERITANCE

Imagine parent promises:

Account
↓
withdraw()

Then subclass says:

FixedTermDepositAccount
↓
withdraw()
↓
UnsupportedOperationException

This is the **exact type of violation used in the source**.

The subclass inherited behavior that it cannot actually fulfill.

---

# 28. SOURCE CODE SMELL #1

## A subtype throws an exception for behavior it cannot fulfill

Source example:

Account
↓
withdraw()

FixedTermDepositAccount
↓
withdraw()
↓
UnsupportedOperationException

The notes call this a weakness in the inheritance hierarchy.

---

# 29. WHY IS THAT BAD?

Caller sees:

Account account

and reasonably believes:

withdraw()
is supported

But if actual object is:

FixedTermDepositAccount

the operation suddenly fails.

Therefore:

subtype
cannot safely substitute
for parent

---

# 30. SOURCE CODE SMELL #2

## Subtype provides no implementation for behavior it cannot fulfill

The notes give:

FileSystem
↓
deleteFile()

but:

ReadOnlyFileSystem

cannot meaningfully support deletion.

Again, the abstraction demands behavior that one subtype cannot honor.

---

# 31. REUSABLE LSP QUESTION

When designing inheritance ask:

> **“Can every subtype fulfill every promise made by the parent abstraction?”**

If answer is:

No, this subclass throws UnsupportedOperationException.

your inheritance design deserves scrutiny.

---

# 32. SOURCE CODE SMELL #3

## Client knows about concrete subtypes

The notes specifically warn about:

instanceof

and:

downcasting

as signs that the client may know too much about subtype details.

The source says this may indicate violations of:

Open/Closed Principle
+
Liskov Substitution Principle

---

# 33. WHY `instanceof` IS A SMELL IN SOURCE

Conceptually:

```
if (shape instanceof Circle) {
   ...
} else if (shape instanceof Rectangle) {
   ...
}
```

The caller is manually deciding subtype behavior.

The notes' recommendation is essentially:

move behavior
into the individual classes

rather than constantly asking which concrete subtype you received.

⚠️ The Shape code is illustrative; the source itself gives the `instanceof`/downcasting rule rather than this exact example.

---

# 34. SOURCE CODE SMELL #4

## Subtype always returns a fixed value

The notes describe a more subtle smell where a subtype:

hardcodes a value

for something that should represent meaningful changing state.

Example mentioned:

return 0

The concern is that the subtype is no longer fully behaving like the parent abstraction promises.

---

# 35. LSP — SOURCE SMELL CHECKLIST

Does subclass throw
UnsupportedOperationException?

Does subclass leave required

behavior effectively unimplemented?

Does caller need instanceof?

Does caller downcast?

Does subclass fake behavior
with meaningless constant values?

If yes:

CHECK LSP

---

# 36. LSP — INTERVIEW ANSWER

> “Liskov Substitution Principle means a subtype should be usable wherever its base type is expected without breaking the parent's behavioral expectations. The notes identify smells such as a subclass throwing UnsupportedOperationException for inherited behavior, providing no real implementation, forcing clients to use instanceof/downcasting, or faking required state with constant values.”

---

# 37. I — INTERFACE SEGREGATION PRINCIPLE

🟢🟢

The source gives two concise rules:

Many client-specific interfaces
are better than
one general-purpose interface.

and:

Clients should not be forced
to implement functions
they do not need.

---

# 38. ISP PROBLEM

Conceptually:

```
interface Worker {
    void work();
    void eat();
    void sleep();
    void fly();
    void swim();
}
```

A class may need:

work()

but be forced to implement unrelated methods.

That is exactly the category of design the source says ISP should avoid.

⚠️ This Worker interface is illustrative; the uploaded notes only state the principle.

---

# 39. FAT INTERFACE

The undesirable shape:

```
        GIANT INTERFACE
      /     |     |     \
     A      B     C      D
```

Everyone depends on everything

The source prefers:

small client-specific interfaces

---

# 40. INTERFACE SEGREGATION MENTAL MODEL

Instead of:

```
MegaInterface
├── A
├── B
├── C
├── D
└── E
```

think:

InterfaceForClient1

InterfaceForClient2

InterfaceForClient3

Clients depend only on the behavior they actually require.

---

# 41. ISP AND LSP ARE RELATED

Bad interface:

FileSystem
├── read()
├── write()
└── delete()

Subtype:

ReadOnlyFileSystem

cannot support:

write()
delete()

That creates the kind of LSP smell the source discusses.

So a useful source-derived connection is:

too broad abstraction
↓
subtype cannot implement all behavior
↓
LSP problem

and ISP tells you to make client-facing abstractions smaller.

---

# 42. ISP — INTERVIEW ANSWER

> “Interface Segregation Principle says clients should not be forced to depend on or implement methods they do not need. The notes prefer multiple client-specific interfaces over one large general-purpose interface.”

---

# 43. D — DEPENDENCY INVERSION PRINCIPLE

🟢🟢🟢

The source gives two short points:

create/use objects through interfaces

concrete class can be passed
through constructor injection

That is the full level of detail provided by the notes.

---

# 44. SOURCE MENTAL MODEL

Instead of:

High-level class
↓
new ConcreteDependency()

the notes want:

High-level class
↓
Interface
↑
ConcreteImplementation

with the concrete implementation supplied dynamically.

---

# 45. CONSTRUCTOR INJECTION

Source framing:

Class depends on interface
↓
constructor receives
concrete implementation

Conceptually:

```
class OrderService {
   private PaymentGateway gateway;

   OrderService(PaymentGateway gateway) {
```

```
        this.gateway = gateway;
    }
}
```

The exact Java snippet is illustrative; the source itself specifically mentions interfaces and constructor injection.

---

# 46. WHY THE INTERFACE MATTERS

The high-level class works against:

PaymentGateway

rather than binding itself directly to:

StripeGateway

or some other concrete class.

Then another implementation can be supplied without changing how the consumer expresses its dependency.

Again, this is the natural implication of the source's interface + constructor-injection formulation.

---

# 47. ⚠️ SOURCE GAP — FORMAL DIP DEFINITION

A common textbook treatment distinguishes:

high-level modules
low-level modules
abstractions
details

but your uploaded SOLID section **does not provide that formal explanation**.

The source only gives:

use interfaces

inject concrete implementation
through constructor

So that is what belongs in this source-faithful version.

---

# 48. DIP — INTERVIEW ANSWER

> “The notes express Dependency Inversion as depending on interfaces rather than directly constructing concrete dependencies, with the concrete implementation supplied dynamically through constructor injection.”

---

# 49. SOLID TOGETHER — ONE EXAMPLE

Suppose:

OrderService

handles:

Order calculation
Database persistence
Email
PDF generation
Payments

Potential questions:

SRP
Are there multiple unrelated responsibilities?

COHESION
Do these methods actually belong together?

OCP
Does every new payment type require editing OrderService?

LSP
Can every Payment implementation honor Payment's contract?

ISP
Are implementations forced to implement unrelated methods?

DIP
Does OrderService depend on a Payment abstraction
or directly construct one concrete gateway?

That is the easiest way to use SOLID in an LLD interview: **as diagnostic questions**.

---

# 50. SOLID IS NOT FIVE INDEPENDENT RULES

The source examples show relationships between them.

Huge responsibility
↓
low cohesion
↓
SRP problem
Huge interface
↓
subclass can't honor methods
↓
ISP + LSP problem
Client checks instanceof
↓
must change for every subtype
↓
LSP + OCP smell
Consumer directly creates concrete class
↓
hard dependency
↓
DIP concern

---

# 51. MOST IMPORTANT INTERVIEW DISTINCTION

## SRP vs ISP

SRP
asks about a CLASS'S RESPONSIBILITY

ISP
asks about what an INTERFACE
forces clients to depend on

---

# 52. OCP vs DIP

OCP
How do I add new behavior

without repeatedly changing old code?

DIP
What kind of dependency
should my class depend on?

DIP can help make extension easier, but the two principles are not identical.

---

# 53. ISP vs LSP

ISP

Don't make interface too broad.

LSP

Any subtype must correctly satisfy
the abstraction it claims to implement.

Poor ISP frequently creates LSP-style implementations that:

throw UnsupportedOperationException

---

# 54. SRP vs COHESION

SRP
ONE PURPOSE / REASON TO CHANGE

COHESION
HOW RELATED THE CLASS'S
METHODS/DATA ARE

They point toward the same design quality from different angles.

---

# 55. CODE-SMELL → PRINCIPLE MAPPING

| Code smell | Principle to inspect |
| --- | --- |
| God class with unrelated functionality | SRP / Cohesion |

| | |
|---|---|
| New feature always edits giant conditional | OCP |
| Subclass throws UnsupportedOperationException | LSP |
| Client uses `instanceof` everywhere | LSP / OCP |
| Class forced to implement useless methods | ISP |
| Consumer directly constructs concrete dependency | DIP |

This table reorganizes the specific smells and rules present in the source.

---

# 56. INTERVIEW Q — WHAT IS SRP?

> “A class should have one cohesive responsibility or one reason to change. Multiple methods are fine if all of them serve that same purpose.”

---

# 57. INTERVIEW Q — DOES SRP MEAN ONE METHOD?

> “No. The notes explicitly say one responsibility may require multiple operations. The issue is unrelated responsibilities, not method count.”

---

# 58. INTERVIEW Q — WHAT IS COHESION?

> “Cohesion measures how related the methods of a class are. A cohesive class performs one related function, whereas a non-cohesive class contains unrelated behavior and may need to be split.”

---

# 59. INTERVIEW Q — WHAT IS OCP?

> “Open/Closed Principle means open for extension and closed for modification. New requirements should preferably be handled through extension rather than repeatedly modifying existing classes.”

---

# 60. INTERVIEW Q — WHAT IS LSP?

> “A subtype must remain substitutable for its base type. The caller should not lose behavior promised by the parent simply because the concrete object is a particular subtype.”

---

# 61. INTERVIEW Q — GIVE AN LSP VIOLATION FROM THE NOTES

> “The notes use an Account hierarchy where `FixedTermDepositAccount` inherits `withdraw()` but cannot support it and throws `UnsupportedOperationException`. That means the subtype cannot fully satisfy the parent's contract.”

---

# 62. INTERVIEW Q — ANOTHER LSP SMELL?

> “A `ReadOnlyFileSystem` implementing a broader FileSystem contract but being unable to implement deletion is another example used in the notes.”

---

# 63. INTERVIEW Q — WHY IS `instanceof` A POSSIBLE SMELL?

> “The source says repeated `instanceof` or downcasting means client code knows about concrete subtypes, which can indicate Liskov and Open/Closed violations. Behavior may belong in the subtype itself instead.”

---

# 64. INTERVIEW Q — WHAT IS ISP?

> “Prefer multiple client-specific interfaces over one general-purpose interface. A client should not be forced to implement functions it doesn't need.”

---

# 65. INTERVIEW Q — WHAT IS DIP?

> “The notes recommend working through interfaces and providing the concrete implementation dynamically using constructor injection rather than hard-wiring the concrete class.”

---

# 66. COMMON BAD ANSWERS

### ❌ “SRP means one method per class.”

No.

The source explicitly allows multiple methods serving the same purpose.

---

### ❌ “A class should be split whenever it becomes large.”

Not the source's criterion.

The important question is:

Are the responsibilities unrelated?

---

### ❌ “OCP means existing code can never change.”

That turns the principle into an unrealistic literal rule.

The source's design direction is:

new requirement
→ extension
rather than repeatedly rewriting stable behavior

---

### ❌ “Inheritance is fine as long as the Java compiler accepts it.”

No.

LSP is about whether the subtype can actually honor the parent's expected behavior.

---

### ❌ “Throwing UnsupportedOperationException is always a perfectly fine implementation.”

In the exact inheritance examples in these notes, it is treated as a code smell indicating a poor hierarchy.

---

### ❌ “One massive interface is convenient.”

The notes explicitly prefer:

many client-specific interfaces

over one general-purpose interface.

---

### ❌ “Dependency injection and DIP are completely unrelated.”

The source specifically explains DIP using interfaces plus constructor injection.

---

# 67. SOLID — ONE-PAGE MEMORY SHEET

SOLID
=====

S — SINGLE RESPONSIBILITY

one class
→ one cohesive purpose
→ one reason to change

NOT
one method per class

BAD
unrelated responsibilities together

BENEFITS IN SOURCE
testing
lower coupling
organization

COHESION

methods/data belong together

HIGH COHESION
→ related behavior

LOW COHESION
→ unrelated behavior
→ consider splitting

O — OPEN/CLOSED

OPEN
→ extension

CLOSED
→ modification

new requirement
→ prefer extending design
→ don't repeatedly rewrite stable class

L — LISKOV SUBSTITUTION

B extends A

Then:
B must work
where A is expected

SOURCE SMELLS

UnsupportedOperationException

missing/no useful implementation

instanceof/downcasting

meaningless constant behavior

I — INTERFACE SEGREGATION

many focused interfaces
>
one giant interface

client should not implement
methods it doesn't need

D — DEPENDENCY INVERSION

depend through interfaces

supply concrete implementation
through constructor injection

---

# 68. SOLID MEMORY PHRASE

S
KEEP A CLASS FOCUSED

O
ADD, DON'T KEEP REWRITING

L
SUBTYPES MUST KEEP THE PROMISE

I
DON'T FORCE UNUSED METHODS

D
DEPEND THROUGH INTERFACES

---

# 69. 60-SECOND SENIOR ANSWER

If interviewer asks:

> **“Explain SOLID without definitions from a textbook.”**

Use:

> “I use SOLID mainly as a way to detect designs that will become hard to change. SRP says a class should have one cohesive responsibility, not necessarily one method. Open/Closed says new requirements should preferably extend the design instead of repeatedly modifying stable code. Liskov says a subtype must honor the behavior expected from its base type—the notes use a subclass throwing `UnsupportedOperationException` for inherited behavior as a violation. Interface Segregation says clients shouldn't depend on methods they don't need, so smaller client-specific interfaces are preferable to one giant interface. Dependency Inversion in these notes means depending through interfaces and injecting the concrete implementation, for example through the constructor.”

That covers the entire source SOLID section at interview depth.

---

# 70. SOLID DIAGNOSTIC CHECKLIST

When looking at an LLD class diagram, run:

CLASS
│
├─ Does it have several unrelated reasons to change?

```
│      → SRP
│
├─ Do its methods/data actually belong together?
│      → Cohesion
│
├─ Will every new type require changing existing logic?
│      → OCP
│
├─ Can every subtype honor the parent contract?
│      → LSP
│
├─ Are implementations forced to support useless methods?
│      → ISP
│
└─ Does high-level code directly bind itself to concrete classes?
       → DIP
```

---

# 71. WHAT THE SOURCE DOES NOT EXPLAIN HERE

To keep this handbook source-faithful:

SRP
→ reasonably explained

Cohesion
→ reasonably explained

OCP
→ definition only

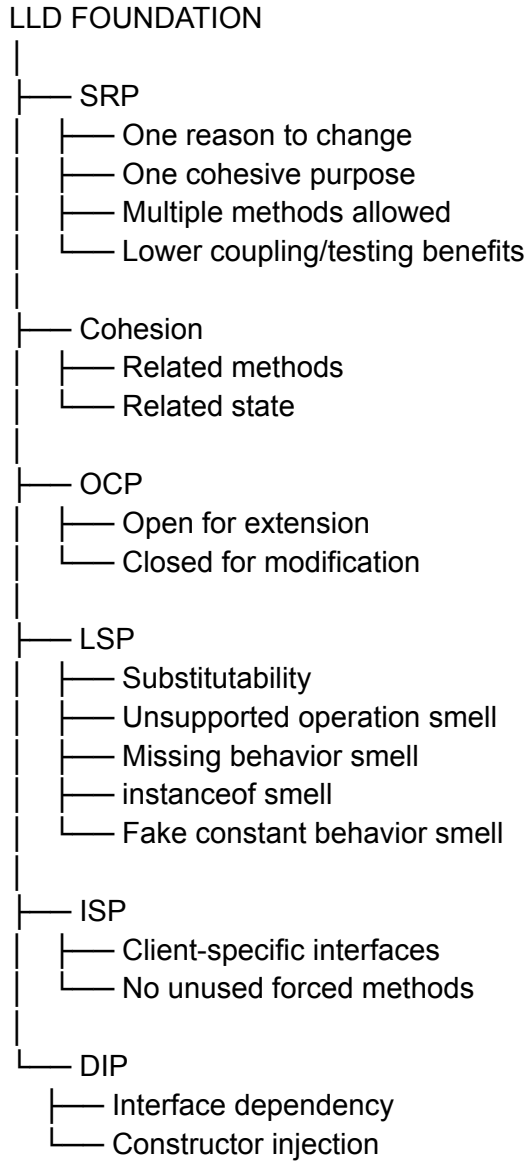
LSP
→ definition + several useful smells

ISP
→ two-line definition

DIP
→ interface + constructor-injection definition

The uploaded SOLID section does **not** provide deeper formal examples for OCP, ISP or DIP, so I have not presented outside textbook material as though it came from the PDF.

# 72. PART 13 MEMORY MAP



# CJ SYSTEM DESIGN INTERVIEW HANDBOOK

## PART 14 — CREATIONAL DESIGN PATTERNS: SINGLETON, FACTORY, ABSTRACT FACTORY & BUILDER

**Target:** Senior Java / Backend Engineer — ~8 YOE
**Goal:** Recognize **who should be responsible for object creation**, when construction itself becomes complicated, and how the source distinguishes the four creational patterns.

> **Priority legend**
> 🟢 **MUST KNOW** — interview floor
> 🟡 **SHOULD KNOW** — recognize and explain in code
> 🔵 **SOURCE-SPECIFIC** — exact framing/example from your notes
> ⚠️ **SOURCE INCONSISTENCY / GAP** — source itself is incomplete or internally inconsistent; do not silently "fix" it

The notes introduce:

CREATIONAL DESIGN PATTERNS
│
├── Singleton
├── Factory
├── Abstract Factory
└── Builder

The source then explicitly contrasts **Builder vs Factory** based on whether creation requires many construction steps or can happen in one factory call.

---

# 1. WHAT IS A CREATIONAL PATTERN?

The source groups these patterns together because they deal with:

OBJECT CREATION

Rather than letting every client freely decide:

new Something(...)

the creation process is controlled by some dedicated structure.

Mental model:

CLIENT
↓
CREATION MECHANISM
↓
OBJECT

---

# 2. THE FOUR PATTERNS — 10-SECOND MEMORY

SINGLETON
→ only one instance

FACTORY
→ choose which object to create

ABSTRACT FACTORY
→ choose which factory / product family to use

BUILDER
→ construct an object step-by-step

That is the source's progression.

---

# 3. SINGLETON — SOURCE DEFINITION

🟢🟢🟢

The notes define Singleton as a creational pattern that ensures:

ONE CLASS
↓
ONE INSTANCE

while providing a:

GLOBAL ACCESS POINT

to that instance.

---

# 4. SOURCE'S SINGLETON STRUCTURE

The notes identify three ingredients:

private constructor

private reference

public static access method

Source reasoning:

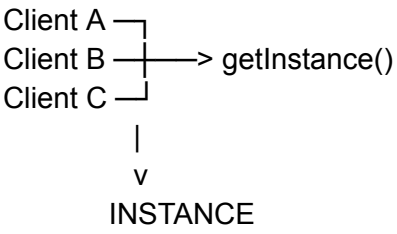
private constructor
→ outside classes cannot instantiate it

private reference
→ outside code cannot modify stored instance directly

public static method
→ controlled way to get instance

---

# 5. SINGLETON MENTAL MODEL




Every client receives the same object.

---

# 6. PRESIDENT ANALOGY

🔵

The source uses:

Country
↓
one President

If a president is requested repeatedly:

getPresident()

should return the existing president rather than constructing another one.

---

# 7. SOURCE JAVA SHAPE

The notes show the equivalent of:

```
public class AmericaPresident {

    private static AmericaPresident thePresident;

    private AmericaPresident() {}

    public static AmericaPresident getPresident() {
        if (thePresident == null) {
            thePresident = new AmericaPresident();
        }
        return thePresident;
    }
}
```

Core idea:

constructor
→ hidden

instance
→ class-owned

access
→ controlled method

---

# 8. LAZY CREATION

The notes say Singleton resources should avoid instantiation until the client actually calls:

getInstance()

So source flow is:

Application starts
↓
no object yet

first getInstance()
↓
create object

later getInstance()
↓
reuse object

This is the source's lazy-instantiation framing.

---

# 9. SOURCE SINGLETON USE CASES

The notes mention resources such as:

File System

Database connections

as Singleton-style examples.

Preserve these as source examples rather than universal recommendations.

---

# 10. THE MULTITHREADING PROBLEM

🟢

The source later makes the concurrency problem explicit.

Unsafe shape:

```
if (instance == null) {
    instance = new Singleton();
}
```

Imagine:

Thread T1 checks
instance == null

Thread T2 checks
instance == null

T1 creates object A

T2 creates object B

Now:

Singleton guarantee broken

The notes call this the concurrent creation problem.

---

# 11. SOURCE VERSION 1 — UNSYNCHRONIZED

The notes explicitly label the first version:

getInstance1()

as able to create multiple instances if two threads enter simultaneously.

Conceptually:

```
public static Singleton getInstance1() {
    if (instance == null) {
        instance = new Singleton();
    }
    return instance;
}
```

---

# 12. SOURCE VERSION 2 — SYNCHRONIZED METHOD

The notes then use:

public static synchronized Singleton getInstance2()

and say this makes creation thread-safe.

---

# 13. SYNCHRONIZED-METHOD COST IN SOURCE

The notes criticize synchronizing the whole accessor because:

every getInstance() call
↓
pays synchronization cost

even though the creation race matters primarily around initial creation.

This motivates:

Double Checked Locking

---

# 14. DOUBLE-CHECKED LOCKING — SOURCE IDEA

The notes say it is called:

DOUBLE CHECKED

because:

check #1
instance == null
before synchronization

check #2
instance == null
inside synchronized block

---

# 15. SOURCE DOUBLE-CHECK FLOW

getInstance()
↓
instance null?
|
YES
↓
acquire lock
↓
instance still null?
|
YES
↓
create

Later callers:

instance already exists
↓
skip synchronization

---

# 16. SOURCE VERSION 3

The notes present approximately:

```
public static Singleton getInstance3() {
    if (instance == null) {
        synchronized (Singleton.class) {
            if (instance == null) {
                instance = new Singleton();
            }
        }
    }
```

```
    return instance;
}
```

and describe its purpose as reducing synchronization cost by locking only the critical creation section.

---

# 17. ⚠️ SOURCE INCONSISTENCY — SINGLETON EXAMPLE

There is an internal inconsistency in the uploaded notes.

The `AmericaPresident` example as extracted already shows:

public synchronized static AmericaPresident getPresident()

but immediately afterward the text says:

> the above code can create multiple instances when called concurrently.

Later, however, the notes clearly distinguish:

Version 1
→ unsynchronized → unsafe

Version 2
→ synchronized → thread-safe

So for memorization use the **later three-version explanation**, because that is the internally coherent sequence in the source.

---

# 18. ⚠️ SOURCE BOUNDARY — DOUBLE-CHECKED LOCKING

The uploaded notes present the double-checked-locking implementation above.

They do **not** discuss any additional Java Memory Model requirements for that implementation in this section.

So this source-only handbook does not silently add them.

---

# 19. SINGLETON INTERVIEW FLOW

Need one instance
↓
private constructor
↓
static stored instance
↓
static accessor
↓
lazy creation
↓
concurrency race?
↓
synchronization
↓
source introduces
double-checked locking

---

# 20. SINGLETON — 30-SECOND ANSWER

> “The source defines Singleton as ensuring one instance of a class with a global access point. It uses a private constructor, a private stored reference and a public static accessor. Lazy creation means the instance isn't created until the accessor is called. The notes then address concurrent initialization: an unsynchronized null check can create multiple objects, synchronizing the whole accessor is thread-safe but adds synchronization cost to every call, so the source introduces double-checked locking to synchronize only the creation path.”

---

# 21. FACTORY DESIGN PATTERN

🟢🟢🟢

The source says Factory is used for:

creating an object
based on different parameters

---

# 22. FACTORY MENTAL MODEL

Without factory:

Client
↓
new Boy()
new Girl()
...

With source factory:

Client
  ↓
HumanFactory
  ↓
parameter
  ↓
Boy / Girl

---

# 23. SOURCE HUMAN EXAMPLE

The notes use:

HumanFactory.createHuman("boy")

to produce:

Boy

and:

HumanFactory.createHuman("girl")

to produce:

Girl

---

# 24. SOURCE FACTORY CODE

Approximately:

```
public class HumanFactory {

    public static Human createHuman(String m) {

        Human p = null;

        if (m.equals("boy")) {
            p = new Boy();
        } else if (m.equals("girl")) {
            p = new Girl();
        }

        return p;
    }
}
```

The creation choice lives in:

HumanFactory

rather than each caller manually deciding which concrete class to instantiate.

---

# 25. WHAT DOES THE FACTORY KNOW?

In the source example, Factory knows:

parameter
→ concrete type

Specifically:

"boy"
→ Boy

"girl"
→ Girl

---

# 26. WHAT DOES CLIENT ASK FOR?

Client essentially says:

Give me the appropriate object
for this parameter.

Factory decides:

which concrete class

---

# 27. FACTORY — CORE MEMORY

INPUT / PARAMETER
↓
FACTORY
↓
CHOOSE CONCRETE OBJECT
↓
RETURN COMMON TYPE

---

# 28. SOURCE TERMINOLOGY NOTE

The uploaded notes call this:

Factory Design Pattern

and use a static `HumanFactory.createHuman(...)` example.

This handbook preserves that terminology rather than reclassifying the source's example into a different pattern taxonomy.

---

# 29. FACTORY INTERVIEW ANSWER

> “In the source, Factory centralizes object creation based on a parameter. The HumanFactory example accepts a value such as `boy` or `girl`, chooses the appropriate concrete implementation and returns it through the common Human type.”

---

# 30. ABSTRACT FACTORY

🟢🟢

The source's shortest definition is:

> **Factory of factories.**

---

# 31. SOURCE CONCEPTUAL EXAMPLE — FURNITURE

The notes ask you to imagine related products:

Chair
Sofa
CoffeeTable

forming a product family.

Then multiple variants exist:

Modern

Victorian

ArtDeco

---

# 32. FAMILY MODEL

MODERN FAMILY
├── ModernChair
├── ModernSofa
└── ModernCoffeeTable

VICTORIAN FAMILY
├── VictorianChair
├── VictorianSofa
└── VictorianCoffeeTable

That is the conceptual explanation present in the notes.

---

# 33. WHY “FACTORY OF FACTORIES”?

Conceptual source model:

Client
↓
choose product-family factory
↓
ModernFactory
or
VictorianFactory
or
ArtDecoFactory

That selected factory creates the related products belonging to its variant.

---

# 34. SOURCE CODE EXAMPLE — CPU FACTORIES

The notes then show a different example:

CPU
│
├── AMDCPU

└── IntelCPU

with:

CPUFactory
│
├── AMDFactory
└── IntelFactory

---

# 35. SOURCE CPU FACTORY INTERFACE

```
interface CPUFactory {
    CPU produceCPU();
}
```

Implementations:

```
class AMDFactory implements CPUFactory {
    public CPU produceCPU() {
        return new AMDCPU();
    }
}
```

and similarly:

IntelFactory
→ IntelCPU

---

# 36. COMPUTER DEPENDS ON FACTORY INTERFACE

Source structure:

```
class Computer {

    CPU cpu;

    public Computer(CPUFactory factory) {
        cpu = factory.produceCPU();
        cpu.process();
    }
}
```

So:

Computer
→ CPUFactory abstraction

rather than directly constructing one fixed CPU type.

---

# 37. CLIENT SELECTS FACTORY

The source client contains a helper such as:

createSpecificFactory()

which returns:

AMDFactory
or
IntelFactory

depending on some requirement.

---

# 38. ABSTRACT FACTORY SOURCE FLOW

CLIENT
  ↓
choose factory
  ↓
CPUFactory
 /  \
AMD  Intel
|  |
v  v
AMDCPU  IntelCPU

---

# 39. ⚠️ SOURCE CONCEPT VS SOURCE CODE

There is an important mismatch in the uploaded notes.

The **conceptual definition** describes:

families of related products

such as:

Chair + Sofa + CoffeeTable

with multiple variants.

But the provided CPU code only shows one product abstraction:

CPU

produced by different factories.

So:

CONCEPT
→ product family

CODE EXAMPLE
→ only one product type demonstrated

Do not claim that the CPU example itself demonstrates the full related-product-family concept.

---

# 40. ABSTRACT FACTORY — INTERVIEW ANSWER

> “The source calls Abstract Factory a factory of factories. Its conceptual example is a family of related products such as Chair, Sofa and CoffeeTable, with variants such as Modern, Victorian and ArtDeco. The Java example then shows a CPUFactory abstraction with AMDFactory and IntelFactory implementations selected by the client.”

---

# 41. FACTORY VS ABSTRACT FACTORY — SOURCE MEMORY

FACTORY

parameter
↓
one factory decides
which concrete object

ABSTRACT FACTORY

client chooses a factory implementation
↓
that factory creates the appropriate product

And conceptually, the Abstract Factory section emphasizes:

families of related products

---

# 42. BUILDER PATTERN

🟢🟢🟢

The source says the key characteristic is:

> **Step-by-step construction.**

---

# 43. BUILDER SOURCE MENTAL MODEL

CREATE PRODUCT
↓
STEP 1
↓
STEP 2
↓
STEP 3
↓
FINISHED PRODUCT

Different builders can use the same broad construction process while making the steps produce different values.

---

# 44. SOURCE BUILDER PROBLEMS

The notes list:

Problem 1
→ too many subclasses

Problem 2
→ too many optional constructor parameters

as motivations around the Builder section.

---

# 45. SOURCE STARBUCKS PRODUCT

The product is:

```
class Starbucks {
    private String size;
```

```
    private String drink;
}
```

with setters for:

size

drink

---

# 46. ABSTRACT BUILDER

The source defines:

abstract class StarbucksBuilder

which stores:

Starbucks starbucks

and defines construction operations such as:

createStarbucks()

buildSize()

buildDrink()

---

# 47. CONCRETE BUILDER — TEA

Source:

TeaBuilder

builds:

size = large

drink = tea

---

# 48. CONCRETE BUILDER — COFFEE

Source:

CoffeeBuilder

builds:

size = medium

drink = coffee

---

# 49. DIRECTOR — WAITER

The source introduces:

Waiter

as the **director**.

It receives a:

StarbucksBuilder

and invokes the construction steps:

createStarbucks()

buildDrink()

buildSize()

---

# 50. DIRECTOR MENTAL MODEL

```
          BUILDER
            ↑
            |
CLIENT → DIRECTOR
            |
        construction steps
            |
            v
          PRODUCT
```

---

# 51. SOURCE CUSTOMER FLOW

Client:

```
Waiter waiter = new Waiter();

StarbucksBuilder coffeeBuilder =
        new CoffeeBuilder();
```

```
waiter.setStarbucksBuilder(coffeeBuilder);

waiter.constructStarbucks();

Starbucks drink =
        waiter.getstarbucksDrink();
```

The client chooses the concrete builder while the director controls the build sequence.

---

# 52. SWITCH BUILDER, KEEP PROCESS

Instead of:

CoffeeBuilder

the source notes you could provide:

TeaBuilder

The director still invokes:

create
buildDrink
buildSize

but the resulting product differs.

---

# 53. BUILDER SOURCE IDEA

SAME CONSTRUCTION FLOW

+

DIFFERENT BUILD IMPLEMENTATIONS

=

DIFFERENT PRODUCT CONFIGURATION

That is the exact structure of the Starbucks example.

---

# 54. BUILDER VS FACTORY

🟢🟢🟢 **MEMORIZE THIS EXACT SOURCE DISTINCTION**

The notes explicitly say:

BUILDER
→ many steps to create object

FACTORY
→ factory can create whole object
within one method call

---

# 55. VISUAL — FACTORY

create(type)
↓
FACTORY
↓
complete object

---

# 56. VISUAL — BUILDER

Builder
↓
create
↓
step 1
↓
step 2
↓
step 3
↓
complete object

---

# 57. THE FOUR PATTERNS SIDE BY SIDE

| Pattern | Source's central idea |
|---|---|
| Singleton | Ensure one instance and global access |
| Factory | Create different concrete objects from parameters |
| Abstract Factory | Factory of factories / related product-family concept |
| Builder | Build an object step-by-step |

---

# 58. WHEN THE SOURCE WOULD MAKE YOU THINK “SINGLETON”

Question:

Should there only be one controlled instance?

Then source points toward:

Singleton

---

# 59. WHEN THE SOURCE WOULD MAKE YOU THINK “FACTORY”

Question:

Do I need to choose
which concrete object to create
based on input/parameters?

Then:

Factory

---

# 60. WHEN THE SOURCE WOULD MAKE YOU THINK “ABSTRACT FACTORY”

Question:

Do I need interchangeable factories,
conceptually for related product variants?

Then:

Abstract Factory

---

# 61. WHEN THE SOURCE WOULD MAKE YOU THINK “BUILDER”

Question:

Does object creation itself consist
of several construction steps?

Then:

Builder

---

# 62. INTERVIEW Q — WHAT IS SINGLETON?

> “Singleton ensures that a class has only one instance while providing a global access point. The source implements that through a private constructor, a stored static reference and a public static accessor.”

---

# 63. INTERVIEW Q — WHY PRIVATE CONSTRUCTOR?

> “So external classes cannot freely create another instance.”

---

# 64. INTERVIEW Q — WHAT IS THE CONCURRENCY PROBLEM WITH LAZY SINGLETON?

> “Two threads may both observe a null instance before either creates it and then both construct an object. The source first uses synchronization and then introduces double-checked locking.”

---

# 65. INTERVIEW Q — WHY DOUBLE-CHECKED LOCKING?

Source answer:

> “To avoid paying the synchronization cost on every accessor call once the instance has already been created. The notes check for null once outside and once inside the synchronized block.”

---

# 66. INTERVIEW Q — WHAT IS FACTORY?

> “The source describes Factory as creating an object based on parameters. Its example takes `boy` or `girl` and returns the corresponding Human implementation.”

---

# 67. INTERVIEW Q — WHAT IS ABSTRACT FACTORY?

> “The notes call it a factory of factories and conceptually describe families of related products with different variants, such as Modern or Victorian furniture.”

---

# 68. INTERVIEW Q — WHAT IS BUILDER?

> “Builder creates an object through a step-by-step process. The Starbucks example uses an abstract builder, concrete Tea and Coffee builders, and a Waiter director that invokes the construction steps.”

---

# 69. INTERVIEW Q — BUILDER VS FACTORY?

Use the source's exact distinction:

> “Use Builder when creation involves many steps. Use Factory when the factory can create the complete object in one method call.”

---

# 70. COMMON BAD ANSWERS

### ❌ “Singleton just means static methods.”

No.

The source specifically talks about:

one object instance

and controlled access to that instance.

---

### ❌ “A private constructor alone creates Singleton.”

No.

Source structure also includes:

stored instance
+
controlled accessor

---

### ❌ “Factory means the client creates the concrete class.”

Opposite of the source example.

The factory performs the concrete creation decision.

---

### ❌ “Abstract Factory is just a longer name for Factory.”

The source distinguishes it as:

Factory of factories

with a conceptual emphasis on related product families.

---

### ❌ “Builder and Factory solve exactly the same construction problem.”

The notes explicitly distinguish:

Builder
→ many creation steps

Factory
→ one-call object creation

---

# 71. SOURCE LIMITATIONS YOU SHOULD KNOW

The uploaded material is useful but not fully consistent.

### Singleton

The President example contains a synchronization inconsistency; the later Version 1/2/3 explanation is clearer.

### Double-checked locking

The source gives the null-check + synchronized-block implementation but does not discuss additional Java Memory Model details.

### Abstract Factory

The conceptual furniture example discusses **families of related products**, but the CPU code demonstrates only one product abstraction per factory.

### Builder

The source uses the classic Director + Abstract Builder style and does not discuss alternative Java Builder styles in this section.

These are **source boundaries**, not silently filled gaps.

---

# 72. ONE-PAGE MEMORY SHEET

CREATIONAL PATTERNS
==========

SINGLETON
---

GOAL
one instance

STRUCTURE
private constructor
private stored instance
public static accessor

LAZY
create on first access

PROBLEM
multithreading can create >1

SOURCE SOLUTIONS
synchronized accessor
double-checked locking

FACTORY
---

GOAL

choose object based on parameter

HumanFactory

"boy"
→ Boy

"girl"
→ Girl

ABSTRACT FACTORY
---

"Factory of factories"

CONCEPT
family of related products

Chair
Sofa
CoffeeTable

variants:
Modern
Victorian
ArtDeco

CODE EXAMPLE
CPUFactory
├── AMDFactory
└── IntelFactory

BUILDER
---

GOAL
step-by-step construction

PRODUCT
Starbucks

ABSTRACT BUILDER
StarbucksBuilder

CONCRETE BUILDERS
TeaBuilder
CoffeeBuilder

DIRECTOR
Waiter

CLIENT
Customer

BUILDER vs FACTORY

Builder
→ many steps

Factory
→ complete object
in one method call

---

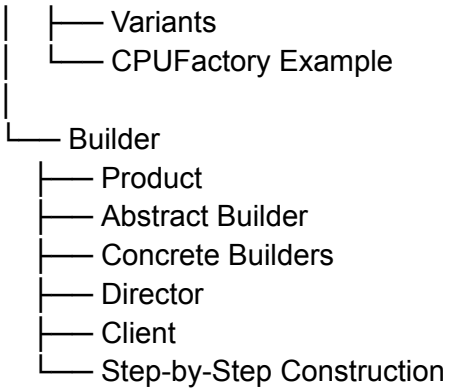
# 73. 60-SECOND SENIOR ANSWER

> “The notes group Singleton, Factory, Abstract Factory and Builder as creational patterns because all of them control object creation. Singleton controls the number of instances and provides one shared access point; the source then discusses lazy initialization, synchronization and double-checked locking. Factory centralizes which concrete object gets created based on input parameters. Abstract Factory is described as a factory of factories and conceptually handles variants of related product families. Builder is for step-by-step object construction: the source's Starbucks example has a product, an abstract builder, Tea and Coffee concrete builders and a Waiter director. The notes' simplest distinction is that Factory can create the object in one method call, whereas Builder is useful when creation requires multiple steps.”

---

# 74. PART 14 MEMORY MAP

CREATIONAL
│
├── Singleton
│ ├── One Instance
│ ├── Private Constructor
│ ├── Static Access
│ ├── Lazy Creation
│ ├── Synchronization
│ └── Double Checked Locking
│
├── Factory
│ ├── Parameter
│ ├── Creation Decision
│ └── Concrete Product
│
├── Abstract Factory
│ ├── Factory of Factories
│ ├── Product Family Concept


Variants
CPUFactory Example
Builder
Product
Abstract Builder
Concrete Builders
Director
Client
Step-by-Step Construction

---